

FOORT

1.0

Generated by Doxygen 1.12.0

1 Namespace Index	1
1.1 Namespace List	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	9
4.1 File List	9
5 Namespace Documentation	11
5.1 Config Namespace Reference	11
5.1.1 Detailed Description	12
5.1.2 Typedef Documentation	12
5.1.2.1 ConfigCollection	12
5.1.2.2 SettingError	12
5.1.3 Function Documentation	12
5.1.3.1 GetGeodesicIntegrator()	12
5.1.3.2 GetMesh()	12
5.1.3.3 GetMetric()	13
5.1.3.4 GetOutputHandler()	13
5.1.3.5 GetSource()	13
5.1.3.6 GetViewScreen()	14
5.1.3.7 InitializeDiagnostics()	14
5.1.3.8 InitializeScreenOutput()	14
5.1.3.9 InitializeTerminations()	15
5.1.4 Variable Documentation	15
5.1.4.1 Output_Important_Default	15
5.1.4.2 Output_Other_Default	15
5.2 ConfigReader Namespace Reference	15
5.2.1 Detailed Description	16
5.2.2 Variable Documentation	16
5.2.2.1 MaxStreamSize	16
5.3 Integrators Namespace Reference	16
5.3.1 Function Documentation	16
5.3.1.1 GetAdaptiveStep()	16
5.3.1.2 GetFullIntegratorDescription()	17
5.3.1.3 IntegrateGeodesicStep_RK4()	17
5.3.1.4 IntegrateGeodesicStep_Verlet()	17
5.3.2 Variable Documentation	18
5.3.2.1 delta_nodiv0	18
5.3.2.2 Derivative_hval	18

5.3.2.3 epsilon	18
5.3.2.4 IntegratorDescription	18
5.3.2.5 SmallestPossibleStepsize	18
5.3.2.6 VerletVelocityTolerance	19
5.4 tk Namespace Reference	19
5.5 tk::internal Namespace Reference	19
5.6 Utilities Namespace Reference	19
5.6.1 Function Documentation	19
5.6.1.1 GetDiagNameStrings()	19
5.6.1.2 GetFirstLineInfoString()	20
5.6.1.3 GetTimeStampString()	20
6 Class Documentation	21
6.1 tk::internal::band_matrix Class Reference	21
6.1.1 Constructor & Destructor Documentation	21
6.1.1.1 band_matrix() [1/2]	21
6.1.1.2 band_matrix() [2/2]	22
6.1.1.3 ~band_matrix()	22
6.1.2 Member Function Documentation	22
6.1.2.1 dim()	22
6.1.2.2 l_solve()	22
6.1.2.3 lu_decompose()	22
6.1.2.4 lu_solve()	22
6.1.2.5 num_lower()	22
6.1.2.6 num_upper()	22
6.1.2.7 operator()() [1/2]	22
6.1.2.8 operator()() [2/2]	23
6.1.2.9 r_solve()	23
6.1.2.10 resize()	23
6.1.2.11 saved_diag() [1/2]	23
6.1.2.12 saved_diag() [2/2]	23
6.1.3 Member Data Documentation	23
6.1.3.1 m_lower	23
6.1.3.2 m_upper	23
6.2 BosonStarMetric Class Reference	24
6.2.1 Detailed Description	25
6.2.2 Constructor & Destructor Documentation	25
6.2.2.1 BosonStarMetric()	25
6.2.3 Member Function Documentation	26
6.2.3.1 getFullDescriptionStr()	26
6.2.3.2 getMetric_dd()	26
6.2.3.3 getMetric_uu()	26

6.2.3.4 read_data()	27
6.2.4 Member Data Documentation	27
6.2.4.1 m_m_filename	27
6.2.4.2 m_mSpline	27
6.2.4.3 m_num_lines	27
6.2.4.4 m_Phi_filename	27
6.2.4.5 m_Phi_infinity	27
6.2.4.6 m_PhiSpline	27
6.3 BoundarySphereTermination Class Reference	28
6.3.1 Detailed Description	29
6.3.2 Constructor & Destructor Documentation	29
6.3.2.1 BoundarySphereTermination()	29
6.3.3 Member Function Documentation	29
6.3.3.1 CheckTermination()	29
6.3.3.2 getFullDescriptionStr()	29
6.3.4 Member Data Documentation	29
6.3.4.1 TermOptions	29
6.4 BoundarySphereTermOptions Struct Reference	30
6.4.1 Detailed Description	30
6.4.2 Constructor & Destructor Documentation	30
6.4.2.1 BoundarySphereTermOptions()	30
6.4.3 Member Data Documentation	31
6.4.3.1 rLogScale	31
6.4.3.2 SphereRadius	31
6.5 ClosestRadiusDiagnostic Class Reference	31
6.5.1 Detailed Description	32
6.5.2 Constructor & Destructor Documentation	32
6.5.2.1 ClosestRadiusDiagnostic()	32
6.5.3 Member Function Documentation	32
6.5.3.1 FinalDataValDistance()	32
6.5.3.2 getFinalDataVal()	33
6.5.3.3 getFullDataStr()	33
6.5.3.4 getFullDescriptionStr()	33
6.5.3.5 getNameStr()	34
6.5.3.6 Reset()	34
6.5.3.7 UpdateData()	34
6.5.4 Member Data Documentation	34
6.5.4.1 DiagOptions	34
6.5.4.2 m_ClosestRadius	34
6.6 ClosestRadiusOptions Struct Reference	35
6.6.1 Detailed Description	35
6.6.2 Constructor & Destructor Documentation	35

6.6.2.1 ClosestRadiusOptions()	35
6.6.3 Member Data Documentation	36
6.6.3.1 RLogScale	36
6.7 ConfigReader::ConfigCollection Class Reference	36
6.7.1 Member Typedef Documentation	37
6.7.1.1 ConfigSettingValue	37
6.7.2 Member Function Documentation	38
6.7.2.1 DisplayCollection()	38
6.7.2.2 DisplaySetting()	38
6.7.2.3 DisplayTabs()	38
6.7.2.4 Exists()	38
6.7.2.5 GetSettingIndex()	39
6.7.2.6 IsCollection() [1/2]	39
6.7.2.7 IsCollection() [2/2]	39
6.7.2.8 LookupValue() [1/2]	40
6.7.2.9 LookupValue() [2/2]	40
6.7.2.10 LookupValueInteger() [1/7]	41
6.7.2.11 LookupValueInteger() [2/7]	41
6.7.2.12 LookupValueInteger() [3/7]	42
6.7.2.13 LookupValueInteger() [4/7]	42
6.7.2.14 LookupValueInteger() [5/7]	42
6.7.2.15 LookupValueInteger() [6/7]	43
6.7.2.16 LookupValueInteger() [7/7]	43
6.7.2.17 NrSettings()	43
6.7.2.18 operator[]() [1/2]	43
6.7.2.19 operator[]() [2/2]	44
6.7.2.20 ReadCollection()	44
6.7.2.21 ReadFile()	44
6.7.2.22 ReadSettingName()	45
6.7.2.23 ReadSettingSpecificChar()	45
6.7.2.24 ReadSettingValue()	45
6.7.3 Member Data Documentation	46
6.7.3.1 m_Settings	46
6.8 ConfigReader::ConfigReaderException Class Reference	46
6.8.1 Constructor & Destructor Documentation	46
6.8.1.1 ConfigReaderException()	46
6.8.2 Member Function Documentation	46
6.8.2.1 trace()	46
6.8.3 Member Data Documentation	47
6.8.3.1 m_settingtrace	47
6.9 ConfigReader::ConfigCollection::ConfigSetting Struct Reference	47
6.9.1 Member Data Documentation	47

6.9.1.1 SettingName	47
6.9.1.2 SettingValue	47
6.10 Diagnostic Class Reference	47
6.10.1 Detailed Description	48
6.10.2 Constructor & Destructor Documentation	48
6.10.2.1 Diagnostic() [1/2]	48
6.10.2.2 Diagnostic() [2/2]	48
6.10.2.3 ~Diagnostic()	48
6.10.3 Member Function Documentation	48
6.10.3.1 DecideUpdate()	48
6.10.3.2 FinalDataValDistance()	49
6.10.3.3 getFinalDataVal()	49
6.10.3.4 getFullDataStr()	49
6.10.3.5 getFullDescriptionStr()	49
6.10.3.6 getNameStr()	50
6.10.3.7 Reset()	50
6.10.3.8 UpdateData()	50
6.10.4 Member Data Documentation	50
6.10.4.1 m_OwnerGeodesic	50
6.10.4.2 m_StepsSinceUpdated	50
6.11 DiagnosticOptions Struct Reference	51
6.11.1 Detailed Description	51
6.11.2 Constructor & Destructor Documentation	51
6.11.2.1 DiagnosticOptions()	51
6.11.2.2 ~DiagnosticOptions()	51
6.11.3 Member Data Documentation	51
6.11.3.1 theUpdateFrequency	51
6.12 EmissionModel Struct Reference	52
6.12.1 Detailed Description	52
6.12.2 Constructor & Destructor Documentation	52
6.12.2.1 ~EmissionModel()	52
6.12.3 Member Function Documentation	52
6.12.3.1 GetEmission()	52
6.12.3.2 getFullDescriptionStr()	53
6.13 EquatorialEmissionDiagnostic Class Reference	53
6.13.1 Detailed Description	55
6.13.2 Constructor & Destructor Documentation	55
6.13.2.1 EquatorialEmissionDiagnostic()	55
6.13.3 Member Function Documentation	55
6.13.3.1 FinalDataValDistance()	55
6.13.3.2 getFinalDataVal()	55
6.13.3.3 getFullDataStr()	56

6.13.3.4 getFullDescriptionStr()	56
6.13.3.5 getNameStr()	56
6.13.3.6 Reset()	56
6.13.3.7 UpdateData()	57
6.13.4 Member Data Documentation	57
6.13.4.1 DiagOptions	57
6.13.4.2 m_Intensity	57
6.14 EquatorialEmissionOptions Struct Reference	57
6.14.1 Detailed Description	58
6.14.2 Constructor & Destructor Documentation	58
6.14.2.1 EquatorialEmissionOptions()	58
6.14.3 Member Data Documentation	59
6.14.3.1 EquatPassUpperBound	59
6.14.3.2 GeometricFudgeFactor	59
6.14.3.3 RedShiftPower	59
6.14.3.4 RLogScale	59
6.14.3.5 TheEmissionModel	59
6.14.3.6 TheFluidVelocityModel	59
6.15 EquatorialPassesDiagnostic Class Reference	59
6.15.1 Detailed Description	61
6.15.2 Constructor & Destructor Documentation	61
6.15.2.1 EquatorialPassesDiagnostic()	61
6.15.3 Member Function Documentation	61
6.15.3.1 FinalDataValDistance()	61
6.15.3.2 getFinalDataVal()	61
6.15.3.3 getFullDataStr()	62
6.15.3.4 getFullDescriptionStr()	62
6.15.3.5 getNameStr()	62
6.15.3.6 Reset()	62
6.15.3.7 UpdateData()	63
6.15.4 Member Data Documentation	63
6.15.4.1 DiagOptions	63
6.15.4.2 m_EquatPasses	63
6.15.4.3 m_PrevTheta	63
6.16 EquatorialPassesOptions Struct Reference	63
6.16.1 Detailed Description	64
6.16.2 Constructor & Destructor Documentation	64
6.16.2.1 EquatorialPassesOptions()	64
6.16.3 Member Data Documentation	64
6.16.3.1 Threshold	64
6.17 FlatSpaceMetric Class Reference	65
6.17.1 Detailed Description	66

6.17.2 Constructor & Destructor Documentation	66
6.17.2.1 FlatSpaceMetric()	66
6.17.3 Member Function Documentation	66
6.17.3.1 getFullDescriptionStr()	66
6.17.3.2 getMetric_dd()	66
6.17.3.3 getMetric_uu()	66
6.18 FluidVelocityModel Struct Reference	67
6.18.1 Detailed Description	67
6.18.2 Constructor & Destructor Documentation	68
6.18.2.1 FluidVelocityModel()	68
6.18.2.2 ~FluidVelocityModel()	68
6.18.3 Member Function Documentation	68
6.18.3.1 GetFourVelocityd()	68
6.18.3.2 getFullDescriptionStr()	68
6.18.4 Member Data Documentation	68
6.18.4.1 m_theMetric	68
6.19 FourColorScreenDiagnostic Class Reference	69
6.19.1 Detailed Description	70
6.19.2 Constructor & Destructor Documentation	70
6.19.2.1 FourColorScreenDiagnostic()	70
6.19.3 Member Function Documentation	70
6.19.3.1 FinalDataValDistance()	70
6.19.3.2 getFinalDataVal()	70
6.19.3.3 getFullDataStr()	71
6.19.3.4 getFullDescriptionStr()	71
6.19.3.5 getNameStr()	71
6.19.3.6 Reset()	71
6.19.3.7 UpdateData()	72
6.19.4 Member Data Documentation	72
6.19.4.1 m_quadrant	72
6.20 GeneralCircularRadialFluid Struct Reference	72
6.20.1 Detailed Description	73
6.20.2 Constructor & Destructor Documentation	73
6.20.2.1 GeneralCircularRadialFluid()	73
6.20.3 Member Function Documentation	74
6.20.3.1 FindISCO()	74
6.20.3.2 GetChrstrRaisedDer()	74
6.20.3.3 GetCircularVelocityd()	74
6.20.3.4 GetFourVelocityd()	74
6.20.3.5 getFullDescriptionStr()	75
6.20.3.6 GetInsideISCOCircularVelocityd()	75
6.20.3.7 GetRadialVelocityd()	75

6.20.4 Member Data Documentation	76
6.20.4.1 m_betaPhi	76
6.20.4.2 m_betaR	76
6.20.4.3 m_ISCOexists	76
6.20.4.4 m_ISCOpphi	76
6.20.4.5 m_ISCOpt	76
6.20.4.6 m_ISCOr	76
6.20.4.7 m_subKeplerParam	76
6.21 GeneralSingularityTermination Class Reference	77
6.21.1 Detailed Description	78
6.21.2 Constructor & Destructor Documentation	78
6.21.2.1 GeneralSingularityTermination()	78
6.21.3 Member Function Documentation	78
6.21.3.1 CheckTermination()	78
6.21.3.2 getFullDescriptionStr()	78
6.21.3.3 SingularityToString()	78
6.21.4 Member Data Documentation	79
6.21.4.1 TermOptions	79
6.22 GeneralSingularityTermOptions Struct Reference	79
6.22.1 Detailed Description	80
6.22.2 Constructor & Destructor Documentation	80
6.22.2.1 GeneralSingularityTermOptions()	80
6.22.3 Member Data Documentation	80
6.22.3.1 Epsilon	80
6.22.3.2 OutputToConsole	80
6.22.3.3 rLogScale	80
6.22.3.4 Singularities	81
6.23 Geodesic Class Reference	81
6.23.1 Detailed Description	82
6.23.2 Constructor & Destructor Documentation	82
6.23.2.1 Geodesic() [1/3]	82
6.23.2.2 Geodesic() [2/3]	82
6.23.2.3 Geodesic() [3/3]	82
6.23.3 Member Function Documentation	83
6.23.3.1 getAllOutputStr()	83
6.23.3.2 getCurrentLambda()	83
6.23.3.3 getCurrentPos()	83
6.23.3.4 getCurrentVel()	84
6.23.3.5 getDiagnosticFinalValue()	84
6.23.3.6 getScreenIndex()	84
6.23.3.7 getTermCondition()	84
6.23.3.8 operator=()	84

6.23.3.9 Reset()	84
6.23.3.10 Update()	85
6.23.4 Member Data Documentation	85
6.23.4.1 m_AllDiagnostics	85
6.23.4.2 m_AllTerminations	85
6.23.4.3 m_curLambda	85
6.23.4.4 m_CurrentPos	85
6.23.4.5 m_CurrentVel	86
6.23.4.6 m_ScreenIndex	86
6.23.4.7 m_TermCond	86
6.23.4.8 m_theIntegrator	86
6.23.4.9 m_theMetric	86
6.23.4.10 m_theSource	86
6.24 GeodesicOutputHandler Class Reference	86
6.24.1 Detailed Description	88
6.24.2 Constructor & Destructor Documentation	88
6.24.2.1 GeodesicOutputHandler() [1/2]	88
6.24.2.2 GeodesicOutputHandler() [2/2]	88
6.24.3 Member Function Documentation	88
6.24.3.1 GetFileName()	88
6.24.3.2 getFullDescriptionStr()	89
6.24.3.3 NewGeodesicOutput()	89
6.24.3.4 OpenForFirstTime()	89
6.24.3.5 OutputFinished()	89
6.24.3.6 PrepareForOutput()	89
6.24.3.7 WriteCachedOutputToFile()	90
6.24.4 Member Data Documentation	90
6.24.4.1 m_AllCachedData	90
6.24.4.2 m_CurrentFullFiles	90
6.24.4.3 m_CurrentGeodesicsInFile	90
6.24.4.4 m_DiagNames	90
6.24.4.5 m_FileExtension	90
6.24.4.6 m_FilePrefix	91
6.24.4.7 m_FirstLineInfoString	91
6.24.4.8 m_nrGeodesicsPerFile	91
6.24.4.9 m_nrOutputsToCache	91
6.24.4.10 m_PrevCached	91
6.24.4.11 m_PrintFirstLineInfo	91
6.24.4.12 m_TimeStamp	91
6.24.4.13 m_WriteToConsole	92
6.25 GeodesicPositionDiagnostic Class Reference	92
6.25.1 Detailed Description	93

6.25.2 Constructor & Destructor Documentation	93
6.25.2.1 GeodesicPositionDiagnostic()	93
6.25.3 Member Function Documentation	93
6.25.3.1 FinalDataValDistance()	93
6.25.3.2 getFinalDataVal()	94
6.25.3.3 getFullDataStr()	94
6.25.3.4 getFullDescriptionStr()	94
6.25.3.5 getNameStr()	95
6.25.3.6 Reset()	95
6.25.3.7 UpdateData()	95
6.25.4 Member Data Documentation	95
6.25.4.1 DiagOptions	95
6.25.4.2 m_AllSavedPoints	95
6.26 GeodesicPositionOptions Struct Reference	96
6.26.1 Detailed Description	96
6.26.2 Constructor & Destructor Documentation	96
6.26.2.1 GeodesicPositionOptions()	96
6.26.3 Member Data Documentation	97
6.26.3.1 OutputNrSteps	97
6.27 GLMJohnsonSUEmission Struct Reference	97
6.27.1 Detailed Description	98
6.27.2 Constructor & Destructor Documentation	98
6.27.2.1 GLMJohnsonSUEmission()	98
6.27.3 Member Function Documentation	98
6.27.3.1 GetEmission()	98
6.27.3.2 getFullDescriptionStr()	98
6.27.4 Member Data Documentation	99
6.27.4.1 m_gamma	99
6.27.4.2 m_mu	99
6.27.4.3 m_sigma	99
6.28 HorizonTermination Class Reference	99
6.28.1 Detailed Description	100
6.28.2 Constructor & Destructor Documentation	100
6.28.2.1 HorizonTermination()	100
6.28.3 Member Function Documentation	100
6.28.3.1 CheckTermination()	100
6.28.3.2 getFullDescriptionStr()	101
6.28.4 Member Data Documentation	101
6.28.4.1 TermOptions	101
6.29 HorizonTermOptions Struct Reference	101
6.29.1 Detailed Description	102
6.29.2 Constructor & Destructor Documentation	102

6.29.2.1 HorizonTermOptions()	102
6.29.3 Member Data Documentation	102
6.29.3.1 AtHorizonEps	102
6.29.3.2 HorizonRadius	102
6.29.3.3 rLogScale	103
6.30 InputCertainPixelsMesh Class Reference	103
6.30.1 Detailed Description	104
6.30.2 Constructor & Destructor Documentation	104
6.30.2.1 InputCertainPixelsMesh() [1/3]	104
6.30.2.2 InputCertainPixelsMesh() [2/3]	104
6.30.2.3 InputCertainPixelsMesh() [3/3]	104
6.30.3 Member Function Documentation	105
6.30.3.1 EndCurrentLoop()	105
6.30.3.2 GeodesicFinished()	105
6.30.3.3 getCurNrGeodesics()	105
6.30.3.4 getFullDescriptionStr()	106
6.30.3.5 getNewInitConds()	106
6.30.3.6 IsFinished()	106
6.30.4 Member Data Documentation	106
6.30.4.1 m_Finished	106
6.30.4.2 m_PixelsToIntegrate	107
6.30.4.3 m_RowColumnSize	107
6.30.4.4 m_TotalPixels	107
6.31 JohannsenMetric Class Reference	107
6.31.1 Detailed Description	109
6.31.2 Constructor & Destructor Documentation	109
6.31.2.1 JohannsenMetric() [1/2]	109
6.31.2.2 JohannsenMetric() [2/2]	109
6.31.3 Member Function Documentation	109
6.31.3.1 getFullDescriptionStr()	109
6.31.3.2 getMetric_dd()	109
6.31.3.3 getMetric_uu()	110
6.31.4 Member Data Documentation	110
6.31.4.1 m_alpha13Param	110
6.31.4.2 m_alpha22Param	110
6.31.4.3 m_alpha52Param	110
6.31.4.4 m_aParam	111
6.31.4.5 m_eps3Param	111
6.32 KerrMetric Class Reference	111
6.32.1 Detailed Description	112
6.32.2 Constructor & Destructor Documentation	113
6.32.2.1 KerrMetric() [1/2]	113

6.32.2.2 KerrMetric() [2/2]	113
6.32.3 Member Function Documentation	113
6.32.3.1 getFullDescriptionStr()	113
6.32.3.2 getMetric_dd()	113
6.32.3.3 getMetric_uu()	113
6.32.4 Member Data Documentation	114
6.32.4.1 m_aParam	114
6.32.4.2 m_mParam	114
6.33 KerrSchildMetric Class Reference	114
6.33.1 Detailed Description	116
6.33.2 Constructor & Destructor Documentation	116
6.33.2.1 KerrSchildMetric() [1/2]	116
6.33.2.2 KerrSchildMetric() [2/2]	116
6.33.3 Member Function Documentation	116
6.33.3.1 getFullDescriptionStr()	116
6.33.3.2 getMetric_dd()	116
6.33.3.3 getMetric_uu()	117
6.33.4 Member Data Documentation	117
6.33.4.1 m_aParam	117
6.34 MankoNovikovMetric Class Reference	117
6.34.1 Detailed Description	119
6.34.2 Constructor & Destructor Documentation	119
6.34.2.1 MankoNovikovMetric() [1/2]	119
6.34.2.2 MankoNovikovMetric() [2/2]	119
6.34.3 Member Function Documentation	119
6.34.3.1 getFullDescriptionStr()	119
6.34.3.2 getMetric_dd()	119
6.34.3.3 getMetric_uu()	120
6.34.4 Member Data Documentation	120
6.34.4.1 m_alpha3Param	120
6.34.4.2 m_alphaParam	120
6.34.4.3 m_aParam	120
6.34.4.4 m_kParam	121
6.35 Mesh Class Reference	121
6.35.1 Detailed Description	121
6.35.2 Constructor & Destructor Documentation	122
6.35.2.1 Mesh()	122
6.35.2.2 ~Mesh()	122
6.35.3 Member Function Documentation	122
6.35.3.1 EndCurrentLoop()	122
6.35.3.2 GeodesicFinished()	122
6.35.3.3 getCurNrGeodesics()	122

6.35.3.4 getFullDescriptionStr()	122
6.35.3.5 getNewInitConds()	123
6.35.3.6 IsFinished()	123
6.35.4 Member Data Documentation	123
6.35.4.1 m_DistanceDiagnostic	123
6.36 Metric Class Reference	123
6.36.1 Detailed Description	124
6.36.2 Constructor & Destructor Documentation	124
6.36.2.1 ~Metric()	124
6.36.2.2 Metric()	124
6.36.3 Member Function Documentation	124
6.36.3.1 getChristoffel_udd()	124
6.36.3.2 getFullDescriptionStr()	125
6.36.3.3 getKretschmann()	125
6.36.3.4 getMetric_dd()	125
6.36.3.5 getMetric_uu()	126
6.36.3.6 getRiemann_uddd()	126
6.36.3.7 getrLogScale()	126
6.36.4 Member Data Documentation	126
6.36.4.1 m_rLogScale	126
6.36.4.2 m_Symmetries	127
6.37 NaNTermination Class Reference	127
6.37.1 Detailed Description	128
6.37.2 Constructor & Destructor Documentation	128
6.37.2.1 NaNTermination()	128
6.37.3 Member Function Documentation	128
6.37.3.1 CheckTermination()	128
6.37.3.2 getFullDescriptionStr()	128
6.37.4 Member Data Documentation	129
6.37.4.1 TermOptions	129
6.38 NaNTermOptions Struct Reference	129
6.38.1 Detailed Description	130
6.38.2 Constructor & Destructor Documentation	130
6.38.2.1 NaNTermOptions()	130
6.38.3 Member Data Documentation	130
6.38.3.1 OutputToConsole	130
6.39 NoSource Class Reference	130
6.39.1 Detailed Description	131
6.39.2 Constructor & Destructor Documentation	131
6.39.2.1 NoSource()	131
6.39.3 Member Function Documentation	131
6.39.3.1 getFullDescriptionStr()	131

6.39.3.2 getSource()	131
6.40 SquareSubdivisionMesh::PixelInfo Struct Reference	132
6.40.1 Detailed Description	132
6.40.2 Constructor & Destructor Documentation	133
6.40.2.1 PixelInfo()	133
6.40.3 Member Data Documentation	133
6.40.3.1 DiagValue	133
6.40.3.2 Index	133
6.40.3.3 LowerNbrIndex	133
6.40.3.4 RightNbrIndex	133
6.40.3.5 SubdivideLevel	133
6.40.3.6 Weight	134
6.41 SquareSubdivisionMeshV2::PixelInfo Struct Reference	134
6.41.1 Detailed Description	134
6.41.2 Constructor & Destructor Documentation	135
6.41.2.1 PixelInfo()	135
6.41.3 Member Data Documentation	135
6.41.3.1 DiagValue	135
6.41.3.2 DownNbr	135
6.41.3.3 Index	135
6.41.3.4 LeftNbr	135
6.41.3.5 RightNbr	135
6.41.3.6 SEdiagNbr	135
6.41.3.7 SubdivideLevel	136
6.41.3.8 UpNbr	136
6.41.3.9 Weight	136
6.42 RasheedLarsenMetric Class Reference	136
6.42.1 Detailed Description	138
6.42.2 Constructor & Destructor Documentation	138
6.42.2.1 RasheedLarsenMetric() [1/2]	138
6.42.2.2 RasheedLarsenMetric() [2/2]	138
6.42.3 Member Function Documentation	138
6.42.3.1 getFullDescriptionStr()	138
6.42.3.2 getMetric_dd()	138
6.42.3.3 getMetric_uu()	139
6.42.4 Member Data Documentation	139
6.42.4.1 m_aParam	139
6.42.4.2 m_mParam	139
6.42.4.3 m_pParam	139
6.42.4.4 m_qParam	140
6.43 SimpleSquareMesh Class Reference	140
6.43.1 Detailed Description	141

6.43.2 Constructor & Destructor Documentation	141
6.43.2.1 SimpleSquareMesh() [1/2]	141
6.43.2.2 SimpleSquareMesh() [2/2]	141
6.43.3 Member Function Documentation	141
6.43.3.1 EndCurrentLoop()	141
6.43.3.2 GeodesicFinished()	141
6.43.3.3 getCurNrGeodesics()	142
6.43.3.4 getFullDescriptionStr()	142
6.43.3.5 getNewInitConds()	142
6.43.3.6 IsFinished()	143
6.43.4 Member Data Documentation	143
6.43.4.1 m_Finished	143
6.43.4.2 m_RowColumnSize	143
6.43.4.3 m_TotalPixels	143
6.44 SingularityMetric Class Reference	143
6.44.1 Detailed Description	144
6.44.2 Constructor & Destructor Documentation	144
6.44.2.1 SingularityMetric()	144
6.44.3 Member Function Documentation	145
6.44.3.1 getSingularities()	145
6.44.4 Member Data Documentation	145
6.44.4.1 m_AllSingularities	145
6.45 Source Class Reference	145
6.45.1 Detailed Description	146
6.45.2 Constructor & Destructor Documentation	146
6.45.2.1 Source()	146
6.45.2.2 ~Source()	146
6.45.3 Member Function Documentation	146
6.45.3.1 getFullDescriptionStr()	146
6.45.3.2 getSource()	147
6.45.4 Member Data Documentation	147
6.45.4.1 m_theMetric	147
6.46 SphericalHorizonMetric Class Reference	147
6.46.1 Detailed Description	148
6.46.2 Constructor & Destructor Documentation	148
6.46.2.1 SphericalHorizonMetric() [1/2]	148
6.46.2.2 SphericalHorizonMetric() [2/2]	148
6.46.3 Member Function Documentation	149
6.46.3.1 getHorizonRadius()	149
6.46.4 Member Data Documentation	149
6.46.4.1 m_HorizonRadius	149
6.47 tk::spline Class Reference	149

6.47.1 Member Enumeration Documentation	150
6.47.1.1 bd_type	150
6.47.1.2 spline_type	150
6.47.2 Constructor & Destructor Documentation	151
6.47.2.1 spline() [1/2]	151
6.47.2.2 spline() [2/2]	151
6.47.3 Member Function Documentation	151
6.47.3.1 deriv()	151
6.47.3.2 find_closest()	151
6.47.3.3 get_x()	151
6.47.3.4 get_x_max()	151
6.47.3.5 get_x_min()	152
6.47.3.6 get_y()	152
6.47.3.7 make_monotonic()	152
6.47.3.8 operator()()	152
6.47.3.9 set_boundary()	152
6.47.3.10 set_coeffs_from_b()	152
6.47.3.11 set_points()	152
6.47.4 Member Data Documentation	152
6.47.4.1 m_b	152
6.47.4.2 m_c	153
6.47.4.3 m_c0	153
6.47.4.4 m_d	153
6.47.4.5 m_left	153
6.47.4.6 m_left_value	153
6.47.4.7 m_made_monotonic	153
6.47.4.8 m_right	153
6.47.4.9 m_right_value	153
6.47.4.10 m_type	153
6.47.4.11 m_x	153
6.47.4.12 m_y	154
6.48 SquareSubdivisionMesh Class Reference	154
6.48.1 Detailed Description	156
6.48.2 Constructor & Destructor Documentation	156
6.48.2.1 SquareSubdivisionMesh() [1/2]	156
6.48.2.2 SquareSubdivisionMesh() [2/2]	156
6.48.3 Member Function Documentation	156
6.48.3.1 EndCurrentLoop()	156
6.48.3.2 ExpInt()	156
6.48.3.3 GeodesicFinished()	157
6.48.3.4 getCurNrGeodesics()	157
6.48.3.5 getFullDescriptionStr()	157

6.48.3.6	getNewInitConds()	157
6.48.3.7	InitializeFirstGrid()	158
6.48.3.8	IsFinished()	158
6.48.3.9	SubdivideAndQueue()	158
6.48.3.10	UpdateAllNeighbors()	158
6.48.3.11	UpdateAllWeights()	159
6.48.4	Member Data Documentation	159
6.48.4.1	m_AllPixels	159
6.48.4.2	m_CurrentPixelQueue	159
6.48.4.3	m_CurrentPixelQueueDone	159
6.48.4.4	m_InfinitePixels	159
6.48.4.5	m_InitialPixels	159
6.48.4.6	m_InitialSubDivideToFinal	160
6.48.4.7	m_IterationPixels	160
6.48.4.8	m_MaxPixels	160
6.48.4.9	m_MaxSubdivide	160
6.48.4.10	m_PixelsLeft	160
6.48.4.11	m_RowColumnSize	160
6.49	SquareSubdivisionMeshV2 Class Reference	161
6.49.1	Detailed Description	163
6.49.2	Constructor & Destructor Documentation	163
6.49.2.1	SquareSubdivisionMeshV2() [1/2]	163
6.49.2.2	SquareSubdivisionMeshV2() [2/2]	163
6.49.3	Member Function Documentation	163
6.49.3.1	EndCurrentLoop()	163
6.49.3.2	ExpInt()	163
6.49.3.3	GeodesicFinished()	164
6.49.3.4	getCurNrGeodesics()	164
6.49.3.5	GetDown()	164
6.49.3.6	getFullDescriptionStr()	165
6.49.3.7	GetLeft()	165
6.49.3.8	getNewInitConds()	165
6.49.3.9	GetRight()	165
6.49.3.10	GetUp()	166
6.49.3.11	InitializeFirstGrid()	166
6.49.3.12	IsFinished()	166
6.49.3.13	SubdivideAndQueue()	166
6.49.3.14	UpdateAllWeights()	167
6.49.4	Member Data Documentation	167
6.49.4.1	m_ActivePixels	167
6.49.4.2	m_AllPixels	167
6.49.4.3	m_CurrentPixelQueue	167

6.49.4.4 m_CurrentPixelQueueDone	167
6.49.4.5 m_CurrentPixelUpdating	168
6.49.4.6 m_InfinitePixels	168
6.49.4.7 m_InitialPixels	168
6.49.4.8 m_InitialSubDivideToFinal	168
6.49.4.9 m_IterationPixels	168
6.49.4.10 m_MaxPixels	168
6.49.4.11 m_MaxSubdivide	168
6.49.4.12 m_PixelsIntegrated	168
6.49.4.13 m_PixelsLeft	169
6.49.4.14 m_RowColumnSize	169
6.50 ST3CrMetric Class Reference	169
6.50.1 Detailed Description	170
6.50.2 Constructor & Destructor Documentation	171
6.50.2.1 ST3CrMetric()	171
6.50.3 Member Function Documentation	171
6.50.3.1 f_om_phi()	171
6.50.3.2 f_phi()	171
6.50.3.3 get_omega()	172
6.50.3.4 getFullDescriptionStr()	172
6.50.3.5 getMetric_dd()	172
6.50.3.6 getMetric_uu()	173
6.50.4 Member Data Documentation	173
6.50.4.1 m_lambda	173
6.50.4.2 m_P	173
6.50.4.3 m_q0	173
6.51 Termination Class Reference	174
6.51.1 Detailed Description	174
6.51.2 Constructor & Destructor Documentation	174
6.51.2.1 Termination() [1/2]	174
6.51.2.2 Termination() [2/2]	174
6.51.2.3 ~Termination()	175
6.51.3 Member Function Documentation	175
6.51.3.1 CheckTermination()	175
6.51.3.2 DecideUpdate()	175
6.51.3.3 getFullDescriptionStr()	175
6.51.3.4 Reset()	175
6.51.4 Member Data Documentation	175
6.51.4.1 m_OwnerGeodesic	175
6.51.4.2 m_StepsSinceUpdated	176
6.52 TerminationOptions Struct Reference	176
6.52.1 Detailed Description	176

6.52.2 Constructor & Destructor Documentation	176
6.52.2.1 TerminationOptions()	176
6.52.2.2 ~TerminationOptions()	177
6.52.3 Member Data Documentation	177
6.52.3.1 UpdateEveryNSteps	177
6.53 ThetaSingularityTermination Class Reference	177
6.53.1 Detailed Description	178
6.53.2 Constructor & Destructor Documentation	178
6.53.2.1 ThetaSingularityTermination()	178
6.53.3 Member Function Documentation	178
6.53.3.1 CheckTermination()	178
6.53.3.2 getFullDescriptionStr()	178
6.53.4 Member Data Documentation	179
6.53.4.1 TermOptions	179
6.54 ThetaSingularityTermOptions Struct Reference	179
6.54.1 Detailed Description	180
6.54.2 Constructor & Destructor Documentation	180
6.54.2.1 ThetaSingularityTermOptions()	180
6.54.3 Member Data Documentation	180
6.54.3.1 ThetaSingEpsilon	180
6.55 TimeOutTermination Class Reference	180
6.55.1 Detailed Description	181
6.55.2 Constructor & Destructor Documentation	181
6.55.2.1 TimeOutTermination()	181
6.55.3 Member Function Documentation	181
6.55.3.1 CheckTermination()	181
6.55.3.2 getFullDescriptionStr()	182
6.55.3.3 Reset()	182
6.55.4 Member Data Documentation	182
6.55.4.1 m_CurNrSteps	182
6.55.4.2 TermOptions	182
6.56 TimeOutTermOptions Struct Reference	182
6.56.1 Detailed Description	183
6.56.2 Constructor & Destructor Documentation	183
6.56.2.1 TimeOutTermOptions()	183
6.56.3 Member Data Documentation	183
6.56.3.1 MaxSteps	183
6.57 Utilities::Timer Class Reference	184
6.57.1 Member Typedef Documentation	184
6.57.1.1 Clock	184
6.57.1.2 Second	184
6.57.2 Member Function Documentation	184

6.57.2.1 elapsed()	184
6.57.2.2 reset()	184
6.57.3 Member Data Documentation	185
6.57.3.1 m_beg	185
6.58 UpdateFrequency Struct Reference	185
6.58.1 Detailed Description	185
6.58.2 Member Data Documentation	185
6.58.2.1 UpdateFinish	185
6.58.2.2 UpdateNSteps	185
6.58.2.3 UpdateStart	185
6.59 ViewScreen Class Reference	186
6.59.1 Constructor & Destructor Documentation	186
6.59.1.1 ViewScreen() [1/2]	186
6.59.1.2 ViewScreen() [2/2]	186
6.59.2 Member Function Documentation	187
6.59.2.1 ConstructVielbein()	187
6.59.2.2 EndCurrentLoop()	187
6.59.2.3 GeodesicFinished()	187
6.59.2.4 getCurNrGeodesics()	187
6.59.2.5 getFullDescriptionStr()	187
6.59.2.6 IsFinished()	187
6.59.2.7 SetNewInitialConditions()	187
6.59.3 Member Data Documentation	187
6.59.3.1 m_Direction	187
6.59.3.2 m_GeodType	188
6.59.3.3 m_Metric_dd	188
6.59.3.4 m_Pos	188
6.59.3.5 m_rLogScale	188
6.59.3.6 m_ScreenCenter	188
6.59.3.7 m_ScreenSize	188
6.59.3.8 m_theMesh	188
6.59.3.9 m_theMetric	188
6.59.3.10 m_Vielbein	188
7 File Documentation	189
7.1 FOORT/src/Config.cpp File Reference	189
7.1.1 Detailed Description	189
7.2 FOORT/src/Config.h File Reference	189
7.2.1 Detailed Description	191
7.2.2 Macro Definition Documentation	191
7.2.2.1 CONFIGURATION_MODE	191
7.3 Config.h	191

7.4 FOORT/src/ConfigReader.cpp File Reference	192
7.4.1 Detailed Description	192
7.5 FOORT/src/ConfigReader.h File Reference	193
7.5.1 Detailed Description	193
7.6 ConfigReader.h	194
7.7 FOORT/src/Diagnostics.cpp File Reference	196
7.7.1 Detailed Description	196
7.7.2 Function Documentation	196
7.7.2.1 CreateDiagnosticVector()	196
7.8 FOORT/src/Diagnostics.h File Reference	197
7.8.1 Detailed Description	198
7.8.2 Typedef Documentation	198
7.8.2.1 DiagBitflag	198
7.8.2.2 DiagnosticUniqueVector	199
7.8.3 Function Documentation	199
7.8.3.1 CreateDiagnosticVector()	199
7.8.4 Variable Documentation	199
7.8.4.1 Diag_ClosestRadius	199
7.8.4.2 Diag_EquatorialEmission	199
7.8.4.3 Diag_EquatorialPasses	199
7.8.4.4 Diag_FourColorScreen	200
7.8.4.5 Diag_GeodesicPosition	200
7.8.4.6 Diag_None	200
7.9 Diagnostics.h	200
7.10 FOORT/src/DiagnosticsEmission.cpp File Reference	205
7.10.1 Detailed Description	205
7.11 FOORT/src/DiagnosticsEmission.h File Reference	206
7.11.1 Detailed Description	206
7.12 DiagnosticsEmission.h	207
7.13 FOORT/src/Geodesic.cpp File Reference	208
7.13.1 Detailed Description	208
7.14 FOORT/src/Geodesic.h File Reference	209
7.14.1 Detailed Description	209
7.15 Geodesic.h	210
7.16 FOORT/src/Geometry.h File Reference	211
7.16.1 Detailed Description	213
7.16.2 Macro Definition Documentation	213
7.16.2.1 LARGE_COUNTER_MAX	213
7.16.2.2 PIXEL_MAX	213
7.16.3 Typedef Documentation	213
7.16.3.1 FourIndex	213
7.16.3.2 largecounter	214

7.16.3.3 OneIndex	214
7.16.3.4 pixelcoord	214
7.16.3.5 Point	214
7.16.3.6 real	214
7.16.3.7 ScreenIndex	214
7.16.3.8 ScreenPoint	214
7.16.3.9 Singularity	214
7.16.3.10 SingularityCoord	215
7.16.3.11 ThreeIndex	215
7.16.3.12 TwoIndex	215
7.16.4 Function Documentation	215
7.16.4.1 operator*() [1/2]	215
7.16.4.2 operator*() [2/2]	215
7.16.4.3 operator+()	216
7.16.4.4 operator-()	216
7.16.4.5 operator/()	217
7.16.4.6 toString() [1/3]	217
7.16.4.7 toString() [2/3]	218
7.16.4.8 toString() [3/3]	218
7.16.5 Variable Documentation	219
7.16.5.1 dimension	219
7.16.5.2 pi	219
7.17 Geometry.h	219
7.18 FOORT/src/InOutOutput.cpp File Reference	221
7.18.1 Detailed Description	221
7.18.2 Function Documentation	222
7.18.2.1 GetLoopMessageFrequency()	222
7.18.2.2 ScreenOutput()	222
7.18.2.3 SetLoopMessageFrequency()	222
7.18.2.4 SetOutputLevel()	222
7.19 FOORT/src/InOutOutput.h File Reference	223
7.19.1 Detailed Description	224
7.19.2 Enumeration Type Documentation	224
7.19.2.1 OutputLevel	224
7.19.3 Function Documentation	224
7.19.3.1 GetLoopMessageFrequency()	224
7.19.3.2 ScreenOutput()	225
7.19.3.3 SetLoopMessageFrequency()	225
7.19.3.4 SetOutputLevel()	225
7.20 InputOutput.h	226
7.21 FOORT/src/Integrators.cpp File Reference	227
7.21.1 Detailed Description	227

7.22 FOORT/src/Integrators.h File Reference	228
7.22.1 Detailed Description	229
7.22.2 Typedef Documentation	229
7.22.2.1 GeodesicIntegratorFunc	229
7.23 Integrators.h	229
7.24 FOORT/src/Main.cpp File Reference	230
7.24.1 Function Documentation	230
7.24.1.1 LoadPrecompiledOptions()	230
7.24.1.2 main()	231
7.25 FOORT/src/Mesh.cpp File Reference	231
7.25.1 Detailed Description	231
7.26 FOORT/src/Mesh.h File Reference	231
7.26.1 Detailed Description	232
7.27 Mesh.h	232
7.28 FOORT/src/Metric.cpp File Reference	237
7.29 FOORT/src/Metric.h File Reference	237
7.29.1 Detailed Description	238
7.30 Metric.h	238
7.31 FOORT/src/Spline.cpp File Reference	241
7.31.1 Detailed Description	242
7.32 FOORT/src/Spline.h File Reference	242
7.32.1 Detailed Description	243
7.33 Spline.h	243
7.34 FOORT/src/Terminations.cpp File Reference	245
7.34.1 Detailed Description	245
7.34.2 Function Documentation	246
7.34.2.1 CreateTerminationVector()	246
7.35 FOORT/src/Terminations.h File Reference	246
7.35.1 Detailed Description	247
7.35.2 Typedef Documentation	248
7.35.2.1 TermBitflag	248
7.35.2.2 TerminationUniqueVector	248
7.35.3 Enumeration Type Documentation	248
7.35.3.1 Term	248
7.35.4 Function Documentation	248
7.35.4.1 CreateTerminationVector()	248
7.35.5 Variable Documentation	249
7.35.5.1 Term_BoundarySphere	249
7.35.5.2 Term_GeneralSingularity	249
7.35.5.3 Term_Horizon	249
7.35.5.4 Term_NaN	249
7.35.5.5 Term_None	249

7.35.5.6 Term_ThetaSingularity	249
7.35.5.7 Term_TimeOut	249
7.36 Terminations.h	250
7.37 FOORT/src/Utilities.cpp File Reference	254
7.38 FOORT/src/Utilities.h File Reference	254
7.39 Utilities.h	255
7.40 FOORT/src/ViewScreen.cpp File Reference	255
7.41 FOORT/src/ViewScreen.h File Reference	255
7.41.1 Enumeration Type Documentation	256
7.41.1.1 GeodesicType	256
7.42 ViewScreen.h	256
Index	259

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

Config	Namespace for all configuration functions that initialize objects based on configuration file . . .	11
ConfigReader	Namespace to put all ConfigReader objects	15
Integrators		16
tk		19
tk::internal		19
Utilities		19

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

tk::internal::band_matrix	21
ConfigReader::ConfigCollection	36
ConfigReader::ConfigCollection::ConfigSetting	47
Diagnostic	47
ClosestRadiusDiagnostic	31
EquatorialPassesDiagnostic	59
EquatorialEmissionDiagnostic	53
FourColorScreenDiagnostic	69
GeodesicPositionDiagnostic	92
DiagnosticOptions	51
ClosestRadiusOptions	35
EquatorialPassesOptions	63
EquatorialEmissionOptions	57
GeodesicPositionOptions	96
EmissionModel	52
GLMJohnsonSUEmission	97
FluidVelocityModel	67
GeneralCircularRadialFluid	72
Geodesic	81
GeodesicOutputHandler	86
Mesh	121
InputCertainPixelsMesh	103
SimpleSquareMesh	140
SquareSubdivisionMesh	154
SquareSubdivisionMeshV2	161
Metric	123
BosonStarMetric	24
FlatSpaceMetric	65
SingularityMetric	143
ST3CrMetric	169
SphericalHorizonMetric	147
JohannsenMetric	107
KerrMetric	111

KerrSchildMetric	114
MankoNovikovMetric	117
RasheedLarsenMetric	136
SquareSubdivisionMesh::PixelInfo	132
SquareSubdivisionMeshV2::PixelInfo	134
std::runtime_error	
ConfigReader::ConfigReaderException	46
Source	145
NoSource	130
tk::spline	149
Termination	174
BoundarySphereTermination	28
GeneralSingularityTermination	77
HorizonTermination	99
NaNTermination	127
ThetaSingularityTermination	177
TimeOutTermination	180
TerminationOptions	176
BoundarySphereTermOptions	30
GeneralSingularityTermOptions	79
HorizonTermOptions	101
NaNTermOptions	129
ThetaSingularityTermOptions	179
TimeOutTermOptions	182
Utilities::Timer	184
UpdateFrequency	185
ViewScreen	186

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

tk::internal::band_matrix	21
BosonStarMetric	
Boson star metric with solitonic potential	24
BoundarySphereTermination	
BoundarySphereTermination : Terminate geodesics if they reach outside of a boundary sphere	28
BoundarySphereTermOptions	
Options for BoundarySphereTermination	30
ClosestRadiusDiagnostic	
Diagnostic that keeps track of the closest (true, not log) radius that the geodesic passes through	31
ClosestRadiusOptions	
Options for ClosestRadiusDiagnostic	35
ConfigReader::ConfigCollection	36
ConfigReader::ConfigReaderException	46
ConfigReader::ConfigCollection::ConfigSetting	47
Diagnostic	
Abstract base class for all diagnostics	47
DiagnosticOptions	
Base class for DiagnosticOptions . Other Diagnostics can inherit from here if they require more options	51
EmissionModel	
Emmision model abstract base class	52
EquatorialEmissionDiagnostic	
Diagnostic that calculates brightness intensity for the geodesic, based on a specified equatorial disc emission model	53
EquatorialEmissionOptions	
Options for EquatorialEmissionDiagnostic	57
EquatorialPassesDiagnostic	
Diagnostic that counts the number of passes through the equatorial plane	59
EquatorialPassesOptions	
Options for EquatorialPassesDiagnostic	63
FlatSpaceMetric	
Flat space metric in 4D	65
FluidVelocityModel	
Fluid velocity model abstract base class	67
FourColorScreenDiagnostic	
The four color screen diagnostic	69

GeneralCircularRadialFluid	
General circular radial fluid velocity model	72
GeneralSingularityTermination	
GeneralSingularityTermination : Terminate geodesics if they get too close to one of a number of singularities (of arbitrary codimension)	77
GeneralSingularityTermOptions	
Options for GeneralSingularityTermination	79
Geodesic	
Geodesic class: an instance of this class is created for each Geodesic that is integrated	81
GeodesicOutputHandler	
// GeodesicOutputHandler handles all of the output to file. It gets passed all of the output strings for every Geodesic , it then stores this data until it eventually writes all data to the appropriate files	86
GeodesicPositionDiagnostic	
Geodesic position tracker	92
GeodesicPositionOptions	
Options for GeodesicPositionDiagnostic	96
GLMJohnsonSUEmission	
The Johnson SU emission model used in GLM	97
HorizonTermination	
HorizonTermination : Terminate geodesics if they get too close to the horizon	99
HorizonTermOptions	
Options for HorizonTermination	101
InputCertainPixelsMesh	
Mesh which integrates only certain user-inputted pixels	103
JohannsenMetric	
Johannsen black hole metric	107
KerrMetric	
The Kerr metric	111
KerrSchildMetric	
Kerr metric in Kerr-Schild coordinates	114
MankoNovikovMetric	
Mano-Novikov metric (with angular momentum and M3 parameter turned on)	117
Mesh	
Abstract base Mesh class	121
Metric	
The abstract base class for all Metrics	123
NaNTermination	
NaNTermination : Terminate geodesics if they contain a NaN in position or velocity	127
NaNTermOptions	
Options for NaNTermination	129
NoSource	
NoSource class	130
SquareSubdivisionMesh::PixelInfo	
A struct the Mesh uses to keep all information about a given pixel	132
SquareSubdivisionMeshV2::PixelInfo	
A struct the Mesh uses to keep all information about a given pixel	134
RasheedLarsenMetric	
Rasheed-Larsen black hole	136
SimpleSquareMesh	
A simple square mesh that will integrate a square of evenly spaced pixels	140
SingularityMetric	
Abstract base class for a metric with an arbitrary number of singularities (of arbitrary codimension)	143
Source	
Abstract Source base class	145
SphericalHorizonMetric	
Abstract base class for a metric with a spherical horizon (i.e. horizon at constant radius r) . . .	147
tk::spline	149

SquareSubdivisionMesh	
Adaptive subdivision Mesh	154
SquareSubdivisionMeshV2	
Adaptive subdivision Mesh	161
ST3CrMetric	
Ring fuzzball metric	169
Termination	
Abstract base class for all Terminations	174
TerminationOptions	
Base class for all TerminationOptions structs. Other TerminationOptions can inherit from here if they require more options	176
ThetaSingularityTermination	
ThetaSingularityTermination : Terminate geodesics if they get too close to a polar coordinate singularity	177
ThetaSingularityTermOptions	
Options for ThetaSingularityTermination	179
TimeOutTermination	
TimeOutTermination : Terminate geodesics if they take too many steps	180
TimeOutTermOptions	
Options for TimeOutTermination	182
Utilities::Timer	184
UpdateFrequency	
Struct to hold the information for a Diagnostic to update itself	185
ViewScreen	186

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

FOORT/src/ Config.cpp	189
FOORT/src/ Config.h	
Functions that read the configuration file and initialize all objects	189
FOORT/src/ ConfigReader.cpp	192
FOORT/src/ ConfigReader.h	
Classes used to read in a configuration file and store the settings	193
FOORT/src/ Diagnostics.cpp	196
FOORT/src/ Diagnostics.h	
Declarations of abstract base Diagnostic class and all its descendants. All Definitions in Diagnostics.cpp	197
FOORT/src/ DiagnosticsEmission.cpp	205
FOORT/src/ DiagnosticsEmission.h	
Declarations of emission and fluid velocity models used for equatorial disc emission	206
FOORT/src/ Geodesic.cpp	208
FOORT/src/ Geodesic.h	
Declarations of abstract base Source class and all its descendants. Declaration of Geodesic class	209
FOORT/src/ Geometry.h	
Definitions and some operations with geometric objects.alignas	211
FOORT/src/ InputOutput.cpp	221
FOORT/src/ InputOutput.h	
Declarations of functions & classes that deal with the output to files and to the screen	223
FOORT/src/ Integrators.cpp	227
FOORT/src/ Integrators.h	
Declarations of integrator functions that integrate geodesic equation	228
FOORT/src/ Main.cpp	230
FOORT/src/ Mesh.cpp	231
FOORT/src/ Mesh.h	
Declarations of abstract base Mesh class and all its descendants	231
FOORT/src/ Metric.cpp	237
FOORT/src/ Metric.h	
Declarations of abstract base Metric class and all its descendants	237
FOORT/src/ Spline.cpp	
Simple cubic spline interpolation library without external dependencies	241
FOORT/src/ Spline.h	
Simple cubic spline interpolation library without external dependencies	242

FOORT/src/ Terminations.cpp	245
FOORT/src/ Terminations.h	
Declarations of abstract base Termination class and all its descendants	246
FOORT/src/ Utilities.cpp	254
FOORT/src/ Utilities.h	254
FOORT/src/ ViewScreen.cpp	255
FOORT/src/ ViewScreen.h	255

Chapter 5

Namespace Documentation

5.1 Config Namespace Reference

Namespace for all configuration functions that initialize objects based on configuration file.

Typedefs

- using [ConfigCollection](#) = [ConfigReader::ConfigCollection](#)
- using [SettingError](#) = [std::invalid_argument](#)

Functions

- void [InitializeScreenOutput](#) (const [ConfigCollection](#) &theCfg)
Initialize screen output options.
- std::unique_ptr< [Metric](#) > [GetMetric](#) (const [ConfigCollection](#) &theCfg)
Use configuration to create the correct [Metric](#) with specified parameters.
- std::unique_ptr< [Source](#) > [GetSource](#) (const [ConfigCollection](#) &theCfg, const [Metric](#) *const theMetric)
Use configuration to create the correct [Source](#) with specified parameters.
- void [InitializeDiagnostics](#) (const [ConfigCollection](#) &theCfg, [DiagBitflag](#) &alldiags, [DiagBitflag](#) &valdiag, const [Metric](#) *const theMetric)
Use configuration to set the [Diagnostics](#) bitflag appropriately; initialize all [DiagnosticOptions](#) for all [Diagnostics](#) that are turned on; and set bitflag for diagnostic to be used for coarseness evaluating in [Mesh](#).
- void [InitializeTerminations](#) (const [ConfigCollection](#) &theCfg, [TermBitflag](#) &allterms, const [Metric](#) *const theMetric)
Use configuration to set the [Termination](#) bitflag appropriately; and initialize all [TerminationOptions](#) for all [Terminations](#) that are turned on.
- std::unique_ptr< [ViewScreen](#) > [GetViewScreen](#) (const [ConfigCollection](#) &theCfg, [DiagBitflag](#) valdiag, const [Metric](#) *const theMetric)
Use configuration to create the [ViewScreen](#) object with specified parameters.
- std::unique_ptr< [Mesh](#) > [GetMesh](#) (const [ConfigCollection](#) &theCfg, [DiagBitflag](#) valdiag)
Use configuration to create the [Mesh](#) object with specified parameters.
- [GeodesicIntegratorFunc](#) [GetGeodesicIntegrator](#) (const [ConfigCollection](#) &theCfg)
Use configuration to set the [GeodesicIntegratorFunc](#) pointer to the correct integrator function.
- std::unique_ptr< [GeodesicOutputHandler](#) > [GetOutputHandler](#) (const [ConfigCollection](#) &theCfg, [DiagBitflag](#) alldiags, [DiagBitflag](#) valdiag, std::string FirstLineInfo)
Use configuration to create the [GeodesicOutputHandler](#) object with specified parameters.

Variables

- constexpr auto `Output_Important_Default` = `OutputLevel::Level_0_WARNING`
- constexpr auto `Output_Other_Default` = `OutputLevel::Level_1_PROC`

5.1.1 Detailed Description

Namespace for all configuration functions that initialize objects based on configuration file.

5.1.2 Typedef Documentation

5.1.2.1 ConfigCollection

```
using Config::ConfigCollection = ConfigReader::ConfigCollection
```

5.1.2.2 SettingError

```
using Config::SettingError = std::invalid_argument
```

5.1.3 Function Documentation

5.1.3.1 GetGeodesicIntegrator()

```
GeodesicIntegratorFunc Config::GetGeodesicIntegrator (
    const ConfigCollection & theCfg)
```

Use configuration to set the GeodesicIntegratorFunc pointer to the correct integrator function.

Parameters

<i>theCfg</i>	Pointer to the configuration object
---------------	-------------------------------------

Returns

GeodesicIntegratorFunc function

5.1.3.2 GetMesh()

```
std::unique_ptr< Mesh > Config::GetMesh (
    const ConfigCollection & theCfg,
    DiagBitflag valdiag)
```

Use configuration to create the `Mesh` object with specified parameters.

Parameters

<i>theCfg</i>	Pointer to the configuration object
<i>valdiag</i>	Diagnostic bitflag used for coarseness evaluation in the Mesh

Returns

Pointer to the [Mesh](#) object

5.1.3.3 GetMetric()

```
std::unique_ptr< Metric > Config::GetMetric (
    const ConfigCollection & theCfg)
```

Use configuration to create the correct [Metric](#) with specified parameters.

Parameters

<i>theCfg</i>	Pointer to the configuration object
---------------	-------------------------------------

Returns

[Metric](#) pointer

5.1.3.4 GetOutputHandler()

```
std::unique_ptr< GeodesicOutputHandler > Config::GetOutputHandler (
    const ConfigCollection & theCfg,
    DiagBitflag alldiags,
    DiagBitflag valdiag,
    std::string FirstLineInfo)
```

Use configuration to create the [GeodesicOutputHandler](#) object with specified parameters.

Parameters

<i>theCfg</i>	Pointer to the configuration object
<i>alldiags</i>	Diagnostics bitflags
<i>valdiag</i>	Diagnostic bitflag used for coarseness evaluation in the Mesh
<i>FirstLineInfo</i>	String to be written as the first line in the output file

Returns

Pointer to the [GeodesicOutputHandler](#) object

5.1.3.5 GetSource()

```
std::unique_ptr< Source > Config::GetSource (
    const ConfigCollection & theCfg,
    const Metric *const theMetric)
```

Use configuration to create the correct [Source](#) with specified parameters.

Parameters

<i>theCfg</i>	Pointer to the configuration object
<i>theMetric</i>	Pointer to the Metric object

Returns

Pointer to the [Source](#) object

5.1.3.6 GetViewScreen()

```
std::unique_ptr< ViewScreen > Config::GetViewScreen (
    const ConfigCollection & theCfg,
    DiagBitflag valdiag,
    const Metric *const theMetric)
```

Use configuration to create the [ViewScreen](#) object with specified parameters.

Parameters

<i>theCfg</i>	Pointer to the configuration object
<i>valdiag</i>	Diagnostic bitflag used for coarseness evaluation in the Mesh
<i>theMetric</i>	Pointer to the Metric object

Returns

Pointer to the [ViewScreen](#) object

5.1.3.7 InitializeDiagnostics()

```
void Config::InitializeDiagnostics (
    const ConfigCollection & theCfg,
    DiagBitflag & alldiags,
    DiagBitflag & valdiag,
    const Metric *const theMetric)
```

Use configuration to set the Diagnostics bitflag appropriately; initialize all [DiagnosticOptions](#) for all Diagnostics that are turned on; and set bitflag for diagnostic to be used for coarseness evaluating in [Mesh](#).

Parameters

<i>theCfg</i>	Pointer to the configuration object
<i>alldiags</i>	Pointer to the diagnostics bitflags
<i>valdiag</i>	Pointer to the diagnostics used for coarseness evaluation in the Mesh
<i>theMetric</i>	Pointer to the Metric object

5.1.3.8 InitializeScreenOutput()

```
void Config::InitializeScreenOutput (
    const ConfigCollection & theCfg)
```

Initialize screen output options.

Parameters

<i>theCfg</i>	Pointer to the configuration object
---------------	-------------------------------------

5.1.3.9 InitializeTerminations()

```
void Config::InitializeTerminations (
    const ConfigCollection & theCfg,
    TermBitflag & allterms,
    const Metric *const theMetric)
```

Use configuration to set the [Termination](#) bitflag appropriately; and initialize all [TerminationOptions](#) for all Terminations that are turned on.

Parameters

<i>theCfg</i>	Pointer to the configuration object
<i>allterms</i>	Pointer to the termination bitflags
<i>theMetric</i>	Pointer to the Metric object

5.1.4 Variable Documentation**5.1.4.1 Output_Important_Default**

```
auto Config::Output_Important_Default = OutputLevel::Level_0_WARNING [constexpr]
```

5.1.4.2 Output_Other_Default

```
auto Config::Output_Other_Default = OutputLevel::Level_1_PROC [constexpr]
```

5.2 ConfigReader Namespace Reference

Namespace to put all [ConfigReader](#) objects.

Classes

- class [ConfigCollection](#)
- class [ConfigReaderException](#)

Variables

- const long long [MaxStreamSize](#) {std::numeric_limits<std::streamsize>::max()}

5.2.1 Detailed Description

Namespace to put all [ConfigReader](#) objects.

5.2.2 Variable Documentation

5.2.2.1 MaxStreamSize

```
const long long ConfigReader::MaxStreamSize {std::numeric_limits<std::streamsize>::max() }
```

5.3 Integrators Namespace Reference

Functions

- `std::string GetFullIntegratorDescription ()`
Description string getter for the Integrator.
- `real GetAdaptiveStep (Point curpos, OneIndex curvel)`
Determine the (affine parameter) step size to take.
- `void IntegrateGeodesicStep_RK4 (Point curpos, OneIndex curvel, Point &nextpos, OneIndex &nextvel, real &stepsize, const Metric *theMetric, const Source *theSource)`
Integrate the geodesic equation by one step using Runge-Kutta-4.
- `void IntegrateGeodesicStep_Verlet (Point curpos, OneIndex curvel, Point &nextpos, OneIndex &nextvel, real &stepsize, const Metric *theMetric, const Source *theSource)`
Integrate the geodesic equation by one step using the Verlet algorithm.

Variables

- `constexpr real delta_nodiv0 = 1e-20`
This is used to avoid dividing by zero.
- `real Derivative_hval {1e-7}`
The amount of any coordinate that we shift to calculate derivatives (using central difference)
- `std::string IntegratorDescription {"RK4"}`
The name of the integrator selected.
- `real epsilon {0.03}`
This is the base step size to be taken (the integrator will adapt this if necessary)
- `real SmallestPossibleStepsize {1e-12}`
The affine parameter must always go forward by at least this amount.
- `real VerletVelocityTolerance {0.001}`
Tolerance on the change in velocity in the iteration loop of the Verlet integration algorithm.

5.3.1 Function Documentation

5.3.1.1 GetAdaptiveStep()

```
real Integrators::GetAdaptiveStep (
    Point curpos,
    OneIndex curvel)
```

Determine the (affine parameter) step size to take.

algorithm as in Raptor (eqs (21)-(24)), which is taken from Noble et al. (2007) & Dolence et al. (2009)

Parameters

<i>curpos</i>	current position
<i>curvel</i>	current velocity

Returns

real

5.3.1.2 GetFullIntegratorDescription()

```
std::string Integrators::GetFullIntegratorDescription ()
```

Description string getter for the Integrator.

Returns

std::string

5.3.1.3 IntegrateGeodesicStep_RK4()

```
void Integrators::IntegrateGeodesicStep_RK4 (
    Point curpos,
    OneIndex curvel,
    Point & nextpos,
    OneIndex & nextvel,
    real & stepsize,
    const Metric * theMetric,
    const Source * theSource)
```

Integrate the geodesic equation by one step using Runge-Kutta-4.

Parameters

<i>curpos</i>	current position
<i>curvel</i>	current velocity
<i>nextpos</i>	reference to next position
<i>nextvel</i>	reference to next velocity
<i>stepsize</i>	reference to the stepsize
<i>theMetric</i>	pointer to the Metric object
<i>theSource</i>	pointer to the Source object

5.3.1.4 IntegrateGeodesicStep_Verlet()

```
void Integrators::IntegrateGeodesicStep_Verlet (
    Point curpos,
    OneIndex curvel,
    Point & nextpos,
    OneIndex & nextvel,
    real & stepsize,
    const Metric * theMetric,
    const Source * theSource)
```

Integrate the geodesic equation by one step using the Verlet algorithm.

Parameters

<i>curpos</i>	current position
<i>curvel</i>	current velocity
<i>nextpos</i>	reference to next position
<i>nextvel</i>	reference to next velocity
<i>stepsize</i>	reference to the stepsize
<i>theMetric</i>	pointer to the Metric object
<i>theSource</i>	pointer to the Source object

5.3.2 Variable Documentation**5.3.2.1 `delta_nodiv0`**

```
real Integrators::delta_nodiv0 = 1e-20 [constexpr]
```

This is used to avoid dividing by zero.

5.3.2.2 `Derivative_hval`

```
real Integrators::Derivative_hval {1e-7} [inline]
```

The amount of any coordinate that we shift to calculate derivatives (using central difference)

5.3.2.3 `epsilon`

```
real Integrators::epsilon {0.03} [inline]
```

This is the base step size to be taken (the integrator will adapt this if necessary)

5.3.2.4 `IntegratorDescription`

```
std::string Integrators::IntegratorDescription {"RK4"} [inline]
```

The name of the integrator selected.

5.3.2.5 `SmallestPossibleStepsize`

```
real Integrators::SmallestPossibleStepsize {1e-12} [inline]
```

The affine parameter must always go forward by at least this amount.

5.3.2.6 VerletVelocityTolerance

```
real Integrators::VerletVelocityTolerance {0.001} [inline]
```

Tolerance on the change in velocity in the iteration loop of the Verlet integration algorithm.

5.4 tk Namespace Reference

Namespaces

- namespace [internal](#)

Classes

- class [spline](#)

5.5 tk::internal Namespace Reference

Classes

- class [band_matrix](#)

5.6 Utilities Namespace Reference

Classes

- class [Timer](#)

Functions

- `std::string GetTimeStampString ()`
Other functions in [Utilities](#).
- `std::vector< std::string > GetDiagNameStrings (DiagBitflag alldiags, DiagBitflag valdiag)`
- `std::string GetFirstLineInfoString (const Metric *theMetric, const Source *theSource, DiagBitflag alldiags, DiagBitflag valdiag, TermBitflag allterms, const ViewScreen *theView)`

5.6.1 Function Documentation

5.6.1.1 GetDiagNameStrings()

```
std::vector< std::string > Utilities::GetDiagNameStrings (
    DiagBitflag alldiags,
    DiagBitflag valdiag)
```

5.6.1.2 GetFirstLineInfoString()

```
std::string Utilities::GetFirstLineInfoString (
    const Metric * theMetric,
    const Source * theSource,
    DiagBitflag alldiags,
    DiagBitflag valdiag,
    TermBitflag allterms,
    const ViewScreen * theView)
```

5.6.1.3 GetTimeStampString()

```
std::string Utilities::GetTimeStampString ()
```

Other functions in [Utilities](#).

Chapter 6

Class Documentation

6.1 tk::internal::band_matrix Class Reference

```
#include <Spline.h>
```

Public Member Functions

- [band_matrix](#) ()
- [band_matrix](#) (int [dim](#), int [n_u](#), int [n_l](#))
- [~band_matrix](#) ()
- void [resize](#) (int [dim](#), int [n_u](#), int [n_l](#))
- int [dim](#) () const
- int [num_upper](#) () const
- int [num_lower](#) () const
- double & [operator\(\)](#) (int [i](#), int [j](#))
- double [operator\(\)](#) (int [i](#), int [j](#)) const
- double & [saved_diag](#) (int [i](#))
- double [saved_diag](#) (int [i](#)) const
- void [lu_decompose](#) ()
- std::vector< double > [r_solve](#) (const std::vector< double > &[b](#)) const
- std::vector< double > [l_solve](#) (const std::vector< double > &[b](#)) const
- std::vector< double > [lu_solve](#) (const std::vector< double > &[b](#), bool [is_lu_decomposed](#)=false)

Private Attributes

- std::vector< std::vector< double > > [m_upper](#)
- std::vector< std::vector< double > > [m_lower](#)

6.1.1 Constructor & Destructor Documentation

6.1.1.1 band_matrix() [1/2]

```
tk::internal::band_matrix::band_matrix () [inline]
```

6.1.1.2 band_matrix() [2/2]

```
tk::internal::band_matrix::band_matrix (
    int dim,
    int n_u,
    int n_l)
```

6.1.1.3 ~band_matrix()

```
tk::internal::band_matrix::~~band_matrix () [inline]
```

6.1.2 Member Function Documentation

6.1.2.1 dim()

```
int tk::internal::band_matrix::dim () const
```

6.1.2.2 l_solve()

```
std::vector< double > tk::internal::band_matrix::l_solve (
    const std::vector< double > & b) const
```

6.1.2.3 lu_decompose()

```
void tk::internal::band_matrix::lu_decompose ()
```

6.1.2.4 lu_solve()

```
std::vector< double > tk::internal::band_matrix::lu_solve (
    const std::vector< double > & b,
    bool is_lu_decomposed = false)
```

6.1.2.5 num_lower()

```
int tk::internal::band_matrix::num_lower () const
```

6.1.2.6 num_upper()

```
int tk::internal::band_matrix::num_upper () const
```

6.1.2.7 operator()() [1/2]

```
double & tk::internal::band_matrix::operator() (
    int i,
    int j)
```

6.1.2.8 operator>() [2/2]

```
double tk::internal::band_matrix::operator() (
    int i,
    int j) const
```

6.1.2.9 r_solve()

```
std::vector< double > tk::internal::band_matrix::r_solve (
    const std::vector< double > & b) const
```

6.1.2.10 resize()

```
void tk::internal::band_matrix::resize (
    int dim,
    int n_u,
    int n_l)
```

6.1.2.11 saved_diag() [1/2]

```
double & tk::internal::band_matrix::saved_diag (
    int i)
```

6.1.2.12 saved_diag() [2/2]

```
double tk::internal::band_matrix::saved_diag (
    int i) const
```

6.1.3 Member Data Documentation

6.1.3.1 m_lower

```
std::vector<std::vector<double> > tk::internal::band_matrix::m_lower [private]
```

6.1.3.2 m_upper

```
std::vector<std::vector<double> > tk::internal::band_matrix::m_upper [private]
```

The documentation for this class was generated from the following files:

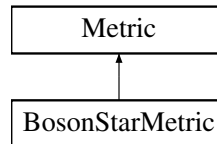
- FOORT/src/[Spline.h](#)
- FOORT/src/[Spline.cpp](#)

6.2 BosonStarMetric Class Reference

Boson star metric with solitonic potential.

```
#include <Metric.h>
```

Inheritance diagram for BosonStarMetric:



Public Member Functions

- [BosonStarMetric](#) (double Phi_infinity, int num_lines, bool rLogScale=false, std::string Phi_filename="Phi.dat", std::string m_filename="m.dat")
Construct a new Boson Star [Metric](#) object.
- [TwoIndex getMetric_dd](#) (const [Point](#) &p) const final
BosonStar metric getter, indices down.
- [TwoIndex getMetric_uu](#) (const [Point](#) &p) const final
BosonStar metric getter, indices up.
- std::string [getFullDescriptionStr](#) () const final
BosonStar metric description string getter.

Public Member Functions inherited from [Metric](#)

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new [Metric](#) object.
- virtual [ThreeIndex getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual [real getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- bool [getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Protected Member Functions

- void [read_data](#) ()
function to read the data

Protected Attributes

- const double [m_Phi_infinity](#)
The value of Phi at infinity, before rescaling.
- const int [m_num_lines](#)
- std::string [m_Phi_filename](#)
filename with Phi data
- std::string [m_m_filename](#)
filename with m data
- tk::spline [m_PhiSpline](#)
The spline interpolator for Phi.
- tk::spline [m_mSpline](#)
The spline interpolator for m.

Protected Attributes inherited from [Metric](#)

- std::vector< int > [m_Symmetries](#) {}
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- const bool [m_rLogScale](#)
Are we using a logarithmic r coordinate?

6.2.1 Detailed Description

Boson star metric with solitonic potential.

Note

Implementation by Seppe Staelens

The default files correspond to $\sigma = 0.06$ and $\phi_c = 0.044$

6.2.2 Constructor & Destructor Documentation

6.2.2.1 BosonStarMetric()

```
BosonStarMetric::BosonStarMetric (
    double Phi_infinity,
    int num_lines,
    bool rLogScale = false,
    std::string Phi_filename = "Phi.dat",
    std::string m_filename = "m.dat")
```

Construct a new Boson Star [Metric](#) object.

Parameters

<i>Phi_infinity</i>	value of $\Phi = \log(\text{lapse})$ at infinity
<i>num_lines</i>	number of lines in the data files
<i>rLogScale</i>	whether we are using a logarithmic radial scale
<i>Phi_filename</i>	filename for the Phi data
<i>m_filename</i>	filename for the mass aspect data

6.2.3 Member Function Documentation

6.2.3.1 getFullDescriptionStr()

```
std::string BosonStarMetric::getFullDescriptionStr () const [final], [virtual]
```

BosonStar metric description string getter.

Returns

std::string

Reimplemented from [Metric](#).

6.2.3.2 getMetric_dd()

```
TwoIndex BosonStarMetric::getMetric_dd (  
    const Point & p) const [final], [virtual]
```

BosonStar metric getter, indices down.

Parameters

p	Point at which to evaluate the metric
-----	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).

6.2.3.3 getMetric_uu()

```
TwoIndex BosonStarMetric::getMetric_uu (  
    const Point & p) const [final], [virtual]
```

BosonStar metric getter, indices up.

Parameters

p	Point at which to evaluate the metric
-----	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).

6.2.3.4 read_data()

```
void BosonStarMetric::read_data () [protected]
```

function to read the data

Read the numerically obtained boson star metric data from the files.

6.2.4 Member Data Documentation

6.2.4.1 m_m_filename

```
std::string BosonStarMetric::m_m_filename [protected]
```

filename with m data

6.2.4.2 m_mSpline

```
tk::spline BosonStarMetric::m_mSpline [protected]
```

The spline interpolator for m.

6.2.4.3 m_num_lines

```
const int BosonStarMetric::m_num_lines [protected]
```

Number of lines to be read from the data files. Mainly to avoid reading the data at extreme distances, which could upset the spline interpolation.

6.2.4.4 m_Phi_filename

```
std::string BosonStarMetric::m_Phi_filename [protected]
```

filename with Phi data

6.2.4.5 m_Phi_infinity

```
const double BosonStarMetric::m_Phi_infinity [protected]
```

The value of Phi at infinity, before rescaling.

6.2.4.6 m_PhiSpline

```
tk::spline BosonStarMetric::m_PhiSpline [protected]
```

The spline interpolator for Phi.

The documentation for this class was generated from the following files:

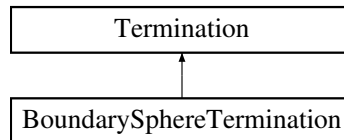
- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.3 BoundarySphereTermination Class Reference

[BoundarySphereTermination](#): Terminate geodesics if they reach outside of a boundary sphere.

```
#include <Terminations.h>
```

Inheritance diagram for [BoundarySphereTermination](#):



Public Member Functions

- [BoundarySphereTermination](#) ([Geodesic](#) *const theGeodesic)
- [Term CheckTermination](#) () final
[BoundarySphereTermination](#) functions.
- std::string [getFullDescriptionStr](#) () const final

Public Member Functions inherited from [Termination](#)

- [Termination](#) ()=delete
- [Termination](#) ([Geodesic](#) *const theGeodesic)
- virtual void [Reset](#) ()
[Termination](#) (abstract base class) functions.
- virtual [~Termination](#) ()=default

Static Public Attributes

- static std::unique_ptr< [BoundarySphereTermOptions](#) > [TermOptions](#)
The options that the [BoundarySphereTermination](#) keeps (contains the radius of the boundary sphere)

Additional Inherited Members

Protected Member Functions inherited from [Termination](#)

- bool [DecideUpdate](#) ([largecounter](#) UpdateNSteps)

Protected Attributes inherited from [Termination](#)

- [Geodesic](#) *const [m_OwnerGeodesic](#)
The geodesic that owns the [Termination](#) (a const pointer to the [Geodesic](#))
- [largecounter](#) [m_StepsSinceUpdated](#) {}
The termination is itself in charge of keeping track of how many steps it has been since it has been updated. The [Termination](#)'s [TerminationOptions](#) struct tells it how many steps it needs to wait between updates.

6.3.1 Detailed Description

[BoundarySphereTermination](#): Terminate geodesics if they reach outside of a boundary sphere.

6.3.2 Constructor & Destructor Documentation

6.3.2.1 BoundarySphereTermination()

```
BoundarySphereTermination::BoundarySphereTermination (  
    Geodesic *const theGeodesic) [inline]
```

6.3.3 Member Function Documentation

6.3.3.1 CheckTermination()

```
Term BoundarySphereTermination::CheckTermination () [final], [virtual]
```

[BoundarySphereTermination](#) functions.

Implements [Termination](#).

6.3.3.2 getFullDescriptionStr()

```
std::string BoundarySphereTermination::getFullDescriptionStr () const [final], [virtual]
```

Implements [Termination](#).

6.3.4 Member Data Documentation

6.3.4.1 TermOptions

```
std::unique_ptr< BoundarySphereTermOptions > BoundarySphereTermination::TermOptions [static]
```

The options that the [BoundarySphereTermination](#) keeps (contains the radius of the boundary sphere)

The documentation for this class was generated from the following files:

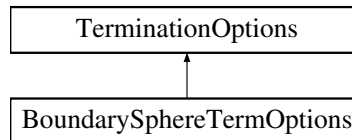
- FOORT/src/[Terminations.h](#)
- FOORT/src/[Config.cpp](#)
- FOORT/src/[Terminations.cpp](#)

6.4 BoundarySphereTermOptions Struct Reference

Options for [BoundarySphereTermination](#).

```
#include <Terminations.h>
```

Inheritance diagram for BoundarySphereTermOptions:



Public Member Functions

- [BoundarySphereTermOptions](#) ([real](#) theRadius, [bool](#) therLogScale, [largecounter](#) Nsteps)

Public Member Functions inherited from [TerminationOptions](#)

- [TerminationOptions](#) ([largecounter](#) Nsteps)
- virtual [~TerminationOptions](#) ()=default

Public Attributes

- const [real](#) [SphereRadius](#)
Radius of the boundary sphere.
- const [bool](#) [rLogScale](#)
Whether we are using a logarithmic r coordinate.

Public Attributes inherited from [TerminationOptions](#)

- const [largecounter](#) [UpdateEveryNSteps](#)
Number of steps between updates.

6.4.1 Detailed Description

Options for [BoundarySphereTermination](#).

Keeps track of the boundary sphere radius, whether we are using a logarithmic r coordinate.

6.4.2 Constructor & Destructor Documentation

6.4.2.1 BoundarySphereTermOptions()

```
BoundarySphereTermOptions::BoundarySphereTermOptions (
    real theRadius,
    bool therLogScale,
    largecounter Nsteps) [inline]
```

6.4.3 Member Data Documentation

6.4.3.1 rLogScale

```
const bool BoundarySphereTermOptions::rLogScale
```

Whether we are using a logarithmic r coordinate.

6.4.3.2 SphereRadius

```
const real BoundarySphereTermOptions::SphereRadius
```

Radius of the boundary sphere.

The documentation for this struct was generated from the following file:

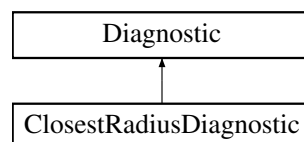
- FOORT/src/[Terminations.h](#)

6.5 ClosestRadiusDiagnostic Class Reference

[Diagnostic](#) that keeps track of the closest (true, not log) radius that the geodesic passes through.

```
#include <Diagnostics.h>
```

Inheritance diagram for ClosestRadiusDiagnostic:



Public Member Functions

- [ClosestRadiusDiagnostic](#) ([Geodesic](#) *const theGeodesic)
- void [Reset](#) () final
ClosestRadiusDiagnostic functions.
- void [UpdateData](#) () final
Check to see if we have travelled closer to the center and update the closest radius.
- std::string [getFullDataStr](#) () const final
Return the closest radius as a string.
- std::vector< [real](#) > [getFinalDataVal](#) () const final
Return the closest radius as a vector of size one.
- [real](#) [FinalDataValDistance](#) (const std::vector< [real](#) > &val1, const std::vector< [real](#) > &val2) const final
Return the absolute value of the difference between the closest radii.
- std::string [getNameStr](#) () const final
Simple name without spaces.
- std::string [getFullDescriptionStr](#) () const final
More descriptive string (with spaces)

Public Member Functions inherited from [Diagnostic](#)

- [Diagnostic](#) ()=delete
- [Diagnostic](#) ([Geodesic](#) *const theGeodesic)
- virtual [~Diagnostic](#) ()=default

Static Public Attributes

- static std::unique_ptr< [ClosestRadiusOptions](#) > [DiagOptions](#)

Private Attributes

- [real](#) [m_ClosestRadius](#) {-1}

Additional Inherited Members

Protected Member Functions inherited from [Diagnostic](#)

- bool [DecideUpdate](#) (const [UpdateFrequency](#) &myUpdateFrequency)
Returns true if the [Diagnostic](#) should update its initial status.

Protected Attributes inherited from [Diagnostic](#)

- [Geodesic](#) *const [m_OwnerGeodesic](#)
- [largecounter](#) [m_StepsSinceUpdated](#) {}

6.5.1 Detailed Description

[Diagnostic](#) that keeps track of the closest (true, not log) radius that the geodesic passes through.

6.5.2 Constructor & Destructor Documentation

6.5.2.1 [ClosestRadiusDiagnostic](#)()

```
ClosestRadiusDiagnostic::ClosestRadiusDiagnostic (
    Geodesic *const theGeodesic) [inline]
```

6.5.3 Member Function Documentation

6.5.3.1 [FinalDataValDistance](#)()

```
real ClosestRadiusDiagnostic::FinalDataValDistance (
    const std::vector< real > & val1,
    const std::vector< real > & val2) const [final], [virtual]
```

Return the absolute value of the difference between the closest radii.

Parameters

<i>val1</i>	first closest radius
<i>val2</i>	second closest radius

Returns

real

Implements [Diagnostic](#).

6.5.3.2 getFinalDataVal()

```
std::vector< real > ClosestRadiusDiagnostic::getFinalDataVal () const [final], [virtual]
```

Return the closest radius as a vector of size one.

Returns

std::vector<real>

Implements [Diagnostic](#).

6.5.3.3 getFullDataStr()

```
std::string ClosestRadiusDiagnostic::getFullDataStr () const [final], [virtual]
```

Return the closest radius as a string.

Returns

std::string

Implements [Diagnostic](#).

6.5.3.4 getFullDescriptionStr()

```
std::string ClosestRadiusDiagnostic::getFullDescriptionStr () const [final], [virtual]
```

More descriptive string (with spaces)

Returns

std::string

Reimplemented from [Diagnostic](#).

6.5.3.5 getNameStr()

```
std::string ClosestRadiusDiagnostic::getNameStr () const [final], [virtual]
```

Simple name without spaces.

Returns

std::string

Implements [Diagnostic](#).

6.5.3.6 Reset()

```
void ClosestRadiusDiagnostic::Reset () [final], [virtual]
```

[ClosestRadiusDiagnostic](#) functions.

Reset the closest radius to -1 and call base class Reset function

Reimplemented from [Diagnostic](#).

6.5.3.7 UpdateData()

```
void ClosestRadiusDiagnostic::UpdateData () [final], [virtual]
```

Check to see if we have travelled closer to the center and update the closest radius.

Implements [Diagnostic](#).

6.5.4 Member Data Documentation

6.5.4.1 DiagOptions

```
std::unique_ptr< ClosestRadiusOptions > ClosestRadiusDiagnostic::DiagOptions [static]
```

6.5.4.2 m_ClosestRadius

```
real ClosestRadiusDiagnostic::m_ClosestRadius {-1} [private]
```

The documentation for this class was generated from the following files:

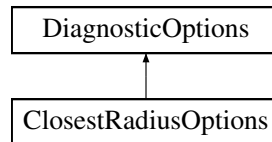
- FOORT/src/[Diagnostics.h](#)
- FOORT/src/[Config.cpp](#)
- FOORT/src/[Diagnostics.cpp](#)

6.6 ClosestRadiusOptions Struct Reference

Options for [ClosestRadiusDiagnostic](#).

```
#include <Diagnostics.h>
```

Inheritance diagram for ClosestRadiusOptions:



Public Member Functions

- [ClosestRadiusOptions](#) (bool rlog, [UpdateFrequency](#) thefrequency)

Public Member Functions inherited from [DiagnosticOptions](#)

- [DiagnosticOptions](#) ([UpdateFrequency](#) thefrequency)
- virtual [~DiagnosticOptions](#) ()=default

Public Attributes

- const bool [RLogScale](#)

Public Attributes inherited from [DiagnosticOptions](#)

- const [UpdateFrequency](#) theUpdateFrequency

6.6.1 Detailed Description

Options for [ClosestRadiusDiagnostic](#).

Inherits from [DiagnosticOptions](#), and adds the flag for using log(r) scale.

6.6.2 Constructor & Destructor Documentation

6.6.2.1 ClosestRadiusOptions()

```
ClosestRadiusOptions::ClosestRadiusOptions (  
    bool rlog,  
    UpdateFrequency thefrequency) [inline]
```

6.6.3 Member Data Documentation

6.6.3.1 RLogScale

```
const bool ClosestRadiusOptions::RLogScale
```

The documentation for this struct was generated from the following file:

- FOORT/src/[Diagnostics.h](#)

6.7 ConfigReader::ConfigCollection Class Reference

```
#include <ConfigReader.h>
```

Classes

- struct [ConfigSetting](#)

Public Member Functions

- bool [Exists](#) (std::string_view SettingName) const
Check if a setting with the given name exists in the collection.
- int [NrSettings](#) () const
Return the total number of settings in the collection.
- bool [IsCollection](#) (std::string_view SettingName) const
Check if a setting with the given name is a collection.
- bool [IsCollection](#) (int SettingIndex) const
Check if a setting with the given index is a collection.
- const [ConfigCollection](#) & [operator\[\]](#) (std::string_view CollectionName) const
Access a subcollection by name.
- const [ConfigCollection](#) & [operator\[\]](#) (int CollectionIndex) const
Access a subcollection by index.
- template<class OutputType >
bool [LookupValue](#) (std::string_view SettingName, OutputType &theOutput) const
Looks up the value of a setting with the given name and puts it in the output variable.
- template<class OutputType >
bool [LookupValue](#) (int SettingIndex, OutputType &theOutput) const
Looks up the value of a setting with the given index and puts it in the output variable.
- bool [LookupValueInteger](#) (std::string_view SettingName, int &theOutput) const
If the output is int, we only look up to see if the value is an int.
- bool [LookupValueInteger](#) (std::string_view SettingName, long &theOutput) const
If the output is long, the setting value can either be a long or an int, so we check for both.
- bool [LookupValueInteger](#) (std::string_view SettingName, long long &theOutput) const
If the output is long long, the setting value can either be a long long, a long, or an int, so we check for all.
- bool [LookupValueInteger](#) (std::string_view SettingName, unsigned int &theOutput) const
If the output is unsigned int, the setting value can either be an unsigned int, an int, a long, or a long long, so we check for all.
- bool [LookupValueInteger](#) (std::string_view SettingName, unsigned long &theOutput) const

If the output is unsigned long, the setting value can either be an unsigned long, an unsigned int, an int, a long, or a long long, so we check for all.

- bool [LookupValueInteger](#) (std::string_view SettingName, unsigned long long &theOutput) const
If the output is unsigned long long, the setting value can either be an unsigned long long, an unsigned long, an unsigned int, an int, a long, or a long long, so we check for all.
- template<class OutputType >
bool [LookupValueInteger](#) (int SettingIndex, OutputType &theOutput) const
This simply defers to the setting-name-based implementation of the function with given output type.
- void [DisplayCollection](#) (std::ostream &OutputStream, int Indent=0) const
Output the entire collection to a given output stream.
- bool [ReadFile](#) (const std::string &FileName)
Read in a configuration file.

Private Types

- using [ConfigSettingValue](#)

Private Member Functions

- int [GetSettingIndex](#) (std::string_view SettingName) const
Return the index of a specific setting in the collection.
- void [DisplaySetting](#) (std::ostream &OutputStream, int SettingIndex, int Indent) const
Output a single setting to the output stream.
- void [DisplayTabs](#) (std::ostream &OutputStream, int NrTabs) const
Output a given number of tabs to the output stream.
- void [ReadCollection](#) (std::ifstream &InputFile)
Read in an entire collection (including subcollections). Pre: stream is positioned right after '{' (or at beginning of file). Post: stream is positioned right after '}' (or at EOF).
- void [ReadSettingName](#) (std::ifstream &InputFile, std::string &theName)
Read in a setting name. Pre: next non-whitespace character in stream is beginning of name. Post: stream has just read in all characters of name (but no more); returns "" if there is no more setting to read in the current collection, in this case '}' has been read in (for subcollections).
- void [ReadSettingSpecificChar](#) (std::ifstream &InputFile, char theChar) const
Read in one specific character only (not '/'). Pre: next non-whitespace character in stream is this character. Post: stream is just after character.
- void [ReadSettingValue](#) (std::ifstream &InputFile, [ConfigSettingValue](#) &theValue)
Read in setting value (could be subcollection). Pre: next non-whitespace character in stream is beginning of value (can be '{' for subcollection, digit or '-' or '.' or 'e' for number, "" for string, or 't'/'f' for boolean). Post: Value has entirely been read in (i.e. next character should be ';').

Private Attributes

- std::vector< [ConfigSetting](#) > [m_Settings](#) {}

6.7.1 Member Typedef Documentation

6.7.1.1 ConfigSettingValue

using [ConfigReader::ConfigCollection::ConfigSettingValue](#) [private]

Initial value:

```
std::variant<bool,
              int, long, long long,
              double,
              std::string,
              std::unique_ptr<ConfigCollection>>
```

6.7.2 Member Function Documentation

6.7.2.1 DisplayCollection()

```
void ConfigCollection::DisplayCollection (
    std::ostream & OutputStream,
    int Indent = 0) const
```

Output the entire collection to a given output stream.

Parameters

<i>OutputStream</i>	pointer to an outputstream
<i>Indent</i>	indentation level

6.7.2.2 DisplaySetting()

```
void ConfigCollection::DisplaySetting (
    std::ostream & OutputStream,
    int SettingIndex,
    int Indent) const [private]
```

Output a single setting to the output stream.

Parameters

<i>OutputStream</i>	pointer to an outputstream
<i>SettingIndex</i>	index of the setting to output
<i>Indent</i>	indentation level

6.7.2.3 DisplayTabs()

```
void ConfigCollection::DisplayTabs (
    std::ostream & OutputStream,
    int NrTabs) const [private]
```

Output a given number of tabs to the output stream.

Parameters

<i>OutputStream</i>	pointer to an outputstream
<i>NrTabs</i>	number of tabs to output

6.7.2.4 Exists()

```
bool ConfigCollection::Exists (
    std::string_view SettingName) const
```

Check if a setting with the given name exists in the collection.

Parameters

<i>SettingName</i>	The name of the setting to check
--------------------	----------------------------------

Returns

true
false

6.7.2.5 GetSettingIndex()

```
int ConfigCollection::GetSettingIndex (
    std::string_view SettingName) const [private]
```

Return the index of a specific setting in the collection.

Parameters

<i>SettingName</i>	The name of the setting to find
--------------------	---------------------------------

Returns

int

6.7.2.6 IsCollection() [1/2]

```
bool ConfigCollection::IsCollection (
    int SettingIndex) const
```

Check if a setting with the given index is a collection.

Parameters

<i>SettingIndex</i>	The index of the setting to check
---------------------	-----------------------------------

Returns

true
false

6.7.2.7 IsCollection() [2/2]

```
bool ConfigCollection::IsCollection (
    std::string_view SettingName) const
```

Check if a setting with the given name is a collection.

Parameters

<i>SettingName</i>	The name of the setting to check
--------------------	----------------------------------

Returns

true
false

6.7.2.8 LookupValue() [1/2]

```
template<class OutputType >
bool ConfigReader::ConfigCollection::LookupValue (
    int SettingIndex,
    OutputType & theOutput) const
```

Looks up the value of a setting with the given index and puts it in the output variable.

Template Parameters

<i>OutputType</i>	
-------------------	--

Parameters

<i>SettingIndex</i>	Setting index
<i>theOutput</i>	Pointer to the output variable

Returns

true
false

6.7.2.9 LookupValue() [2/2]

```
template<class OutputType >
bool ConfigReader::ConfigCollection::LookupValue (
    std::string_view SettingName,
    OutputType & theOutput) const
```

Looks up the value of a setting with the given name and puts it in the output variable.

Template Parameters

<i>OutputType</i>	
-------------------	--

Parameters

<i>SettingName</i>	Setting name
<i>theOutput</i>	Pointer to the output variable

Returns

true
false

6.7.2.10 LookupValueInteger() [1/7]

```
template<class OutputType >  
bool ConfigReader::ConfigCollection::LookupValueInteger (  
    int SettingIndex,  
    OutputType & theOutput) const
```

This simply defers to the setting-name-based implementation of the function with given output type.

Template Parameters

<i>OutputType</i>	
-------------------	--

Parameters

<i>SettingIndex</i>	Setting index
<i>theOutput</i>	Pointer to the output variable

Returns

true
false

6.7.2.11 LookupValueInteger() [2/7]

```
bool ConfigCollection::LookupValueInteger (  
    std::string_view SettingName,  
    int & theOutput) const
```

If the output is int, we only look up to see if the value is an int.

Parameters

<i>SettingName</i>	Name of the setting to look up
<i>theOutput</i>	integer output

Returns

true
false

6.7.2.12 LookupValueInteger() [3/7]

```
bool ConfigCollection::LookupValueInteger (
    std::string_view SettingName,
    long & theOutput) const
```

If the output is long, the setting value can either be a long or an int, so we check for both.

Parameters

<i>SettingName</i>	Name of the setting to look up
<i>theOutput</i>	long output

Returns

true
false

6.7.2.13 LookupValueInteger() [4/7]

```
bool ConfigCollection::LookupValueInteger (
    std::string_view SettingName,
    long long & theOutput) const
```

If the output is long long, the setting value can either be a long long, a long, or an int, so we check for all.

Parameters

<i>SettingName</i>	Name of the setting to look up
<i>theOutput</i>	long long output

Returns

true
false

6.7.2.14 LookupValueInteger() [5/7]

```
bool ConfigCollection::LookupValueInteger (
    std::string_view SettingName,
    unsigned int & theOutput) const
```

If the output is unsigned int, the setting value can either be an unsigned int, an int, a long, or a long long, so we check for all.

Parameters

<i>SettingName</i>	Name of the setting to look up
<i>theOutput</i>	unsigned int output

Returns

true
false

6.7.2.15 LookupValueInteger() [6/7]

```
bool ConfigCollection::LookupValueInteger (
    std::string_view SettingName,
    unsigned long & theOutput) const
```

If the output is unsigned long, the setting value can either be an unsigned long, an unsigned int, an int, a long, or a long long, so we check for all.

Parameters

<i>SettingName</i>	Name of the setting to look up
<i>theOutput</i>	unsigned long output

Returns

true
false

6.7.2.16 LookupValueInteger() [7/7]

```
bool ConfigCollection::LookupValueInteger (
    std::string_view SettingName,
    unsigned long long & theOutput) const
```

If the output is unsigned long long, the setting value can either be an unsigned long long, an unsigned long, an unsigned int, an int, a long, or a long long, so we check for all.

Parameters

<i>SettingName</i>	Name of the setting to look up
<i>theOutput</i>	unsigned long long output

Returns

true
false

6.7.2.17 NrSettings()

```
int ConfigCollection::NrSettings () const
```

Return the total number of settings in the collection.

Returns

int

6.7.2.18 operator[]() [1/2]

```
const ConfigCollection & ConfigCollection::operator[] (
    int CollectionIndex) const
```

Access a subcollection by index.

Parameters

<i>CollectionIndex</i>	The index of the subcollection to access
------------------------	--

Returns

const [ConfigCollection](#)&

6.7.2.19 operator[]() [2/2]

```
const ConfigCollection & ConfigCollection::operator[] (
    std::string_view CollectionName) const
```

Access a subcollection by name.

Parameters

<i>CollectionName</i>	The name of the subcollection to access
-----------------------	---

Returns

const [ConfigCollection](#)&

6.7.2.20 ReadCollection()

```
void ConfigCollection::ReadCollection (
    std::ifstream & InputFile) [private]
```

Read in an entire collection (including subcollections). Pre: stream is positioned right after '{' (or at beginning of file). Post: stream is positioned right after '}' (or at EOF).

Parameters

<i>InputFile</i>	pointer to an input file stream
------------------	---------------------------------

6.7.2.21 ReadFile()

```
bool ConfigCollection::ReadFile (
    const std::string & FileName)
```

Read in a configuration file.

Parameters

<i>FileName</i>	File name of the configuration file
-----------------	-------------------------------------

Returns

true if succesful

false if the file does not exist, or if there is a parse/syntax error

6.7.2.22 ReadSettingName()

```
void ConfigCollection::ReadSettingName (
    std::ifstream & InputFile,
    std::string & theName) [private]
```

Read in a setting name. Pre: next non-whitespace character in stream is beginning of name. Post: stream has just read in all characters of name (but no more); returns "" if there is no more setting to read in the current collection, in this case '}' has been read in (for subcollections).

Parameters

<i>InputFile</i>	Pointer to the input file stream
<i>theName</i>	Pointer to the name of the setting

6.7.2.23 ReadSettingSpecificChar()

```
void ConfigCollection::ReadSettingSpecificChar (
    std::ifstream & InputFile,
    char theChar) const [private]
```

Read in one specific character only (not '/'). Pre: next non-whitespace character in stream is this character. Post: stream is just after character.

Parameters

<i>InputFile</i>	Pointer to the input file stream
<i>theChar</i>	The character to read in

6.7.2.24 ReadSettingValue()

```
void ConfigCollection::ReadSettingValue (
    std::ifstream & InputFile,
    ConfigSettingValue & theValue) [private]
```

Read in setting value (could be subcollection). Pre: next non-whitespace character in stream is beginning of value (can be '{' for subcollection, digit or '-' or '.' or 'e' for number, "" for string, or 't'/'f' for boolean). Post: Value has entirely been read in (i.e. next character should be ';').

Parameters

<i>InputFile</i>	Pointer to the input file stream
<i>theValue</i>	Pointer to the setting value

6.7.3 Member Data Documentation

6.7.3.1 m_Settings

```
std::vector<ConfigSetting> ConfigReader::ConfigCollection::m_Settings {} [private]
```

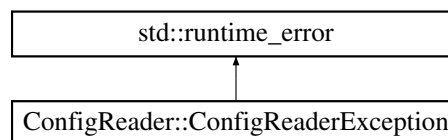
The documentation for this class was generated from the following files:

- FOORT/src/[ConfigReader.h](#)
- FOORT/src/[ConfigReader.cpp](#)

6.8 ConfigReader::ConfigReaderException Class Reference

```
#include <ConfigReader.h>
```

Inheritance diagram for ConfigReader::ConfigReaderException:



Public Member Functions

- [ConfigReaderException](#) (const std::string &error, std::vector< int > settingtrace={})
- std::vector< int > [trace](#) () const

Private Attributes

- std::vector< int > [m_settingtrace](#)

6.8.1 Constructor & Destructor Documentation

6.8.1.1 ConfigReaderException()

```
ConfigReader::ConfigReaderException::ConfigReaderException (
    const std::string & error,
    std::vector< int > settingtrace = {}) [inline]
```

6.8.2 Member Function Documentation

6.8.2.1 trace()

```
std::vector< int > ConfigReader::ConfigReaderException::trace () const [inline]
```

6.8.3 Member Data Documentation

6.8.3.1 m_settingtrace

```
std::vector<int> ConfigReader::ConfigReaderException::m_settingtrace [private]
```

The documentation for this class was generated from the following file:

- FOORT/src/[ConfigReader.h](#)

6.9 ConfigReader::ConfigCollection::ConfigSetting Struct Reference

Public Attributes

- std::string [SettingName](#)
- [ConfigSettingValue](#) [SettingValue](#)

6.9.1 Member Data Documentation

6.9.1.1 SettingName

```
std::string ConfigReader::ConfigCollection::ConfigSetting::SettingName
```

6.9.1.2 SettingValue

```
ConfigSettingValue ConfigReader::ConfigCollection::ConfigSetting::SettingValue
```

The documentation for this struct was generated from the following file:

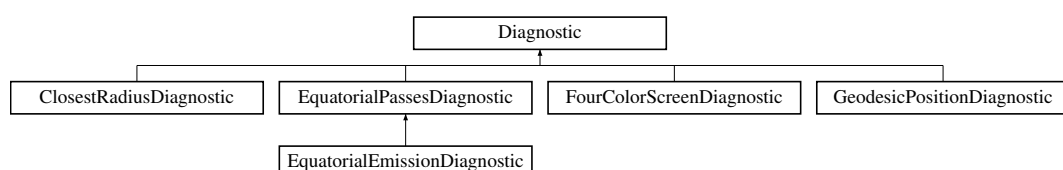
- FOORT/src/[ConfigReader.h](#)

6.10 Diagnostic Class Reference

Abstract base class for all diagnostics.

```
#include <Diagnostics.h>
```

Inheritance diagram for Diagnostic:



Public Member Functions

- [Diagnostic](#) ()=delete
- [Diagnostic](#) ([Geodesic](#) *const theGeodesic)
- virtual void [Reset](#) ()
- *[Diagnostic](#) (abstract base class) functions.*
- virtual [~Diagnostic](#) ()=default
- virtual void [UpdateData](#) ()=0
- virtual std::string [getFullDataStr](#) () const =0
- virtual std::vector< [real](#) > [getFinalDataVal](#) () const =0
- virtual [real](#) [FinalDataValDistance](#) (const std::vector< [real](#) > &val1, const std::vector< [real](#) > &val2) const =0
- virtual std::string [getNameStr](#) () const =0
- virtual std::string [getFullDescriptionStr](#) () const

Base class just returns the short name of the [Diagnostic](#).

Protected Member Functions

- bool [DecideUpdate](#) (const [UpdateFrequency](#) &myUpdateFrequency)

Returns true if the [Diagnostic](#) should update its initial status.

Protected Attributes

- [Geodesic](#) *const [m_OwnerGeodesic](#)
- [largecounter](#) [m_StepsSinceUpdated](#) {}

6.10.1 Detailed Description

Abstract base class for all diagnostics.

6.10.2 Constructor & Destructor Documentation

6.10.2.1 [Diagnostic](#)() [1/2]

```
Diagnostic::Diagnostic () [delete]
```

6.10.2.2 [Diagnostic](#)() [2/2]

```
Diagnostic::Diagnostic (
    Geodesic *const theGeodesic) [inline]
```

6.10.2.3 [~Diagnostic](#)()

```
virtual Diagnostic::~~Diagnostic () [virtual], [default]
```

6.10.3 Member Function Documentation

6.10.3.1 [DecideUpdate](#)()

```
bool Diagnostic::DecideUpdate (
    const UpdateFrequency & myUpdateFrequency) [protected]
```

Returns true if the [Diagnostic](#) should update its initial status.

Should be called from within [UpdateData\(\)](#) with the appropriate [DiagnosticOptions::theUpdateFrequency](#).

Parameters

<i>myUpdateFrequency</i>	UpdateFrequency struct
--------------------------	--

Returns

true
false

6.10.3.2 FinalDataValDistance()

```
virtual real Diagnostic::FinalDataValDistance (
    const std::vector< real > & val1,
    const std::vector< real > & val2) const [pure virtual]
```

Implemented in [ClosestRadiusDiagnostic](#), [EquatorialEmissionDiagnostic](#), [EquatorialPassesDiagnostic](#), [FourColorScreenDiagnostic](#), and [GeodesicPositionDiagnostic](#).

6.10.3.3 getFinalDataVal()

```
virtual std::vector< real > Diagnostic::getFinalDataVal () const [pure virtual]
```

Implemented in [ClosestRadiusDiagnostic](#), [EquatorialEmissionDiagnostic](#), [EquatorialPassesDiagnostic](#), [FourColorScreenDiagnostic](#), and [GeodesicPositionDiagnostic](#).

6.10.3.4 getFullDataStr()

```
virtual std::string Diagnostic::getFullDataStr () const [pure virtual]
```

Implemented in [ClosestRadiusDiagnostic](#), [EquatorialEmissionDiagnostic](#), [EquatorialPassesDiagnostic](#), [FourColorScreenDiagnostic](#), and [GeodesicPositionDiagnostic](#).

6.10.3.5 getFullDescriptionStr()

```
std::string Diagnostic::getFullDescriptionStr () const [virtual]
```

Base class just returns the short name of the [Diagnostic](#).

This function is pure virtual in the base class.

Returns

std::string

Reimplemented in [ClosestRadiusDiagnostic](#), [EquatorialEmissionDiagnostic](#), [EquatorialPassesDiagnostic](#), [FourColorScreenDiagnostic](#), and [GeodesicPositionDiagnostic](#).

6.10.3.6 getNameStr()

```
virtual std::string Diagnostic::getNameStr () const [pure virtual]
```

Implemented in [ClosestRadiusDiagnostic](#), [EquatorialEmissionDiagnostic](#), [EquatorialPassesDiagnostic](#), [FourColorScreenDiagnostic](#), and [GeodesicPositionDiagnostic](#).

6.10.3.7 Reset()

```
void Diagnostic::Reset () [virtual]
```

[Diagnostic](#) (abstract base class) functions.

Reset [Diagnostic](#) object steps to zero.

Reimplemented in [ClosestRadiusDiagnostic](#), [EquatorialEmissionDiagnostic](#), [EquatorialPassesDiagnostic](#), [FourColorScreenDiagnostic](#), and [GeodesicPositionDiagnostic](#).

6.10.3.8 UpdateData()

```
virtual void Diagnostic::UpdateData () [pure virtual]
```

Implemented in [ClosestRadiusDiagnostic](#), [EquatorialEmissionDiagnostic](#), [EquatorialPassesDiagnostic](#), [FourColorScreenDiagnostic](#), and [GeodesicPositionDiagnostic](#).

6.10.4 Member Data Documentation

6.10.4.1 m_OwnerGeodesic

```
Geodesic* const Diagnostic::m_OwnerGeodesic [protected]
```

6.10.4.2 m_StepsSinceUpdated

```
largecounter Diagnostic::m_StepsSinceUpdated {} [protected]
```

The documentation for this class was generated from the following files:

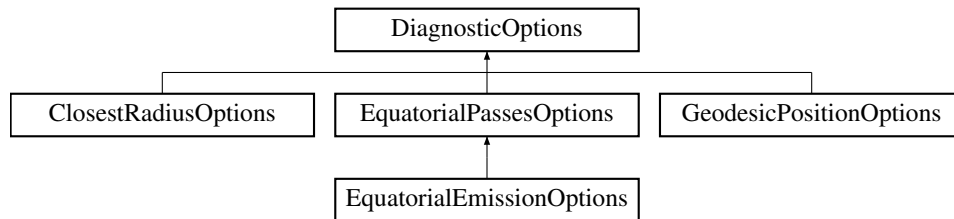
- FOORT/src/[Diagnostics.h](#)
- FOORT/src/[Diagnostics.cpp](#)

6.11 DiagnosticOptions Struct Reference

Base class for [DiagnosticOptions](#). Other Diagnostics can inherit from here if they require more options.

```
#include <Diagnostics.h>
```

Inheritance diagram for DiagnosticOptions:



Public Member Functions

- [DiagnosticOptions](#) ([UpdateFrequency](#) thefrequency)
- virtual [~DiagnosticOptions](#) ()=default

Public Attributes

- const [UpdateFrequency](#) theUpdateFrequency

6.11.1 Detailed Description

Base class for [DiagnosticOptions](#). Other Diagnostics can inherit from here if they require more options.

6.11.2 Constructor & Destructor Documentation

6.11.2.1 DiagnosticOptions()

```
DiagnosticOptions::DiagnosticOptions (
    UpdateFrequency thefrequency) [inline]
```

6.11.2.2 ~DiagnosticOptions()

```
virtual DiagnosticOptions::~~DiagnosticOptions () [virtual], [default]
```

6.11.3 Member Data Documentation

6.11.3.1 theUpdateFrequency

```
const UpdateFrequency DiagnosticOptions::theUpdateFrequency
```

The documentation for this struct was generated from the following file:

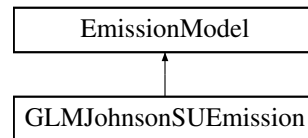
- FOORT/src/[Diagnostics.h](#)

6.12 EmissionModel Struct Reference

Emission model abstract base class.

```
#include <DiagnosticsEmission.h>
```

Inheritance diagram for EmissionModel:



Public Member Functions

- virtual [~EmissionModel](#) ()=default
- virtual [real GetEmission](#) (const [Point](#) &p) const =0
- virtual std::string [getFullDescriptionStr](#) () const

Emission model functions.

6.12.1 Detailed Description

Emission model abstract base class.

6.12.2 Constructor & Destructor Documentation

6.12.2.1 ~EmissionModel()

```
virtual EmissionModel::~~EmissionModel () [virtual], [default]
```

6.12.3 Member Function Documentation

6.12.3.1 GetEmission()

```
virtual real EmissionModel::GetEmission (
    const Point & p) const [pure virtual]
```

Implemented in [GLMJohnsonSUEmission](#).

6.12.3.2 getFullDescriptionStr()

```
std::string EmissionModel::getFullDescriptionStr () const [virtual]
```

Emission model functions.

Base class default string getter

Returns

std::string

Reimplemented in [GLMJohnsonSUEmission](#).

The documentation for this struct was generated from the following files:

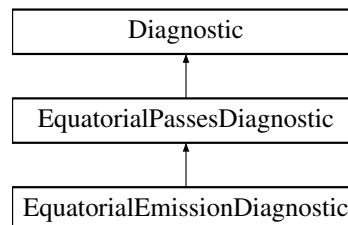
- FOORT/src/[DiagnosticsEmission.h](#)
- FOORT/src/[DiagnosticsEmission.cpp](#)

6.13 EquatorialEmissionDiagnostic Class Reference

[Diagnostic](#) that calculates brightness intensity for the geodesic, based on a specified equatorial disc emission model.

```
#include <Diagnostics.h>
```

Inheritance diagram for EquatorialEmissionDiagnostic:



Public Member Functions

- [EquatorialEmissionDiagnostic](#) ([Geodesic](#) *const theGeodesic)
- void [Reset](#) () final
EquatorialEmissionDiagnostic functions.
- void [UpdateData](#) () final
Update the intensity based on the emission model and the number of equatorial passes.
- std::string [getFullDataStr](#) () const final
Return the intensity and number of equatorial passes as a string.
- std::vector< [real](#) > [getFinalDataVal](#) () const final
Return the intensity and number of equatorial passes as a vector of size two.
- [real](#) [FinalDataValDistance](#) (const std::vector< [real](#) > &val1, const std::vector< [real](#) > &val2) const final
Return the distance between two geodesics based on their intensity and number of equatorial passes.
- std::string [getNameStr](#) () const final
Simple name without spaces.
- std::string [getFullDescriptionStr](#) () const final
More descriptive string (with spaces)

Public Member Functions inherited from [EquatorialPassesDiagnostic](#)

- [EquatorialPassesDiagnostic](#) ([Geodesic](#) *const theGeodesic)
- void [Reset](#) () override
[EquatorialPassesDiagnostic](#) functions.
- void [UpdateData](#) () override
Check to see if we have crossed the equatorial plane and update the number of passes.
- std::string [getFullDataStr](#) () const override
Return the number of equatorial passes as a string.
- std::vector< [real](#) > [getFinalDataVal](#) () const override
Return the number of equatorial passes as a vector of size one.
- [real](#) [FinalDataValDistance](#) (const std::vector< [real](#) > &val1, const std::vector< [real](#) > &val2) const override
Return the absolute value of the difference between the number of equatorial passes.
- std::string [getNameStr](#) () const override
Simple name without spaces.
- std::string [getFullDescriptionStr](#) () const override
More descriptive string (with spaces)

Public Member Functions inherited from [Diagnostic](#)

- [Diagnostic](#) ()=delete
- [Diagnostic](#) ([Geodesic](#) *const theGeodesic)
- virtual [~Diagnostic](#) ()=default

Static Public Attributes

- static std::unique_ptr< [EquatorialEmissionOptions](#) > [DiagOptions](#)

Static Public Attributes inherited from [EquatorialPassesDiagnostic](#)

- static std::unique_ptr< [EquatorialPassesOptions](#) > [DiagOptions](#)
Needs the extra option of a threshold.

Private Attributes

- [real](#) [m_Intensity](#) {0.0}

Additional Inherited Members

Protected Member Functions inherited from [Diagnostic](#)

- bool [DecideUpdate](#) (const [UpdateFrequency](#) &myUpdateFrequency)
Returns true if the [Diagnostic](#) should update its initial status.

Protected Attributes inherited from [EquatorialPassesDiagnostic](#)

- int [m_EquatPasses](#) {0}
Keeps track of how many passes have been made.

Protected Attributes inherited from [Diagnostic](#)

- [Geodesic](#) *const `m_OwnerGeodesic`
- `largecounter` `m_StepsSinceUpdated` {}

6.13.1 Detailed Description

[Diagnostic](#) that calculates brightness intensity for the geodesic, based on a specified equatorial disc emission model.

Inherits from [EquatorialPassesDiagnostic](#), since it needs to keep track of when it passes through the equatorial plane. As a result it also keeps track of the number of equatorial passes. Both the intensity profile and fluid velocity profile are specified in the options struct.

6.13.2 Constructor & Destructor Documentation

6.13.2.1 EquatorialEmissionDiagnostic()

```
EquatorialEmissionDiagnostic::EquatorialEmissionDiagnostic (
    Geodesic *const theGeodesic) [inline]
```

6.13.3 Member Function Documentation

6.13.3.1 FinalDataValDistance()

```
real EquatorialEmissionDiagnostic::FinalDataValDistance (
    const std::vector< real > & val1,
    const std::vector< real > & val2) const [final], [virtual]
```

Return the distance between two geodesics based on their intensity and number of equatorial passes.

Parameters

<i>val1</i>	first geodesic intensity and number of equatorial passes
<i>val2</i>	second geodesic intensity and number of equatorial passes

Returns

`real`

Implements [Diagnostic](#).

6.13.3.2 getFinalDataVal()

```
std::vector< real > EquatorialEmissionDiagnostic::getFinalDataVal () const [final], [virtual]
```

Return the intensity and number of equatorial passes as a vector of size two.

Returns

`std::vector<real>`

Implements [Diagnostic](#).

6.13.3.3 `getFullDataStr()`

```
std::string EquatorialEmissionDiagnostic::getFullDataStr () const [final], [virtual]
```

Return the intensity and number of equatorial passes as a string.

Returns

std::string

Implements [Diagnostic](#).

6.13.3.4 `getFullDescriptionStr()`

```
std::string EquatorialEmissionDiagnostic::getFullDescriptionStr () const [final], [virtual]
```

More descriptive string (with spaces)

Returns

std::string

Reimplemented from [Diagnostic](#).

6.13.3.5 `getNameStr()`

```
std::string EquatorialEmissionDiagnostic::getNameStr () const [final], [virtual]
```

Simple name without spaces.

Returns

std::string

Implements [Diagnostic](#).

6.13.3.6 `Reset()`

```
void EquatorialEmissionDiagnostic::Reset () [final], [virtual]
```

[EquatorialEmissionDiagnostic](#) functions.

Reset the intensity and number of equatorial passes to zero and call base class Reset function

Reimplemented from [Diagnostic](#).

6.13.3.7 UpdateData()

```
void EquatorialEmissionDiagnostic::UpdateData () [final], [virtual]
```

Update the intensity based on the emission model and the number of equatorial passes.

Implements [Diagnostic](#).

6.13.4 Member Data Documentation

6.13.4.1 DiagOptions

```
std::unique_ptr< EquatorialEmissionOptions > EquatorialEmissionDiagnostic::DiagOptions [static]
```

6.13.4.2 m_Intensity

```
real EquatorialEmissionDiagnostic::m_Intensity {0.0} [private]
```

The documentation for this class was generated from the following files:

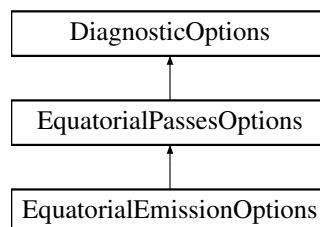
- FOORT/src/[Diagnostics.h](#)
- FOORT/src/[Config.cpp](#)
- FOORT/src/[Diagnostics.cpp](#)

6.14 EquatorialEmissionOptions Struct Reference

Options for [EquatorialEmissionDiagnostic](#).

```
#include <Diagnostics.h>
```

Inheritance diagram for EquatorialEmissionOptions:



Public Member Functions

- [EquatorialEmissionOptions](#) ([real](#) thefudgefactor, int equatupper, std::unique_ptr< [EmissionModel](#) > theemission, std::unique_ptr< [FluidVelocityModel](#) > thefluidmodel, bool rlog, int theredshiftpower, [real](#) thethreshold, [UpdateFrequency](#) thefrequency)

Public Member Functions inherited from [EquatorialPassesOptions](#)

- [EquatorialPassesOptions](#) ([real](#) thethreshold, [UpdateFrequency](#) thefrequency)

Public Member Functions inherited from [DiagnosticOptions](#)

- [DiagnosticOptions](#) ([UpdateFrequency](#) thefrequency)
- virtual [~DiagnosticOptions](#) ()=default

Public Attributes

- const [real](#) GeometricFudgeFactor
- const int [EquatPassUpperBound](#)
- const bool [RLogScale](#)
- const int [RedShiftPower](#)
- const std::unique_ptr< [EmissionModel](#) > [TheEmissionModel](#)
- const std::unique_ptr< [FluidVelocityModel](#) > [TheFluidVelocityModel](#)

Public Attributes inherited from [EquatorialPassesOptions](#)

- const [real](#) Threshold

Public Attributes inherited from [DiagnosticOptions](#)

- const [UpdateFrequency](#) theUpdateFrequency

6.14.1 Detailed Description

Options for [EquatorialEmissionDiagnostic](#).

Inherits from [EquatorialPassesOptions](#), and adds the fudge factor, upper bound, emission model, fluid velocity model, redshift power, and threshold.

6.14.2 Constructor & Destructor Documentation

6.14.2.1 [EquatorialEmissionOptions](#)()

```
EquatorialEmissionOptions::EquatorialEmissionOptions (
    real thefudgefactor,
    int equatupper,
    std::unique_ptr< EmissionModel > theemission,
    std::unique_ptr< FluidVelocityModel > thefluidmodel,
    bool rlog,
    int theredshiftpower,
    real thethreshold,
    UpdateFrequency thefrequency) [inline]
```

6.14.3 Member Data Documentation

6.14.3.1 EquatPassUpperBound

```
const int EquatorialEmissionOptions::EquatPassUpperBound
```

6.14.3.2 GeometricFudgeFactor

```
const real EquatorialEmissionOptions::GeometricFudgeFactor
```

6.14.3.3 RedShiftPower

```
const int EquatorialEmissionOptions::RedShiftPower
```

6.14.3.4 RLogScale

```
const bool EquatorialEmissionOptions::RLogScale
```

6.14.3.5 TheEmissionModel

```
const std::unique_ptr<EmissionModel> EquatorialEmissionOptions::TheEmissionModel
```

6.14.3.6 TheFluidVelocityModel

```
const std::unique_ptr<FluidVelocityModel> EquatorialEmissionOptions::TheFluidVelocityModel
```

The documentation for this struct was generated from the following file:

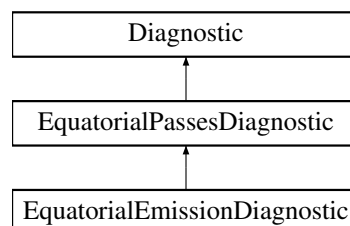
- FOORT/src/[Diagnostics.h](#)

6.15 EquatorialPassesDiagnostic Class Reference

[Diagnostic](#) that counts the number of passes through the equatorial plane.

```
#include <Diagnostics.h>
```

Inheritance diagram for EquatorialPassesDiagnostic:



Public Member Functions

- [EquatorialPassesDiagnostic](#) ([Geodesic](#) *const theGeodesic)
- void [Reset](#) () override
[EquatorialPassesDiagnostic](#) functions.
- void [UpdateData](#) () override
Check to see if we have crossed the equatorial plane and update the number of passes.
- std::string [getFullDataStr](#) () const override
Return the number of equatorial passes as a string.
- std::vector< [real](#) > [getFinalDataVal](#) () const override
Return the number of equatorial passes as a vector of size one.
- [real](#) [FinalDataValDistance](#) (const std::vector< [real](#) > &val1, const std::vector< [real](#) > &val2) const override
Return the absolute value of the difference between the number of equatorial passes.
- std::string [getNameStr](#) () const override
Simple name without spaces.
- std::string [getFullDescriptionStr](#) () const override
More descriptive string (with spaces)

Public Member Functions inherited from [Diagnostic](#)

- [Diagnostic](#) ()=delete
- [Diagnostic](#) ([Geodesic](#) *const theGeodesic)
- virtual [~Diagnostic](#) ()=default

Static Public Attributes

- static std::unique_ptr< [EquatorialPassesOptions](#) > [DiagOptions](#)
Needs the extra option of a threshold.

Protected Attributes

- int [m_EquatPasses](#) {0}
Keeps track of how many passes have been made.

Protected Attributes inherited from [Diagnostic](#)

- [Geodesic](#) *const [m_OwnerGeodesic](#)
- [largecounter](#) [m_StepsSinceUpdated](#) {}

Private Attributes

- [real](#) [m_PrevTheta](#) {-1}
Keeps track of the previous theta angle, so that we can compare with current theta angle.

Additional Inherited Members

Protected Member Functions inherited from [Diagnostic](#)

- bool [DecideUpdate](#) (const [UpdateFrequency](#) &myUpdateFrequency)
Returns true if the [Diagnostic](#) should update its initial status.

6.15.1 Detailed Description

[Diagnostic](#) that counts the number of passes through the equatorial plane.

6.15.2 Constructor & Destructor Documentation

6.15.2.1 EquatorialPassesDiagnostic()

```
EquatorialPassesDiagnostic::EquatorialPassesDiagnostic (
    Geodesic *const theGeodesic) [inline]
```

6.15.3 Member Function Documentation

6.15.3.1 FinalDataValDistance()

```
real EquatorialPassesDiagnostic::FinalDataValDistance (
    const std::vector< real > & val1,
    const std::vector< real > & val2) const [override], [virtual]
```

Return the absolute value of the difference between the number of equatorial passes.

Parameters

<i>val1</i>	first number of equatorial passes
<i>val2</i>	second number of equatorial passes

Returns

[real](#)

Implements [Diagnostic](#).

6.15.3.2 getFinalDataVal()

```
std::vector< real > EquatorialPassesDiagnostic::getFinalDataVal () const [override], [virtual]
```

Return the number of equatorial passes as a vector of size one.

Returns

std::vector<real>

Implements [Diagnostic](#).

6.15.3.3 getFullDataStr()

```
std::string EquatorialPassesDiagnostic::getFullDataStr () const [override], [virtual]
```

Return the number of equatorial passes as a string.

Returns

std::string

Implements [Diagnostic](#).

6.15.3.4 getFullDescriptionStr()

```
std::string EquatorialPassesDiagnostic::getFullDescriptionStr () const [override], [virtual]
```

More descriptive string (with spaces)

Returns

std::string

Reimplemented from [Diagnostic](#).

6.15.3.5 getNameStr()

```
std::string EquatorialPassesDiagnostic::getNameStr () const [override], [virtual]
```

Simple name without spaces.

Returns

std::string

Implements [Diagnostic](#).

6.15.3.6 Reset()

```
void EquatorialPassesDiagnostic::Reset () [override], [virtual]
```

[EquatorialPassesDiagnostic](#) functions.

Reset the number of equatorial passes and the previous theta angle and call base class Reset function

Reimplemented from [Diagnostic](#).

6.15.3.7 UpdateData()

```
void EquatorialPassesDiagnostic::UpdateData () [override], [virtual]
```

Check to see if we have crossed the equatorial plane and update the number of passes.

Implements [Diagnostic](#).

6.15.4 Member Data Documentation

6.15.4.1 DiagOptions

```
std::unique_ptr< EquatorialPassesOptions > EquatorialPassesDiagnostic::DiagOptions [static]
```

Needs the extra option of a threshold.

6.15.4.2 m_EquatPasses

```
int EquatorialPassesDiagnostic::m_EquatPasses {0} [protected]
```

Keeps track of how many passes have been made.

6.15.4.3 m_PrevTheta

```
real EquatorialPassesDiagnostic::m_PrevTheta {-1} [private]
```

Keeps track of the previous theta angle, so that we can compare with current theta angle.

The documentation for this class was generated from the following files:

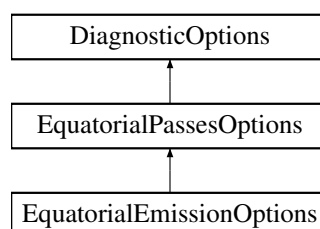
- FOORT/src/[Diagnostics.h](#)
- FOORT/src/[Config.cpp](#)
- FOORT/src/[Diagnostics.cpp](#)

6.16 EquatorialPassesOptions Struct Reference

Options for [EquatorialPassesDiagnostic](#).

```
#include <Diagnostics.h>
```

Inheritance diagram for EquatorialPassesOptions:



Public Member Functions

- [EquatorialPassesOptions](#) ([real](#) thethreshold, [UpdateFrequency](#) thefrequency)

Public Member Functions inherited from [DiagnosticOptions](#)

- [DiagnosticOptions](#) ([UpdateFrequency](#) thefrequency)
- virtual [~DiagnosticOptions](#) ()=default

Public Attributes

- const [real](#) Threshold

Public Attributes inherited from [DiagnosticOptions](#)

- const [UpdateFrequency](#) theUpdateFrequency

6.16.1 Detailed Description

Options for [EquatorialPassesDiagnostic](#).

Inherits from [DiagnosticOptions](#), and adds the threshold for counting passes.

6.16.2 Constructor & Destructor Documentation

6.16.2.1 [EquatorialPassesOptions\(\)](#)

```
EquatorialPassesOptions::EquatorialPassesOptions (
    real thethreshold,
    UpdateFrequency thefrequency) [inline]
```

6.16.3 Member Data Documentation

6.16.3.1 Threshold

```
const real EquatorialPassesOptions::Threshold
```

The documentation for this struct was generated from the following file:

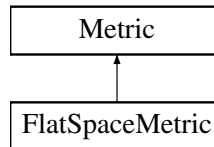
- FOORT/src/[Diagnostics.h](#)

6.17 FlatSpaceMetric Class Reference

Flat space metric in 4D.

```
#include <Metric.h>
```

Inheritance diagram for FlatSpaceMetric:



Public Member Functions

- [FlatSpaceMetric](#) (bool rlogscale=false)
Construct a new Flat Space [Metric](#) object.
- [TwoIndex getMetric_dd](#) (const [Point](#) &p) const final
Flat metric getter, indices down.
- [TwoIndex getMetric_uu](#) (const [Point](#) &p) const final
Flat metric getter, indices up.
- std::string [getFullDescriptionStr](#) () const final
Flat metric description string getter.

Public Member Functions inherited from [Metric](#)

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new [Metric](#) object.
- virtual [ThreeIndex getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual [real getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- bool [getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Additional Inherited Members

Protected Attributes inherited from [Metric](#)

- std::vector< int > [m_Symmetries](#) {}
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- const bool [m_rLogScale](#)
Are we using a logarithmic r coordinate?

6.17.1 Detailed Description

Flat space metric in 4D.

6.17.2 Constructor & Destructor Documentation

6.17.2.1 FlatSpaceMetric()

```
FlatSpaceMetric::FlatSpaceMetric (
    bool rlogscale = false)
```

Construct a new Flat Space [Metric](#) object.

Parameters

<i>rlogscale</i>	
------------------	--

6.17.3 Member Function Documentation

6.17.3.1 getFullDescriptionStr()

```
std::string FlatSpaceMetric::getFullDescriptionStr () const [final], [virtual]
```

Flat metric description string getter.

Returns

std::string

Reimplemented from [Metric](#).

6.17.3.2 getMetric_dd()

```
TwoIndex FlatSpaceMetric::getMetric_dd (
    const Point & p) const [final], [virtual]
```

Flat metric getter, indices down.

Parameters

<i>p</i>	Point at which to evaluate the metric
----------	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).

6.17.3.3 getMetric_uu()

```
TwoIndex FlatSpaceMetric::getMetric_uu (
    const Point & p) const [final], [virtual]
```

Flat metric getter, indices up.

Parameters

p	Point at which to evaluate the metric
-----	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).

The documentation for this class was generated from the following files:

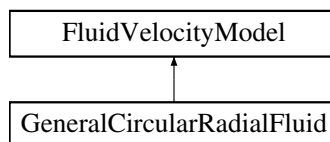
- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.18 FluidVelocityModel Struct Reference

Fluid velocity model abstract base class.

```
#include <DiagnosticsEmission.h>
```

Inheritance diagram for FluidVelocityModel:



Public Member Functions

- [FluidVelocityModel](#) (const [Metric](#) *const theMetric)
- virtual [~FluidVelocityModel](#) ()=default
- virtual [OneIndex GetFourVelocityd](#) (const [Point](#) &p) const =0
- virtual std::string [getFullDescriptionStr](#) () const

FluidVelocityModel functions.

Protected Attributes

- const [Metric](#) *const [m_theMetric](#)

6.18.1 Detailed Description

Fluid velocity model abstract base class.

6.18.2 Constructor & Destructor Documentation

6.18.2.1 FluidVelocityModel()

```
FluidVelocityModel::FluidVelocityModel (
    const Metric *const theMetric) [inline]
```

6.18.2.2 ~FluidVelocityModel()

```
virtual FluidVelocityModel::~~FluidVelocityModel () [virtual], [default]
```

6.18.3 Member Function Documentation

6.18.3.1 GetFourVelocityd()

```
virtual OneIndex FluidVelocityModel::GetFourVelocityd (
    const Point & p) const [pure virtual]
```

Implemented in [GeneralCircularRadialFluid](#).

6.18.3.2 getFullDescriptionStr()

```
std::string FluidVelocityModel::getFullDescriptionStr () const [virtual]
```

[FluidVelocityModel](#) functions.

Base class default string getter

Returns

std::string

Reimplemented in [GeneralCircularRadialFluid](#).

6.18.4 Member Data Documentation

6.18.4.1 m_theMetric

```
const Metric* const FluidVelocityModel::m_theMetric [protected]
```

The documentation for this struct was generated from the following files:

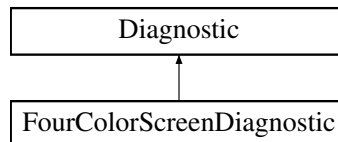
- FOORT/src/[DiagnosticsEmission.h](#)
- FOORT/src/[DiagnosticsEmission.cpp](#)

6.19 FourColorScreenDiagnostic Class Reference

The four color screen diagnostic.

```
#include <Diagnostics.h>
```

Inheritance diagram for FourColorScreenDiagnostic:



Public Member Functions

- [FourColorScreenDiagnostic](#) ([Geodesic](#) *const theGeodesic)
- void [Reset](#) () final
FourColorScreen functions.
- void [UpdateData](#) () override
Update the quadrant based on the geodesic's final position.
- std::string [getFullDataStr](#) () const final
Return the value of the quadrant as a string.
- std::vector< [real](#) > [getFinalDataVal](#) () const final
Return the value of the quadrant as a vector of size one.
- [real](#) [FinalDataValDistance](#) (const std::vector< [real](#) > &val1, const std::vector< [real](#) > &val2) const final
Discrete metric for distance: return 0 if the quadrants are the same, 1 if they are not.
- std::string [getNameStr](#) () const final
Simple name without spaces.
- std::string [getFullDescriptionStr](#) () const final
Return full name, possibly with spaces.

Public Member Functions inherited from [Diagnostic](#)

- [Diagnostic](#) ()=delete
- [Diagnostic](#) ([Geodesic](#) *const theGeodesic)
- virtual [~Diagnostic](#) ()=default

Private Attributes

- int [m_quadrant](#) {0}
Note initialization to 0; this means the default value returned will be 0 (e.g. if [Term::BoundarySphere](#) is not reached)

Additional Inherited Members

Protected Member Functions inherited from [Diagnostic](#)

- bool [DecideUpdate](#) (const [UpdateFrequency](#) &myUpdateFrequency)
Returns true if the [Diagnostic](#) should update its initial status.

Protected Attributes inherited from [Diagnostic](#)

- [Geodesic](#) *const [m_OwnerGeodesic](#)
- [largecounter](#) [m_StepsSinceUpdated](#) {}

6.19.1 Detailed Description

The four color screen diagnostic.

This diagnostic associates one of four colors based on the quadrant that the geodesic finishes in. Will ONLY return a color if the geodesic indeed finishes because it passes through the boundary sphere!

6.19.2 Constructor & Destructor Documentation

6.19.2.1 FourColorScreenDiagnostic()

```
FourColorScreenDiagnostic::FourColorScreenDiagnostic (
    Geodesic *const theGeodesic) [inline]
```

6.19.3 Member Function Documentation

6.19.3.1 FinalDataValDistance()

```
real FourColorScreenDiagnostic::FinalDataValDistance (
    const std::vector< real > & val1,
    const std::vector< real > & val2) const [final], [virtual]
```

Discrete metric for distance: return 0 if the quadrants are the same, 1 if they are not.

Parameters

<i>val1</i>	first quadrant value
<i>val2</i>	second quadrant value

Returns

[real](#)

Implements [Diagnostic](#).

6.19.3.2 getFinalDataVal()

```
std::vector< real > FourColorScreenDiagnostic::getFinalDataVal () const [final], [virtual]
```

Return the value of the quadrant as a vector of size one.

Returns

std::vector<real>

Implements [Diagnostic](#).

6.19.3.3 getFullDataStr()

```
std::string FourColorScreenDiagnostic::getFullDataStr () const [final], [virtual]
```

Return the value of the quadrant as a string.

Returns

std::string

Implements [Diagnostic](#).

6.19.3.4 getFullDescriptionStr()

```
std::string FourColorScreenDiagnostic::getFullDescriptionStr () const [final], [virtual]
```

Return full name, possibly with spaces.

Returns

std::string

Reimplemented from [Diagnostic](#).

6.19.3.5 getNameStr()

```
std::string FourColorScreenDiagnostic::getNameStr () const [final], [virtual]
```

Simple name without spaces.

Returns

std::string

Implements [Diagnostic](#).

6.19.3.6 Reset()

```
void FourColorScreenDiagnostic::Reset () [final], [virtual]
```

FourColorScreen functions.

Reset the quadrant to its default option; also call base class Reset function

Reimplemented from [Diagnostic](#).

6.19.3.7 UpdateData()

```
void FourColorScreenDiagnostic::UpdateData () [override], [virtual]
```

Update the quadrant based on the geodesic's final position.

Implements [Diagnostic](#).

6.19.4 Member Data Documentation

6.19.4.1 m_quadrant

```
int FourColorScreenDiagnostic::m_quadrant {0} [private]
```

Note initialization to 0; this means the default value returned will be 0 (e.g. if [Term::BoundarySphere](#) is not reached)

The documentation for this class was generated from the following files:

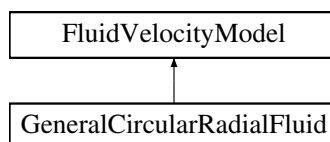
- FOORT/src/[Diagnostics.h](#)
- FOORT/src/[Diagnostics.cpp](#)

6.20 GeneralCircularRadialFluid Struct Reference

General circular radial fluid velocity model.

```
#include <DiagnosticsEmission.h>
```

Inheritance diagram for GeneralCircularRadialFluid:



Public Member Functions

- [GeneralCircularRadialFluid](#) ([real](#) subKeplerParam, [real](#) betar, [real](#) betaphi, const [Metric](#) *const theMetric)
- [OneIndex GetFourVelocityd](#) (const [Point](#) &p) const final
Get the local four-velocity of the fluid according to the GeneralCircularRadial model.
- [std::string getFullDescriptionStr](#) () const final
Description string getter for the GeneralCircularRadial fluid velocity model.

Public Member Functions inherited from [FluidVelocityModel](#)

- [FluidVelocityModel](#) (const [Metric](#) *const theMetric)
- virtual [~FluidVelocityModel](#) ()=default

Private Member Functions

- [OneIndex GetCircularVelocityd](#) (const [Point](#) &p, bool subKeplerianOn=true) const
Get the circular velocity for a point, with optional subKeplerian rescaling.
- [OneIndex GetInsideISCOCircularVelocityd](#) (const [Point](#) &p) const
Returns velocity for non-geodetic motion where $E, L = (E, L)_{ISCO}$ and radially falling inward.
- [OneIndex GetRadialVelocityd](#) (const [Point](#) &p) const
Returns radial velocity for pure infalling matter with $E = 1$ and $L = 0$ at infinity.
- void [FindISCO](#) ()
Helper function to find the ISCO.
- [TwoIndex GetChrstrRaisedDer](#) (real r) const
Helper function to get $\partial_r(g^{ab})\Gamma^c_{bc}g^{cd}$

Private Attributes

- const [real m_subKeplerParam](#)
Sub-Keplerian parameter (0 = Keplerian, 1 = sub-Keplerian)
- const [real m_betaR](#)
Radial velocity parameter.
- const [real m_betaPhi](#)
Azimuthal velocity parameter.
- bool [m_ISCOexists](#) {false}
Flag to indicate if ISCO exists for this [Metric](#).
- [real m_ISCO](#) {-1.0}
ISCO radius.
- [real m_ISCOpt](#) {}
ISCO t momentum.
- [real m_ISCOpphi](#) {}
ISCO phi momentum.

Additional Inherited Members

Protected Attributes inherited from [FluidVelocityModel](#)

- const [Metric](#) *const [m_theMetric](#)

6.20.1 Detailed Description

General circular radial fluid velocity model.

This fluid velocity model has three tuneable parameters and represents fluid travelling at a mix of (sub)Keplerian circular orbits and radially infalling orbits in the equatorial plane

6.20.2 Constructor & Destructor Documentation

6.20.2.1 GeneralCircularRadialFluid()

```
GeneralCircularRadialFluid::GeneralCircularRadialFluid (
    real subKeplerParam,
    real betar,
    real betaphi,
    const Metric *const theMetric) [inline]
```

6.20.3 Member Function Documentation

6.20.3.1 FindISCO()

```
void GeneralCircularRadialFluid::FindISCO () [private]
```

Helper function to find the ISCO.

This helper function finds the ISCO. We use a binary search to converge on the ISCO value; the initial outer bounds for a metric with horizon are the horizon radius and $10 \times (\text{horizon radius})$ (in true radii, not $\log(r)$ coordinates)

6.20.3.2 GetChrstrRaisedDer()

```
TwoIndex GeneralCircularRadialFluid::GetChrstrRaisedDer (
    real r) const [private]
```

Helper function to get $\partial_r(g^{ab})\Gamma^r_{bc}g^{cd}$

This function calculates $\partial_r(g^{ab})\Gamma^r_{bc}g^{cd}$. We do so by calculating the central difference with $O(h^4)$ accuracy, but taking the sqrt of the usual h used for derivatives

Parameters

r	Radius at which to calculate the derivative
-----	---

Returns

TwoIndex

6.20.3.3 GetCircularVelocityd()

```
OneIndex GeneralCircularRadialFluid::GetCircularVelocityd (
    const Point & p,
    bool subKeplerianOn = true) const [private]
```

Get the circular velocity for a point, with optional subKeplerian rescaling.

Returns the velocity for circular (sub)Keplerian motion (outside the ISCO). First the circular geodesic is calculated, then if `subKeplerianOn == true`, the orbit is reparametrized to non-geodetic subKeplerian circular motion according to the subKeplerian parameter

Parameters

p	Point at which to calculate the velocity
<code>subKeplerianOn</code>	Boolean to indicate whether to rescale the angular momentum for sub-Keplerian motion

Returns

OneIndex circular velocity with index down. If no solution, returns all zeros

6.20.3.4 GetFourVelocityd()

```
OneIndex GeneralCircularRadialFluid::GetFourVelocityd (
    const Point & p) const [final], [virtual]
```

Get the local four-velocity of the fluid according to the GeneralCircularRadial model.

Parameters

p	Point at which to calculate the four-velocity
-----	---

Returns

OneIndex Local four-velocity with index down

Implements [FluidVelocityModel](#).

6.20.3.5 getFullDescriptionStr()

```
std::string GeneralCircularRadialFluid::getFullDescriptionStr () const [final], [virtual]
```

Description string getter for the GeneralCircularRadial fluid velocity model.

Returns

std::string

Reimplemented from [FluidVelocityModel](#).

6.20.3.6 GetInsideISCOCircularVelocityd()

```
OneIndex GeneralCircularRadialFluid::GetInsideISCOCircularVelocityd (
    const Point & p) const [private]
```

Returns velocity for non-geodetic motion where $E, L = (E, L)_{\text{ISCO}}$ and radially falling inward.

Parameters

p	Point at which to calculate the velocity
-----	--

Returns

OneIndex

6.20.3.7 GetRadialVelocityd()

```
OneIndex GeneralCircularRadialFluid::GetRadialVelocityd (
    const Point & p) const [private]
```

Returns radial velocity for pure infalling matter with $E = 1$ and $L = 0$ at infinity.

Parameters

p	Point at which to calculate the velocity
-----	--

Returns

OneIndex

6.20.4 Member Data Documentation

6.20.4.1 m_betaPhi

```
const real GeneralCircularRadialFluid::m_betaPhi [private]
```

Azimuthal velocity parameter.

6.20.4.2 m_betaR

```
const real GeneralCircularRadialFluid::m_betaR [private]
```

Radial velocity parameter.

6.20.4.3 m_ISCOexists

```
bool GeneralCircularRadialFluid::m_ISCOexists {false} [private]
```

Flag to indicate if ISCO exists for this [Metric](#).

6.20.4.4 m_ISCOpphi

```
real GeneralCircularRadialFluid::m_ISCOpphi {} [private]
```

ISCO phi momentum.

6.20.4.5 m_ISCOpt

```
real GeneralCircularRadialFluid::m_ISCOpt {} [private]
```

ISCO t momentum.

6.20.4.6 m_ISCOr

```
real GeneralCircularRadialFluid::m_ISCOr {-1.0} [private]
```

ISCO radius.

6.20.4.7 m_subKeplerParam

```
const real GeneralCircularRadialFluid::m_subKeplerParam [private]
```

Sub-Keplerian parameter (0 = Keplerian, 1 = sub-Keplerian)

The documentation for this struct was generated from the following files:

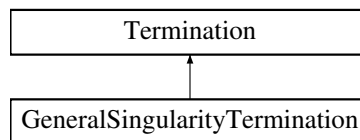
- FOORT/src/[DiagnosticsEmission.h](#)
- FOORT/src/[DiagnosticsEmission.cpp](#)

6.21 GeneralSingularityTermination Class Reference

[GeneralSingularityTermination](#): Terminate geodesics if they get too close to one of a number of singularities (of arbitrary codimension)

```
#include <Terminations.h>
```

Inheritance diagram for GeneralSingularityTermination:



Public Member Functions

- [GeneralSingularityTermination](#) ([Geodesic](#) *const theGeodesic)
- [Term CheckTermination](#) () final
GeneralSingularityTermination functions.
- std::string [getFullDescriptionStr](#) () const final

Public Member Functions inherited from [Termination](#)

- [Termination](#) ()=delete
- [Termination](#) ([Geodesic](#) *const theGeodesic)
- virtual void [Reset](#) ()
Termination (abstract base class) functions.
- virtual [~Termination](#) ()=default

Static Public Attributes

- static std::unique_ptr< [GeneralSingularityTermOptions](#) > [TermOptions](#)
Options (contains horizon radius, if we are using logarithmic r coordinate, and distance allowed from the horizon)

Private Member Functions

- std::string [SingularityToString](#) (int singnr) const

Additional Inherited Members

Protected Member Functions inherited from [Termination](#)

- bool [DecideUpdate](#) ([largecounter](#) UpdateNSteps)

Protected Attributes inherited from [Termination](#)

- [Geodesic](#) *const [m_OwnerGeodesic](#)

The geodesic that owns the [Termination](#) (a const pointer to the [Geodesic](#))

- [largecounter](#) [m_StepsSinceUpdated](#) {}

The termination is itself in charge of keeping track of how many steps it has been since it has been updated. The [Termination](#)'s [TerminationOptions](#) struct tells it how many steps it needs to wait between updates.

6.21.1 Detailed Description

[GeneralSingularityTermination](#): Terminate geodesics if they get too close to one of a number of singularities (of arbitrary codimension)

6.21.2 Constructor & Destructor Documentation

6.21.2.1 [GeneralSingularityTermination\(\)](#)

```
GeneralSingularityTermination::GeneralSingularityTermination (
    Geodesic *const theGeodesic) [inline]
```

6.21.3 Member Function Documentation

6.21.3.1 [CheckTermination\(\)](#)

```
Term GeneralSingularityTermination::CheckTermination () [final], [virtual]
```

[GeneralSingularityTermination](#) functions.

Implements [Termination](#).

6.21.3.2 [getFullDescriptionStr\(\)](#)

```
std::string GeneralSingularityTermination::getFullDescriptionStr () const [final], [virtual]
```

Implements [Termination](#).

6.21.3.3 [SingularityToString\(\)](#)

```
std::string GeneralSingularityTermination::SingularityToString (
    int singnr) const [private]
```

6.21.4 Member Data Documentation

6.21.4.1 TermOptions

```
std::unique_ptr< GeneralSingularityTermOptions > GeneralSingularityTermination::TermOptions
[static]
```

Options (contains horizon radius, if we are using logarithmic r coordinate, and distance allowed from the horizon)

The documentation for this class was generated from the following files:

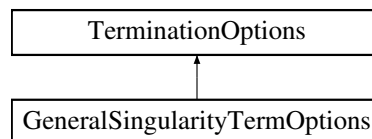
- FOORT/src/Terminations.h
- FOORT/src/Config.cpp
- FOORT/src/Terminations.cpp

6.22 GeneralSingularityTermOptions Struct Reference

Options for [GeneralSingularityTermination](#).

```
#include <Terminations.h>
```

Inheritance diagram for GeneralSingularityTermOptions:



Public Member Functions

- [GeneralSingularityTermOptions](#) (std::vector< [Singularity](#) > sings, [real](#) eps, bool consoleoutputon, bool therlogscale, [largecounter](#) Nsteps)

Public Member Functions inherited from [TerminationOptions](#)

- [TerminationOptions](#) ([largecounter](#) Nsteps)
- virtual [~TerminationOptions](#) ()=default

Public Attributes

- const std::vector< [Singularity](#) > [Singularities](#)
Vector of singularities.
- const [real](#) [Epsilon](#)
Minimum flat distance the geodesic is allowed to be from any singularity.
- const bool [OutputToConsole](#)
Whether to output a message to the console when a singularity is encountered.
- const bool [rLogScale](#)
Whether we are using a logarithmic r coordinate or not.

Public Attributes inherited from [TerminationOptions](#)

- const [largecounter](#) [UpdateEveryNSteps](#)
Number of steps between updates.

6.22.1 Detailed Description

Options for [GeneralSingularityTermination](#).

Keeps track of the singularities, allowed distance and whether we are using a logarithmic r coordinate.

6.22.2 Constructor & Destructor Documentation

6.22.2.1 [GeneralSingularityTermOptions\(\)](#)

```
GeneralSingularityTermOptions::GeneralSingularityTermOptions (
    std::vector< Singularity > sings,
    real eps,
    bool consoleoutputon,
    bool therlogscale,
    largecounter Nsteps) [inline]
```

6.22.3 Member Data Documentation

6.22.3.1 [Epsilon](#)

```
const real GeneralSingularityTermOptions::Epsilon
```

Minimum flat distance the geodesic is allowed to be from any singularity.

6.22.3.2 [OutputToConsole](#)

```
const bool GeneralSingularityTermOptions::OutputToConsole
```

Whether to output a message to the console when a singularity is encountered.

6.22.3.3 [rLogScale](#)

```
const bool GeneralSingularityTermOptions::rLogScale
```

Whether we are using a logarithmic r coordinate or not.

6.22.3.4 Singularities

```
const std::vector<Singularity> GeneralSingularityTermOptions::Singularities
```

Vector of singularities.

The documentation for this struct was generated from the following file:

- FOORT/src/[Terminations.h](#)

6.23 Geodesic Class Reference

[Geodesic](#) class: an instance of this class is created for each [Geodesic](#) that is integrated.

```
#include <Geodesic.h>
```

Public Member Functions

- [Geodesic](#) ()=delete
- [Geodesic](#) (const [Geodesic](#) &)=delete
- [Geodesic](#) & operator= (const [Geodesic](#) &)=delete
- [Geodesic](#) (const [Metric](#) *const theMetric, const [Source](#) *const theSource, [DiagBitflag](#) diagbit, [DiagBitflag](#) valdiagbit, [TermBitflag](#) termbit, [GeodesicIntegratorFunc](#) theIntegrator)

Construct a new [Geodesic](#) object. Arguments initialize private member variables that remain the same over all geodesics.
- void [Reset](#) ([ScreenIndex](#) scrindex, [Point](#) initpos, [OneIndex](#) initvel)

[Geodesic](#) (and descendant classes) functions.
- [Term Update](#) ()

This makes the [Geodesic](#) integrate itself one step; then the [Geodesic](#) loops through all Terminations and Diagnostics to update.
- [Term](#) [getTermCondition](#) () const

Get the current termination condition.
- [Point](#) [getCurrentPos](#) () const

Get the current position.
- [OneIndex](#) [getCurrentVel](#) () const

Get the current velocity.
- [real](#) [getCurrentLambda](#) () const

Get the current value of the affine parameter.
- [ScreenIndex](#) [getScreenIndex](#) () const

Get the screen index.
- std::vector< std::string > [getAllOutputStr](#) () const

Get the complete output that should be written to the output files.
- std::vector< [real](#) > [getDiagnosticFinalValue](#) () const

Get the "value" (from the [Diagnostic](#) that was set to the value [Diagnostic](#)) that is associated to the [Geodesic](#).

Private Attributes

- [Term](#) `m_TermCond` {[Term::Uninitialized](#)}
As long as this is [Term::Continue](#), not done integrating yet.
- [Point](#) `m_CurrentPos` {}
Current position.
- [OneIndex](#) `m_CurrentVel` {}
Current proper velocity.
- [real](#) `m_curLambda` {0.0}
Current value of affine parameter (starts at 0.0)
- [ScreenIndex](#) `m_ScreenIndex` {}
- `const` [Metric](#) `*const m_theMetric`
const pointer to the [Metric](#)
- `const` [Source](#) `*const m_theSource`
const pointer to the [Source](#) for the rhs of the geodesic equation
- `const` [DiagnosticUniqueVector](#) `m_AllDiagnostics`
Vector of unique pointers to [Diagnostics](#). An instance of each [Diagnostic](#) is created for the [Geodesic](#), so the [Geodesic](#) is the owner of these objects.
- `const` [TerminationUniqueVector](#) `m_AllTerminations`
Vector of unique pointers to [Terminations](#). An instance of each [Termination](#) is created for the [Geodesic](#), so the [Geodesic](#) is the owner of these objects.
- `const` [GeodesicIntegratorFunc](#) `m_theIntegrator`
This is the function that will integrate the geodesic equation one step.

6.23.1 Detailed Description

[Geodesic](#) class: an instance of this class is created for each [Geodesic](#) that is integrated.

The [Geodesic](#) is in charge of integrating itself until termination, updating its [Diagnostics](#) accordingly, and (after termination) returning the appropriate output

6.23.2 Constructor & Destructor Documentation

6.23.2.1 [Geodesic\(\)](#) [1/3]

```
Geodesic::Geodesic () [delete]
```

6.23.2.2 [Geodesic\(\)](#) [2/3]

```
Geodesic::Geodesic (
    const Geodesic & ) [delete]
```

6.23.2.3 [Geodesic\(\)](#) [3/3]

```
Geodesic::Geodesic (
    const Metric *const theMetric,
    const Source *const theSource,
    DiagBitflag diagbit,
    DiagBitflag valdiagbit,
    TermBitflag termbit,
    GeodesicIntegratorFunc theIntegrator) [inline]
```

Construct a new [Geodesic](#) object. Arguments initialize private member variables that remain the same over all geodesics.

Parameters

<i>theMetric</i>	Pointer to the Metric object.
<i>theSource</i>	Pointer to the Source object.
<i>diagbit</i>	Diagnostic bitflag.
<i>valdiagbit</i>	Value diagnostic bitflag.
<i>termbit</i>	Termination bitflag.
<i>theIntegrator</i>	Geoodesic integrator function to use for integrating geodesic equation.

6.23.3 Member Function Documentation

6.23.3.1 getAllOutputStr()

```
std::vector< std::string > Geodesic::getAllOutputStr () const
```

Get the complete output that should be written to the output files.

There is one string more than the count of Diagnostics: one string per [Diagnostic](#), PLUS the first string is the screen index.

Returns

```
std::vector<std::string>
```

6.23.3.2 getCurrentLambda()

```
real Geodesic::getCurrentLambda () const
```

Get the current value of the affine parameter.

Returns

```
real
```

6.23.3.3 getCurrentPos()

```
Point Geodesic::getCurrentPos () const
```

Get the current position.

Returns

```
Point
```

6.23.3.4 `getCurrentVel()`

```
OneIndex Geodesic::getCurrentVel () const
```

Get the current velocity.

Returns

OneIndex

6.23.3.5 `getDiagnosticFinalValue()`

```
std::vector< real > Geodesic::getDiagnosticFinalValue () const
```

Get the "value" (from the [Diagnostic](#) that was set to the value [Diagnostic](#)) that is associated to the [Geodesic](#).

Will be used to determine "distance" between Geodesics which is used in [Mesh](#) refinement.

Returns

std::vector<real>

6.23.3.6 `getScreenIndex()`

```
ScreenIndex Geodesic::getScreenIndex () const
```

Get the screen index.

Returns

ScreenIndex

6.23.3.7 `getTermCondition()`

```
Term Geodesic::getTermCondition () const
```

Get the current termination condition.

Returns

Term

6.23.3.8 `operator=()`

```
Geodesic & Geodesic::operator= (
    const Geodesic & ) [delete]
```

6.23.3.9 `Reset()`

```
void Geodesic::Reset (
    ScreenIndex scrindex,
    Point initpos,
    OneIndex initvel)
```

[Geodesic](#) (and descendant classes) functions.

This initializes/resets the geodesic with a given ScreenIndex, initial position, and initial velocity

Also resets all Diagnostics and Terminations, resets the TermCondition to [Term::Continue](#), and puts the [Geodesic](#) back to $\lambda = 0.0$.

Parameters

<i>scrindex</i>	ScreenIndex
<i>initpos</i>	Initial position
<i>initvel</i>	Initial velocity

6.23.3.10 Update()

Term `Geodesic::Update ()`

This makes the [Geodesic](#) integrate itself one step; then the [Geodesic](#) loops through all Terminations and Diagnostics to update.

Returns

Term

6.23.4 Member Data Documentation**6.23.4.1 m_AllDiagnostics**

const [DiagnosticUniqueVector](#) `Geodesic::m_AllDiagnostics` [private]

Vector of unique pointers to Diagnostics. An instance of each [Diagnostic](#) is created for the [Geodesic](#), so the [Geodesic](#) is the owner of these objects.

6.23.4.2 m_AllTerminations

const [TerminationUniqueVector](#) `Geodesic::m_AllTerminations` [private]

Vector of unique pointers to Terminations. An instance of each [Termination](#) is created for the [Geodesic](#), so the [Geodesic](#) is the owner of these objects.

6.23.4.3 m_curLambda

real `Geodesic::m_curLambda {0.0}` [private]

Current value of affine parameter (starts at 0.0)

6.23.4.4 m_CurrentPos

[Point](#) `Geodesic::m_CurrentPos {}` [private]

Current position.

6.23.4.5 m_CurrentVel

```
OneIndex Geodesic::m_CurrentVel {} [private]
```

Current proper velocity.

6.23.4.6 m_ScreenIndex

```
ScreenIndex Geodesic::m_ScreenIndex {} [private]
```

The [Geodesic](#) keeps track of what index it has been assigned; it outputs this information in its final output string

6.23.4.7 m_TermCond

```
Term Geodesic::m_TermCond {Term::Uninitialized} [private]
```

As long as this is [Term::Continue](#), not done integrating yet.

6.23.4.8 m_theIntegrator

```
const GeodesicIntegratorFunc Geodesic::m_theIntegrator [private]
```

This is the function that will integrate the geodesic equation one step.

6.23.4.9 m_theMetric

```
const Metric* const Geodesic::m_theMetric [private]
```

const pointer to the [Metric](#)

6.23.4.10 m_theSource

```
const Source* const Geodesic::m_theSource [private]
```

const pointer to the [Source](#) for the rhs of the geodesic equation

The documentation for this class was generated from the following files:

- FOORT/src/[Geodesic.h](#)
- FOORT/src/[Geodesic.cpp](#)

6.24 GeodesicOutputHandler Class Reference

// [GeodesicOutputHandler](#) handles all of the output to file. It gets passed all of the output strings for every [Geodesic](#), it then stores this data until it eventually writes all data to the appropriate files

```
#include <InputOutput.h>
```

Public Member Functions

- [GeodesicOutputHandler](#) ()=delete
- [GeodesicOutputHandler](#) (std::string FilePrefix, std::string TimeStamp, std::string FileExtension, std::vector< std::string > DiagNames, [largecounter](#) nrOutputstocache=[LARGECOUNTER_MAX](#) - 1, [largecounter](#) geodperfile=[LARGECOUNTER_MAX](#), std::string firstlineinfo="")
Construct a new [Geodesic](#) Output Handler object and initialize all const member variables.
- void [PrepareForOutput](#) ([largecounter](#) nrOutputToCome)
prepare the cache for output
- void [NewGeodesicOutput](#) ([largecounter](#) index, std::vector< std::string > theOutput)
- void [OutputFinished](#) ()
There is no more output, so we write anything that is cached to file to clean up and finalize.
- std::string [getFullDescriptionStr](#) () const
full description string getter for the geodesic output handler

Private Member Functions

- void [WriteCachedOutputToFile](#) ()
Write cached output to a file.
- void [OpenForFirstTime](#) (const std::string &filename)
We check here to see if the files are being put in a (sub)directory; if so, we create the directory/directories first to make sure creating/opening the file will succeed.
- std::string [GetFileName](#) (int diagnr, unsigned short filenr) const
Get the file name based on the various parts of the file name stored in the member variables.

Private Attributes

- const std::string [m_FilePrefix](#)
Const string for file prefix.
- const std::string [m_TimeStamp](#)
Const string for timestamp.
- const std::string [m_FileExtension](#)
Const string for file extension.
- const std::vector< std::string > [m_DiagNames](#)
Const string for the diagnostic names.
- const bool [m_PrintFirstLineInfo](#)
Const variable that control whether we write a descriptive first line in every output file or not.
- const std::string [m_FirstLineInfoString](#)
Const string for the first line info.
- const [largecounter](#) [m_nrOutputsToCache](#) {}
- const [largecounter](#) [m_nrGeodesicsPerFile](#) {}
Const setting the max number of geodesics per file.
- bool [m_WriteToConsole](#) {false}
If this is false, then we are writing to file(s). At any time, if a file I/O error occurs, the output handler switches to outputting everything to the console.
- [largecounter](#) [m_PrevCached](#) {0}
How many outputs are already cached in m_nrOutputsToCache before the current iteration of output.
- [largecounter](#) [m_CurrentGeodesicsInFile](#) {0}
The number of geodesics already written to the current file (once this hits m_nrGeodesicsPerFile, this file is full)
- unsigned short [m_CurrentFullFiles](#) {0}
The current counter of completely full files (this is kept track of so that it knows what the next file to write output to is). (We had better not have more than 60k files!)
- std::vector< std::vector< std::string > > [m_AllCachedData](#) {}
Cached data that has not been written to a file yet (once this hits a size of > m_nrOutputsToCache, this must be written to file(s))

6.24.1 Detailed Description

// [GeodesicOutputHandler](#) handles all of the output to file. It gets passed all of the output strings for every [Geodesic](#), it then stores this data until it eventually writes all data to the appropriate files

6.24.2 Constructor & Destructor Documentation

6.24.2.1 GeodesicOutputHandler() [1/2]

```
GeodesicOutputHandler::GeodesicOutputHandler () [delete]
```

6.24.2.2 GeodesicOutputHandler() [2/2]

```
GeodesicOutputHandler::GeodesicOutputHandler (
    std::string FilePrefix,
    std::string TimeStamp,
    std::string FileExtension,
    std::vector< std::string > DiagNames,
    largecounter nrouterstocache = LARGE_COUNTER_MAX - 1,
    largecounter geodperfile = LARGE_COUNTER_MAX,
    std::string firstlineinfo = "")
```

Construct a new [Geodesic](#) Output Handler object and initialize all const member variables.

Parameters

<i>FilePrefix</i>	file prefix
<i>TimeStamp</i>	time stamp
<i>FileExtension</i>	file extensions
<i>DiagNames</i>	diagnostics names
<i>nrouterstocache</i>	number of geodesics to put to the cache
<i>geodperfile</i>	max number of geodesics per output file
<i>firstlineinfo</i>	the info to print on the first line

6.24.3 Member Function Documentation

6.24.3.1 GetFileName()

```
std::string GeodesicOutputHandler::GetFileName (
    int diagnr,
    unsigned short filenr) const [private]
```

Get the file name based on the various parts of the file name stored in the member variables.

Parameters

<i>diagnr</i>	
<i>filenr</i>	

Returns

std::string

6.24.3.2 getFullDescriptionStr()

```
std::string GeodesicOutputHandler::getFullDescriptionStr () const
```

full description string getter for the geodesic output handler

Returns

std::string

6.24.3.3 NewGeodesicOutput()

```
void GeodesicOutputHandler::NewGeodesicOutput (
    largecounter index,
    std::vector< std::string > theOutput)
```

Parameters

<i>index</i>	
<i>theOutput</i>	

6.24.3.4 OpenForFirstTime()

```
void GeodesicOutputHandler::OpenForFirstTime (
    const std::string & filename) [private]
```

We check here to see if the files are being put in a (sub)directory; if so, we create the directory/directories first to make sure creating/opening the file will succeed.

Parameters

<i>filename</i>	Filename to be found / created
-----------------	--------------------------------

6.24.3.5 OutputFinished()

```
void GeodesicOutputHandler::OutputFinished ()
```

There is no more output, so we write anything that is cached to file to clean up and finalize.

6.24.3.6 PrepareForOutput()

```
void GeodesicOutputHandler::PrepareForOutput (
    largecounter nrOutputToCome)
```

prepare the cache for output

Parameters

<i>nrOutputToCome</i>	number of outputs to come into the cache
-----------------------	--

6.24.3.7 WriteCachedOutputToFile()

```
void GeodesicOutputHandler::WriteCachedOutputToFile () [private]
```

Write cached output to a file.

6.24.4 Member Data Documentation**6.24.4.1 m_AllCachedData**

```
std::vector<std::vector<std::string> > GeodesicOutputHandler::m_AllCachedData {} [private]
```

Cached data that has not been written to a file yet (once this hits a size of $> m_nrOutputsToCache$, this must be written to file(s))

6.24.4.2 m_CurrentFullFiles

```
unsigned short GeodesicOutputHandler::m_CurrentFullFiles {0} [private]
```

The current counter of completely full files (this is kept track of so that it knows what the next file to write output to is). (We had better not have more than 60k files!)

6.24.4.3 m_CurrentGeodesicsInFile

```
largecounter GeodesicOutputHandler::m_CurrentGeodesicsInFile {0} [private]
```

The number of geodesics already written to the current file (once this hits $m_nrGeodesicsPerFile$, this file is full)

6.24.4.4 m_DiagNames

```
const std::vector<std::string> GeodesicOutputHandler::m_DiagNames [private]
```

Const string for the diagnostic names.

6.24.4.5 m_FileExtension

```
const std::string GeodesicOutputHandler::m_FileExtension [private]
```

Const string for file extension.

6.24.4.6 m_FilePrefix

```
const std::string GeodesicOutputHandler::m_FilePrefix [private]
```

Const string for file prefix.

6.24.4.7 m_FirstLineInfoString

```
const std::string GeodesicOutputHandler::m_FirstLineInfoString [private]
```

Const string for the first line info.

6.24.4.8 m_nrGeodesicsPerFile

```
const largecounter GeodesicOutputHandler::m_nrGeodesicsPerFile {} [private]
```

Const setting the max number of geodesics per file.

6.24.4.9 m_nrOutputsToCache

```
const largecounter GeodesicOutputHandler::m_nrOutputsToCache {} [private]
```

6.24.4.10 m_PrevCached

```
largecounter GeodesicOutputHandler::m_PrevCached {0} [private]
```

How many outputs are already cached in m_nrOutputsToCache before the current iteration of output.

6.24.4.11 m_PrintFirstLineInfo

```
const bool GeodesicOutputHandler::m_PrintFirstLineInfo [private]
```

Const variable that control whether we write a descriptive first line in every output file or not.

6.24.4.12 m_TimeStamp

```
const std::string GeodesicOutputHandler::m_TimeStamp [private]
```

Const string for timestamp.

6.24.4.13 m_WriteToConsole

```
bool GeodesicOutputHandler::m_WriteToConsole {false} [private]
```

If this is false, then we are writing to file(s). At any time, if a file I/O error occurs, the output handler switches to outputting everything to the console.

The documentation for this class was generated from the following files:

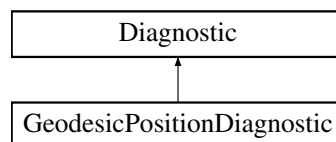
- FOORT/src/InputOutput.h
- FOORT/src/InputOutput.cpp

6.25 GeodesicPositionDiagnostic Class Reference

[Geodesic](#) position tracker.

```
#include <Diagnostics.h>
```

Inheritance diagram for GeodesicPositionDiagnostic:



Public Member Functions

- [GeodesicPositionDiagnostic](#) ([Geodesic](#) *const theGeodesic)
- void [Reset](#) () final
[GeodesicPositionDiagnostic](#) functions.
- void [UpdateData](#) () final
Update the saved points vector with the current position of the geodesic.
- std::string [getFullDataStr](#) () const final
Return full data string of all saved points.
- std::vector< [real](#) > [getFinalDataVal](#) () const final
Return the final (theta, phi) value of the geodesic.
- [real](#) [FinalDataValDistance](#) (const std::vector< [real](#) > &val1, const std::vector< [real](#) > &val2) const final
Return the angular distance between two geodesics.
- std::string [getNameStr](#) () const final
Simple name without spaces.
- std::string [getFullDescriptionStr](#) () const final
More descriptive string (with spaces)

Public Member Functions inherited from [Diagnostic](#)

- [Diagnostic](#) ()=delete
- [Diagnostic](#) ([Geodesic](#) *const theGeodesic)
- virtual [~Diagnostic](#) ()=default

Static Public Attributes

- static std::unique_ptr< [GeodesicPositionOptions](#) > [DiagOptions](#)

Private Attributes

- std::vector< [Point](#) > [m_AllSavedPoints](#) {}

Additional Inherited Members

Protected Member Functions inherited from [Diagnostic](#)

- bool [DecideUpdate](#) (const [UpdateFrequency](#) &myUpdateFrequency)
Returns true if the [Diagnostic](#) should update its initial status.

Protected Attributes inherited from [Diagnostic](#)

- [Geodesic](#) *const [m_OwnerGeodesic](#)
- [largecounter](#) [m_StepsSinceUpdated](#) {}

6.25.1 Detailed Description

[Geodesic](#) position tracker.

6.25.2 Constructor & Destructor Documentation

6.25.2.1 GeodesicPositionDiagnostic()

```
GeodesicPositionDiagnostic::GeodesicPositionDiagnostic (
    Geodesic *const theGeodesic) [inline]
```

6.25.3 Member Function Documentation

6.25.3.1 FinalDataValDistance()

```
real GeodesicPositionDiagnostic::FinalDataValDistance (
    const std::vector< real > & val1,
    const std::vector< real > & val2) const [final], [virtual]
```

Return the angular distance between two geodesics.

This is based on their final angles on the boundary sphere (theta, phi)

Parameters

<i>val1</i>	first geodesic final position
<i>val2</i>	second geodesic final position

Returns

real

Implements [Diagnostic](#).

6.25.3.2 getFinalDataVal()

```
std::vector< real > GeodesicPositionDiagnostic::getFinalDataVal () const [final], [virtual]
```

Return the final (theta, phi) value of the geodesic.

Returns

std::vector<real>

Implements [Diagnostic](#).

6.25.3.3 getFullDataStr()

```
std::string GeodesicPositionDiagnostic::getFullDataStr () const [final], [virtual]
```

Return full data string of all saved points.

The full output string looks like this: "(total nr steps) ;; (step 1) (step 2) (step 3) ..." where each step is a (space-separated) output of the geodesic coordinates at that step

Returns

std::string

Implements [Diagnostic](#).

6.25.3.4 getFullDescriptionStr()

```
std::string GeodesicPositionDiagnostic::getFullDescriptionStr () const [final], [virtual]
```

More descriptive string (with spaces)

Returns

std::string

Reimplemented from [Diagnostic](#).

6.25.3.5 getNameStr()

```
std::string GeodesicPositionDiagnostic::getNameStr () const [final], [virtual]
```

Simple name without spaces.

Returns

std::string

Implements [Diagnostic](#).

6.25.3.6 Reset()

```
void GeodesicPositionDiagnostic::Reset () [final], [virtual]
```

[GeodesicPositionDiagnostic](#) functions.

Empty out vector of points and call base Reset function

Reimplemented from [Diagnostic](#).

6.25.3.7 UpdateData()

```
void GeodesicPositionDiagnostic::UpdateData () [final], [virtual]
```

Update the saved points vector with the current position of the geodesic.

Implements [Diagnostic](#).

6.25.4 Member Data Documentation

6.25.4.1 DiagOptions

```
std::unique_ptr< GeodesicPositionOptions > GeodesicPositionDiagnostic::DiagOptions [static]
```

6.25.4.2 m_AllSavedPoints

```
std::vector<Point> GeodesicPositionDiagnostic::m_AllSavedPoints {} [private]
```

The documentation for this class was generated from the following files:

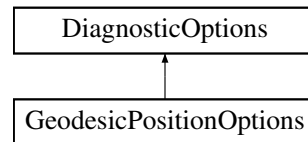
- FOORT/src/[Diagnostics.h](#)
- FOORT/src/[Config.cpp](#)
- FOORT/src/[Diagnostics.cpp](#)

6.26 GeodesicPositionOptions Struct Reference

Options for [GeodesicPositionDiagnostic](#).

```
#include <Diagnostics.h>
```

Inheritance diagram for GeodesicPositionOptions:



Public Member Functions

- [GeodesicPositionOptions](#) ([largecounter](#) outputsteps, [UpdateFrequency](#) thefrequency)

Public Member Functions inherited from [DiagnosticOptions](#)

- [DiagnosticOptions](#) ([UpdateFrequency](#) thefrequency)
- virtual [~DiagnosticOptions](#) ()=default

Public Attributes

- const [largecounter](#) OutputNrSteps

Public Attributes inherited from [DiagnosticOptions](#)

- const [UpdateFrequency](#) theUpdateFrequency

6.26.1 Detailed Description

Options for [GeodesicPositionDiagnostic](#).

Inherits from [DiagnosticOptions](#), and adds the number of steps to output.

6.26.2 Constructor & Destructor Documentation

6.26.2.1 GeodesicPositionOptions()

```
GeodesicPositionOptions::GeodesicPositionOptions (
    largecounter outputsteps,
    UpdateFrequency thefrequency) [inline]
```

6.26.3 Member Data Documentation

6.26.3.1 OutputNrSteps

```
const largecounter GeodesicPositionOptions::OutputNrSteps
```

The documentation for this struct was generated from the following file:

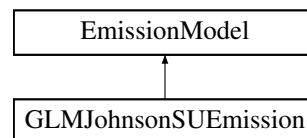
- FOORT/src/[Diagnostics.h](#)

6.27 GLMJohnsonSUEmission Struct Reference

The Johnson SU emission model used in GLM.

```
#include <DiagnosticsEmission.h>
```

Inheritance diagram for GLMJohnsonSUEmission:



Public Member Functions

- [GLMJohnsonSUEmission](#) ([real](#) mu, [real](#) gamma, [real](#) sigma)
- [real](#) [GetEmission](#) (const [Point](#) &p) const final
Get the emission at a point according to the Johnson SU model used in GLM.
- std::string [getFullDescriptionStr](#) () const final
Description string getter for the Johnson SU emission model.

Public Member Functions inherited from [EmissionModel](#)

- virtual [~EmissionModel](#) ()=default

Private Attributes

- const [real](#) [m_mu](#)
mu parameter
- const [real](#) [m_gamma](#)
gamma parameter
- const [real](#) [m_sigma](#)
sigma parameter

6.27.1 Detailed Description

The Johnson SU emission model used in GLM.

6.27.2 Constructor & Destructor Documentation

6.27.2.1 GLMJohnsonSUEmission()

```
GLMJohnsonSUEmission::GLMJohnsonSUEmission (
    real mu,
    real gamma,
    real sigma) [inline]
```

6.27.3 Member Function Documentation

6.27.3.1 GetEmission()

```
real GLMJohnsonSUEmission::GetEmission (
    const Point & p) const [final], [virtual]
```

Get the emission at a point according to the Johnson SU model used in GLM.

Parameters

<i>p</i>	Point at which to calculate the emission
----------	--

Returns

real Emission at the point

Implements [EmissionModel](#).

6.27.3.2 getFullDescriptionStr()

```
std::string GLMJohnsonSUEmission::getFullDescriptionStr () const [final], [virtual]
```

Description string getter for the Johnson SU emission model.

Returns

std::string

Reimplemented from [EmissionModel](#).

6.27.4 Member Data Documentation

6.27.4.1 m_gamma

```
const real GLMJohnsonSUEmission::m_gamma [private]
```

gamma parameter

6.27.4.2 m_mu

```
const real GLMJohnsonSUEmission::m_mu [private]
```

mu parameter

6.27.4.3 m_sigma

```
const real GLMJohnsonSUEmission::m_sigma [private]
```

sigma parameter

The documentation for this struct was generated from the following files:

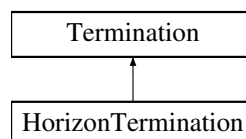
- FOORT/src/[DiagnosticsEmission.h](#)
- FOORT/src/[DiagnosticsEmission.cpp](#)

6.28 HorizonTermination Class Reference

[HorizonTermination](#): Terminate geodesics if they get too close to the horizon.

```
#include <Terminations.h>
```

Inheritance diagram for HorizonTermination:



Public Member Functions

- [HorizonTermination](#) ([Geodesic](#) *const theGeodesic)
- [Term CheckTermination](#) () final
- *[HorizonTermination](#) functions.*
- std::string [getFullDescriptionStr](#) () const final

Public Member Functions inherited from [Termination](#)

- [Termination](#) ()=delete
- [Termination](#) ([Geodesic](#) *const theGeodesic)
- virtual void [Reset](#) ()
[Termination](#) (abstract base class) functions.
- virtual $\sim$ [Termination](#) ()=default

Static Public Attributes

- static std::unique_ptr< [HorizonTermOptions](#) > [TermOptions](#)
Options (contains horizon radius, if we are using logarithmic r coordinate, and distance allowed from the horizon)

Additional Inherited Members

Protected Member Functions inherited from [Termination](#)

- bool [DecideUpdate](#) ([largecounter](#) UpdateNSteps)

Protected Attributes inherited from [Termination](#)

- [Geodesic](#) *const [m_OwnerGeodesic](#)
The geodesic that owns the [Termination](#) (a const pointer to the [Geodesic](#))
- [largecounter](#) [m_StepsSinceUpdated](#) {}
The termination is itself in charge of keeping track of how many steps it has been since it has been updated. The [Termination](#)'s [TerminationOptions](#) struct tells it how many steps it needs to wait between updates.

6.28.1 Detailed Description

[HorizonTermination](#): Terminate geodesics if they get too close to the horizon.

6.28.2 Constructor & Destructor Documentation

6.28.2.1 [HorizonTermination](#)()

```
HorizonTermination::HorizonTermination (
    Geodesic *const theGeodesic) [inline]
```

6.28.3 Member Function Documentation

6.28.3.1 [CheckTermination](#)()

```
Term HorizonTermination::CheckTermination () [final], [virtual]
```

[HorizonTermination](#) functions.

Implements [Termination](#).

6.28.3.2 getFullDescriptionStr()

```
std::string HorizonTermination::getFullDescriptionStr () const [final], [virtual]
```

Implements [Termination](#).

6.28.4 Member Data Documentation

6.28.4.1 TermOptions

```
std::unique_ptr< HorizonTermOptions > HorizonTermination::TermOptions [static]
```

Options (contains horizon radius, if we are using logarithmic r coordinate, and distance allowed from the horizon)

The documentation for this class was generated from the following files:

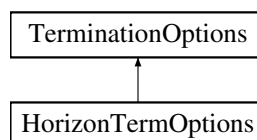
- FOORT/src/[Terminations.h](#)
- FOORT/src/[Config.cpp](#)
- FOORT/src/[Terminations.cpp](#)

6.29 HorizonTermOptions Struct Reference

Options for [HorizonTermination](#).

```
#include <Terminations.h>
```

Inheritance diagram for HorizonTermOptions:



Public Member Functions

- [HorizonTermOptions](#) ([real](#) theHorizonRadius, bool therLogScale, [real](#) theAtHorizonEps, [largecounter](#) Nsteps)

Public Member Functions inherited from [TerminationOptions](#)

- [TerminationOptions](#) ([largecounter](#) Nsteps)
- virtual [~TerminationOptions](#) ()=default

Public Attributes

- const [real](#) [HorizonRadius](#)
The radius of the horizon.
- const [real](#) [AtHorizonEps](#)
The distance allowed from the horizon.
- const [bool](#) [rLogScale](#)
Whether we are using a logarithmic r coordinate.

Public Attributes inherited from [TerminationOptions](#)

- const [largecounter](#) [UpdateEveryNSteps](#)
Number of steps between updates.

6.29.1 Detailed Description

Options for [HorizonTermination](#).

Keeps track of the horizon radius, whether we are using a logarithmic r coordinate, and the distance allowed from the horizon.

6.29.2 Constructor & Destructor Documentation

6.29.2.1 [HorizonTermOptions\(\)](#)

```
HorizonTermOptions::HorizonTermOptions (  
    real theHorizonRadius,  
    bool therLogScale,  
    real theAtHorizonEps,  
    largecounter Nsteps) [inline]
```

6.29.3 Member Data Documentation

6.29.3.1 [AtHorizonEps](#)

```
const real HorizonTermOptions::AtHorizonEps
```

The distance allowed from the horizon.

6.29.3.2 [HorizonRadius](#)

```
const real HorizonTermOptions::HorizonRadius
```

The radius of the horizon.

6.29.3.3 rLogScale

```
const bool HorizonTermOptions::rLogScale
```

Whether we are using a logarithmic r coordinate.

The documentation for this struct was generated from the following file:

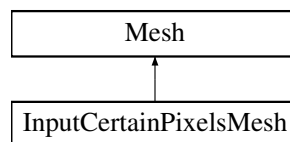
- FOORT/src/[Terminations.h](#)

6.30 InputCertainPixelsMesh Class Reference

[Mesh](#) which integrates only certain user-inputted pixels.

```
#include <Mesh.h>
```

Inheritance diagram for InputCertainPixelsMesh:



Public Member Functions

- [InputCertainPixelsMesh](#) ()=delete
- [InputCertainPixelsMesh](#) (const [InputCertainPixelsMesh](#) &)=delete
- [InputCertainPixelsMesh](#) ([largecounter](#) totalPixels, [DiagBitflag](#) valdiag)
Constructor for [InputCertainPixelsMesh](#).
- [largecounter](#) getCurNrGeodesics () const final
Get the current number of geodesics to be integrated.
- void [getNewInitConds](#) ([largecounter](#) index, [ScreenPoint](#) &newunitpoint, [ScreenIndex](#) &newscreenindex) const final
Get the initial conditions for the next pixel to be integrated.
- void [GeodesicFinished](#) ([largecounter](#) index, std::vector< [real](#) > finalValues) final
Function that is called when a geodesic is finished integrating.
- void [EndCurrentLoop](#) () final
End the current loop of integrating pixels.
- bool [IsFinished](#) () const final
Check whether the [Mesh](#) is finished integrating.
- std::string [getFullDescriptionStr](#) () const final
Description string getter.

Public Member Functions inherited from [Mesh](#)

- [Mesh](#) ([DiagBitflag](#) valdiag)
- virtual [~Mesh](#) ()=default

Private Attributes

- const pixelcoord `m_RowColumnSize`
Total pixel size of the screen (square grid)
- largecounter `m_TotalPixels` {0}
How many pixels have been inputted in total, i.e. need integrating.
- std::vector< ScreenIndex > `m_PixelsToIntegrate` {}
All pixels' location.
- bool `m_Finished` {false}
Are we finished integrating?

Additional Inherited Members

Protected Attributes inherited from `Mesh`

- const std::unique_ptr< const Diagnostic > `m_DistanceDiagnostic`
The `Diagnostic` (a const pointer to a const `Diagnostic` object) that is used to calculate distances (using `FinalData`↔`ValDistance()`) between the "values" that are assigned to Geodesics.

6.30.1 Detailed Description

`Mesh` which integrates only certain user-inputted pixels.

6.30.2 Constructor & Destructor Documentation

6.30.2.1 `InputCertainPixelsMesh()` [1/3]

```
InputCertainPixelsMesh::InputCertainPixelsMesh () [delete]
```

6.30.2.2 `InputCertainPixelsMesh()` [2/3]

```
InputCertainPixelsMesh::InputCertainPixelsMesh (
    const InputCertainPixelsMesh & ) [delete]
```

6.30.2.3 `InputCertainPixelsMesh()` [3/3]

```
InputCertainPixelsMesh::InputCertainPixelsMesh (
    largecounter totalPixels,
    DiagBitflag valdiag)
```

Constructor for `InputCertainPixelsMesh`.

Constructor will ask for pixels to be inputted by the user through the console

Parameters

<i>totalPixels</i>	total number of pixels in the grid
<i>valdiag</i>	value diagnostic bitflag

6.30.3 Member Function Documentation

6.30.3.1 EndCurrentLoop()

```
void InputCertainPixelsMesh::EndCurrentLoop () [final], [virtual]
```

End the current loop of integrating pixels.

This [Mesh](#) only has one loop; we are now finished integrating

Implements [Mesh](#).

6.30.3.2 GeodesicFinished()

```
void InputCertainPixelsMesh::GeodesicFinished (  
    largecounter index,  
    std::vector< real > finalValues) [final], [virtual]
```

Function that is called when a geodesic is finished integrating.

This [Mesh](#) doesn't actually have to do anything with the geodesic (values) when it is done!

Parameters

<i>index</i>	index of the geodesic that is finished
<i>finalValues</i>	final values of the geodesic

Implements [Mesh](#).

6.30.3.3 getCurNrGeodesics()

```
largecounter InputCertainPixelsMesh::getCurNrGeodesics () const [final], [virtual]
```

Get the current number of geodesics to be integrated.

This is the total number of pixels in the grid

Returns

largecounter

Implements [Mesh](#).

6.30.3.4 getFullDescriptionStr()

```
std::string InputCertainPixelsMesh::getFullDescriptionStr () const [final], [virtual]
```

Description string getter.

Returns

std::string

Reimplemented from [Mesh](#).

6.30.3.5 getNewInitConds()

```
void InputCertainPixelsMesh::getNewInitConds (
    largecounter index,
    ScreenPoint & newunitpoint,
    ScreenIndex & newscreenindex) const [final], [virtual]
```

Get the initial conditions for the next pixel to be integrated.

The next pixel is the one with index m_CurrentPixel

Parameters

<i>index</i>	index of the pixel to be initialized
<i>newunitpoint</i>	new unit point to be initialized
<i>newscreenindex</i>	new screen index to be initialized

Implements [Mesh](#).

6.30.3.6 IsFinished()

```
bool InputCertainPixelsMesh::IsFinished () const [final], [virtual]
```

Check whether the [Mesh](#) is finished integrating.

Returns

true

false

Implements [Mesh](#).

6.30.4 Member Data Documentation

6.30.4.1 m_Finished

```
bool InputCertainPixelsMesh::m_Finished {false} [private]
```

Are we finished integrating?

6.30.4.2 m_PixelsToIntegrate

```
std::vector<ScreenIndex> InputCertainPixelsMesh::m_PixelsToIntegrate {} [private]
```

All pixels' location.

6.30.4.3 m_RowColumnSize

```
const pixelcoord InputCertainPixelsMesh::m_RowColumnSize [private]
```

Total pixel size of the screen (square grid)

6.30.4.4 m_TotalPixels

```
largecounter InputCertainPixelsMesh::m_TotalPixels {0} [private]
```

How many pixels have been inputted in total, i.e. need integrating.

The documentation for this class was generated from the following files:

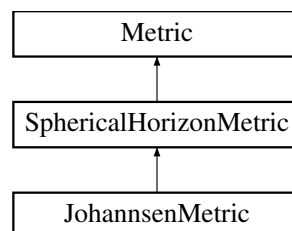
- FOORT/src/[Mesh.h](#)
- FOORT/src/[Mesh.cpp](#)

6.31 JohannsenMetric Class Reference

Johannsen black hole metric.

```
#include <Metric.h>
```

Inheritance diagram for JohannsenMetric:



Public Member Functions

- [JohannsenMetric](#) ()=delete
- [JohannsenMetric](#) (real aParam, real alpha13Param, real alpha22Param, real alpha52Param, real eps3Param, bool rLogScale=false)
Construct a new Johannsen [Metric](#) object. We always have a $M = 1$ BH.
- [TwoIndex getMetric_dd](#) (const [Point](#) &p) const final
Johannsen metric getter, indices down.
- [TwoIndex getMetric_uu](#) (const [Point](#) &p) const final
Johannsen metric getter, indices up.
- std::string [getFullDescriptionStr](#) () const final
Johannsen metric description string getter.

Public Member Functions inherited from [SphericalHorizonMetric](#)

- [SphericalHorizonMetric](#) ()=delete
- [SphericalHorizonMetric](#) (real HorizonRadius, bool rLogScale)
Construct a new Spherical Horizon [Metric](#) object.
- [real getHorizonRadius](#) () const
Get the radius of the horizon.

Public Member Functions inherited from [Metric](#)

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new [Metric](#) object.
- virtual [ThreeIndex getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual [real getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- bool [getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Private Attributes

- const [real m_aParam](#)
a parameter for the Johannsen metric
- const [real m_alpha13Param](#)
alpha13 parameter for the Johannsen metric
- const [real m_alpha22Param](#)
alpha22 parameter for the Johannsen metric
- const [real m_alpha52Param](#)
alpha52 parameter for the Johannsen metric
- const [real m_eps3Param](#)
eps3 parameter for the Johannsen metric

Additional Inherited Members

Protected Attributes inherited from [SphericalHorizonMetric](#)

- const [real m_HorizonRadius](#)
Radius of the horizon.

Protected Attributes inherited from [Metric](#)

- std::vector< int > [m_Symmetries](#) {}
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- const bool [m_rLogScale](#)
Are we using a logarithmic r coordinate?

6.31.1 Detailed Description

Johanssen black hole metric.

Note

Implementation by Seppe Staelens

6.31.2 Constructor & Destructor Documentation

6.31.2.1 JohanssenMetric() [1/2]

```
JohanssenMetric::JohanssenMetric () [delete]
```

6.31.2.2 JohanssenMetric() [2/2]

```
JohanssenMetric::JohanssenMetric (
    real aParam,
    real alpha13Param,
    real alpha22Param,
    real alpha52Param,
    real eps3Param,
    bool rLogScale = false)
```

Construct a new Johanssen [Metric](#) object. We always have a $M = 1$ BH.

Parameters

<i>aParam</i>	a parameter for the Johanssen metric
<i>alpha13Param</i>	alpha13 parameter for the Johanssen metric
<i>alpha22Param</i>	alpha22 parameter for the Johanssen metric
<i>alpha52Param</i>	alpha52 parameter for the Johanssen metric
<i>eps3Param</i>	eps3 parameter for the Johanssen metric
<i>rLogScale</i>	whether we are using a logarithmic radial scale

Note

Implementation by Seppe Staelens

6.31.3 Member Function Documentation

6.31.3.1 getFullDescriptionStr()

```
std::string JohanssenMetric::getFullDescriptionStr () const [final], [virtual]
```

Johanssen metric description string getter.

Returns

std::string

Reimplemented from [Metric](#).

6.31.3.2 getMetric_dd()

```
TwoIndex JohanssenMetric::getMetric_dd (
    const Point & p) const [final], [virtual]
```

Johanssen metric getter, indices down.

Parameters

p	Point at which to evaluate the metric
-----	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).

6.31.3.3 getMetric_uu()

```
TwoIndex JohannsenMetric::getMetric_uu (  
    const Point & p) const [final], [virtual]
```

Johannsen metric getter, indices up.

Parameters

p	Point at which to evaluate the metric
-----	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).

6.31.4 Member Data Documentation**6.31.4.1 m_alpha13Param**

```
const real JohannsenMetric::m_alpha13Param [private]
```

alpha13 parameter for the Johannsen metric

6.31.4.2 m_alpha22Param

```
const real JohannsenMetric::m_alpha22Param [private]
```

alpha22 parameter for the Johannsen metric

6.31.4.3 m_alpha52Param

```
const real JohannsenMetric::m_alpha52Param [private]
```

alpha52 parameter for the Johannsen metric

6.31.4.4 m_aParam

```
const real JohannsenMetric::m_aParam [private]
```

a parameter for the Johannsen metric

6.31.4.5 m_eps3Param

```
const real JohannsenMetric::m_eps3Param [private]
```

eps3 parameter for the Johannsen metric

The documentation for this class was generated from the following files:

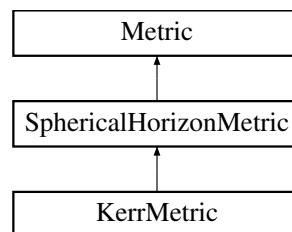
- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.32 KerrMetric Class Reference

The Kerr metric.

```
#include <Metric.h>
```

Inheritance diagram for KerrMetric:



Public Member Functions

- [KerrMetric](#) ()=delete
- [KerrMetric](#) (real aParam, bool rLogScale=false, real mParam=1.)
Construct a new Kerr [Metric](#) object.
- [TwoIndex getMetric_dd](#) (const [Point](#) &p) const final
Kerr metric getter, indices down.
- [TwoIndex getMetric_uu](#) (const [Point](#) &p) const final
Kerr metric getter, indices up.
- std::string [getFullDescriptionStr](#) () const final
Kerr metric description string getter.

Public Member Functions inherited from [SphericalHorizonMetric](#)

- [SphericalHorizonMetric](#) ()=delete
- [SphericalHorizonMetric](#) (real HorizonRadius, bool rLogScale)
Construct a new Spherical Horizon [Metric](#) object.
- [real getHorizonRadius](#) () const
Get the radius of the horizon.

Public Member Functions inherited from [Metric](#)

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new [Metric](#) object.
- virtual [ThreeIndex getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual [real getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- bool [getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Private Attributes

- const [real m_aParam](#)
Mass-rescaled rotation parameter for Kerr. Note that this should be between -1 and 1.
- const [real m_mParam](#)
Mass parameter for Kerr. Default is 1.

Additional Inherited Members

Protected Attributes inherited from [SphericalHorizonMetric](#)

- const [real m_HorizonRadius](#)
Radius of the horizon.

Protected Attributes inherited from [Metric](#)

- [std::vector< int > m_Symmetries](#) {}
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- const bool [m_rLogScale](#)
Are we using a logarithmic r coordinate?

6.32.1 Detailed Description

The Kerr metric.

6.32.2 Constructor & Destructor Documentation

6.32.2.1 KerrMetric() [1/2]

```
KerrMetric::KerrMetric () [delete]
```

6.32.2.2 KerrMetric() [2/2]

```
KerrMetric::KerrMetric (
    real aParam,
    bool rLogScale = false,
    real mParam = 1.)
```

Construct a new Kerr [Metric](#) object.

Parameters

<i>aParam</i>	a parameter for the Kerr metric
<i>rLogScale</i>	Whether we are using a logarithmic radial scale
<i>mParam</i>	mass parameter for the Kerr metric

6.32.3 Member Function Documentation

6.32.3.1 getFullDescriptionStr()

```
std::string KerrMetric::getFullDescriptionStr () const [final], [virtual]
```

Kerr metric description string getter.

Returns

std::string

Reimplemented from [Metric](#).

6.32.3.2 getMetric_dd()

```
TwoIndex KerrMetric::getMetric_dd (
    const Point & p) const [final], [virtual]
```

Kerr metric getter, indices down.

Parameters

<i>p</i>	Point at which to evaluate the metric
----------	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).

6.32.3.3 getMetric_uu()

```
TwoIndex KerrMetric::getMetric_uu (
    const Point & p) const [final], [virtual]
```

Kerr metric getter, indices up.

Parameters

p	Point at which to evaluate the metric
-----	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).

6.32.4 Member Data Documentation**6.32.4.1 m_aParam**

```
const real KerrMetric::m_aParam [private]
```

Mass-rescaled rotation parameter for Kerr. Note that this should be between -1 and 1.

6.32.4.2 m_mParam

```
const real KerrMetric::m_mParam [private]
```

Mass parameter for Kerr. Default is 1.

The documentation for this class was generated from the following files:

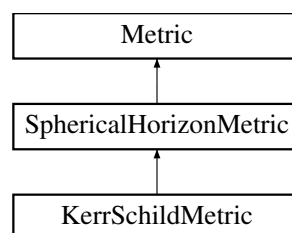
- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.33 KerrSchildMetric Class Reference

Kerr metric in Kerr-Schild coordinates.

```
#include <Metric.h>
```

Inheritance diagram for KerrSchildMetric:



Public Member Functions

- [KerrSchildMetric](#) ()=delete
- [KerrSchildMetric](#) (real aParam, bool rLogScale=false)
Construct a new Kerr Schild [Metric](#) object. We always have a $M = 1$ BH.
- [TwoIndex getMetric_dd](#) (const [Point](#) &p) const final
Kerr-Schild metric getter, indices down.
- [TwoIndex getMetric_uu](#) (const [Point](#) &p) const final
Kerr-Schild metric getter, indices up.
- std::string [getFullDescriptionStr](#) () const final
Kerr-Schild metric description string getter.

Public Member Functions inherited from [SphericalHorizonMetric](#)

- [SphericalHorizonMetric](#) ()=delete
- [SphericalHorizonMetric](#) (real HorizonRadius, bool rLogScale)
Construct a new Spherical Horizon [Metric](#) object.
- real [getHorizonRadius](#) () const
Get the radius of the horizon.

Public Member Functions inherited from [Metric](#)

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new [Metric](#) object.
- virtual [ThreeIndex getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual real [getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- bool [getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Private Attributes

- const real m_aParam
Rotation parameter for Kerr. Note that this should be between -1 and 1 since $M=1$.

Additional Inherited Members

Protected Attributes inherited from [SphericalHorizonMetric](#)

- const real m_HorizonRadius
Radius of the horizon.

Protected Attributes inherited from [Metric](#)

- `std::vector< int > m\_Symmetries {}`
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- `const bool m\_rLogScale`
Are we using a logarithmic r coordinate?

6.33.1 Detailed Description

Kerr metric in Kerr-Schild coordinates.

Note

Normalized so that $M = 1$

6.33.2 Constructor & Destructor Documentation

6.33.2.1 `KerrSchildMetric()` [1/2]

```
KerrSchildMetric::KerrSchildMetric () [delete]
```

6.33.2.2 `KerrSchildMetric()` [2/2]

```
KerrSchildMetric::KerrSchildMetric (
    real aParam,
    bool rLogScale = false)
```

Construct a new Kerr Schild [Metric](#) object. We always have a $M = 1$ BH.

Parameters

<i>aParam</i>	a parameter for the Kerr-Schild metric
<i>rLogScale</i>	whether we are using a logarithmic radial scale

6.33.3 Member Function Documentation

6.33.3.1 `getFullDescriptionStr()`

```
std::string KerrSchildMetric::getFullDescriptionStr () const [final], [virtual]
```

Kerr-Schild metric description string getter.

Returns

`std::string`

Reimplemented from [Metric](#).

6.33.3.2 `getMetric_dd()`

```
TwoIndex KerrSchildMetric::getMetric_dd (
    const Point & p) const [final], [virtual]
```

Kerr-Schild metric getter, indices down.

Parameters

p	Point at which to evaluate the metric
-----	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).

6.33.3.3 getMetric_uu()

```
TwoIndex KerrSchildMetric::getMetric_uu (
    const Point & p) const [final], [virtual]
```

Kerr-Schild metric getter, indices up.

Parameters

p	Point at which to evaluate the metric
-----	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).

6.33.4 Member Data Documentation

6.33.4.1 m_aParam

```
const real KerrSchildMetric::m_aParam [private]
```

Rotation parameter for Kerr. Note that this should be between -1 and 1 since $M=1$.

The documentation for this class was generated from the following files:

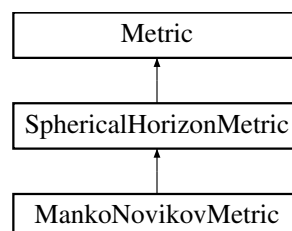
- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.34 MankoNovikovMetric Class Reference

Mano-Novikov metric (with angular momentum and M3 parameter turned on)

#include <Metric.h>

Inheritance diagram for MankoNovikovMetric:



Public Member Functions

- [MankoNovikovMetric](#) ()=delete
- [MankoNovikovMetric](#) (real aParam, real alpha3Param, bool rLogScale=false)
Construct a new Manko Novikov [Metric](#) object. We always have a $M = 1$ BH.
- [TwoIndex getMetric_dd](#) (const [Point](#) &p) const final
Manko-Novikov metric getter, indices down.
- [TwoIndex getMetric_uu](#) (const [Point](#) &p) const final
Manko-Novikov metric getter, indices up.
- std::string [getFullDescriptionStr](#) () const final
Manko-Novikov metric description string getter.

Public Member Functions inherited from [SphericalHorizonMetric](#)

- [SphericalHorizonMetric](#) ()=delete
- [SphericalHorizonMetric](#) (real HorizonRadius, bool rLogScale)
Construct a new Spherical Horizon [Metric](#) object.
- real [getHorizonRadius](#) () const
Get the radius of the horizon.

Public Member Functions inherited from [Metric](#)

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new [Metric](#) object.
- virtual [ThreeIndex getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual real [getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- bool [getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Private Attributes

- const real [m_aParam](#)
a parameter for the Manko-Novikov metric
- const real [m_alpha3Param](#)
alpha3 parameter for the Manko-Novikov metric
- const real [m_alphaParam](#)
Derived alpha parameter.
- const real [m_kParam](#)
Derived k parameter.

Additional Inherited Members

Protected Attributes inherited from [SphericalHorizonMetric](#)

- const real [m_HorizonRadius](#)
Radius of the horizon.

Protected Attributes inherited from [Metric](#)

- `std::vector< int > m_Symmetries {}`
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- `const bool m_rLogScale`
Are we using a logarithmic r coordinate?

6.34.1 Detailed Description

Mano-Novikov metric (with angular momentum and M3 parameter turned on)

Note

Implementation by Seppe Staelens

6.34.2 Constructor & Destructor Documentation

6.34.2.1 MankoNovikovMetric() [1/2]

```
MankoNovikovMetric::MankoNovikovMetric () [delete]
```

6.34.2.2 MankoNovikovMetric() [2/2]

```
MankoNovikovMetric::MankoNovikovMetric (
    real aParam,
    real alpha3Param,
    bool rLogScale = false)
```

Construct a new Manko Novikov [Metric](#) object. We always have a $M = 1$ BH.

The horizon is at $r = M + k$

Parameters

<i>aParam</i>	a parameter for the Manko-Novikov metric. The angular momentum $a = J/M^2$
<i>alpha3Param</i>	alpha3 parameter for the Manko-Novikov metric
<i>rLogScale</i>	whether we are using a logarithmic radial scale

6.34.3 Member Function Documentation

6.34.3.1 getFullDescriptionStr()

```
std::string MankoNovikovMetric::getFullDescriptionStr () const [final], [virtual]
```

Manko-Novikov metric description string getter.

Returns

`std::string`

Reimplemented from [Metric](#).

6.34.3.2 getMetric_dd()

```
TwoIndex MankoNovikovMetric::getMetric_dd (
    const Point & p) const [final], [virtual]
```

Manko-Novikov metric getter, indices down.

Parameters

p	Point at which to evaluate the metric
-----	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).

6.34.3.3 getMetric_uu()

```
TwoIndex MankoNovikovMetric::getMetric_uu (  
    const Point & p) const [final], [virtual]
```

Manko-Novikov metric getter, indices up.

Parameters

p	Point at which to evaluate the metric
-----	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).

6.34.4 Member Data Documentation

6.34.4.1 m_alpha3Param

```
const real MankoNovikovMetric::m_alpha3Param [private]
```

alpha3 parameter for the Manko-Novikov metric

6.34.4.2 m_alphaParam

```
const real MankoNovikovMetric::m_alphaParam [private]
```

Derived alpha parameter.

6.34.4.3 m_aParam

```
const real MankoNovikovMetric::m_aParam [private]
```

a parameter for the Manko-Novikov metric

6.34.4.4 m_kParam

```
const real MankoNovikovMetric::m_kParam [private]
```

Derived k parameter.

The documentation for this class was generated from the following files:

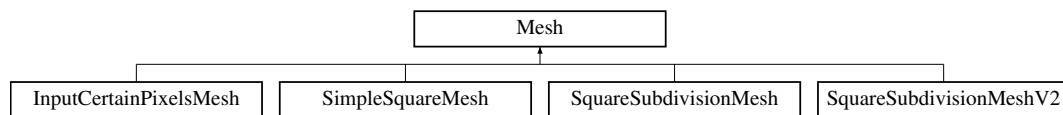
- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.35 Mesh Class Reference

Abstract base [Mesh](#) class.

```
#include <Mesh.h>
```

Inheritance diagram for Mesh:



Public Member Functions

- [Mesh](#) ([DiagBitflag](#) valdiag)
- virtual [~Mesh](#) ()=default
- virtual [largecounter](#) [getCurNrGeodesics](#) () const =0
- virtual void [getNewInitConds](#) ([largecounter](#) index, [ScreenPoint](#) &newunitpoint, [ScreenIndex](#) &newscreenindex) const =0
- virtual void [GeodesicFinished](#) ([largecounter](#) index, std::vector< [real](#) > finalValues)=0
- virtual void [EndCurrentLoop](#) ()=0
- virtual bool [IsFinished](#) () const =0
- virtual std::string [getFullDescriptionStr](#) () const

Basic description string getter.

Protected Attributes

- const std::unique_ptr< const [Diagnostic](#) > [m_DistanceDiagnostic](#)

The [Diagnostic](#) (a const pointer to a const [Diagnostic](#) object) that is used to calculate distances (using [FinalData](#)↔[ValDistance\(\)](#)) between the "values" that are assigned to Geodesics.

6.35.1 Detailed Description

Abstract base [Mesh](#) class.

6.35.2 Constructor & Destructor Documentation

6.35.2.1 Mesh()

```
Mesh::Mesh (
    DiagBitflag valdiag) [inline]
```

6.35.2.2 ~Mesh()

```
virtual Mesh::~Mesh () [virtual], [default]
```

6.35.3 Member Function Documentation

6.35.3.1 EndCurrentLoop()

```
virtual void Mesh::EndCurrentLoop () [pure virtual]
```

Implemented in [InputCertainPixelsMesh](#), [SimpleSquareMesh](#), [SquareSubdivisionMesh](#), and [SquareSubdivisionMeshV2](#).

6.35.3.2 GeodesicFinished()

```
virtual void Mesh::GeodesicFinished (
    largecounter index,
    std::vector< real > finalValues) [pure virtual]
```

Implemented in [InputCertainPixelsMesh](#), [SimpleSquareMesh](#), [SquareSubdivisionMesh](#), and [SquareSubdivisionMeshV2](#).

6.35.3.3 getCurNrGeodesics()

```
virtual largecounter Mesh::getCurNrGeodesics () const [pure virtual]
```

Implemented in [InputCertainPixelsMesh](#), [SimpleSquareMesh](#), [SquareSubdivisionMesh](#), and [SquareSubdivisionMeshV2](#).

6.35.3.4 getFullDescriptionStr()

```
std::string Mesh::getFullDescriptionStr () const [virtual]
```

Basic description string getter.

Returns

std::string

Reimplemented in [InputCertainPixelsMesh](#), [SimpleSquareMesh](#), [SquareSubdivisionMesh](#), and [SquareSubdivisionMeshV2](#).

6.35.3.5 getNewInitConds()

```
virtual void Mesh::getNewInitConds (
    largecounter index,
    ScreenPoint & newunitpoint,
    ScreenIndex & newscreenindex) const [pure virtual]
```

Implemented in [InputCertainPixelsMesh](#), [SimpleSquareMesh](#), [SquareSubdivisionMesh](#), and [SquareSubdivisionMeshV2](#).

6.35.3.6 IsFinished()

```
virtual bool Mesh::IsFinished () const [pure virtual]
```

Implemented in [InputCertainPixelsMesh](#), [SimpleSquareMesh](#), [SquareSubdivisionMesh](#), and [SquareSubdivisionMeshV2](#).

6.35.4 Member Data Documentation

6.35.4.1 m_DistanceDiagnostic

```
const std::unique_ptr<const Diagnostic> Mesh::m_DistanceDiagnostic [protected]
```

The [Diagnostic](#) (a const pointer to a const [Diagnostic](#) object) that is used to calculate distances (using `FinalData↔ValDistance()`) between the "values" that are assigned to Geodesics.

The documentation for this class was generated from the following files:

- FOORT/src/[Mesh.h](#)
- FOORT/src/[Mesh.cpp](#)

6.36 Metric Class Reference

The abstract base class for all Metrics.

```
#include <Metric.h>
```

Inheritance diagram for Metric:



Public Member Functions

- virtual `~Metric()`=default
- `Metric` (bool rlogscale=false)
Construct a new `Metric` object.
- virtual `TwoIndex getMetric_dd` (const `Point` &p) const =0
- virtual `TwoIndex getMetric_uu` (const `Point` &p) const =0
- virtual `ThreeIndex getChristoffel_udd` (const `Point` &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual `FourIndex getRiemann_uddd` (const `Point` &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual `real getKretschmann` (const `Point` &p) const
Get the Kretschmann scalar at a point.
- virtual std::string `getFullDescriptionStr` () const
Base class description string getter.
- bool `getrLogScale` () const
Get whether we are using a logarithmic radial scale.

Protected Attributes

- std::vector< int > `m_Symmetries` {}
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- const bool `m_rLogScale`
Are we using a logarithmic r coordinate?

6.36.1 Detailed Description

The abstract base class for all Metrics.

6.36.2 Constructor & Destructor Documentation

6.36.2.1 ~Metric()

```
virtual Metric::~Metric () [virtual], [default]
```

6.36.2.2 Metric()

```
Metric::Metric (
    bool rlogscale = false)
```

Construct a new `Metric` object.

Parameters

<code>rlogscale</code>	Whether we are using a logarithmic radial scale
------------------------	---

6.36.3 Member Function Documentation

6.36.3.1 getChristoffel_udd()

```
ThreeIndex Metric::getChristoffel_udd (
    const Point & p) const [virtual]
```

Return the Christoffel symbols of the metric (indices up, down, down)

Parameters

p	Point at which to evaluate the Christoffel symbols
-----	--

Returns

ThreeIndex

6.36.3.2 getFullDescriptionStr()

```
std::string Metric::getFullDescriptionStr () const [virtual]
```

Base class description string getter.

Returns

std::string

Reimplemented in [BosonStarMetric](#), [FlatSpaceMetric](#), [JohannsenMetric](#), [KerrMetric](#), [KerrSchildMetric](#), [MankoNovikovMetric](#), [RasheedLarsenMetric](#), and [ST3CrMetric](#).

6.36.3.3 getKretschmann()

```
real Metric::getKretschmann (
    const Point & p) const [virtual]
```

Get the Kretschmann scalar at a point.

The Kretschmann scalar is defined as $R_{\{abcd\}} R^{\{abcd\}}$

Note

This needs to be implemented still.

Parameters

p	Point at which to evaluate the Kretschmann scalar
-----	---

Returns

real

6.36.3.4 getMetric_dd()

```
virtual TwoIndex Metric::getMetric_dd (
    const Point & p) const [pure virtual]
```

Implemented in [BosonStarMetric](#), [FlatSpaceMetric](#), [JohannsenMetric](#), [KerrMetric](#), [KerrSchildMetric](#), [MankoNovikovMetric](#), [RasheedLarsenMetric](#), and [ST3CrMetric](#).

6.36.3.5 getMetric_uu()

```
virtual TwoIndex Metric::getMetric_uu (
    const Point & p) const [pure virtual]
```

Implemented in [BosonStarMetric](#), [FlatSpaceMetric](#), [JohannsenMetric](#), [KerrMetric](#), [KerrSchildMetric](#), [MankoNovikovMetric](#), [RasheedLarsenMetric](#), and [ST3CrMetric](#).

6.36.3.6 getRiemann_uddd()

```
FourIndex Metric::getRiemann_uddd (
    const Point & p) const [virtual]
```

Riemann tensor of the metric (indices up, down, down, down)

Parameters

p	Point at which to evaluate the Riemann tensor
-----	---

Note

This needs to be implemented still.

Returns

FourIndex

6.36.3.7 getrLogScale()

```
bool Metric::getrLogScale () const
```

Get whether we are using a logarithmic radial scale.

Returns

true

false

6.36.4 Member Data Documentation

6.36.4.1 m_rLogScale

```
const bool Metric::m_rLogScale [protected]
```

Are we using a logarithmic r coordinate?

6.36.4.2 m_Symmetries

```
std::vector<int> Metric::m_Symmetries {} [protected]
```

The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.

The documentation for this class was generated from the following files:

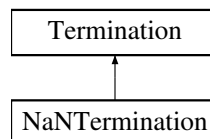
- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.37 NaNTermination Class Reference

[NaNTermination](#): Terminate geodesics if they contain a NaN in position or velocity.

```
#include <Terminations.h>
```

Inheritance diagram for NaNTermination:



Public Member Functions

- [NaNTermination](#) ([Geodesic](#) *const theGeodesic)
- [Term CheckTermination](#) () final
- *[NaNTermination](#) functions.*
- std::string [getFullDescriptionStr](#) () const final

Public Member Functions inherited from [Termination](#)

- [Termination](#) ()=delete
- [Termination](#) ([Geodesic](#) *const theGeodesic)
- virtual void [Reset](#) ()
- *[Termination](#) (abstract base class) functions.*
- virtual [~Termination](#) ()=default

Static Public Attributes

- static std::unique_ptr< [NaNTermOptions](#) > [TermOptions](#)
Options (contains horizon radius, if we are using logarithmic r coordinate, and distance allowed from the horizon)

Additional Inherited Members

Protected Member Functions inherited from [Termination](#)

- bool [DecideUpdate](#) ([largecounter](#) UpdateNSteps)

Protected Attributes inherited from [Termination](#)

- [Geodesic](#) *const [m_OwnerGeodesic](#)
The geodesic that owns the [Termination](#) (a const pointer to the [Geodesic](#))
- [largecounter](#) [m_StepsSinceUpdated](#) {}
The termination is itself in charge of keeping track of how many steps it has been since it has been updated. The [Termination](#)'s [TerminationOptions](#) struct tells it how many steps it needs to wait between updates.

6.37.1 Detailed Description

[NaNTermination](#): Terminate geodesics if they contain a NaN in position or velocity.

6.37.2 Constructor & Destructor Documentation

6.37.2.1 NaNTermination()

```
NaNTermination::NaNTermination (
    Geodesic *const theGeodesic) [inline]
```

6.37.3 Member Function Documentation

6.37.3.1 CheckTermination()

```
Term NaNTermination::CheckTermination () [final], [virtual]
```

[NaNTermination](#) functions.

Implements [Termination](#).

6.37.3.2 getFullDescriptionStr()

```
std::string NaNTermination::getFullDescriptionStr () const [final], [virtual]
```

Implements [Termination](#).

6.37.4 Member Data Documentation

6.37.4.1 TermOptions

```
std::unique_ptr< NaNTermOptions > NaNTermination::TermOptions [static]
```

Options (contains horizon radius, if we are using logarithmic r coordinate, and distance allowed from the horizon)

The documentation for this class was generated from the following files:

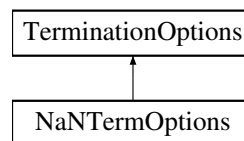
- FOORT/src/Terminations.h
- FOORT/src/Config.cpp
- FOORT/src/Terminations.cpp

6.38 NaNTermOptions Struct Reference

Options for [NaNTermination](#).

```
#include <Terminations.h>
```

Inheritance diagram for NaNTermOptions:



Public Member Functions

- [NaNTermOptions](#) (bool consoleoutputon, [largecounter](#) Nsteps)

Public Member Functions inherited from [TerminationOptions](#)

- [TerminationOptions](#) ([largecounter](#) Nsteps)
- virtual [~TerminationOptions](#) ()=default

Public Attributes

- const bool [OutputToConsole](#)
Whether to output to the console when a NaN is encountered.

Public Attributes inherited from [TerminationOptions](#)

- const [largecounter](#) [UpdateEveryNSteps](#)
Number of steps between updates.

6.38.1 Detailed Description

Options for [NaNTermination](#).

6.38.2 Constructor & Destructor Documentation

6.38.2.1 NaNTermOptions()

```
NaNTermOptions::NaNTermOptions (
    bool consoleoutputon,
    largecounter Nsteps) [inline]
```

6.38.3 Member Data Documentation

6.38.3.1 OutputToConsole

```
const bool NaNTermOptions::OutputToConsole
```

Whether to output to the console when a NaN is encountered.

The documentation for this struct was generated from the following file:

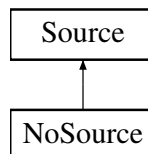
- FOORT/src/[Terminations.h](#)

6.39 NoSource Class Reference

[NoSource](#) class.

```
#include <Geodesic.h>
```

Inheritance diagram for NoSource:



Public Member Functions

- [NoSource](#) (const [Metric](#) *const theMetric)
- [OneIndex getSource](#) ([Point](#) pos, [OneIndex](#) vel) const final
Returns zero source for the geodesic equation: no force felt by geodesic.
- std::string [getFullDescriptionStr](#) () const final
Full description string getter for [NoSource](#) class.

Public Member Functions inherited from [Source](#)

- [Source](#) (const [Metric](#) *const theMetric)
- virtual [~Source](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [Source](#)

- const [Metric](#) *const m_theMetric
A const pointer to a const metric.

6.39.1 Detailed Description

[NoSource](#) class.

There is no source, i.e. the geodesic is indeed a geodesic (and feels no force)

6.39.2 Constructor & Destructor Documentation

6.39.2.1 NoSource()

```
NoSource::NoSource (
    const Metric *const theMetric) [inline]
```

6.39.3 Member Function Documentation

6.39.3.1 getFullDescriptionStr()

```
std::string NoSource::getFullDescriptionStr () const [final], [virtual]
```

Full description string getter for [NoSource](#) class.

Returns

std::string

Reimplemented from [Source](#).

6.39.3.2 getSource()

```
OneIndex NoSource::getSource (
    Point pos,
    OneIndex vel) const [final], [virtual]
```

Returns zero source for the geodesic equation: no force felt by geodesic.

Parameters

<i>pos</i>	Position of the geodesic
<i>vel</i>	Velocity of the geodesic

Returns

OneIndex [Source](#) for the geodesic

Implements [Source](#).

The documentation for this class was generated from the following files:

- FOORT/src/[Geodesic.h](#)
- FOORT/src/[Geodesic.cpp](#)

6.40 SquareSubdivisionMesh::PixelInfo Struct Reference

A struct the [Mesh](#) uses to keep all information about a given pixel.

Public Member Functions

- [PixelInfo](#) ([ScreenIndex](#) ind, int subdiv)

Public Attributes

- [ScreenIndex](#) Index {}
The pixel's screenindex.
- int [SubdivideLevel](#) {}
The level at which the pixel has been subdivided. Note: initial grid pixels are at 1; pixels at 0 are pixels that cannot be subdivided (for example, at the right or lower edges)
- [real](#) [Weight](#) {-1}
Weight of the pixel: if negative, this signifies that it needs to be updated/calculated! The weight is determined as the max of the distance (as calculated by the value [Diagnostic](#)) between its values and those of its right, lower, and right-lower neighbors.
- std::vector< [real](#) > [DiagValue](#) {}
The values associated to this pixel (as calculated by the value [Diagnostic](#))
- [largecounter](#) [LowerNbrIndex](#) {0}
Where its lower neighbor is located in m_AllPixels. Note: the pixel with index 0 is (0,0) and can never be the lower or right neighbor of any other pixel!
- [largecounter](#) [RightNbrIndex](#) {0}
Where its right neighbor is located in m_AllPixels. Note: the pixel with index 0 is (0,0) and can never be the lower or right neighbor of any other pixel!

6.40.1 Detailed Description

A struct the [Mesh](#) uses to keep all information about a given pixel.

6.40.2 Constructor & Destructor Documentation

6.40.2.1 PixelInfo()

```
SquareSubdivisionMesh::PixelInfo::PixelInfo (
    ScreenIndex ind,
    int subdiv) [inline]
```

6.40.3 Member Data Documentation

6.40.3.1 DiagValue

```
std::vector<real> SquareSubdivisionMesh::PixelInfo::DiagValue {}
```

The values associated to this pixel (as calculated by the value [Diagnostic](#))

6.40.3.2 Index

```
ScreenIndex SquareSubdivisionMesh::PixelInfo::Index {}
```

The pixel's screenindex.

6.40.3.3 LowerNbrIndex

```
largecounter SquareSubdivisionMesh::PixelInfo::LowerNbrIndex {0}
```

Where its lower neighbor is located in m_AllPixels. Note: the pixel with index 0 is (0,0) and can never be the lower or right neighbor of any other pixel!

6.40.3.4 RightNbrIndex

```
largecounter SquareSubdivisionMesh::PixelInfo::RightNbrIndex {0}
```

Where its right neighbor is located in m_AllPixels. Note: the pixel with index 0 is (0,0) and can never be the lower or right neighbor of any other pixel!

6.40.3.5 SubdivideLevel

```
int SquareSubdivisionMesh::PixelInfo::SubdivideLevel {}
```

The level at which the pixel has been subdivided. Note: initial grid pixels are at 1; pixels at 0 are pixels that cannot be subdivided (for example, at the right or lower edges)

6.40.3.6 Weight

```
real SquareSubdivisionMesh::PixelInfo::Weight {-1}
```

Weight of the pixel: if negative, this signifies that it needs to be updated/calculated! The weight is determined as the max of the distance (as calculated by the value [Diagnostic](#)) between its values and those of its right, lower, and right-lower neighbors.

The documentation for this struct was generated from the following file:

- FOORT/src/[Mesh.h](#)

6.41 SquareSubdivisionMeshV2::PixelInfo Struct Reference

A struct the [Mesh](#) uses to keep all information about a given pixel.

Public Member Functions

- [PixelInfo](#) ([ScreenIndex](#) ind, int subdiv)

Public Attributes

- const [ScreenIndex](#) Index {}
The pixel's screenindex: this gets set by the constructor and cannot change anymore.
- int [SubdivideLevel](#) {}
The level at which the pixel has been subdivided. Note: initial grid pixels are at 1; pixels at 0 are pixels that cannot be subdivided (for example, at the right or lower edges)
- [real](#) Weight {-1}
Weight of the pixel: if negative, this signifies that it needs to be updated/calculated! The weight is determined as the max of the distance (as calculated by the value [Diagnostic](#)) between its values and those of its right, lower, and right-lower neighbors.
- std::vector< [real](#) > [DiagValue](#) {}
The values associated to this pixel (as calculated by the value [Diagnostic](#))
- [PixelInfo](#) * [LeftNbr](#) {nullptr}
Pointer to its left neighbor.
- [PixelInfo](#) * [RightNbr](#) {nullptr}
Pointer to its right neighbor.
- [PixelInfo](#) * [UpNbr](#) {nullptr}
Pointer to its upper neighbor.
- [PixelInfo](#) * [DownNbr](#) {nullptr}
Pointer to its lower neighbor.
- [PixelInfo](#) * [SEdiagNbr](#) {nullptr}
Pointer to its lower-right neighbor.

6.41.1 Detailed Description

A struct the [Mesh](#) uses to keep all information about a given pixel.

6.41.2 Constructor & Destructor Documentation

6.41.2.1 PixelInfo()

```
SquareSubdivisionMeshV2::PixelInfo::PixelInfo (
    ScreenIndex ind,
    int subdiv) [inline]
```

6.41.3 Member Data Documentation

6.41.3.1 DiagValue

```
std::vector<real> SquareSubdivisionMeshV2::PixelInfo::DiagValue {}
```

The values associated to this pixel (as calculated by the value [Diagnostic](#))

6.41.3.2 DownNbr

```
PixelInfo* SquareSubdivisionMeshV2::PixelInfo::DownNbr {nullptr}
```

Pointer to its lower neighbor.

6.41.3.3 Index

```
const ScreenIndex SquareSubdivisionMeshV2::PixelInfo::Index {}
```

The pixel's screenindex: this gets set by the constructor and cannot change anymore.

6.41.3.4 LeftNbr

```
PixelInfo* SquareSubdivisionMeshV2::PixelInfo::LeftNbr {nullptr}
```

Pointer to its left neighbor.

6.41.3.5 RightNbr

```
PixelInfo* SquareSubdivisionMeshV2::PixelInfo::RightNbr {nullptr}
```

Pointer to its right neighbor.

6.41.3.6 SEdiagNbr

```
PixelInfo* SquareSubdivisionMeshV2::PixelInfo::SEdiagNbr {nullptr}
```

Pointer to its lower-righttt neighbor.

6.41.3.7 SubdivideLevel

```
int SquareSubdivisionMeshV2::PixelInfo::SubdivideLevel {}
```

The level at which the pixel has been subdivided. Note: initial grid pixels are at 1; pixels at 0 are pixels that cannot be subdivided (for example, at the right or lower edges)

6.41.3.8 UpNbr

```
PixelInfo* SquareSubdivisionMeshV2::PixelInfo::UpNbr {nullptr}
```

Pointer to its upper neighbor.

6.41.3.9 Weight

```
real SquareSubdivisionMeshV2::PixelInfo::Weight {-1}
```

Weight of the pixel: if negative, this signifies that it needs to be updated/calculated! The weight is determined as the max of the distance (as calculated by the value [Diagnostic](#)) between its values and those of its right, lower, and right-lower neighbors.

The documentation for this struct was generated from the following file:

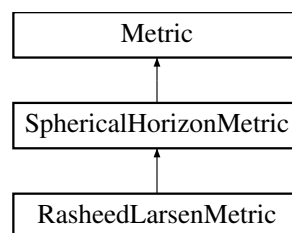
- FOORT/src/[Mesh.h](#)

6.42 RasheedLarsenMetric Class Reference

Rasheed-Larsen black hole.

```
#include <Metric.h>
```

Inheritance diagram for RasheedLarsenMetric:



Public Member Functions

- [RasheedLarsenMetric](#) ()=delete
- [RasheedLarsenMetric](#) (real mParam, real aParam, real pParam, real qParam, bool rLogScale=false)
Construct a new Rasheed Larsen [Metric](#) object. All parameters are rescaled by the mass to end up with a $M = 1$ BH.
- [TwoIndex getMetric_dd](#) (const [Point](#) &p) const final
Rasheed-Larsen metric getter, indices down.
- [TwoIndex getMetric_uu](#) (const [Point](#) &p) const final
Rasheed-Larsen metric getter, indices up.
- std::string [getFullDescriptionStr](#) () const final
Rasheed-Larsen metric description string getter.

Public Member Functions inherited from SphericalHorizonMetric

- [SphericalHorizonMetric](#) ()=delete
- [SphericalHorizonMetric](#) (real HorizonRadius, bool rLogScale)
Construct a new Spherical Horizon Metric object.
- [real getHorizonRadius](#) () const
Get the radius of the horizon.

Public Member Functions inherited from Metric

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new Metric object.
- virtual [ThreeIndex getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual [real getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- bool [getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Private Attributes

- const [real m_aParam](#)
a parameter for the Rasheed-Larsen metric
- const [real m_mParam](#)
m parameter for the Rasheed-Larsen metric
- const [real m_pParam](#)
p parameter for the Rasheed-Larsen metric
- const [real m_qParam](#)
q parameter for the Rasheed-Larsen metric

Additional Inherited Members

Protected Attributes inherited from SphericalHorizonMetric

- const [real m_HorizonRadius](#)
Radius of the horizon.

Protected Attributes inherited from Metric

- [std::vector< int > m_Symmetries](#) {}
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- const bool [m_rLogScale](#)
Are we using a logarithmic r coordinate?

6.42.1 Detailed Description

Rasheed-Larsen black hole.

6.42.2 Constructor & Destructor Documentation

6.42.2.1 RasheedLarsenMetric() [1/2]

```
RasheedLarsenMetric::RasheedLarsenMetric () [delete]
```

6.42.2.2 RasheedLarsenMetric() [2/2]

```
RasheedLarsenMetric::RasheedLarsenMetric (
    real mParam,
    real aParam,
    real pParam,
    real qParam,
    bool rLogScale = false)
```

Construct a new Rasheed Larsen [Metric](#) object. All parameters are rescaled by the mass to end up with a $M = 1$ BH.

Parameters

<i>mParam</i>	m parameter for the Rasheed-Larsen metric
<i>aParam</i>	a parameter for the Rasheed-Larsen metric
<i>pParam</i>	p parameter for the Rasheed-Larsen metric
<i>qParam</i>	q parameter for the Rasheed-Larsen metric
<i>rLogScale</i>	whether we are using a logarithmic radial scale

6.42.3 Member Function Documentation

6.42.3.1 getFullDescriptionStr()

```
std::string RasheedLarsenMetric::getFullDescriptionStr () const [final], [virtual]
```

Rasheed-Larsen metric description string getter.

Returns

std::string

Reimplemented from [Metric](#).

6.42.3.2 getMetric_dd()

```
TwoIndex RasheedLarsenMetric::getMetric_dd (
    const Point & p) const [final], [virtual]
```

Rasheed-Larsen metric getter, indices down.

Parameters

p	Point at which to evaluate the metric
-----	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).

6.42.3.3 getMetric_uu()

```
TwoIndex RasheedLarsenMetric::getMetric_uu (  
    const Point & p) const [final], [virtual]
```

Rasheed-Larsen metric getter, indices up.

Parameters

p	Point at which to evaluate the metric
-----	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).

6.42.4 Member Data Documentation

6.42.4.1 m_aParam

```
const real RasheedLarsenMetric::m_aParam [private]
```

a parameter for the Rasheed-Larsen metric

6.42.4.2 m_mParam

```
const real RasheedLarsenMetric::m_mParam [private]
```

m parameter for the Rasheed-Larsen metric

6.42.4.3 m_pParam

```
const real RasheedLarsenMetric::m_pParam [private]
```

p parameter for the Rasheed-Larsen metric

6.42.4.4 m_qParam

```
const real RasheedLarsenMetric::m_qParam [private]
```

q parameter for the Rasheed-Larsen metric

The documentation for this class was generated from the following files:

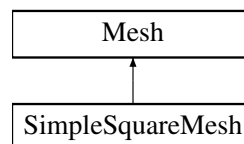
- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.43 SimpleSquareMesh Class Reference

A simple square mesh that will integrate a square of evenly spaced pixels.

```
#include <Mesh.h>
```

Inheritance diagram for SimpleSquareMesh:



Public Member Functions

- [SimpleSquareMesh](#) ()=delete
- [SimpleSquareMesh](#) (largecounter totalPixels, [DiagBitflag](#) valdiag)
- [largecounter](#) [getCurNrGeodesics](#) () const final
Get the current number of geodesics to be integrated.
- void [getNewInitConds](#) (largecounter index, [ScreenPoint](#) &newunitpoint, [ScreenIndex](#) &newscreenindex) const final
Get the initial conditions for the next pixel to be integrated.
- void [GeodesicFinished](#) (largecounter index, std::vector< [real](#) > finalValues) final
Function that is called when a geodesic is finished integrating.
- void [EndCurrentLoop](#) () final
End the current loop of integrating pixels.
- bool [IsFinished](#) () const final
Check whether the [Mesh](#) is finished integrating.
- std::string [getFullDescriptionStr](#) () const final
Description string getter.

Public Member Functions inherited from [Mesh](#)

- [Mesh](#) ([DiagBitflag](#) valdiag)
- virtual [~Mesh](#) ()=default

Private Attributes

- const [largecounter](#) `m_TotalPixels`
Total amount of pixels in grid (is the square of `m_RowColumnSize`)
- const [pixelcoord](#) `m_RowColumnSize`
Amount of pixels per row or column (square grid)
- bool `m_Finished` {false}
Are we done integrating or not?

Additional Inherited Members**Protected Attributes inherited from [Mesh](#)**

- const `std::unique_ptr< const Diagnostic > m_DistanceDiagnostic`
The [Diagnostic](#) (a const pointer to a const [Diagnostic](#) object) that is used to calculate distances (using `FinalData↔ValDistance()`) between the "values" that are assigned to Geodesics.

6.43.1 Detailed Description

A simple square mesh that will integrate a square of evenly spaced pixels.

6.43.2 Constructor & Destructor Documentation**6.43.2.1 SimpleSquareMesh() [1/2]**

```
SimpleSquareMesh::SimpleSquareMesh () [delete]
```

6.43.2.2 SimpleSquareMesh() [2/2]

```
SimpleSquareMesh::SimpleSquareMesh (
    largecounter totalPixels,
    DiagBitflag valdiag) [inline]
```

6.43.3 Member Function Documentation**6.43.3.1 EndCurrentLoop()**

```
void SimpleSquareMesh::EndCurrentLoop () [final], [virtual]
```

End the current loop of integrating pixels.

We are done integrating after one loop

Implements [Mesh](#).

6.43.3.2 GeodesicFinished()

```
void SimpleSquareMesh::GeodesicFinished (
    largecounter index,
    std::vector< real > finalValues) [final], [virtual]
```

Function that is called when a geodesic is finished integrating.

This [Mesh](#) doesn't actually have to do anything with the geodesic (values) when it is done!

Parameters

<i>index</i>	index of the geodesic that is finished
<i>finalValues</i>	final values of the geodesic

Implements [Mesh](#).

6.43.3.3 getCurNrGeodesics()

```
largecounter SimpleSquareMesh::getCurNrGeodesics () const [final], [virtual]
```

Get the current number of geodesics to be integrated.

This is the total number of pixels in the grid

Returns

largecounter

Implements [Mesh](#).

6.43.3.4 getFullDescriptionStr()

```
std::string SimpleSquareMesh::getFullDescriptionStr () const [final], [virtual]
```

Description string getter.

Returns

std::string

Reimplemented from [Mesh](#).

6.43.3.5 getNewInitConds()

```
void SimpleSquareMesh::getNewInitConds (
    largecounter index,
    ScreenPoint & newunitpoint,
    ScreenIndex & newscreenindex) const [final], [virtual]
```

Get the initial conditions for the next pixel to be integrated.

The next pixel is the one with index m_CurrentPixel

Parameters

<i>index</i>	index of the pixel to be initialized
<i>newunitpoint</i>	new unit point to be initialized
<i>newscreenindex</i>	new screen index to be initialized

Implements [Mesh](#).

6.43.3.6 IsFinished()

```
bool SimpleSquareMesh::IsFinished () const [final], [virtual]
```

Check whether the [Mesh](#) is finished integrating.

We are done if we have sent all pixels to be integrated (there is only one iteration of pixels)

Returns

```
true
false
```

Implements [Mesh](#).

6.43.4 Member Data Documentation

6.43.4.1 m_Finished

```
bool SimpleSquareMesh::m_Finished {false} [private]
```

Are we done integrating or not?

6.43.4.2 m_RowColumnSize

```
const pixelcoord SimpleSquareMesh::m_RowColumnSize [private]
```

Amount of pixels per row or column (square grid)

6.43.4.3 m_TotalPixels

```
const largecounter SimpleSquareMesh::m_TotalPixels [private]
```

Total amount of pixels in grid (is the square of m_RowColumnSize)

The documentation for this class was generated from the following files:

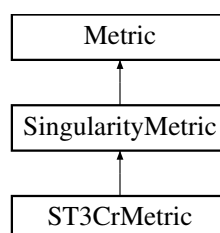
- FOORT/src/[Mesh.h](#)
- FOORT/src/[Mesh.cpp](#)

6.44 SingularityMetric Class Reference

Abstract base class for a metric with an arbitrary number of singularities (of arbitrary codimension)

```
#include <Metric.h>
```

Inheritance diagram for SingularityMetric:



Public Member Functions

- [SingularityMetric](#) (std::vector< [Singularity](#) > thesings, bool rLogScale)
Construct a new Singularity [Metric](#) object.
- std::vector< [Singularity](#) > [getSingularities](#) () const
Getters for the singularities.

Public Member Functions inherited from [Metric](#)

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new [Metric](#) object.
- virtual [TwoIndex](#) [getMetric_dd](#) (const [Point](#) &p) const =0
- virtual [TwoIndex](#) [getMetric_uu](#) (const [Point](#) &p) const =0
- virtual [ThreeIndex](#) [getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex](#) [getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual [real](#) [getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- virtual std::string [getFullDescriptionStr](#) () const
Base class description string getter.
- bool [getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Protected Attributes

- const std::vector< [Singularity](#) > [m_AllSingularities](#)
All singularities of the metric.

Protected Attributes inherited from [Metric](#)

- std::vector< int > [m_Symmetries](#) {}
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- const bool [m_rLogScale](#)
Are we using a logarithmic r coordinate?

6.44.1 Detailed Description

Abstract base class for a metric with an arbitrary number of singularities (of arbitrary codimension)

6.44.2 Constructor & Destructor Documentation

6.44.2.1 SingularityMetric()

```
SingularityMetric::SingularityMetric (
    std::vector< Singularity > thesings,
    bool rLogScale)
```

Construct a new Singularity [Metric](#) object.

Parameters

<i>thesings</i>	The singularities to include in the metric
<i>rLogScale</i>	whether we are using a logarithmic radial scale

6.44.3 Member Function Documentation

6.44.3.1 getSingularities()

```
std::vector< Singularity > SingularityMetric::getSingularities () const
```

Getters for the singularities.

Returns

```
std::vector<Singularity>
```

6.44.4 Member Data Documentation

6.44.4.1 m_AllSingularities

```
const std::vector<Singularity> SingularityMetric::m_AllSingularities [protected]
```

All singularities of the metric.

The documentation for this class was generated from the following files:

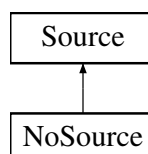
- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.45 Source Class Reference

Abstract [Source](#) base class.

```
#include <Geodesic.h>
```

Inheritance diagram for Source:



Public Member Functions

- [Source](#) (const [Metric](#) *const theMetric)
- virtual [~Source](#) ()=default
- virtual [OneIndex](#) [getSource](#) ([Point](#) pos, [OneIndex](#) vel) const =0
- virtual std::string [getFullDescriptionStr](#) () const

Basic full description string getter for [Source](#) base class.

Protected Attributes

- const [Metric](#) *const [m_theMetric](#)

A const pointer to a const metric.

6.45.1 Detailed Description

Abstract [Source](#) base class.

6.45.2 Constructor & Destructor Documentation

6.45.2.1 Source()

```
Source::Source (  
    const Metric *const theMetric) [inline]
```

6.45.2.2 ~Source()

```
virtual Source::~Source () [virtual], [default]
```

6.45.3 Member Function Documentation

6.45.3.1 getFullDescriptionStr()

```
std::string Source::getFullDescriptionStr () const [virtual]
```

Basic full description string getter for [Source](#) base class.

Returns

std::string

Reimplemented in [NoSource](#).

6.45.3.2 getSource()

```
virtual OneIndex Source::getSource (
    Point pos,
    OneIndex vel) const [pure virtual]
```

Implemented in [NoSource](#).

6.45.4 Member Data Documentation

6.45.4.1 m_theMetric

```
const Metric* const Source::m_theMetric [protected]
```

A const pointer to a const metric.

The documentation for this class was generated from the following files:

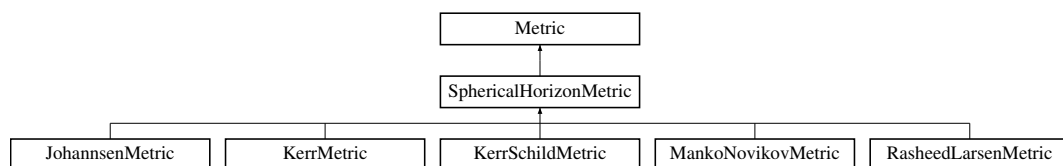
- [FOORT/src/Geodesic.h](#)
- [FOORT/src/Geodesic.cpp](#)

6.46 SphericalHorizonMetric Class Reference

Abstract base class for a metric with a spherical horizon (i.e. horizon at constant radius r)

```
#include <Metric.h>
```

Inheritance diagram for SphericalHorizonMetric:



Public Member Functions

- [SphericalHorizonMetric](#) ()=delete
- [SphericalHorizonMetric](#) (real HorizonRadius, bool rLogScale)
Construct a new Spherical Horizon [Metric](#) object.
- [real getHorizonRadius](#) () const
Get the radius of the horizon.

Public Member Functions inherited from [Metric](#)

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new [Metric](#) object.
- virtual [TwoIndex](#) getMetric_dd (const [Point](#) &p) const =0
- virtual [TwoIndex](#) getMetric_uu (const [Point](#) &p) const =0
- virtual [ThreeIndex](#) getChristoffel_udd (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex](#) getRiemann_uddd (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual [real](#) getKretschmann (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- virtual std::string getFullDescriptionStr () const
Base class description string getter.
- bool getLogScale () const
Get whether we are using a logarithmic radial scale.

Protected Attributes

- const [real](#) m_HorizonRadius
Radius of the horizon.

Protected Attributes inherited from [Metric](#)

- std::vector< int > m_Symmetries {}
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- const bool m_rLogScale
Are we using a logarithmic r coordinate?

6.46.1 Detailed Description

Abstract base class for a metric with a spherical horizon (i.e. horizon at constant radius r)

6.46.2 Constructor & Destructor Documentation

6.46.2.1 SphericalHorizonMetric() [1/2]

```
SphericalHorizonMetric::SphericalHorizonMetric () [delete]
```

6.46.2.2 SphericalHorizonMetric() [2/2]

```
SphericalHorizonMetric::SphericalHorizonMetric (
    real HorizonRadius,
    bool rLogScale)
```

Construct a new Spherical Horizon [Metric](#) object.

Parameters

<i>HorizonRadius</i>	Radius of the horizon
<i>rLogScale</i>	Whether we are using a logarithmic radial scale

6.46.3 Member Function Documentation

6.46.3.1 getHorizonRadius()

```
real SphericalHorizonMetric::getHorizonRadius () const
```

Get the radius of the horizon.

Returns

real

6.46.4 Member Data Documentation

6.46.4.1 m_HorizonRadius

```
const real SphericalHorizonMetric::m_HorizonRadius [protected]
```

Radius of the horizon.

The documentation for this class was generated from the following files:

- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.47 tk::spline Class Reference

```
#include <Spline.h>
```

Public Types

- enum [spline_type](#) { [linear](#) = 10 , [cspline](#) = 30 , [cspline_hermite](#) = 31 }
- enum [bd_type](#) { [first_deriv](#) = 1 , [second_deriv](#) = 2 }

Public Member Functions

- [spline](#) ()
- [spline](#) (const std::vector< double > &X, const std::vector< double > &Y, [spline_type](#) type=[cspline](#), bool [make_monotonic](#)=false, [bd_type](#) left=[second_deriv](#), double left_value=0.0, [bd_type](#) right=[second_deriv](#), double right_value=0.0)
- void [set_boundary](#) ([bd_type](#) left, double left_value, [bd_type](#) right, double right_value)
- void [set_points](#) (const std::vector< double > &x, const std::vector< double > &y, [spline_type](#) type=[cspline](#))
- bool [make_monotonic](#) ()
- double [operator\(\)](#) (double x) const
- double [deriv](#) (int order, double x) const
- std::vector< double > [get_x](#) () const
- std::vector< double > [get_y](#) () const
- double [get_x_min](#) () const
- double [get_x_max](#) () const

Protected Member Functions

- void [set_coeffs_from_b](#) ()
- size_t [find_closest](#) (double x) const

Protected Attributes

- std::vector< double > [m_x](#)
- std::vector< double > [m_y](#)
- std::vector< double > [m_b](#)
- std::vector< double > [m_c](#)
- std::vector< double > [m_d](#)
- double [m_c0](#)
- [spline_type](#) [m_type](#)
- [bd_type](#) [m_left](#)
- [bd_type](#) [m_right](#)
- double [m_left_value](#)
- double [m_right_value](#)
- bool [m_made_monotonic](#)

6.47.1 Member Enumeration Documentation

6.47.1.1 [bd_type](#)

```
enum tk::spline::bd_type
```

Enumerator

first_deriv	
second_deriv	

6.47.1.2 [spline_type](#)

```
enum tk::spline::spline_type
```

Enumerator

linear	
cspline	
cspline_hermite	

6.47.2 Constructor & Destructor Documentation

6.47.2.1 spline() [1/2]

```
tk::spline::spline ()
```

6.47.2.2 spline() [2/2]

```
tk::spline::spline (
    const std::vector< double > & X,
    const std::vector< double > & Y,
    spline_type type = cspline,
    bool make_monotonic = false,
    bd_type left = second_deriv,
    double left_value = 0.0,
    bd_type right = second_deriv,
    double right_value = 0.0)
```

6.47.3 Member Function Documentation

6.47.3.1 deriv()

```
double tk::spline::deriv (
    int order,
    double x) const
```

6.47.3.2 find_closest()

```
size_t tk::spline::find_closest (
    double x) const [protected]
```

6.47.3.3 get_x()

```
std::vector< double > tk::spline::get_x () const
```

6.47.3.4 get_x_max()

```
double tk::spline::get_x_max () const
```

6.47.3.5 get_x_min()

```
double tk::spline::get_x_min () const
```

6.47.3.6 get_y()

```
std::vector< double > tk::spline::get_y () const
```

6.47.3.7 make_monotonic()

```
bool tk::spline::make_monotonic ()
```

6.47.3.8 operator()()

```
double tk::spline::operator() (
    double x) const
```

6.47.3.9 set_boundary()

```
void tk::spline::set_boundary (
    spline::bd_type left,
    double left_value,
    spline::bd_type right,
    double right_value)
```

6.47.3.10 set_coeffs_from_b()

```
void tk::spline::set_coeffs_from_b () [protected]
```

6.47.3.11 set_points()

```
void tk::spline::set_points (
    const std::vector< double > & x,
    const std::vector< double > & y,
    spline_type type = cspline)
```

6.47.4 Member Data Documentation

6.47.4.1 m_b

```
std::vector<double> tk::spline::m_b [protected]
```

6.47.4.2 m_c

`std::vector<double> tk::spline::m_c` [protected]

6.47.4.3 m_c0

`double tk::spline::m_c0` [protected]

6.47.4.4 m_d

`std::vector<double> tk::spline::m_d` [protected]

6.47.4.5 m_left

`bd_type tk::spline::m_left` [protected]

6.47.4.6 m_left_value

`double tk::spline::m_left_value` [protected]

6.47.4.7 m_made_monotonic

`bool tk::spline::m_made_monotonic` [protected]

6.47.4.8 m_right

`bd_type tk::spline::m_right` [protected]

6.47.4.9 m_right_value

`double tk::spline::m_right_value` [protected]

6.47.4.10 m_type

`spline_type tk::spline::m_type` [protected]

6.47.4.11 m_x

`std::vector<double> tk::spline::m_x` [protected]

6.47.4.12 m_y

```
std::vector<double> tk::spline::m_y [protected]
```

The documentation for this class was generated from the following files:

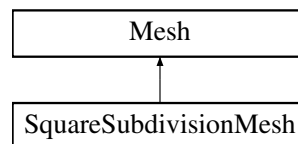
- FOORT/src/[Spline.h](#)
- FOORT/src/[Spline.cpp](#)

6.48 SquareSubdivisionMesh Class Reference

Adaptive subdivision [Mesh](#).

```
#include <Mesh.h>
```

Inheritance diagram for SquareSubdivisionMesh:



Classes

- struct [PixelInfo](#)
A struct the [Mesh](#) uses to keep all information about a given pixel.

Public Member Functions

- [SquareSubdivisionMesh](#) ()=delete
- [SquareSubdivisionMesh](#) ([largecounter](#) maxPixels, [largecounter](#) initialPixels, int maxSubdivide, [largecounter](#) iterationPixels, bool initialSubToFinal, [DiagBitflag](#) valdiag)
- [largecounter](#) getCurNrGeodesics () const final
Return the current number of geodesics to be integrated.
- void [getNewInitConds](#) ([largecounter](#) index, [ScreenPoint](#) &newunitpoint, [ScreenIndex](#) &newscreenindex) const final
Get the initial conditions for the next pixel to be integrated.
- void [GeodesicFinished](#) ([largecounter](#) index, std::vector< [real](#) > finalValues) final
Called when a geodesic is finished integrating: the final values are stored.
- void [EndCurrentLoop](#) () final
End the current loop of integrating pixels.
- bool [IsFinished](#) () const final
Check whether the [Mesh](#) is finished integrating.
- std::string [getFullDescriptionStr](#) () const final
Description string getter.

Public Member Functions inherited from [Mesh](#)

- [Mesh](#) ([DiagBitflag](#) valdiag)
- virtual [~Mesh](#) ()=default

Private Member Functions

- void [InitializeFirstGrid](#) ()
Helper function to initialize the first grid of pixels to integrate.
- void [UpdateAllNeighbors](#) ()
Helper function to update all pixel neighbors (if needed)
- void [UpdateAllWeights](#) ()
Helper function to update all pixel weights.
- void [SubdivideAndQueue](#) ([largecounter](#) ind)
Helper function to subdivide a pixel and add up to 5 new pixels to the queue.
- [pixelcoord](#) [Explnt](#) (int base, int exp)
Helper function to exponentiate ints.

Private Attributes

- const [largecounter](#) [m_InitialPixels](#)
How many initial pixels (spread uniformly over the grid) do we integrate?
- const int [m_MaxSubdivide](#)
How many times are we allowed to subdivide a square? Note: the initial grid is already at 1.
- const [pixelcoord](#) [m_RowColumnSize](#)
The total size in pixels of a row or column (square grid)
- const [largecounter](#) [m_IterationPixels](#)
How many pixels per iteration can we subdivide?
- const [largecounter](#) [m_MaxPixels](#)
How many pixels can we integrate in total over all iterations?
- const bool [m_InitialSubDividideToFinal](#)
If we decide to subdivide a square, do we automatically subdivide it further to the max level?
- const bool [m_InfinitePixels](#)
Are we allowed to integrate as many pixels as we want? (m_MaxPixels == 0)
- [largecounter](#) [m_PixelsLeft](#)
How many pixels are we still allowed to integrate (if !m_InfinitePixels)?
- std::vector< [PixelInfo](#) > [m_CurrentPixelQueue](#) {}
The current queue of pixels to be integrated.
- std::vector< bool > [m_CurrentPixelQueueDone](#) {}
A bool for every pixel in the current queue: gets set to true when the pixel is done integrating and gets its values returned.
- std::vector< [PixelInfo](#) > [m_AllPixels](#) {}
All pixels that have been integrated already (so does not include the pixels in the current queue)

Additional Inherited Members

Protected Attributes inherited from [Mesh](#)

- const std::unique_ptr< const [Diagnostic](#) > [m_DistanceDiagnostic](#)
The [Diagnostic](#) (a const pointer to a const [Diagnostic](#) object) that is used to calculate distances (using [FinalData](#)↔[ValDistance\(\)](#)) between the "values" that are assigned to Geodesics.

6.48.1 Detailed Description

Adaptive subdivision [Mesh](#).

starts with evenly spaced, square [Mesh](#), then decides to subdivide certain squares of pixels into smaller squares, based on which pixels have a bigger "weight", which is defined as the maximum "distance" (using the [Diagnostic](#) value distance) between the upper-left vertex of the square with the other three vertices of the square.

6.48.2 Constructor & Destructor Documentation

6.48.2.1 SquareSubdivisionMesh() [1/2]

```
SquareSubdivisionMesh::SquareSubdivisionMesh () [delete]
```

6.48.2.2 SquareSubdivisionMesh() [2/2]

```
SquareSubdivisionMesh::SquareSubdivisionMesh (
    largecounter maxPixels,
    largecounter initialPixels,
    int maxSubdivide,
    largecounter iterationPixels,
    bool initialSubToFinal,
    DiagBitflag valdiag) [inline]
```

6.48.3 Member Function Documentation

6.48.3.1 EndCurrentLoop()

```
void SquareSubdivisionMesh::EndCurrentLoop () [final], [virtual]
```

End the current loop of integrating pixels.

This function is called at the end of each integration iteration loop. We must wrap up the current iteration and initialize the next one.

Implements [Mesh](#).

6.48.3.2 ExpInt()

```
pixelcoord SquareSubdivisionMesh::ExpInt (
    int base,
    int exp) [private]
```

Helper function to exponentiate ints.

Note that the result may be larger than fits in an int, but this is only called with int arguments

Parameters

<i>base</i>	base
<i>exp</i>	exponent

Returns

pixelcoord

6.48.3.3 GeodesicFinished()

```
void SquareSubdivisionMesh::GeodesicFinished (
    largecounter index,
    std::vector< real > finalValues) [final], [virtual]
```

Called when a geodesic is finished integrating: the final values are stored.

Parameters

<i>index</i>	index of the geodesic that is finished
<i>finalValues</i>	final values of the diagnostics

Implements [Mesh](#).

6.48.3.4 getCurNrGeodesics()

```
largecounter SquareSubdivisionMesh::getCurNrGeodesics () const [final], [virtual]
```

Return the current number of geodesics to be integrated.

Returns

largecounter

Implements [Mesh](#).

6.48.3.5 getFullDescriptionStr()

```
std::string SquareSubdivisionMesh::getFullDescriptionStr () const [final], [virtual]
```

Description string getter.

Returns

std::string

Reimplemented from [Mesh](#).

6.48.3.6 getNewInitConds()

```
void SquareSubdivisionMesh::getNewInitConds (
    largecounter index,
    ScreenPoint & newunitpoint,
    ScreenIndex & newscreenindex) const [final], [virtual]
```

Get the initial conditions for the next pixel to be integrated.

The next pixel is the one with index `m_CurrentPixel`

Parameters

<i>index</i>	index of the pixel to be initialized
<i>newunitpoint</i>	new unit point to be initialized
<i>newscreenindex</i>	new screen index to be initialized

Implements [Mesh](#).

6.48.3.7 InitializeFirstGrid()

```
void SquareSubdivisionMesh::InitializeFirstGrid () [private]
```

Helper function to initialize the first grid of pixels to integrate.

6.48.3.8 IsFinished()

```
bool SquareSubdivisionMesh::IsFinished () const [final], [virtual]
```

Check whether the [Mesh](#) is finished integrating.

Returns

true
false

Implements [Mesh](#).

6.48.3.9 SubdivideAndQueue()

```
void SquareSubdivisionMesh::SubdivideAndQueue (  
    largecounter ind) [private]
```

Helper function to subdivide a pixel and add up to 5 new pixels to the queue.

This will take the pixel `m_AllPixels[ind]` and subdivide it, adding up to ≤ 5 pixels to the `CurrentPixelQueue`

Parameters

<i>ind</i>	index of the pixel to subdivide
------------	---------------------------------

6.48.3.10 UpdateAllNeighbors()

```
void SquareSubdivisionMesh::UpdateAllNeighbors () [private]
```

Helper function to update all pixel neighbors (if needed)

6.48.3.11 UpdateAllWeights()

```
void SquareSubdivisionMesh::UpdateAllWeights () [private]
```

Helper function to update all pixel weights.

Updates all weights of the pixels in m_AllPixels with weight < 0 and subdiv > 0 and subdiv < m_MaxSubdivide

Note

Assumes all squares have neighbors assigned correctly!

6.48.4 Member Data Documentation

6.48.4.1 m_AllPixels

```
std::vector<PixelInfo> SquareSubdivisionMesh::m_AllPixels {} [private]
```

All pixels that have been integrated already (so does not include the pixels in the current queue)

6.48.4.2 m_CurrentPixelQueue

```
std::vector<PixelInfo> SquareSubdivisionMesh::m_CurrentPixelQueue {} [private]
```

The current queue of pixels to be integrated.

6.48.4.3 m_CurrentPixelQueueDone

```
std::vector<bool> SquareSubdivisionMesh::m_CurrentPixelQueueDone {} [private]
```

A bool for every pixel in the current queue: gets set to true when the pixel is done integrating and gets its values returned.

6.48.4.4 m_InfinitePixels

```
const bool SquareSubdivisionMesh::m_InfinitePixels [private]
```

Are we allowed to integrate as many pixels as we want? (m_MaxPixels == 0)

6.48.4.5 m_InitialPixels

```
const largecounter SquareSubdivisionMesh::m_InitialPixels [private]
```

How many initial pixels (spread uniformly over the grid) do we integrate?

6.48.4.6 m_InitialSubDividideToFinal

```
const bool SquareSubdivisionMesh::m_InitialSubDividideToFinal [private]
```

If we decide to subdivide a square, do we automatically subdivide it further to the max level?

6.48.4.7 m_IterationPixels

```
const largecounter SquareSubdivisionMesh::m_IterationPixels [private]
```

How many pixels per iteration can we subdivide?

6.48.4.8 m_MaxPixels

```
const largecounter SquareSubdivisionMesh::m_MaxPixels [private]
```

How many pixels can we integrate in total over all iterations?

6.48.4.9 m_MaxSubdivide

```
const int SquareSubdivisionMesh::m_MaxSubdivide [private]
```

How many times are we allowed to subdivide a square? Note: the initial grid is already at 1.

6.48.4.10 m_PixelsLeft

```
largecounter SquareSubdivisionMesh::m_PixelsLeft [private]
```

How many pixels are we still allowed to integrate (if !m_InfinitePixels)?

6.48.4.11 m_RowColumnSize

```
const pixelcoord SquareSubdivisionMesh::m_RowColumnSize [private]
```

The total size in pixels of a row or column (square grid)

The documentation for this class was generated from the following files:

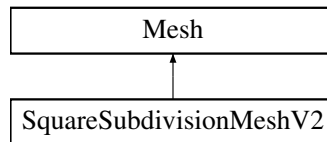
- FOORT/src/[Mesh.h](#)
- FOORT/src/[Mesh.cpp](#)

6.49 SquareSubdivisionMeshV2 Class Reference

Adaptive subdivision [Mesh](#).

```
#include <Mesh.h>
```

Inheritance diagram for SquareSubdivisionMeshV2:



Classes

- struct [PixelInfo](#)
A struct the [Mesh](#) uses to keep all information about a given pixel.

Public Member Functions

- [SquareSubdivisionMeshV2](#) ()=delete
- [SquareSubdivisionMeshV2](#) ([largecounter](#) maxPixels, [largecounter](#) initialPixels, int maxSubdivide, [largecounter](#) iterationPixels, bool initialSubToFinal, [DiagBitflag](#) valdiag)
- [largecounter](#) [getCurNrGeodesics](#) () const final
Return the current number of geodesics to be integrated.
- void [getNewInitConds](#) ([largecounter](#) index, [ScreenPoint](#) &newunitpoint, [ScreenIndex](#) &newscreenindex) const final
Get the initial conditions for the next pixel to be integrated.
- void [GeodesicFinished](#) ([largecounter](#) index, std::vector< [real](#) > finalValues) final
Called when a geodesic is finished integrating: the final values are stored.
- void [EndCurrentLoop](#) () final
Called at the end of each integration iteration loop. Wraps up the current iteration and initializes the next one.
- bool [IsFinished](#) () const final
Check whether the [Mesh](#) is finished integrating.
- std::string [getFullDescriptionStr](#) () const final
Description string getter.

Public Member Functions inherited from [Mesh](#)

- [Mesh](#) ([DiagBitflag](#) valdiag)
- virtual [~Mesh](#) ()=default

Private Member Functions

- void `InitializeFirstGrid ()`
Helper function to initialize the first grid of pixels to integrate.
- void `UpdateAllWeights ()`
Helper function to update all pixel weights in `m_CurrentPixelUpdating`.
- `PixelInfo * GetUp (PixelInfo *p, int subdiv) const`
Get the upper neighbor of a pixel at a certain subdivision level.
- `PixelInfo * GetDown (PixelInfo *p, int subdiv) const`
Get the lower neighbor of a pixel at a certain subdivision level.
- `PixelInfo * GetRight (PixelInfo *p, int subdiv) const`
Get the right neighbor of a pixel at a certain subdivision level.
- `PixelInfo * GetLeft (PixelInfo *p, int subdiv) const`
Get the left neighbor of a pixel at a certain subdivision level.
- void `SubdivideAndQueue (largecounter ind)`
Subdivide the pixel at index `ind` and add up to 5 new pixels to the integration queue.
- `pixelcoord Explnt (int base, int exp) const`
Helper function to exponentiate ints.

Private Attributes

- const `largecounter m_InitialPixels`
How many initial pixels (spread uniformly over the grid) do we integrate?
- const int `m_MaxSubdivide`
How many times are we allowed to subdivide a square? Note: the initial grid is already at 1.
- const `pixelcoord m_RowColumnSize`
The total size in pixels of a row or column (square grid)
- const `largecounter m_IterationPixels`
How many pixels per iteration can we subdivide?
- const `largecounter m_MaxPixels`
How many pixels can we integrate in total over all iterations?
- const bool `m_InitialSubDivideToFinal`
If we decide to subdivide a square, do we automatically subdivide it further to the max level?
- const bool `m_InfinitePixels`
Are we allowed to integrate as many pixels as we want? (`m_MaxPixels == 0`)
- `largecounter m_PixelsLeft`
How many pixels are we still allowed to integrate (if `!m_InfinitePixels`)?
- `largecounter m_PixelsIntegrated {0}`
How many pixels we have integrated so far.
- `std::forward_list< std::unique_ptr< PixelInfo > > m_AllPixels {}`
Master list of all pixels. `std::forward_list` is more space-efficient than `std::list`, bidirectional iteration is not needed, no random access supported. This list is only used to store the owner pointers (and thus the objects) of all pixels. The other pixel vectors are used to iterate through (and need random access)
- `std::vector< PixelInfo * > m_ActivePixels {}`
List of active pixels, i.e. those that can be subdivided and have non-zero weight.
- `std::vector< PixelInfo * > m_CurrentPixelQueue {}`
List of current queue of pixels to be sent to be integrated.
- `std::vector< bool > m_CurrentPixelQueueDone {}`
A bool for every pixel in the current queue: gets set to true when the pixel is done integrating and gets its values returned.
- `std::vector< PixelInfo * > m_CurrentPixelUpdating {}`
List of pixels that are already integrated but need updating weights after current queue is all integrated.

Additional Inherited Members

Protected Attributes inherited from [Mesh](#)

- `const std::unique_ptr< const Diagnostic > m_DistanceDiagnostic`

The [Diagnostic](#) (a `const` pointer to a `const Diagnostic` object) that is used to calculate distances (using `FinalData↔ValDistance()`) between the "values" that are assigned to Geodesics.

6.49.1 Detailed Description

Adaptive subdivision [Mesh](#).

starts with evenly spaced, square [Mesh](#), then decides to subdivide certain squares of pixels into smaller squares, based on which pixels have a bigger "weight", which is defined as the maximum "distance" (using the [Diagnostic](#) value distance) between the upper-left vertex of the square with the other three vertices of the square. V2: new way of dealing with neighbors and looping over pixels

6.49.2 Constructor & Destructor Documentation

6.49.2.1 SquareSubdivisionMeshV2() [1/2]

```
SquareSubdivisionMeshV2::SquareSubdivisionMeshV2 () [delete]
```

6.49.2.2 SquareSubdivisionMeshV2() [2/2]

```
SquareSubdivisionMeshV2::SquareSubdivisionMeshV2 (
    largecounter maxPixels,
    largecounter initialPixels,
    int maxSubdivide,
    largecounter iterationPixels,
    bool initialSubToFinal,
    DiagBitflag valdiag) [inline]
```

6.49.3 Member Function Documentation

6.49.3.1 EndCurrentLoop()

```
void SquareSubdivisionMeshV2::EndCurrentLoop () [final], [virtual]
```

Called at the end of each integration iteration loop. Wraps up the current iteration and initializes the next one.

Implements [Mesh](#).

6.49.3.2 ExpInt()

```
pixelcoord SquareSubdivisionMeshV2::ExpInt (
    int base,
    int exp) const [private]
```

Helper function to exponentiate ints.

Note

The result may be larger than fits in an int, but this is only called with int arguments

Parameters

<i>base</i>	base
<i>exp</i>	exponent

Returns

pixelcoord

6.49.3.3 GeodesicFinished()

```
void SquareSubdivisionMeshV2::GeodesicFinished (
    largecounter index,
    std::vector< real > finalValues) [final], [virtual]
```

Called when a geodesic is finished integrating: the final values are stored.

Parameters

<i>index</i>	index of the geodesic that is finished
<i>finalValues</i>	final values of the diagnostics

Implements [Mesh](#).

6.49.3.4 getCurNrGeodesics()

```
largecounter SquareSubdivisionMeshV2::getCurNrGeodesics () const [final], [virtual]
```

Return the current number of geodesics to be integrated.

Returns

largecounter

Implements [Mesh](#).

6.49.3.5 GetDown()

```
SquareSubdivisionMeshV2::PixelInfo * SquareSubdivisionMeshV2::GetDown (
    PixelInfo * p,
    int subdiv) const [private]
```

Get the lower neighbor of a pixel at a certain subdivision level.

Parameters

<i>p</i>	PixelInfo pointer to the pixel
<i>subdiv</i>	subdivision level of the neighbor

Returns

[SquareSubdivisionMeshV2::PixelInfo*](#)

6.49.3.6 getFullDescriptionStr()

```
std::string SquareSubdivisionMeshV2::getFullDescriptionStr () const [final], [virtual]
```

Description string getter.

Returns

std::string

Reimplemented from [Mesh](#).

6.49.3.7 GetLeft()

```
SquareSubdivisionMeshV2::PixelInfo * SquareSubdivisionMeshV2::GetLeft (
    PixelInfo * p,
    int subdiv) const [private]
```

Get the left neighbor of a pixel at a certain subdivision level.

Parameters

<i>p</i>	PixelInfo pointer to the pixel
<i>subdiv</i>	subdivision level of the neighbor

Returns

[SquareSubdivisionMeshV2::PixelInfo*](#)

6.49.3.8 getNewInitConds()

```
void SquareSubdivisionMeshV2::getNewInitConds (
    largecounter index,
    ScreenPoint & newunitpoint,
    ScreenIndex & newscreenindex) const [final], [virtual]
```

Get the initial conditions for the next pixel to be integrated.

The next pixel is the one with index m_CurrentPixel

Parameters

<i>index</i>	index of the pixel to be initialized
<i>newunitpoint</i>	new unit point to be initialized
<i>newscreenindex</i>	new screen index to be initialized

Implements [Mesh](#).

6.49.3.9 GetRight()

```
SquareSubdivisionMeshV2::PixelInfo * SquareSubdivisionMeshV2::GetRight (
    PixelInfo * p,
    int subdiv) const [private]
```

Get the right neighbor of a pixel at a certain subdivision level.

Parameters

<i>p</i>	PixelInfo pointer to the pixel
<i>subdiv</i>	subdivision level of the neighbor

Returns

[SquareSubdivisionMeshV2::PixelInfo*](#)

6.49.3.10 GetUp()

```

SquareSubdivisionMeshV2::PixelInfo * SquareSubdivisionMeshV2::GetUp (
    PixelInfo * p,
    int subdiv) const [private]

```

Get the upper neighbor of a pixel at a certain subdivision level.

Parameters

<i>p</i>	PixelInfo pointer to the pixel
<i>subdiv</i>	subdivision level of the neighbor

Returns

[SquareSubdivisionMeshV2::PixelInfo*](#)

6.49.3.11 InitializeFirstGrid()

```

void SquareSubdivisionMeshV2::InitializeFirstGrid () [private]

```

Helper function to initialize the first grid of pixels to integrate.

6.49.3.12 IsFinished()

```

bool SquareSubdivisionMeshV2::IsFinished () const [final], [virtual]

```

Check whether the [Mesh](#) is finished integrating.

Returns

true
false

Implements [Mesh](#).

6.49.3.13 SubdivideAndQueue()

```

void SquareSubdivisionMeshV2::SubdivideAndQueue (
    largecounter ind) [private]

```

Subdivide the pixel at index *ind* and add up to 5 new pixels to the integration queue.

The pixels are numbered as 1 2 3 4 5 6 7 8 9 i.e. the initial square is given by (1, 3, 7, 9), and the new pixels are (2, 4, 5, 6, 8).

Parameters

<i>ind</i>	index of the pixel to subdivide
------------	---------------------------------

6.49.3.14 UpdateAllWeights()

```
void SquareSubdivisionMeshV2::UpdateAllWeights () [private]
```

Helper function to update all pixel weights in m_CurrentPixelUpdating.

All pixels with weight > 0 will be added to m_ActivePixels

Note

Assumes all pixels have subdiv > 0 and subdiv < m_MaxSubdivide, and all their neighbors assigned correctly

Assumes all pixels have their (and their neighbor's) values assigned correctly

6.49.4 Member Data Documentation**6.49.4.1 m_ActivePixels**

```
std::vector<PixelInfo *> SquareSubdivisionMeshV2::m_ActivePixels {} [private]
```

List of active pixels, i.e. those that can be subdivided and have non-zero weight.

6.49.4.2 m_AllPixels

```
std::forward_list<std::unique_ptr<PixelInfo> > SquareSubdivisionMeshV2::m_AllPixels {} [private]
```

Master list of all pixels. std::forward_list is more space-efficient than std::list, bidirectional iteration is not needed, no random access supported. This list is only used to store the owner pointers (and thus the objects) of all pixels. The other pixel vectors are used to iterate through (and need random access)

6.49.4.3 m_CurrentPixelQueue

```
std::vector<PixelInfo *> SquareSubdivisionMeshV2::m_CurrentPixelQueue {} [private]
```

List of current queue of pixels to be sent to be integrated.

6.49.4.4 m_CurrentPixelQueueDone

```
std::vector<bool> SquareSubdivisionMeshV2::m_CurrentPixelQueueDone {} [private]
```

A bool for every pixel in the current queue: gets set to true when the pixel is done integrating and gets its values returned.

6.49.4.5 m_CurrentPixelUpdating

```
std::vector<PixelInfo *> SquareSubdivisionMeshV2::m_CurrentPixelUpdating {} [private]
```

List of pixels that are already integrated but need updating weights after current queue is all integrated.

6.49.4.6 m_InfinitePixels

```
const bool SquareSubdivisionMeshV2::m_InfinitePixels [private]
```

Are we allowed to integrate as many pixels as we want? (m_MaxPixels == 0)

6.49.4.7 m_InitialPixels

```
const largecounter SquareSubdivisionMeshV2::m_InitialPixels [private]
```

How many initial pixels (spread uniformly over the grid) do we integrate?

6.49.4.8 m_InitialSubDividideToFinal

```
const bool SquareSubdivisionMeshV2::m_InitialSubDividideToFinal [private]
```

If we decide to subdivide a square, do we automatically subdivide it further to the max level?

6.49.4.9 m_IterationPixels

```
const largecounter SquareSubdivisionMeshV2::m_IterationPixels [private]
```

How many pixels per iteration can we subdivide?

6.49.4.10 m_MaxPixels

```
const largecounter SquareSubdivisionMeshV2::m_MaxPixels [private]
```

How many pixels can we integrate in total over all iterations?

6.49.4.11 m_MaxSubdivide

```
const int SquareSubdivisionMeshV2::m_MaxSubdivide [private]
```

How many times are we allowed to subdivide a square? Note: the initial grid is already at 1.

6.49.4.12 m_PixelsIntegrated

```
largecounter SquareSubdivisionMeshV2::m_PixelsIntegrated {0} [private]
```

How many pixels we have integrated so far.

6.49.4.13 m_PixelsLeft

`largecounter` SquareSubdivisionMeshV2::m_PixelsLeft [private]

How many pixels are we still allowed to integrate (if !m_InfinitePixels)?

6.49.4.14 m_RowColumnSize

`const pixelcoord` SquareSubdivisionMeshV2::m_RowColumnSize [private]

The total size in pixels of a row or column (square grid)

The documentation for this class was generated from the following files:

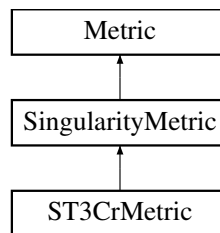
- FOORT/src/[Mesh.h](#)
- FOORT/src/[Mesh.cpp](#)

6.50 ST3CrMetric Class Reference

Ring fuzzball metric.

```
#include <Metric.h>
```

Inheritance diagram for ST3CrMetric:



Public Member Functions

- [ST3CrMetric](#) (`real` P, `real` q0, `real` lambda, `bool` rlogscale=false)
Construct a new ST3Cr [Metric](#) object.
- `TwoIndex` [getMetric_dd](#) (`const` [Point](#) &p) `const` final
ST3Cr metric getter, indices down.
- `TwoIndex` [getMetric_uu](#) (`const` [Point](#) &p) `const` final
ST3Cr metric getter, indices up.
- `std::string` [getFullDescriptionStr](#) () `const` final
ST3Cr metric description string getter.

Public Member Functions inherited from [SingularityMetric](#)

- [SingularityMetric](#) (`std::vector`< [Singularity](#) > thesings, `bool` rLogScale)
Construct a new [Singularity Metric](#) object.
- `std::vector`< [Singularity](#) > [getSingularities](#) () `const`
Getters for the singularities.

Public Member Functions inherited from [Metric](#)

- virtual [~Metric](#) ()=default
- [Metric](#) (bool rlogscale=false)
Construct a new [Metric](#) object.
- virtual [ThreeIndex](#) [getChristoffel_udd](#) (const [Point](#) &p) const
Return the Christoffel symbols of the metric (indices up, down, down)
- virtual [FourIndex](#) [getRiemann_uddd](#) (const [Point](#) &p) const
Riemann tensor of the metric (indices up, down, down, down)
- virtual [real](#) [getKretschmann](#) (const [Point](#) &p) const
Get the Kretschmann scalar at a point.
- bool [getrLogScale](#) () const
Get whether we are using a logarithmic radial scale.

Private Member Functions

- [real](#) [get_omega](#) ([real](#) r, [real](#) theta, [real](#) l) const
Get omega for the ST3Cr metric.
- [real](#) [f_phi](#) ([real](#) phi, [real](#) r, [real](#) theta, [real](#) l, [real](#) R) const
*function for prefactor * dphi' for ring*
- [real](#) [f_om_phi](#) ([real](#) phi, [real](#) r, [real](#) theta, [real](#) l, [real](#) R) const
omega_phi ring functions

Private Attributes

- const [real](#) [m_P](#)
- const [real](#) [m_q0](#)
- const [real](#) [m_lambda](#)

Additional Inherited Members

Protected Attributes inherited from [SingularityMetric](#)

- const std::vector< [Singularity](#) > [m_AllSingularities](#)
All singularities of the metric.

Protected Attributes inherited from [Metric](#)

- std::vector< int > [m_Symmetries](#) {}
The symmetries (coordinate Killing vectors) of the metric. Should be set by descendant constructor.
- const bool [m_rLogScale](#)
Are we using a logarithmic r coordinate?

6.50.1 Detailed Description

Ring fuzzball metric.

Note

Implementation by Lies Van Dael

6.50.2 Constructor & Destructor Documentation

6.50.2.1 ST3CrMetric()

```
ST3CrMetric::ST3CrMetric (
    real P,
    real q0,
    real lambda,
    bool rlogscale = false)
```

Construct a new ST3Cr [Metric](#) object.

Parameters

<i>P</i>	P parameter for the ST3Cr metric
<i>q0</i>	q0 parameter for the ST3Cr metric
<i>lambda</i>	lambda parameter for the ST3Cr metric
<i>rlogscale</i>	whether we are using a logarithmic radial scale

6.50.3 Member Function Documentation

6.50.3.1 f_om_phi()

```
real ST3CrMetric::f_om_phi (
    real phi,
    real r,
    real theta,
    real l,
    real R) const [private]
```

omega_phi ring functions

Parameters

<i>phi</i>	
<i>r</i>	
<i>theta</i>	
<i>l</i>	
<i>R</i>	

Returns

real

6.50.3.2 f_phi()

```
real ST3CrMetric::f_phi (
    real phi,
    real r,
    real theta,
    real l,
    real R) const [private]
```

function for prefactor * dphi' for ring

Parameters

<i>phi</i>	
<i>r</i>	
<i>theta</i>	
<i>l</i>	
<i>R</i>	

Returns

real

6.50.3.3 get_omega()

```
real ST3CrMetric::get_omega (
    real r,
    real theta,
    real l) const [private]
```

Get omega for the ST3Cr metric.

Parameters

<i>r</i>	<i>r</i> _{ij}
<i>theta</i>	<i>theta</i> _{↔ ij}
<i>l</i>	<i>l</i> _{ij}

Returns

real

6.50.3.4 getFullDescriptionStr()

```
std::string ST3CrMetric::getFullDescriptionStr () const [final], [virtual]
```

ST3Cr metric description string getter.

Returns

std::string

Reimplemented from [Metric](#).

6.50.3.5 getMetric_dd()

```
TwoIndex ST3CrMetric::getMetric_dd (
    const Point & p) const [final], [virtual]
```

ST3Cr metric getter, indices down.

Parameters

p	Point at which to evaluate the metric
-----	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).**6.50.3.6 getMetric_uu()**

```
TwoIndex ST3CrMetric::getMetric_uu (
    const Point & p) const [final], [virtual]
```

ST3Cr metric getter, indices up.

Parameters

p	Point at which to evaluate the metric
-----	---------------------------------------

Returns

TwoIndex

Implements [Metric](#).**6.50.4 Member Data Documentation****6.50.4.1 m_lambda**

```
const real ST3CrMetric::m_lambda [private]
```

6.50.4.2 m_P

```
const real ST3CrMetric::m_P [private]
```

6.50.4.3 m_q0

```
const real ST3CrMetric::m_q0 [private]
```

The documentation for this class was generated from the following files:

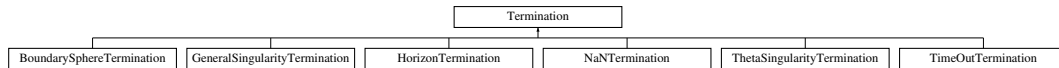
- FOORT/src/[Metric.h](#)
- FOORT/src/[Metric.cpp](#)

6.51 Termination Class Reference

Abstract base class for all Terminations.

```
#include <Terminations.h>
```

Inheritance diagram for Termination:



Public Member Functions

- [Termination](#) ()=delete
- [Termination](#) ([Geodesic](#) *const theGeodesic)
- virtual void [Reset](#) ()
[Termination](#) (abstract base class) functions.
- virtual [~Termination](#) ()=default
- virtual [Term CheckTermination](#) ()=0
- virtual std::string [getFullDescriptionStr](#) () const =0

Protected Member Functions

- bool [DecideUpdate](#) ([largecounter](#) UpdateNSteps)

Protected Attributes

- [Geodesic](#) *const [m_OwnerGeodesic](#)
The geodesic that owns the [Termination](#) (a const pointer to the [Geodesic](#))
- [largecounter](#) [m_StepsSinceUpdated](#) {}
The termination is itself in charge of keeping track of how many steps it has been since it has been updated. The [Termination](#)'s [TerminationOptions](#) struct tells it how many steps it needs to wait between updates.

6.51.1 Detailed Description

Abstract base class for all Terminations.

6.51.2 Constructor & Destructor Documentation

6.51.2.1 Termination() [1/2]

```
Termination::Termination () [delete]
```

6.51.2.2 Termination() [2/2]

```
Termination::Termination (  
    Geodesic *const theGeodesic) [inline]
```

6.51.2.3 ~Termination()

```
virtual Termination::~~Termination () [virtual], [default]
```

6.51.3 Member Function Documentation

6.51.3.1 CheckTermination()

```
virtual Term Termination::CheckTermination () [pure virtual]
```

Implemented in [BoundarySphereTermination](#), [GeneralSingularityTermination](#), [HorizonTermination](#), [NaNTermination](#), [ThetaSingularityTermination](#), and [TimeOutTermination](#).

6.51.3.2 DecideUpdate()

```
bool Termination::DecideUpdate (  
    largecounter UpdateNSteps) [protected]
```

6.51.3.3 getFullDescriptionStr()

```
virtual std::string Termination::getFullDescriptionStr () const [pure virtual]
```

Implemented in [BoundarySphereTermination](#), [GeneralSingularityTermination](#), [HorizonTermination](#), [NaNTermination](#), [ThetaSingularityTermination](#), and [TimeOutTermination](#).

6.51.3.4 Reset()

```
void Termination::Reset () [virtual]
```

[Termination](#) (abstract base class) functions.

Reimplemented in [TimeOutTermination](#).

6.51.4 Member Data Documentation

6.51.4.1 m_OwnerGeodesic

```
Geodesic* const Termination::m_OwnerGeodesic [protected]
```

The geodesic that owns the [Termination](#) (a const pointer to the [Geodesic](#))

6.51.4.2 m_StepsSinceUpdated

```
largecounter Termination::m_StepsSinceUpdated {} [protected]
```

The termination is itself in charge of keeping track of how many steps it has been since it has been updated. The [Termination](#)'s [TerminationOptions](#) struct tells it how many steps it needs to wait between updates.

The documentation for this class was generated from the following files:

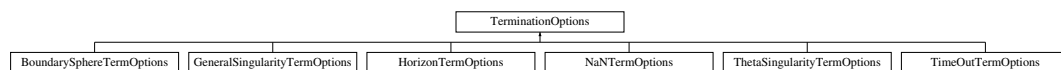
- FOORT/src/[Terminations.h](#)
- FOORT/src/[Terminations.cpp](#)

6.52 TerminationOptions Struct Reference

Base class for all [TerminationOptions](#) structs. Other [TerminationOptions](#) can inherit from here if they require more options.

```
#include <Terminations.h>
```

Inheritance diagram for TerminationOptions:



Public Member Functions

- [TerminationOptions](#) ([largecounter](#) Nsteps)
- virtual [~TerminationOptions](#) ()=default

Public Attributes

- const [largecounter](#) UpdateEveryNSteps
Number of steps between updates.

6.52.1 Detailed Description

Base class for all [TerminationOptions](#) structs. Other [TerminationOptions](#) can inherit from here if they require more options.

6.52.2 Constructor & Destructor Documentation

6.52.2.1 TerminationOptions()

```
TerminationOptions::TerminationOptions (  
    largecounter Nsteps) [inline]
```

6.52.2.2 ~TerminationOptions()

```
virtual TerminationOptions::~~TerminationOptions () [virtual], [default]
```

6.52.3 Member Data Documentation

6.52.3.1 UpdateEveryNSteps

```
const largecounter TerminationOptions::UpdateEveryNSteps
```

Number of steps between updates.

The documentation for this struct was generated from the following file:

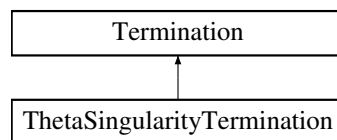
- FOORT/src/[Terminations.h](#)

6.53 ThetaSingularityTermination Class Reference

[ThetaSingularityTermination](#): Terminate geodesics if they get too close to a polar coordinate singularity.

```
#include <Terminations.h>
```

Inheritance diagram for ThetaSingularityTermination:



Public Member Functions

- [ThetaSingularityTermination](#) ([Geodesic](#) *const theGeodesic)
- [Term CheckTermination](#) () final
- *[ThetaSingularityTermination](#) functions.*
- std::string [getFullDescriptionStr](#) () const final

Public Member Functions inherited from [Termination](#)

- [Termination](#) ()=delete
- [Termination](#) ([Geodesic](#) *const theGeodesic)
- virtual void [Reset](#) ()
- *[Termination](#) (abstract base class) functions.*
- virtual [~Termination](#) ()=default

Static Public Attributes

- static std::unique_ptr< [ThetaSingularityTermOptions](#) > [TermOptions](#)

The options that the [Termination](#) keeps (will probably be a descendant struct instead, which specifies any additional options the [Termination](#) needs)

Additional Inherited Members

Protected Member Functions inherited from [Termination](#)

- bool [DecideUpdate](#) ([largecounter](#) UpdateNSteps)

Protected Attributes inherited from [Termination](#)

- [Geodesic](#) *const [m_OwnerGeodesic](#)

The geodesic that owns the [Termination](#) (a const pointer to the [Geodesic](#))

- [largecounter](#) [m_StepsSinceUpdated](#) {}

The termination is itself in charge of keeping track of how many steps it has been since it has been updated. The [Termination](#)'s [TerminationOptions](#) struct tells it how many steps it needs to wait between updates.

6.53.1 Detailed Description

[ThetaSingularityTermination](#): Terminate geodesics if they get too close to a polar coordinate singularity.

6.53.2 Constructor & Destructor Documentation

6.53.2.1 [ThetaSingularityTermination\(\)](#)

```
ThetaSingularityTermination::ThetaSingularityTermination (
    Geodesic *const theGeodesic) [inline]
```

6.53.3 Member Function Documentation

6.53.3.1 [CheckTermination\(\)](#)

```
Term ThetaSingularityTermination::CheckTermination () [final], [virtual]
```

[ThetaSingularityTermination](#) functions.

Implements [Termination](#).

6.53.3.2 [getFullDescriptionStr\(\)](#)

```
std::string ThetaSingularityTermination::getFullDescriptionStr () const [final], [virtual]
```

Implements [Termination](#).

6.53.4 Member Data Documentation

6.53.4.1 TermOptions

```
std::unique_ptr< ThetaSingularityTermOptions > ThetaSingularityTermination::TermOptions [static]
```

The options that the [Termination](#) keeps (will probably be a descendant struct instead, which specifies any additional options the [Termination](#) needs)

The documentation for this class was generated from the following files:

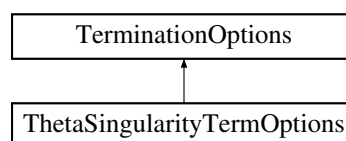
- FOORT/src/[Terminations.h](#)
- FOORT/src/[Config.cpp](#)
- FOORT/src/[Terminations.cpp](#)

6.54 ThetaSingularityTermOptions Struct Reference

Options for [ThetaSingularityTermination](#).

```
#include <Terminations.h>
```

Inheritance diagram for ThetaSingularityTermOptions:



Public Member Functions

- [ThetaSingularityTermOptions](#) ([real](#) epsilon, [largecounter](#) Nsteps)

Public Member Functions inherited from [TerminationOptions](#)

- [TerminationOptions](#) ([largecounter](#) Nsteps)
- virtual [~TerminationOptions](#) ()=default

Public Attributes

- const [real](#) [ThetaSingEpsilon](#)
Minimum difference between theta and 0 or pi.

Public Attributes inherited from [TerminationOptions](#)

- const [largecounter](#) [UpdateEveryNSteps](#)
Number of steps between updates.

6.54.1 Detailed Description

Options for [ThetaSingularityTermination](#).

Keeps track of the minimum required distance from the singularity

6.54.2 Constructor & Destructor Documentation

6.54.2.1 ThetaSingularityTermOptions()

```
ThetaSingularityTermOptions::ThetaSingularityTermOptions (
    real epsilon,
    largecounter Nsteps) [inline]
```

6.54.3 Member Data Documentation

6.54.3.1 ThetaSingEpsilon

```
const real ThetaSingularityTermOptions::ThetaSingEpsilon
```

Minimum difference between theta and 0 or pi.

The documentation for this struct was generated from the following file:

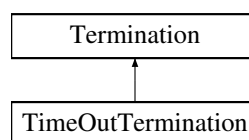
- FOORT/src/[Terminations.h](#)

6.55 TimeOutTermination Class Reference

[TimeOutTermination](#): Terminate geodesics if they take too many steps.

```
#include <Terminations.h>
```

Inheritance diagram for TimeOutTermination:



Public Member Functions

- [TimeOutTermination](#) ([Geodesic](#) *const theGeodesic)
- void [Reset](#) () final
- *[TimeOutTermination](#) functions.*
- [Term CheckTermination](#) () final
- std::string [getFullDescriptionStr](#) () const final

Public Member Functions inherited from Termination

- [Termination](#) ()=delete
- [Termination](#) ([Geodesic](#) *const theGeodesic)
- virtual $\sim$ [Termination](#) ()=default

Static Public Attributes

- static std::unique_ptr< [TimeOutTermOptions](#) > [TermOptions](#)
The options that the [TimeOutTermination](#) keeps (contains max number of steps allowed)

Private Attributes

- [largecounter](#) m_CurNrSteps {0}
Keep track of the number of steps that the geodesic has taken so far.

Additional Inherited Members

Protected Member Functions inherited from Termination

- bool [DecideUpdate](#) ([largecounter](#) UpdateNSteps)

Protected Attributes inherited from Termination

- [Geodesic](#) *const m_OwnerGeodesic
The geodesic that owns the [Termination](#) (a const pointer to the [Geodesic](#))
- [largecounter](#) m_StepsSinceUpdated {}
The termination is itself in charge of keeping track of how many steps it has been since it has been updated. The [Termination](#)'s [TerminationOptions](#) struct tells it how many steps it needs to wait between updates.

6.55.1 Detailed Description

[TimeOutTermination](#): Terminate geodesics if they take too many steps.

6.55.2 Constructor & Destructor Documentation

6.55.2.1 TimeOutTermination()

```
TimeOutTermination::TimeOutTermination (
    Geodesic *const theGeodesic) [inline]
```

6.55.3 Member Function Documentation

6.55.3.1 CheckTermination()

```
Term TimeOutTermination::CheckTermination () [final], [virtual]
```

Implements [Termination](#).

6.55.3.2 getFullDescriptionStr()

```
std::string TimeOutTermination::getFullDescriptionStr () const [final], [virtual]
```

Implements [Termination](#).

6.55.3.3 Reset()

```
void TimeOutTermination::Reset () [final], [virtual]
```

[TimeOutTermination](#) functions.

Reimplemented from [Termination](#).

6.55.4 Member Data Documentation

6.55.4.1 m_CurNrSteps

```
largecounter TimeOutTermination::m_CurNrSteps {0} [private]
```

Keep track of the number of steps that the geodesic has taken so far.

6.55.4.2 TermOptions

```
std::unique_ptr< TimeOutTermOptions > TimeOutTermination::TermOptions [static]
```

The options that the [TimeOutTermination](#) keeps (contains max number of steps allowed)

The documentation for this class was generated from the following files:

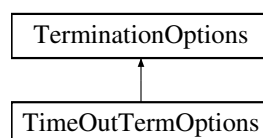
- FOORT/src/[Terminations.h](#)
- FOORT/src/[Config.cpp](#)
- FOORT/src/[Terminations.cpp](#)

6.56 TimeOutTermOptions Struct Reference

Options for [TimeOutTermination](#).

```
#include <Terminations.h>
```

Inheritance diagram for TimeOutTermOptions:



Public Member Functions

- [TimeOutTermOptions](#) ([largecounter](#) MaxStepsAllowed, [largecounter](#) Nsteps)

Public Member Functions inherited from [TerminationOptions](#)

- [TerminationOptions](#) ([largecounter](#) Nsteps)
- virtual [~TerminationOptions](#) ()=default

Public Attributes

- const [largecounter](#) MaxSteps
Max. number of integration steps allowed.

Public Attributes inherited from [TerminationOptions](#)

- const [largecounter](#) UpdateEveryNSteps
Number of steps between updates.

6.56.1 Detailed Description

Options for [TimeOutTermination](#).

Keeps track of the max. number of integration steps allowed

6.56.2 Constructor & Destructor Documentation

6.56.2.1 TimeOutTermOptions()

```
TimeOutTermOptions::TimeOutTermOptions (
    largecounter MaxStepsAllowed,
    largecounter Nsteps) [inline]
```

6.56.3 Member Data Documentation

6.56.3.1 MaxSteps

```
const largecounter TimeOutTermOptions::MaxSteps
```

Max. number of integration steps allowed.

The documentation for this struct was generated from the following file:

- FOORT/src/[Terminations.h](#)

6.57 Utilities::Timer Class Reference

```
#include <Utilities.h>
```

Public Member Functions

- void [reset](#) ()
Utilities::Timer functions.
- double [elapsed](#) () const

Private Types

- using [Clock](#) = std::chrono::steady_clock
- using [Second](#) = std::chrono::duration<double, std::ratio<1>>

Private Attributes

- std::chrono::time_point< [Clock](#) > [m_beg](#) {Clock::now()}

6.57.1 Member Typedef Documentation

6.57.1.1 Clock

```
using Utilities::Timer::Clock = std::chrono::steady_clock [private]
```

6.57.1.2 Second

```
using Utilities::Timer::Second = std::chrono::duration<double, std::ratio<1>> [private]
```

6.57.2 Member Function Documentation

6.57.2.1 elapsed()

```
double Utilities::Timer::elapsed () const
```

6.57.2.2 reset()

```
void Utilities::Timer::reset ()
```

[Utilities::Timer](#) functions.

6.57.3 Member Data Documentation

6.57.3.1 m_beg

```
std::chrono::time_point<Clock> Utilities::Timer::m_beg {Clock::now()} [private]
```

The documentation for this class was generated from the following files:

- FOORT/src/[Utilities.h](#)
- FOORT/src/[Utilities.cpp](#)

6.58 UpdateFrequency Struct Reference

Struct to hold the information for a [Diagnostic](#) to update itself.

```
#include <Diagnostics.h>
```

Public Attributes

- [largecounter](#) [UpdateNSteps](#) {0}
- bool [UpdateStart](#) {false}
- bool [UpdateFinish](#) {false}

6.58.1 Detailed Description

Struct to hold the information for a [Diagnostic](#) to update itself.

If `UpdateNSteps > 0`, then every so many steps. If `UpdateNSteps == 0`, then it only updates at the start and/or finish of integration if the appropriate bool is set.

6.58.2 Member Data Documentation

6.58.2.1 UpdateFinish

```
bool UpdateFrequency::UpdateFinish {false}
```

6.58.2.2 UpdateNSteps

```
largecounter UpdateFrequency::UpdateNSteps {0}
```

6.58.2.3 UpdateStart

```
bool UpdateFrequency::UpdateStart {false}
```

The documentation for this struct was generated from the following file:

- FOORT/src/[Diagnostics.h](#)

6.59 ViewScreen Class Reference

```
#include <ViewScreen.h>
```

Public Member Functions

- [ViewScreen](#) ()=delete
- [ViewScreen](#) ([Point](#) pos, [OneIndex](#) dir, [ScreenPoint](#) screensize, [ScreenPoint](#) screencenter, std::unique_ptr< [Mesh](#) > theMesh, const [Metric](#) *const theMetric, [GeodesicType](#) thegeodtype=[GeodesicType::Null](#))
- void [SetNewInitialConditions](#) ([largecounter](#) index, [Point](#) &pos, [OneIndex](#) &vel, [ScreenIndex](#) &scrIndex) const
- bool [IsFinished](#) () const
- [largecounter](#) [getCurNrGeodesics](#) () const
- void [EndCurrentLoop](#) ()
- void [GeodesicFinished](#) ([largecounter](#) index, std::vector< [real](#) > finalValues)
- std::string [getFullDescriptionStr](#) () const

Private Member Functions

- void [ConstructVielbein](#) ()
[ViewScreen](#) functions.

Private Attributes

- [TwoIndex](#) m_Metric_dd {}
- [TwoIndex](#) m_Vielbein {}
- const [Point](#) m_Pos
- const [OneIndex](#) m_Direction
- const [ScreenPoint](#) m_ScreenSize
- const [ScreenPoint](#) m_ScreenCenter
- const bool m_rLogScale
- const [Metric](#) *const m_theMetric
- const [GeodesicType](#) m_GeodType {[GeodesicType::Null](#)}
- const std::unique_ptr< [Mesh](#) > m_theMesh

6.59.1 Constructor & Destructor Documentation

6.59.1.1 ViewScreen() [1/2]

```
ViewScreen::ViewScreen () [delete]
```

6.59.1.2 ViewScreen() [2/2]

```
ViewScreen::ViewScreen (
    Point pos,
    OneIndex dir,
    ScreenPoint screensize,
    ScreenPoint screencenter,
    std::unique_ptr< Mesh > theMesh,
    const Metric *const theMetric,
    GeodesicType thegeodtype = GeodesicType::Null) [inline]
```

6.59.2 Member Function Documentation

6.59.2.1 ConstructVielbein()

```
void ViewScreen::ConstructVielbein () [private]
```

[ViewScreen](#) functions.

6.59.2.2 EndCurrentLoop()

```
void ViewScreen::EndCurrentLoop ()
```

6.59.2.3 GeodesicFinished()

```
void ViewScreen::GeodesicFinished (
    largecounter index,
    std::vector< real > finalValues)
```

6.59.2.4 getCurNrGeodesics()

```
largecounter ViewScreen::getCurNrGeodesics () const
```

6.59.2.5 getFullDescriptionStr()

```
std::string ViewScreen::getFullDescriptionStr () const
```

6.59.2.6 IsFinished()

```
bool ViewScreen::IsFinished () const
```

6.59.2.7 SetNewInitialConditions()

```
void ViewScreen::SetNewInitialConditions (
    largecounter index,
    Point & pos,
    OneIndex & vel,
    ScreenIndex & scrIndex) const
```

6.59.3 Member Data Documentation

6.59.3.1 m_Direction

```
const OneIndex ViewScreen::m_Direction [private]
```

6.59.3.2 m_GeodType

```
const GeodesicType ViewScreen::m_GeodType {GeodesicType::Null} [private]
```

6.59.3.3 m_Metric_dd

```
TwoIndex ViewScreen::m_Metric_dd {} [private]
```

6.59.3.4 m_Pos

```
const Point ViewScreen::m_Pos [private]
```

6.59.3.5 m_rLogScale

```
const bool ViewScreen::m_rLogScale [private]
```

6.59.3.6 m_ScreenCenter

```
const ScreenPoint ViewScreen::m_ScreenCenter [private]
```

6.59.3.7 m_ScreenSize

```
const ScreenPoint ViewScreen::m_ScreenSize [private]
```

6.59.3.8 m_theMesh

```
const std::unique_ptr<Mesh> ViewScreen::m_theMesh [private]
```

6.59.3.9 m_theMetric

```
const Metric* const ViewScreen::m_theMetric [private]
```

6.59.3.10 m_Vielbein

```
TwoIndex ViewScreen::m_Vielbein {} [private]
```

The documentation for this class was generated from the following files:

- FOORT/src/[ViewScreen.h](#)
- FOORT/src/[ViewScreen.cpp](#)

Chapter 7

File Documentation

7.1 FOORT/src/Config.cpp File Reference

```
#include "Config.h"  
#include "Utilities.h"  
#include <algorithm>  
#include <cctype>  
#include <utility>
```

7.1.1 Detailed Description

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-12

Copyright

Copyright (c) 2024

7.2 FOORT/src/Config.h File Reference

Functions that read the configuration file and initialize all objects.

```
#include "Geometry.h"  
#include "Metric.h"  
#include "Diagnostics.h"  
#include "Terminations.h"  
#include "ViewScreen.h"  
#include "Geodesic.h"  
#include "Integrators.h"  
#include <memory>  
#include <string>  
#include <exception>  
#include "ConfigReader.h"
```

Namespaces

- namespace [Config](#)

Namespace for all configuration functions that initialize objects based on configuration file.

Macros

- #define [CONFIGURATION_MODE](#)

Typedefs

- using [Config::ConfigCollection](#) = [ConfigReader::ConfigCollection](#)
- using [Config::SettingError](#) = `std::invalid_argument`

Functions

- void [Config::InitializeScreenOutput](#) (const [ConfigCollection](#) &theCfg)
Initialize screen output options.
- std::unique_ptr< [Metric](#) > [Config::GetMetric](#) (const [ConfigCollection](#) &theCfg)
Use configuration to create the correct [Metric](#) with specified parameters.
- std::unique_ptr< [Source](#) > [Config::GetSource](#) (const [ConfigCollection](#) &theCfg, const [Metric](#) *const theMetric)
Use configuration to create the correct [Source](#) with specified parameters.
- void [Config::InitializeDiagnostics](#) (const [ConfigCollection](#) &theCfg, [DiagBitflag](#) &alldiags, [DiagBitflag](#) &valdiag, const [Metric](#) *const theMetric)
Use configuration to set the [Diagnostics](#) bitflag appropriately; initialize all [DiagnosticOptions](#) for all [Diagnostics](#) that are turned on; and set bitflag for diagnostic to be used for coarseness evaluating in [Mesh](#).
- void [Config::InitializeTerminations](#) (const [ConfigCollection](#) &theCfg, [TermBitflag](#) &allterms, const [Metric](#) *const theMetric)
Use configuration to set the [Termination](#) bitflag appropriately; and initialize all [TerminationOptions](#) for all [Terminations](#) that are turned on.
- std::unique_ptr< [ViewScreen](#) > [Config::GetViewScreen](#) (const [ConfigCollection](#) &theCfg, [DiagBitflag](#) valdiag, const [Metric](#) *const theMetric)
Use configuration to create the [ViewScreen](#) object with specified parameters.
- std::unique_ptr< [Mesh](#) > [Config::GetMesh](#) (const [ConfigCollection](#) &theCfg, [DiagBitflag](#) valdiag)
Use configuration to create the [Mesh](#) object with specified parameters.
- [GeodesicIntegratorFunc](#) [Config::GetGeodesicIntegrator](#) (const [ConfigCollection](#) &theCfg)
Use configuration to set the [GeodesicIntegratorFunc](#) pointer to the correct integrator function.
- std::unique_ptr< [GeodesicOutputHandler](#) > [Config::GetOutputHandler](#) (const [ConfigCollection](#) &theCfg, [DiagBitflag](#) alldiags, [DiagBitflag](#) valdiag, std::string FirstLineInfo)
Use configuration to create the [GeodesicOutputHandler](#) object with specified parameters.

Variables

- constexpr auto [Config::Output_Important_Default](#) = [OutputLevel::Level_0_WARNING](#)
- constexpr auto [Config::Output_Other_Default](#) = [OutputLevel::Level_1_PROC](#)

7.2.1 Detailed Description

Functions that read the configuration file and initialize all objects.

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-12

Copyright

Copyright (c) 2024

7.2.2 Macro Definition Documentation

7.2.2.1 CONFIGURATION_MODE

```
#define CONFIGURATION_MODE
```

7.3 Config.h

[Go to the documentation of this file.](#)

```
00001 #ifndef _FOORT_CONFIG_H
00002 #define _FOORT_CONFIG_H
00003
00014 // OUTDATED: should remain defined for now
00015 #define CONFIGURATION_MODE
00017
00018 #include "Geometry.h"
00019 #include "Metric.h"
00020 #include "Diagnostics.h"
00021 #include "Terminations.h"
00022 #include "ViewScreen.h"
00023 #include "Geodesic.h"
00024 #include "Integrators.h"
00025
00026 #include <memory> // std::unique_ptr
00027 #include <string> // std::string
00028
00029 #include <exception> // needed to define our own configuration error
00030
00031 #include "ConfigReader.h"
00032
00036 namespace Config
00037 {
00038     // Output level for important missing information that will default
00039     // (e.g. no metric selected, no diagnostics selected, ...)
00040     constexpr auto Output_Important_Default = OutputLevel::Level_0_WARNING;
00041     // Output level for less important information that will default
00042     // (e.g. Kerr metric a parameter not specified)
00043     constexpr auto Output_Other_Default = OutputLevel::Level_1_PROC;
00044
00045     // Total configuration object, as loaded from configuration file
00046     using ConfigCollection = ConfigReader::ConfigCollection;
```

```

00047
00048 // An exception to throw whenever an important setting is not found
00049 // Always should be caught and then reverted to default settings
00050 using SettingError = std::invalid_argument;
00051
00052
00053
00054
00055 // Use configuration to initialize the screen output level
00056 void InitializeScreenOutput(const ConfigCollection &theCfg);
00057
00058 // Use configuration to create the correct Metric with specified parameters
00059 std::unique_ptr<Metric> GetMetric(const ConfigCollection &theCfg);
00060
00061 // Use configuration to create the correct Source with specified parameters
00062 std::unique_ptr<Source> GetSource(const ConfigCollection &theCfg, const Metric *const theMetric);
00063
00064 // Use configuration to set the Diagnostics bitflag appropriately;
00065 // initialize all DiagnosticOptions for all Diagnostics that are turned on;
00066 // and set bitflag for diagnostic to be used for coarseness evaluating in Mesh
00067 void InitializeDiagnostics(const ConfigCollection &theCfg, DiagBitflag &alldiags, DiagBitflag
&valdiag, const Metric *const theMetric);
00068
00069 // Use configuration to set the Terminations bitflag appropriately;
00070 // initialize all TerminationOptions for all Terminations that are turned on;
00071 void InitializeTerminations(const ConfigCollection &theCfg, TermBitflag &allterms, const Metric
*const theMetric);
00072
00073 // Use configuration to create ViewScreen appropriately
00074 std::unique_ptr<ViewScreen> GetViewScreen(const ConfigCollection &theCfg, DiagBitflag valdiag,
const Metric *const theMetric);
00075 // GetViewScreen calls GetMesh to create the correct Mesh
00076 std::unique_ptr<Mesh> GetMesh(const ConfigCollection &theCfg, DiagBitflag valdiag);
00077
00078 // Use configuration to return a pointer to the correct integration function to use
00079 GeodesicIntegratorFunc GetGeodesicIntegrator(const ConfigCollection &theCfg);
00080
00081 // Use configuration to initialize the output handler
00082 std::unique_ptr<GeodesicOutputHandler> GetOutputHandler(const ConfigCollection &theCfg,
DiagBitflag alldiags, DiagBitflag valdiag,
00083 std::string FirstLineInfo);
00084
00085 } // end namespace Config
00086
00087 #endif

```

7.4 FOORT/src/ConfigReader.cpp File Reference

```

#include "ConfigReader.h"
#include <ios>
#include <type_traits>

```

7.4.1 Detailed Description

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-12

Copyright

Copyright (c) 2024

7.5 FOORT/src/ConfigReader.h File Reference

Classes used to read in a configuration file and store the settings.

```
#include <string_view>
#include <string>
#include <iostream>
#include <fstream>
#include <vector>
#include <variant>
#include <stdexcept>
#include <limits>
#include <memory>
```

Classes

- class [ConfigReader::ConfigReaderException](#)
- class [ConfigReader::ConfigCollection](#)
- struct [ConfigReader::ConfigCollection::ConfigSetting](#)

Namespaces

- namespace [ConfigReader](#)
Namespace to put all [ConfigReader](#) objects.

Variables

- const long long [ConfigReader::MaxStreamSize](#) {std::numeric_limits<std::streamsize>::max() }

7.5.1 Detailed Description

Classes used to read in a configuration file and store the settings.

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-12

Copyright

Copyright (c) 2024

7.6 ConfigReader.h

[Go to the documentation of this file.](#)

```

00001 #ifndef _CONFIGREADER_H
00002 #define _CONFIGREADER_H
00003
00013 #include <string_view> // std::string_view
00014 #include <string>      // std::string
00015 #include <iostream>    // needed for file and console output
00016 #include <fstream>     // needed for file input
00017 #include <vector>      // needed to create vectors of Settings
00018 #include <variant>     // needed for std::variant
00019 #include <stdexcept>   // needed for exceptions
00020 #include <limits>      // for std::numeric_limits
00021 #include <memory>      // for std::unique_ptr
00022
00026 namespace ConfigReader
00027 {
00028     // Shorthand for the maximum number of characters we can ignore in a stream
00029     const long long MaxStreamSize{std::numeric_limits<std::streamsize>::max()};
00030
00031     // Exception that is thrown if anything bad happens when reading in configuration file
00032     class ConfigReaderException : public std::runtime_error
00033     {
00034     public:
00035         ConfigReaderException(const std::string &error, std::vector<int> settingtrace = {})
00036             : std::runtime_error{error.c_str()}, m_settingtrace{settingtrace}
00037         {
00038         }
00039         // no need to override what() since we can just use std::runtime_error::what()
00040
00041         std::vector<int> trace() const
00042         {
00043             return m_settingtrace;
00044         };
00045
00046     private:
00047         std::vector<int> m_settingtrace;
00048     };
00049
00050     // A collection of configuration settings
00051     class ConfigCollection
00052     {
00053     public:
00054         // Returns true if the collection contains a setting with the given name
00055         bool Exists(std::string_view SettingName) const;
00056         // Returns total number of settings in this collection
00057         int NrSettings() const;
00058
00059         // Returns true if the setting with the given name (resp. index) is a collection of
00060         // subsettings
00061         // (Returns false if this setting name does not exist, or index is out of bounds)
00062         bool IsCollection(std::string_view SettingName) const;
00063         bool IsCollection(int SettingIndex) const;
00064         // If CollectionName (resp. CollectionIndex) is the name (resp. index)
00065         // of a setting that is a subcollection, then this returns
00066         // a const reference to this subcollection; otherwise throws ConfigReaderException
00067         const ConfigCollection &operator[](std::string_view CollectionName) const;
00068         const ConfigCollection &operator[](int CollectionIndex) const;
00069
00070         // Generic functions that look up the value of a given setting
00071         // and put it in the output variable
00072         // Returns true if successful, false if unsuccessful
00073         // (either due to type mismatch of output and setting value, or if setting doesn't exist)
00074         // Templated functions implemented below in .h file!
00075         template <class OutputType>
00076         bool LookupValue(std::string_view SettingName, OutputType &theOutput) const;
00077         template <class OutputType>
00078         bool LookupValue(int SettingIndex, OutputType &theOutput) const;
00079
00080         // Specific functions for looking up signed integral types
00081         // These will first see if the setting value is of the given type, then
00082         // they will try smaller types and then recast to the larger type
00083         // (so e.g. for long long, first long long is tried, then long, then int)
00084         bool LookupValueInteger(std::string_view SettingName, int &theOutput) const;
00085         bool LookupValueInteger(std::string_view SettingName, long &theOutput) const;
00086         bool LookupValueInteger(std::string_view SettingName, long long &theOutput) const;
00087         // Specific functions for looking up unsigned integral types
00088         // These will first test if the setting value is of the signed case, if it is and it is >=0
00089         // then it always fits in
00090         // the unsigned version; then it will test larger (signed) types and see if the result still
00091         // fits in the unsigned version
00092         bool LookupValueInteger(std::string_view SettingName, unsigned int &theOutput) const;
00093         bool LookupValueInteger(std::string_view SettingName, unsigned long &theOutput) const;
00094         bool LookupValueInteger(std::string_view SettingName, unsigned long long &theOutput) const;

```

```

00094         // Templated functions implemented below in .h file!
00095         template <class OutputType>
00096         bool LookupValueInteger(int SettingIndex, OutputType &theOutput) const;
00097
00099         void DisplayCollection(std::ostream &OutputStream, int Indent = 0) const;
00100
00102         // this overwrites the current collection with the contents of the config file
00103         // Will throw ConfigReaderException if parse/syntax error in configuration file
00104         // Returns true if successful
00105         bool ReadFile(const std::string &FileName);
00106
00107     private:
00108         // A configuration setting value can be an integral type, bool, double, string, or a
00109         (sub)collection of settings
00109         // Note: when reading in a configuration file, an integral value will always be stored in the
00110         smallest possible type!
00110         using ConfigSettingValue = std::variant<bool,
00111             int, long, long long,
00112             double,
00113             std::string,
00114             std::unique_ptr<ConfigCollection>;
00115
00115         // A setting is a name and setting value
00116         struct ConfigSetting
00117         {
00118             std::string SettingName;
00119             ConfigSettingValue SettingValue;
00120         };
00121
00121         // The vector of all settings in this collection
00122         std::vector<ConfigSetting> m_Settings{};
00123
00124         // Helper function that returns the index of the setting with the given name
00125         // returns -1 if setting not found
00126         int GetSettingIndex(std::string_view SettingName) const;
00127
00128         // Helper functions for DisplayCollection
00129         // DisplaySetting displays one setting; if it's a subcollection then it calls
00130         // DisplayCollection on the subcollection
00131         void DisplaySetting(std::ostream &OutputStream, int SettingIndex, int Indent) const;
00132         // Output given number of tabs
00133         void DisplayTabs(std::ostream &OutputStream, int NrTabs) const;
00134
00135         // Helper functions for ReadFile()
00136         // These all take an ifstream at a given location and will read in one specific object
00137         //
00138         // Read in entire collection (including subcollections)
00139         // Pre: stream is positioned right after { (or at beginning of file)
00140         // Post: stream is positioned right after } (or at EOF)
00141         void ReadCollection(std::ifstream &InputFile);
00142         // Read in a setting name
00143         // Pre: next non-whitespace character in stream is beginning of name
00144         // Post: stream has just read in all characters of name (but no more);
00145         // returns "" if there is no more setting to read in the current collection, in this case
00146         // '}' has been read in (for subcollections)
00147         void ReadSettingName(std::ifstream &InputFile, std::string &theName);
00148         // Read in one specific character only (not '/')
00149         // Pre: next non-whitespace character in stream is this character
00150         // Post: stream is just after character
00151         void ReadSettingSpecificChar(std::ifstream &InputFile, char theChar) const;
00152         // Read in setting value (could be subcollection)
00153         // Pre: next non-whitespace character in stream is beginning of value
00154         // (can be '{' for subcollection, digit for number, '"' for string, or 't'/'f' for boolean)
00155         // Post: Value has entirely been read in (i.e. next character should be ';')
00156         void ReadSettingValue(std::ifstream &InputFile, ConfigSettingValue &theValue);
00157     };
00158
00159 } // end namespace declarations
00160
00163
00173 template <class OutputType>
00174 bool ConfigReader::ConfigCollection::LookupValue(int SettingIndex, OutputType &theOutput) const
00175 {
00176     // Get the setting value, if it is indeed of this type
00177     const OutputType *OutputPointer{std::get_if<OutputType>(&m_Settings[SettingIndex].SettingValue)};
00178     if (OutputPointer)
00179     {
00180         // Success, the setting is indeed of this type; set the output accordingly
00181         theOutput = *OutputPointer;
00182         return true;
00183     }
00184     return false;
00185 }
00186
00196 template <class OutputType>
00197 bool ConfigReader::ConfigCollection::LookupValue(std::string_view SettingName, OutputType &theOutput)
00198     const
00199 {
00199     // Look up setting index and then defer to index-based implementation, if setting exists

```

```

00200     int SettingIndex{GetSettingIndex(SettingName)};
00201     if (SettingIndex >= 0)
00202     {
00203         return LookupValue(SettingIndex, theOutput);
00204     }
00205     return false;
00206 }
00207
00218 template <class OutputType>
00219 bool ConfigReader::ConfigCollection::LookupValueInteger(int SettingIndex, OutputType &theOutput) const
00220 {
00221     return LookupValueInteger(m_Settings[SettingIndex].SettingName, theOutput);
00222 }
00223
00224 #endif

```

7.7 FOORT/src/Diagnostics.cpp File Reference

```

#include "Diagnostics.h"
#include "DiagnosticsEmission.h"
#include "Geodesic.h"
#include "InputOutput.h"
#include "Integrators.h"
#include <algorithm>
#include <cmath>

```

Functions

- [DiagnosticUniqueVector CreateDiagnosticVector](#) ([DiagBitflag](#) diagflags, [DiagBitflag](#) valdiag, [Geodesic](#) *const theGeodesic)
Diagnostic helper function.

7.7.1 Detailed Description

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-16

Copyright

Copyright (c) 2024

7.7.2 Function Documentation

7.7.2.1 CreateDiagnosticVector()

```

DiagnosticUniqueVector CreateDiagnosticVector (
    DiagBitflag diagflags,
    DiagBitflag valdiag,
    Geodesic *const theGeodesic)

```

[Diagnostic](#) helper function.

Create a [Diagnostic](#) Vector object

Parameters

<i>diagflags</i>	Diagnostic bitflags
<i>valdiag</i>	Value Diagnostic bitflags
<i>theGeodesic</i>	Pointer to the Geodesic object

Returns

DiagnosticUniqueVector

7.8 FOORT/src/Diagnostics.h File Reference

Declarations of abstract base [Diagnostic](#) class and all its descendants. All Definitions in [Diagnostics.cpp](#).

```
#include "Geometry.h"
#include "Metric.h"
#include "InputOutput.h"
#include "DiagnosticsEmission.h"
#include <stdint>
#include <string>
#include <memory>
#include <vector>
```

Classes

- struct [UpdateFrequency](#)
Struct to hold the information for a [Diagnostic](#) to update itself.
- class [Diagnostic](#)
Abstract base class for all diagnostics.
- class [FourColorScreenDiagnostic](#)
The four color screen diagnostic.
- class [GeodesicPositionDiagnostic](#)
Geodesic position tracker.
- class [EquatorialPassesDiagnostic](#)
Diagnostic that counts the number of passes through the equatorial plane.
- class [ClosestRadiusDiagnostic](#)
Diagnostic that keeps track of the closest (true, not log) radius that the geodesic passes through.
- class [EquatorialEmissionDiagnostic](#)
Diagnostic that calculates brightness intensity for the geodesic, based on a specified equatorial disc emission model.
- struct [DiagnosticOptions](#)
Base class for [DiagnosticOptions](#). Other Diagnostics can inherit from here if they require more options.
- struct [GeodesicPositionOptions](#)
Options for [GeodesicPositionDiagnostic](#).
- struct [EquatorialPassesOptions](#)
Options for [EquatorialPassesDiagnostic](#).
- struct [ClosestRadiusOptions](#)
Options for [ClosestRadiusDiagnostic](#).
- struct [EquatorialEmissionOptions](#)
Options for [EquatorialEmissionDiagnostic](#).

Typedefs

- using [DiagBitflag](#) = std::uint16_t
Diagnostic bitflags.
- using [DiagnosticUniqueVector](#) = std::vector<std::unique_ptr<[Diagnostic](#)>>
Owner vector of derived Diagnostics classes.

Functions

- [DiagnosticUniqueVector CreateDiagnosticVector](#) ([DiagBitflag](#) diagflags, [DiagBitflag](#) valdiag, [Geodesic](#) *const theGeodesic)
Diagnostic helper function.

Variables

- constexpr [DiagBitflag](#) [Diag_None](#) {0b0000'0000'0000'0000}
- constexpr [DiagBitflag](#) [Diag_GeodesicPosition](#) {0b0000'0000'0000'0001}
- constexpr [DiagBitflag](#) [Diag_FourColorScreen](#) {0b0000'0000'0000'0010}
- constexpr [DiagBitflag](#) [Diag_EquatorialPasses](#) {0b0000'0000'0000'0100}
- constexpr [DiagBitflag](#) [Diag_ClosestRadius](#) {0b0000'0000'0000'1000}
- constexpr [DiagBitflag](#) [Diag_EquatorialEmission](#) {0b0000'0000'0001'0000}

7.8.1 Detailed Description

Declarations of abstract base [Diagnostic](#) class and all its descendants. All Definitions in [Diagnostics.cpp](#).

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-16

Copyright

Copyright (c) 2024

7.8.2 Typedef Documentation

7.8.2.1 DiagBitflag

using [DiagBitflag](#) = std::uint16_t

[Diagnostic](#) bitflags.

Used for constructing vector of Diagnostics ...

7.8.2.2 DiagnosticUniqueVector

```
using DiagnosticUniqueVector = std::vector<std::unique_ptr<Diagnostic>>
```

Owner vector of derived Diagnostics classes.

7.8.3 Function Documentation

7.8.3.1 CreateDiagnosticVector()

```
DiagnosticUniqueVector CreateDiagnosticVector (
    DiagBitflag diagflags,
    DiagBitflag valdiag,
    Geodesic *const theGeodesic)
```

[Diagnostic](#) helper function.

Helper to create a new vector of [Diagnostic](#) options, based on the bitflag The first diagnostic is the value diagnostic

Create a [Diagnostic](#) Vector object

Parameters

<i>diagflags</i>	Diagnostic bitflags
<i>valdiag</i>	Value Diagnostic bitflags
<i>theGeodesic</i>	Pointer to the Geodesic object

Returns

DiagnosticUniqueVector

7.8.4 Variable Documentation

7.8.4.1 Diag_ClosestRadius

```
DiagBitflag Diag_ClosestRadius {0b0000'0000'0000'1000} [constexpr]
```

7.8.4.2 Diag_EquatorialEmission

```
DiagBitflag Diag_EquatorialEmission {0b0000'0000'0001'0000} [constexpr]
```

7.8.4.3 Diag_EquatorialPasses

```
DiagBitflag Diag_EquatorialPasses {0b0000'0000'0000'0100} [constexpr]
```

7.8.4.4 Diag_FourColorScreen

```
DiagBitflag Diag_FourColorScreen {0b0000'0000'0000'0010} [constexpr]
```

7.8.4.5 Diag_GeodesicPosition

```
DiagBitflag Diag_GeodesicPosition {0b0000'0000'0000'0001} [constexpr]
```

7.8.4.6 Diag_None

```
DiagBitflag Diag_None {0b0000'0000'0000'0000} [constexpr]
```

7.9 Diagnostics.h

[Go to the documentation of this file.](#)

```
00001 #ifndef _FOORT_DIAGNOSTICS_H
00002 #define _FOORT_DIAGNOSTICS_H
00003
00013 #include "Geometry.h" // for tensors
00014 #include "Metric.h" // for metric
00015 #include "InputOutput.h" // for ScreenOutput
00016
00017 #include "DiagnosticsEmission.h" // Emission models
00018
00019 #include <cstdint> // for std::uint16_t
00020 #include <string> // for strings
00021 #include <memory> // for std::unique_ptr
00022 #include <vector> // for std::vector
00023
00024 // Forward declaration of Geodesic class needed here, since Diagnostics are passed a pointer to their
    owner Geodesic
00025 // (note "Geodesic.h" is NOT included to avoid header loop, and we do not need Geodesic member
    functions here!)
00026 class Geodesic;
00027
00030
00031 // Diagnostic bitflags
00032 // Used for constructing vector of Diagnostics
00033 // Note that this means every diagnostic is either "on" or "off";
00034 // it is not possible to have a Diagnostic "on" more than once
00035 // Note: why not std::bitset<size>? Because in this way we can use expressions with DiagBitflag in
    conditional expressions,
00036 // e.g. if ( mydiagbitflag & Diag_FourColorScreen)
00041 using DiagBitflag = std::uint16_t;
00042
00043 // Define a bitflag per existing diagnostic
00044 constexpr DiagBitflag Diag_None{0b0000'0000'0000'0000};
00045 constexpr DiagBitflag Diag_GeodesicPosition{0b0000'0000'0000'0001};
00046 constexpr DiagBitflag Diag_FourColorScreen{0b0000'0000'0000'0010};
00047 constexpr DiagBitflag Diag_EquatorialPasses{0b0000'0000'0000'0100};
00048 constexpr DiagBitflag Diag_ClosestRadius{0b0000'0000'0000'1000};
00049 constexpr DiagBitflag Diag_EquatorialEmission{0b0000'0000'0001'0000};
00050
00052 // Add a DiagBitflag for your new diagnostic. Make sure you use a bitflag that has not been used
    before!
00053 // Sample code:
00054 /*
00055 constexpr DiagBitflag Diag_MyDiag { 0b0000'0000'0000'1000 };
00056 */
00058
00063 struct UpdateFrequency
00064 {
00065     largecounter UpdateNSteps{0};
00066     bool UpdateStart{false};
00067     bool UpdateFinish{false};
00068 };
00069
00074 class Diagnostic
00075 {
00076 public:
```

```

00077     // Constructor must initialize the pointer to its owner Geodesic
00078     Diagnostic() = delete;
00079     Diagnostic(Geodesic *const theGeodesic) : m_OwnerGeodesic{theGeodesic}
00080     {
00081     }
00082
00083     // Resets Diagnostic object. This is called when the owner Geodesic is reset in order to start
integrating
00084     // a new geodesic.
00085     // The base class implementation only resets m_StepsSinceUpdated
00086     // Descendants can override this if they need to reset additional internal variables
00087     virtual void Reset();
00088
00089     // virtual destructor to ensure correct destruction of descendants
00090     virtual ~Diagnostic() = default;
00091
00092     // This is the heart of the Diagnostic. It calls the helper function DecideUpdate(), and if this
00093     // returns true, will update its internal status based on the current (new) state of its owner
geodesic.
00094     virtual void UpdateData() = 0;
00095
00096     // These functions are for use at the end of integration of a geodesic.
00097     // getFullData() returns all the data stored in the Diagnostic as a string (for output to file)
00098     virtual std::string getFullDataStr() const = 0;
00099     // getFinalDataVal() associates a value to the geodesic corresponding to the final value of this
diagnostic
00100     // (used for determining coarseness of nearby geodesics)
00101     virtual std::vector<real> getFinalDataVal() const = 0;
00102
00103     // Function used to determine distance between two values obtained from getFinalDataVal()
00104     // (for determining coarseness of nearby geodesics)
00105     // This should return a number >=0
00106     virtual real FinalDataValDistance(const std::vector<real> &val1, const std::vector<real> &val2)
const = 0;
00107
00108     // Getters for descriptions
00109     // This returns the name (only) of the Diagnostic, as a string without spaces that will be
appended
00110     // to an output file (e.g. prefix_DiagName.ext). Must be implemented!
00111     virtual std::string getNameStr() const = 0;
00112     // This returns the full description of the Diagnostic. Default implementation
00113     // returns GetDiagNameStr()
00114     virtual std::string getFullDescriptionStr() const;
00115
00116 protected:
00117     // The geodesic that owns the Diagnostic (a const pointer to the Geodesic)
00118     Geodesic *const m_OwnerGeodesic;
00119
00120     // Helper function to decide if the Diagnostic should indeed update its status, based on
00121     // its UpdateFrequency struct information (which will come from the Diagnostics's
DiagnosticOptions)
00122     bool DecideUpdate(const UpdateFrequency &myUpdateFrequency);
00123
00124     // The diagnostic is itself in charge of keeping track of how many steps it has been since it has
been updated
00125     // The Diagnostic's DiagnosticOptions struct tells it how many steps it needs to wait between
updates
00126     largecounter m_StepsSinceUpdated{};
00127 };
00128
00129 using DiagnosticUniqueVector = std::vector<std::unique_ptr<Diagnostic>;
00130
00131 DiagnosticUniqueVector CreateDiagnosticVector(DiagBitflag diagflags, DiagBitflag valdiag, Geodesic
*const theGeodesic);
00132
00133 class FourColorScreenDiagnostic final : public Diagnostic
00134 {
00135 public:
00136     // Basic constructor only passes on Geodesic pointer to base class constructor
00137     FourColorScreenDiagnostic(Geodesic *const theGeodesic) : Diagnostic(theGeodesic) {}
00138
00139     // Reset needs to be overridden in order to also reset m_quadrant
00140     void Reset() final;
00141
00142     // Sets the quadrant to the appropriate value, IF the boundary sphere has been reached
00143     void UpdateData() override;
00144
00145     // Both of these output functions simply returns the quadrant number associated with the
geodesic's end position
00146     std::string getFullDataStr() const final;
00147     std::vector<real> getFinalDataVal() const final;
00148
00149     // Discrete metric for distance: returns 0 if the quadrants are the same, 1 if they are not
00150     real FinalDataValDistance(const std::vector<real> &val1, const std::vector<real> &val2) const
final;
00151
00152

```

```

00163     // Description string getters
00164     std::string getNameStr() const final;
00165     std::string getFullDescriptionStr() const final;
00166
00167     // FourColorScreen does not need any (static) options!
00168
00169 private:
00171     int m_quadrant{0};
00172 };
00173
00175 struct GeodesicPositionOptions;
00179 class GeodesicPositionDiagnostic final : public Diagnostic
00180 {
00181 public:
00182     // Basic constructor only passes on Geodesic pointer to base class constructor
00183     GeodesicPositionDiagnostic(Geodesic *const theGeodesic) : Diagnostic(theGeodesic) {}
00184
00185     // Override Reset to also clear m_AllSavedPoints
00186     void Reset() final;
00187
00188     // Stores the current position of the geodesic
00189     void UpdateData() final;
00190
00191     // This returns as many stored positions as is specified in the options struct
00192     std::string getFullDataStr() const final;
00193     // This returns the final (theta,phi) value of the geodesic
00194     std::vector<real> getFinalDataVal() const final;
00195
00196     // This determines the angular distance between two geodesic
00197     // (based on their final angles on the boundary sphere (theta, phi))
00198     real FinalDataValDistance(const std::vector<real> &vall, const std::vector<real> &val2) const
00199     final;
00200
00201     // Description string getters
00202     std::string getNameStr() const final;
00203     std::string getFullDescriptionStr() const final;
00204
00205     // The options: specifies the update frequency but also how many points we want to output in the
00206     end
00207     static std::unique_ptr<GeodesicPositionOptions> DiagOptions;
00208
00209 private:
00210     // Keeps track of the points that are saved
00211     std::vector<Point> m_AllSavedPoints{};
00212 };
00213
00214 struct EquatorialPassesOptions;
00217 class EquatorialPassesDiagnostic : public Diagnostic
00218 {
00219 public:
00220     // Basic constructor only passes on Geodesic pointer to base class constructor
00221     EquatorialPassesDiagnostic(Geodesic *const theGeodesic) : Diagnostic(theGeodesic) {}
00222
00223     // Override Reset to also reset m_EquatPasses and m_PrevTheta
00224     void Reset() override;
00225
00226     // Checks to see if we have a new cross over the equatorial plane
00227     void UpdateData() override;
00228
00229     // Returns the number of passes over the equatorial plane
00230     std::string getFullDataStr() const override;
00231     std::vector<real> getFinalDataVal() const override;
00232
00233     // Simple absolute value of difference of passes
00234     real FinalDataValDistance(const std::vector<real> &vall, const std::vector<real> &val2) const
00235     override;
00236
00237     // Description string getters
00238     std::string getNameStr() const override;
00239     std::string getFullDescriptionStr() const override;
00240
00241     static std::unique_ptr<EquatorialPassesOptions> DiagOptions;
00242
00243 protected:
00244     int m_EquatPasses{0};
00245
00246 private:
00247     real m_PrevTheta{-1};
00248 };
00249
00250 // Forward declaration needed before Diagnostic
00251 struct ClosestRadiusOptions;
00252 class ClosestRadiusDiagnostic final : public Diagnostic
00253 {
00254 public:
00255     // Basic constructor only passes on Geodesic pointer to base class constructor
00256     ClosestRadiusDiagnostic(Geodesic *const theGeodesic) : Diagnostic(theGeodesic) {}

```

```

00262
00263     // re-initialize closest radius
00264     void Reset() final;
00265
00266     // Check to see if we have travelled closer
00267     void UpdateData() final;
00268
00269     // Return closest radius travelled
00270     std::string getFullDataStr() const final;
00271     std::vector<real> getFinalDataVal() const final;
00272
00273     // Returns the absolute value of the difference between closest radii
00274     real FinalDataValDistance(const std::vector<real> &val1, const std::vector<real> &val2) const
00275     final;
00276
00277     // Description string getters
00278     std::string getNameStr() const final;
00279     std::string getFullDescriptionStr() const final;
00280
00281     // Option struct to keep track of RLogScale of Metric
00282     static std::unique_ptr<ClosestRadiusOptions> DiagOptions;
00283 private:
00284     // Keeps track of closest radius travelled
00285     real m_ClosestRadius{-1};
00286 };
00287
00288 // Forward declaration needed before Diagnostic
00289 struct EquatorialEmissionOptions;
00290 class EquatorialEmissionDiagnostic final : public EquatorialPassesDiagnostic
00291 {
00292 public:
00293     // Basic constructor only passes on Geodesic pointer to parent class constructor
00294     EquatorialEmissionDiagnostic(Geodesic *const theGeodesic) :
00295     EquatorialPassesDiagnostic(theGeodesic) {}
00296
00297     // Reset intensity back to 0.0, and call parent class Reset()
00298     void Reset() final;
00299
00300     // Use parent class UpdateData() to decide whether the update the intensity
00301     // using the specified emission model
00302     void UpdateData() final;
00303
00304     // Returns total intensity and total number of equatorial passes (i.e. normal output from
00305     EquatorialPassesDiagnostic)
00306     std::string getFullDataStr() const final;
00307     std::vector<real> getFinalDataVal() const final;
00308
00309     // Distance is difference in intensities multiplied by a factor magnifying difference in
00310     equatorial passes
00311     real FinalDataValDistance(const std::vector<real> &val1, const std::vector<real> &val2) const
00312     final;
00313
00314     // Description string getters
00315     std::string getNameStr() const final;
00316     std::string getFullDescriptionStr() const final;
00317
00318     // Option struct to keep track emission model specifics
00319     static std::unique_ptr<EquatorialEmissionOptions> DiagOptions;
00320 private:
00321     // Brightness intensity of the geodesic
00322     real m_Intensity{0.0};
00323 };
00324
00325 // Declare your Diagnostic class here, inheriting from Diagnostic.
00326 // Sample code:
00327 /*
00328 // Forward declaration needed before Diagnostic (if using base class DiagnosticOptions, this is
00329 strictly speaking not necessary)
00330 struct DiagnosticOptions;
00331 // class definition
00332 class MyDiagnostic final : public Diagnostic // good practice to make the class final unless
00333 descendant classes are possible
00334 {
00335 public:
00336     // constructor must at least take and pass along the const pointer to the owner Geodesic
00337     MyDiagnostic(Geodesic* const theGeodesic) : Diagnostic(theGeodesic) {}
00338
00339     // If you have MyDiagnostic-specific member variables, then they probably need to be reset at the
00340     beginning
00341     // of integration for each new geodesic; in that case, you need to override Reset() to do so.
00342     // Note: make sure to call the base class implementation Diagnostic::Reset()
00343     // from within your implementation of MyDiagnostic::Reset(), so that the base class internal
00344     variable is also reset!
00345     void Reset() final;
00346

```

```

00347     // This is the heart of the Diagnostic: here the Diagnostic updates its internal state according
to
00348     // the owner Geodesic's current state
00349     void UpdateData() final;
00350
00351     // This should return the string that is to be outputted to the file as the final output of this
Diagnostic
00352     // for its owner Geodesic
00353     std::string getFullDataStr() const final;
00354     // This should return a vector of real numbers that indicates the final "value" that should be
associated to
00355     // the owner Geodesic --- this is then used in FinalDataValDistance() to find "distances" between
geodesics.
00356     std::vector<real> getFinalDataVal() const final;
00357
00358     // This should return a (positive) distance of two values returned by getFinalDataVal(), indicated
the
00359     // "distance" of two geodesics (this is used for Mesh refinement)
00360     real FinalDataValDistance(const std::vector<real>& val1, const std::vector<real>& val2) const
final;
00361
00362     // Must implement getNameStr() (and recommended also to implement getFullDescriptionStr)
00363     // getNameStr() is a simple, short string (without spaces) that will be appended to the file name
00364     // where this Diagnostic's output is written. getFullDescriptionStr() is a descriptive string that
should list
00365     // all relevant options set; this is outputted to e.g. the screen at runtime.
00366     std::string getNameStr() const final;
00367     std::string getFullDescriptionStr() const final;
00368
00369     // Diagnostic options: basic DiagnosticOptions only contains UpdateFrequency information,
00370     // if needed, can use a descendant class with more options (see e.g. GeodesicPositionDiagnostic)
00371     static std::unique_ptr<DiagnosticOptions> DiagOptions;
00372
00373 private:
00374     // will probably want some private member variable(s) to keep track of whatever geodesic property
is desired
00375 };
00376 */
00377
00378
00381
00385 struct DiagnosticOptions
00386 {
00387 public:
00388     // Basic constructor only sets the number of steps between updates
00389     DiagnosticOptions(UpdateFrequency thefrequency) : theUpdateFrequency{thefrequency}
00390     {
00391     }
00392
00393     virtual ~DiagnosticOptions() = default;
00394
00395     const UpdateFrequency theUpdateFrequency;
00396 };
00402 struct GeodesicPositionOptions : public DiagnosticOptions
00403 {
00404 public:
00405     GeodesicPositionOptions(largecounter outputsteps, UpdateFrequency thefrequency) :
OutputNrSteps{outputsteps},
00406     DiagnosticOptions(thefrequency)
00407     {
00408     }
00409
00410     const largecounter OutputNrSteps;
00411 };
00412
00418 struct EquatorialPassesOptions : public DiagnosticOptions
00419 {
00420 public:
00421     EquatorialPassesOptions(real thethreshold, UpdateFrequency thefrequency) :
Threshold{thethreshold},
00422     DiagnosticOptions(thefrequency)
00423     {
00424     }
00425
00426     const real Threshold;
00427 };
00428
00434 struct ClosestRadiusOptions : public DiagnosticOptions
00435 {
00436 public:
00437     ClosestRadiusOptions(bool rlog, UpdateFrequency thefrequency) : RLogScale{rlog},
DiagnosticOptions(thefrequency)
00438     {
00439     }
00440
00441
00442     const bool RLogScale;

```

```

00443 };
00444
00445 // Forward declarations of abstract EmissionModel and FluidVelocityModel classes.
00446 // See Diagnostics_Emission.h and .cpp for declarations and definitions of these and their descendant
    classes!
00447 struct EmissionModel;
00448 struct FluidVelocityModel;
00454 struct EquatorialEmissionOptions : public EquatorialPassesOptions
00455 {
00456 public:
00457     EquatorialEmissionOptions(real thefudgefactor, int equatupper,
00458                             std::unique_ptr<EmissionModel> theemission,
00459                             std::unique_ptr<FluidVelocityModel> thefluidmodel,
00460                             bool rlog, int theredshiftpower, real thethreshold, UpdateFrequency
    thefrequency) : RedShiftPower{theredshiftpower}, RLogScale{rlog},
    GeometricFudgeFactor{thefudgefactor},
00461     EquatPassUpperBound{equatupper},
00462     TheEmissionModel{std::move(theemission)}, TheFluidVelocityModel{std::move(thefluidmodel)},
00463     EquatorialPassesOptions(thethreshold, thefrequency) // constructor for parent class
00464     {
00465
00466         // Geometric fudge factor for n>0 passes
00467         const real GeometricFudgeFactor;
00468         // Upper bound allowed for contribution to intensity
00469         const int EquatPassUpperBound;
00470
00471         // Logarithmic radius scale used or not
00472         const bool RLogScale;
00473
00474         // Power of redshift in intensity contribution (should be 3 or 4)
00475         const int RedShiftPower;
00476
00477         // The emission model used (see Diagnostics_Emission.h and .cpp for specific models)
00478         const std::unique_ptr<EmissionModel> TheEmissionModel;
00479         // The fluid velocity model used (see Diagnostics_Emission.h and .cpp for specific models)
00480         const std::unique_ptr<FluidVelocityModel> TheFluidVelocityModel;
00481 };
00482
00484 // if necessary, define your new DiagnosticOptions class here
00485 // Sample code:
00486 /*
00487 struct MyDiagnosticOptions : public DiagnosticOptions
00488 {
00489 public:
00490     // Constructor should pass along UpdateFrequency information to base class
00491     MyDiagnosticOptions(UpdateFrequency thefrequency) : DiagnosticOptions(thefrequency) //, (...)
    other initializations
00492     {}
00493
00494     // other member variables here - make them const!
00495     // ...
00496 };
00497 */
00499
00500 #endif

```

7.10 FOORT/src/DiagnosticsEmission.cpp File Reference

```

#include "DiagnosticsEmission.h"
#include "Integrators.h"
#include <cmath>

```

7.10.1 Detailed Description

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-16

Copyright

Copyright (c) 2024

7.11 FOORT/src/DiagnosticsEmission.h File Reference

Declarations of emission and fluid velocity models used for equatorial disc emission.

```
#include "Geometry.h"
#include "Metric.h"
#include "InputOutput.h"
#include <string>
#include <cmath>
```

Classes

- struct [EmissionModel](#)
Emission model abstract base class.
- struct [GLMJohnsonSUEmission](#)
The Johnson SU emission model used in GLM.
- struct [FluidVelocityModel](#)
Fluid velocity model abstract base class.
- struct [GeneralCircularRadialFluid](#)
General circular radial fluid velocity model.

7.11.1 Detailed Description

Declarations of emission and fluid velocity models used for equatorial disc emission.

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-16

Copyright

Copyright (c) 2024

7.12 DiagnosticsEmission.h

[Go to the documentation of this file.](#)

```

00001 #ifndef _FOORT_DIAGNOSTICS_EMISSION_H
00002 #define _FOORT_DIAGNOSTICS_EMISSION_H
00003
00004 #include "Geometry.h" // Tensor objects
00005 #include "Metric.h" // for Metric functions
00006 #include "InputOutput.h" // for ScreenOutput
00007
00008 #include <string> // for strings
00009 #include <cmath> // for fmax, fmin
00010
00024 struct EmissionModel
00025 {
00026 public:
00027     // Virtual destructor to ensure correct destruction
00028     virtual ~EmissionModel() = default;
00029
00030     // Function which returns the emitted brightness intensity at Point p
00031     // Note: always pass the true radius (not log(r)) to EmissionModel!
00032     virtual real GetEmission(const Point &p) const = 0;
00033
00034     // Description string getter
00035     virtual std::string getFullDescriptionStr() const;
00036 };
00037
00042 struct GLMJohnsonSUEmission final : public EmissionModel
00043 {
00044 public:
00045     // Constructor
00046     GLMJohnsonSUEmission(real mu, real gamma, real sigma) : m_mu{mu}, m_gamma{gamma}, m_sigma{sigma}
00047     {
00048     }
00049
00050     // Emitted brightness (only depends on radius)
00051     real GetEmission(const Point &p) const final;
00052
00053     // Description string getter
00054     std::string getFullDescriptionStr() const final;
00055
00056 private:
00057     const real m_mu;
00060     const real m_gamma;
00062     const real m_sigma;
00063 };
00064
00066
00071 struct FluidVelocityModel
00072 {
00073 public:
00074     // Constructor is passed Metric pointer
00075     FluidVelocityModel(const Metric *const theMetric) : m_theMetric{theMetric} {}
00076
00077     // Virtual destructor to ensure correct destruction
00078     virtual ~FluidVelocityModel() = default;
00079
00080     // Get the local four-velocity (with index down!) of the fluid at Point p
00081     virtual OneIndex GetFourVelocityd(const Point &p) const = 0;
00082
00083     // Description string getter
00084     virtual std::string getFullDescriptionStr() const;
00085
00086 protected:
00087     // Metric pointer, used for e.g. calculating geodesic orbits
00088     const Metric *const m_theMetric;
00089 };
00090
00097 struct GeneralCircularRadialFluid final : public FluidVelocityModel
00098 {
00099     // Constructor with three parameters and Metric pointer (which is passed to base class
00100     constructor)
00101     GeneralCircularRadialFluid(real subKeplerParam, real betar, real betaphi, const Metric *const
00102     theMetric) : m_subKeplerParam{fmin(fmax(subKeplerParam, 0.0), 1.0)}, m_betaR{fmin(fmax(betar, 0.0),
00103     1.0)},
00104     m_betaPhi{fmin(fmax(betaphi, 0.0), 1.0)}, FluidVelocityModel(theMetric)
00105     {
00106         // Do some checks on three params, which must lie between 0.0 and 1.0 (note that they are
00107         adjusted as such in
00108         // initializer above)
00109         if (subKeplerParam < 0.0)
00110             ScreenOutput("Sub-Keplerian parameter must be between 0 and 1; adjusting to 0",
00111             OutputLevel::Level_0_WARNING);
00112         if (subKeplerParam > 1.0)

```

```

00108         ScreenOutput("Sub-Keplerian parameter must be between 0 and 1; adjusting to 1",
OutputLevel::Level_0_WARNING);
00109
00110         if (betar < 0.0)
00111             ScreenOutput("beta_r parameter must be between 0 and 1; adjusting to 0",
OutputLevel::Level_0_WARNING);
00112         if (betar > 1.0)
00113             ScreenOutput("beta_r parameter must be between 0 and 1; adjusting to 1",
OutputLevel::Level_0_WARNING);
00114
00115         if (betaphi < 0.0)
00116             ScreenOutput("beta_phi parameter must be between 0 and 1; adjusting to 0",
OutputLevel::Level_0_WARNING);
00117         if (betaphi > 1.0)
00118             ScreenOutput("beta_phi parameter must be between 0 and 1; adjusting to 1",
OutputLevel::Level_0_WARNING);
00119
00120         // Find the (equatorial) ISCO for this Metric
00121         FindISCO();
00122     }
00123
00124     // Get the local four-velocity of the fluid according to this model
00125     // Note: will always calculate with Point p exactly on the equator theta=pi/2, despite what p[2]
may be passed
00126     OneIndex GetFourVelocityd(const Point &p) const final;
00127
00128     // Description string getter
00129     std::string getFullDescriptionStr() const final;
00130
00131 private:
00132     // Three parameters determining the flow
00133     const real m_subKeplerParam;
00134     const real m_betaR;
00135     const real m_betaPhi;
00136
00137     // Helper function to get circular (sub)Keplerian velocity outside the ISCO
00138     OneIndex GetCircularVelocityd(const Point &p, bool subKeplerianOn = true) const;
00139     // Helper function to get circular and infalling (sub)Keplerian velocity inside ISCO
00140     // (according to prescription of Cunningham)
00141     OneIndex GetInsideISCOCircularVelocityd(const Point &p) const;
00142
00143     // Helper function to get radial infalling velocity
00144     OneIndex GetRadialVelocityd(const Point &p) const;
00145
00146     // Helper function to find the ISCO (called in Constructor)
00147     void FindISCO();
00148     bool m_ISCOexists{false};
00149     real m_ISCO{ -1.0 };
00150     real m_ISCOpt{};
00151     real m_ISCOpphi{};
00152
00153     // Helper function which returns  $\partial_r(g^{ab}\Gamma^c_{ab})$ ,
00154     // whose sign (after contracted with  $p_a p_d$  of the corresponding circular orbit)
00155     // tells us if the circular orbit considered is stable or not
00156     TwoIndex GetChristRaisingDer(real r) const;
00157 };
00158
00159 #endif

```

7.13 FOORT/src/Geodesic.cpp File Reference

```

#include "Geodesic.h"
#include "InputOutput.h"

```

7.13.1 Detailed Description

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-16

Copyright

Copyright (c) 2024

7.14 FOORT/src/Geodesic.h File Reference

Declarations of abstract base [Source](#) class and all its descendants. Declaration of [Geodesic](#) class.

```
#include "Geometry.h"
#include "Metric.h"
#include "Diagnostics.h"
#include "Terminations.h"
#include "Integrators.h"
#include <string>
#include <vector>
```

Classes

- class [Source](#)
Abstract [Source](#) base class.
- class [NoSource](#)
[NoSource](#) class.
- class [Geodesic](#)
[Geodesic](#) class: an instance of this class is created for each [Geodesic](#) that is integrated.

7.14.1 Detailed Description

Declarations of abstract base [Source](#) class and all its descendants. Declaration of [Geodesic](#) class.

Author

Daniel R. Mayerson

Version

1.0

Date

2024-12-16

Copyright

Copyright (c) 2024

7.15 Geodesic.h

[Go to the documentation of this file.](#)

```

00001 #ifndef _FOORT_GEODESIC_H
00002 #define _FOORT_GEODESIC_H
00003
00004 #include "Geometry.h" // for tensor objects
00005 #include "Metric.h" // for the metric
00006 #include "Diagnostics.h" // Geodesics own Diagnostics
00007 #include "Terminations.h" // Geodesics own Terminations
00008 #include "Integrators.h" // Geodesics use an GeodesicIntegratorFunc to integrate itself
00009
00010 #include <string> // for strings
00011 #include <vector> // for std::vector
00012
00024
00029 class Source
00030 {
00031 public:
00032     // Constructor initializes Metric
00033     Source(const Metric *const theMetric) : m_theMetric{theMetric} {}
00034
00035     // Virtual destructor to ensure correct descendant destruction
00036     virtual ~Source() = default;
00037
00038     // Get the source for the current geodesic position and velocity
00039     virtual OneIndex getSource(Point pos, OneIndex vel) const = 0;
00040
00041     // Full description string (space allowed), to be outputted to file
00042     virtual std::string getFullDescriptionStr() const;
00043
00044 protected:
00045     const Metric *const m_theMetric;
00046 };
00047
00048
00054 class NoSource final : public Source
00055 {
00056 public:
00057     // Simple constructor passes on the Metric pointer to the base constructor
00058     NoSource(const Metric *const theMetric) : Source(theMetric) {}
00059
00060     // Returns zero source
00061     OneIndex getSource(Point pos, OneIndex vel) const final;
00062
00063     // Description string getter
00064     std::string getFullDescriptionStr() const final;
00065 };
00066
00069
00076 class Geodesic
00077 {
00078 public:
00079     // Default constructor not allowed
00080     Geodesic() = delete;
00081     // Copy constructor or copy assignment not allowed
00082     Geodesic(const Geodesic &) = delete;
00083     Geodesic &operator=(const Geodesic &) = delete;
00084
00085     Geodesic(const Metric *const theMetric, const Source *const theSource,
00086             DiagBitflag diagbit, DiagBitflag valdiagbit,
00087             TermBitflag termbit, GeodesicIntegratorFunc theIntegrator) : m_theMetric{theMetric},
00088             m_theSource{theSource},
00089             m_AllDiagnostics{CreateDiagnosticVector(diagbit, valdiagbit, this)},
00090             m_AllTerminations{CreateTerminationVector(termbit, this)},
00091             m_theIntegrator{theIntegrator}
00092     {
00093     }
00094
00095     // This initializes/resets the geodesic with a given ScreenIndex, initial position, and initial
00096     // velocity
00097     // Also resets all Diagnostics and Terminations, resets the TermCondition to Term::Continue,
00098     // and puts the Geodesic back to lambda = 0.0.
00099     void Reset(ScreenIndex scrindex, Point initpos, OneIndex initvel);
00100
00101     // This makes the Geodesic integrate itself one step; then the Geodesic loops through all
00102     // Terminations and Diagnostics to update
00103     Term Update();
00104
00105     // Getters for properties of its internal state
00106     Term getTermCondition() const; // Current termination condition (Term::Continue if not done
00107     // integrating)
00108     Point getCurrentPos() const; // Current position

```

```

00115     OneIndex getCurrentVel() const;      // Current velocity
00116     real getCurrentLambda() const;      // Current value of affine parameter
00117     ScreenIndex getScreenIndex() const; // screen index
00118
00119     // Output getters, to be called after the Geodesic terminates
00120     // This gets the complete output that should be written to the output files;
00121     // there is one string more than the count of Diagnostics: one string per Diagnostic,
00122     // PLUS the first string is the screen index.
00123     std::vector<std::string> getAllOutputStr() const;
00124     // This returns the "value" (from the Diagnostic that was set to the value Diagnostic) that is
    associated
00125     // to the Geodesic. Will be used to determine "distance" between Geodesics which is used in Mesh
    refinement.
00126     std::vector<real> getDiagnosticFinalValue() const;
00127
00128 private:
00129     // These variables define its internal state
00131     Term m_TermCond{Term::Uninitialized};
00133     Point m_CurrentPos{};
00135     OneIndex m_CurrentVel{};
00137     real m_curLambda{0.0};
00138
00141     ScreenIndex m_ScreenIndex{};
00142
00143     // These are const pointers (or const vectors of pointers) that contain all the information the
    Geodesic needs
00145     const Metric *const m_theMetric;
00147     const Source *const m_theSource;
00149     const DiagnosticUniqueVector m_AllDiagnostics;
00151     const TerminationUniqueVector m_AllTerminations;
00153     const GeodesicIntegratorFunc m_theIntegrator;
00154 };
00155
00156 #endif

```

7.16 FOORT/src/Geometry.h File Reference

Definitions and some operations with geometric objects.alignas.

```

#include <limits>
#include <string>
#include <array>
#include <utility>
#include <vector>

```

Macros

- #define `LARGECOUNTER_MAX` `std::numeric_limits<largecounter>::max()`
Macro definition of maximum value that can be held in this large counter.
- #define `PIXEL_MAX` `std::numeric_limits<pixelcoord>::max()`
Macro definition of maximum value that can be held in this large counter.

Typedefs

- using `real` = `double`
A real number (could be changed to use arbitrary precision in the future).
- using `largecounter` = `unsigned long`
This type is used to count geodesics integrated.
- using `pixelcoord` = `largecounter`
A pixel coordinate: always ≥ 0 and integer; we use the largecounter type.
- using `Point` = `std::array<real, dimension>`
A point in spacetime. Note that coordinates are always assumed to be (t, r, theta, phi)

- using `ScreenPoint` = `std::array<real, dimension - 2>`
A point on the `ViewScreen`; this does not have a time or radial extent.
- using `ScreenIndex` = `std::array<pixelcoord, dimension - 2>`
An index on the `ViewScreen` (row, column)
- using `OneIndex` = `Point`
Object with one index has the same structure as a `Point`.
- using `TwoIndex` = `std::array<OneIndex, dimension>`
Object with two indices is an array of `OneIndex` objects.
- using `ThreeIndex` = `std::array<TwoIndex, dimension>`
Object with three indices is an array of `TwoIndex` objects.
- using `FourIndex` = `std::array<ThreeIndex, dimension>`
Object with four indices is an array of `ThreeIndex` objects.
- using `SingularityCoord` = `std::pair<int, real>`
SingularityCoord: pair of (coordinate number, coordinate value)
- using `Singularity` = `std::vector<SingularityCoord>`
Singularity: a number of `SingularityCoords` together that define a arbitrary codimension surface/line/point.

Functions

- `template<size_t TensorDim>`
`std::string toString (const std::array< largecounter, TensorDim > &theTensor)`
Base case for single index tensor of unsigned integers (`ScreenIndex`).
- `template<size_t TensorDim>`
`std::string toString (const std::array< real, TensorDim > &theTensor)`
Base case for single index tensor of reals (`Point`, `OneIndex`, `ScreenPoint`).
- `template<typename Tensor, size_t TensorDim>`
`std::string toString (const std::array< Tensor, TensorDim > &theTensor)`
General case for tensors with more than one index (`TwoIndex`, `ThreeIndex`, `FourIndex`).
- `template<typename t, size_t TensorDim>`
`std::array< t, TensorDim > operator+ (const std::array< t, TensorDim > &a1, const std::array< t, TensorDim > &a2)`
Function to recursively call + on the lower rank tensor (OR the underlying reals/ints, if the tensor is rank 1)
- `template<typename t, size_t TensorDim>`
`std::array< t, TensorDim > operator- (const std::array< t, TensorDim > &a1, const std::array< t, TensorDim > &a2)`
Function to recursively call - on the lower rank tensor (OR the underlying reals/ints, if the tensor is rank 1)
- `template<typename t, size_t TensorDim>`
`std::array< t, TensorDim > operator* (const std::array< t, TensorDim > &t1, real lambda)`
Function to recursively scalar multiply the lower-rank tensors (OR the underlying reals/ints for the rank-1 tensor)
- `template<typename t, size_t TensorDim>`
`std::array< t, TensorDim > operator* (real lambda, const std::array< t, TensorDim > &t1)`
Function to recursively scalar multiply the lower-rank tensors (OR the underlying reals/ints for the rank-1 tensor)
- `template<typename t, size_t TensorDim>`
`std::array< t, TensorDim > operator/ (const std::array< t, TensorDim > &t1, real lambda)`
Function to recursively scalar divide the lower-rank tensors (OR the underlying reals/ints for the rank-1 tensor)

Variables

- `constexpr real pi {3.1415926535}`
The value of pi.
- `constexpr int dimension {4}`
The spacetime dimension.

7.16.1 Detailed Description

Definitions and some operations with geometric objects.

Author

Daniel R. Mayerson

Defines Point, tensors with 1-4 indices, and the operator toString for them. Also defines basic tensor arithmetic (+, -, *, /). No .cpp with implementations; all functions are inline.

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.16.2 Macro Definition Documentation

7.16.2.1 LARGE_COUNTER_MAX

```
#define LARGE_COUNTER_MAX std::numeric_limits<largecounter>::max()
```

Macro definition of maximum value that can be held in this large counter.

7.16.2.2 PIXEL_MAX

```
#define PIXEL_MAX std::numeric_limits<pixelcoord>::max()
```

Macro definition of maximum value that can be held in this large counter.

7.16.3 Typedef Documentation

7.16.3.1 FourIndex

```
using FourIndex = std::array<ThreeIndex, dimension>
```

Object with four indices is an array of ThreeIndex objects.

7.16.3.2 largecounter

```
using largecounter = unsigned long
```

This type is used to count geodesics integrated.

7.16.3.3 OneIndex

```
using OneIndex = Point
```

Object with one index has the same structure as a Point.

7.16.3.4 pixelcoord

```
using pixelcoord = largecounter
```

A pixel coordinate: always ≥ 0 and integer; we use the largecounter type.

7.16.3.5 Point

```
using Point = std::array<real, dimension>
```

A point in spacetime. Note that coordinates are always assumed to be (t, r, theta, phi)

7.16.3.6 real

```
using real = double
```

A real number (could be changed to use arbitrary precision in the future).

7.16.3.7 ScreenIndex

```
using ScreenIndex = std::array<pixelcoord, dimension - 2>
```

An index on the [ViewScreen](#) (row, column)

7.16.3.8 ScreenPoint

```
using ScreenPoint = std::array<real, dimension - 2>
```

A point on the [ViewScreen](#); this does not have a time or radial extent.

7.16.3.9 Singularity

```
using Singularity = std::vector<SingularityCoord>
```

Singularity: a number of SingularityCoords together that define an arbitrary codimension surface/line/point.

7.16.3.10 SingularityCoord

```
using SingularityCoord = std::pair<int, real>
```

SingularityCoord: pair of (coordinate number, coordinate value)

7.16.3.11 ThreeIndex

```
using ThreeIndex = std::array<TwoIndex, dimension>
```

Object with three indices is an array of TwoIndex objects.

7.16.3.12 TwoIndex

```
using TwoIndex = std::array<OneIndex, dimension>
```

Object with two indices is an array of OneIndex objects.

7.16.4 Function Documentation

7.16.4.1 operator*() [1/2]

```
template<typename t , size_t TensorDim>
std::array< t, TensorDim > operator* (
    const std::array< t, TensorDim > & t1,
    real lambda)
```

Function to recursively scalar multiply the lower-rank tensors (OR the underlying reals/ints for the rank-1 tensor)

Template Parameters

<i>t</i>	Type of the tensor
<i>TensorDim</i>	Dimension of the tensor

Parameters

<i>t1</i>	Tensor
<i>lambda</i>	Scalar

Returns

std::array<t, TensorDim> Scalar product of t1 and lambda

7.16.4.2 operator*() [2/2]

```
template<typename t , size_t TensorDim>
std::array< t, TensorDim > operator* (
    real lambda,
    const std::array< t, TensorDim > & t1)
```

Function to recursively scalar multiply the lower-rank tensors (OR the underlying reals/ints for the rank-1 tensor)

This function calls the right multiplication operator.

Template Parameters

<i>t</i>	Type of the tensor
<i>TensorDim</i>	Dimension of the tensor

Parameters

<i>lambda</i>	Scalar
<i>t1</i>	Tensor

Returns

`std::array<t, TensorDim>` Scalar product of lambda and t1

7.16.4.3 `operator+()`

```
template<typename t , size_t TensorDim>
std::array< t, TensorDim > operator+ (
    const std::array< t, TensorDim > & a1,
    const std::array< t, TensorDim > & a2)
```

Function to recursively call + on the lower rank tensor (OR the underlying reals/ints, if the tensor is rank 1)

Template Parameters

<i>t</i>	data type of the tensor
<i>TensorDim</i>	Dimension of the tensor

Parameters

<i>a1</i>	Tensor 1
<i>a2</i>	Tensor 2

Returns

`std::array<t, TensorDim>` Tensor sum of a1 and a2

7.16.4.4 `operator-()`

```
template<typename t , size_t TensorDim>
std::array< t, TensorDim > operator- (
    const std::array< t, TensorDim > & a1,
    const std::array< t, TensorDim > & a2)
```

Function to recursively call - on the lower rank tensor (OR the underlying reals/ints, if the tensor is rank 1)

Template Parameters

<i>t</i>	data type of the tensor
<i>TensorDim</i>	Dimension of the tensor

Parameters

<i>a1</i>	Tensor 1
<i>a2</i>	Tensor 2

Returns

std::array<t, TensorDim> Tensor difference of a1 and a2

7.16.4.5 operator/()

```
template<typename t , size_t TensorDim>
std::array< t, TensorDim > operator/ (
    const std::array< t, TensorDim > & t1,
    real lambda)
```

Function to recursively scalar divide the lower-rank tensors (OR the underlying reals/ints for the rank-1 tensor)

This function calls the right multiplication operator.

Template Parameters

<i>t</i>	Type of the tensor
<i>TensorDim</i>	Dimension of the tensor

Parameters

<i>t1</i>	Tensor
<i>lambda</i>	Scalar

Returns

std::array<t, TensorDim> Scalar division of t1 by lambda

7.16.4.6 toString() [1/3]

```
template<size_t TensorDim>
std::string toString (
    const std::array< largecounter, TensorDim > & theTensor)
```

Base case for single index tensor of unsigned integers (ScreenIndex).

We do not want toString(ScreenIndex) to convert its entries to reals and use the implementation for a single index tensor of reals, because we do not want decimal points in our string for the ints!

Template Parameters

<i>TensorDim</i>	Dimension of the tensor
------------------	-------------------------

Parameters

<i>theTensor</i>	Tensor object
------------------	---------------

Returns

std::string

7.16.4.7 toString() [2/3]

```
template<size_t TensorDim>
std::string toString (
    const std::array< real, TensorDim > & theTensor)
```

Base case for single index tensor of reals (Point, OneIndex, ScreenPoint).

Template Parameters

<i>TensorDim</i>	Dimension of the tensor
------------------	-------------------------

Parameters

<i>theTensor</i>	Tensor object
------------------	---------------

Returns

std::string

7.16.4.8 toString() [3/3]

```
template<typename Tensor , size_t TensorDim>
std::string toString (
    const std::array< Tensor, TensorDim > & theTensor)
```

General case for tensors with more than one index (TwoIndex, ThreeIndex, FourIndex).

This function recursively calls the lower rank tensor to print itself.

Template Parameters

<i>Tensor</i>	The type of the tensor
<i>TensorDim</i>	Dimension of the tensor

Parameters

<i>theTensor</i>	Tensor object
------------------	---------------

Returns

std::string

7.16.5 Variable Documentation

7.16.5.1 dimension

```
int dimension {4} [inline], [constexpr]
```

The spacetime dimension.

7.16.5.2 pi

```
real pi {3.1415926535} [inline], [constexpr]
```

The value of pi.

7.17 Geometry.h

[Go to the documentation of this file.](#)

```
00001 #ifndef _FOORT_GEOMETRY_H
00002 #define _FOORT_GEOMETRY_H
00003
00015 #include <limits> // for std::numeric_limits
00016 #include <string> // needed for toString(...) to convert tensors to strings
00017 #include <array> // needed to define tensors as fixed-size arrays of real or pixelcoord
00018 #include <utility> // needed for std::pair
00019 #include <vector> // for std::vector
00020
00022 using real = double;
00023
00024 // Note: An unsigned long is guaranteed to be able to hold at least 4 294 967 295 (4.10^10).
00025 // An unsigned int is only guaranteed to be able to hold 65 535, although
00026 // many modern-day implementations will actually make the int 32-bit and so much larger
00027
00029 using largecounter = unsigned long;
00031 using pixelcoord = largecounter;
00032
00034 #ifndef LARGE_COUNTER_MAX
00035 #define LARGE_COUNTER_MAX std::numeric_limits<largecounter>::max()
00036 #endif
00038 #ifndef PIXEL_MAX
00039 #define PIXEL_MAX std::numeric_limits<pixelcoord>::max()
00040 #endif
00041
00042 // CONSTANTS
00043
00045 inline constexpr real pi{3.1415926535};
00046
00048 inline constexpr int dimension{4};
00049
00050 // TENSOR DEFINITIONS
00051
00053 using Point = std::array<real, dimension>;
00054
00056 using ScreenPoint = std::array<real, dimension - 2>;
00057
00059 using ScreenIndex = std::array<pixelcoord, dimension - 2>;
```

```

00060
00062 using OneIndex = Point;
00064 using TwoIndex = std::array<OneIndex, dimension>;
00066 using ThreeIndex = std::array<TwoIndex, dimension>;
00068 using FourIndex = std::array<ThreeIndex, dimension>;
00069
00070 // Definition used to define singularity of arbitrary codimension
00072 using SingularityCoord = std::pair<int, real>;
00074 using Singularity = std::vector<SingularityCoord>;
00075
00076 // PRINTING TENSORS TO STRING
00077
00087 template <size_t TensorDim>
00088 std::string toString(const std::array<largecounter, TensorDim> &theTensor)
00089 {
00090     std::string theStr{"("}; // no spaces for the innermost brackets
00091     for (int i = 0; i < TensorDim - 1; ++i)
00092     {
00093         // Here we use the std::to_string to convert the real to string
00094         theStr += std::to_string(theTensor[i]);
00095         theStr += ", ";
00096     }
00097     theStr += std::to_string(theTensor[TensorDim - 1]);
00098     theStr += ")"; // no spaces for the innermost brackets
00099
00100     return theStr;
00101 }
00102
00110 template <size_t TensorDim>
00111 std::string toString(const std::array<real, TensorDim> &theTensor)
00112 {
00113     std::string theStr{"("}; // no spaces for the innermost brackets
00114
00115     for (int i = 0; i < TensorDim - 1; ++i)
00116     {
00117         // Here we use the std::to_string to convert the real to string
00118         theStr += std::to_string(theTensor[i]);
00119         theStr += ", ";
00120     }
00121     theStr += std::to_string(theTensor[TensorDim - 1]);
00122     theStr += ")"; // no spaces for the innermost brackets
00123
00124     return theStr;
00125 }
00126
00135 template <typename Tensor, size_t TensorDim>
00136 std::string toString(const std::array<Tensor, TensorDim> &theTensor)
00137 {
00138     std::string theStr{" ("}; // All but the innermost brackets have an extra space padding the
00139     // bracket
00140     for (int i = 0; i < TensorDim - 1; ++i)
00141     {
00142         theStr += toString(theTensor[i]);
00143         theStr += ", ";
00144     }
00145     theStr += toString(theTensor[TensorDim - 1]); // the last element doesn't have a comma after it
00146
00147     theStr += ")"; // All but the innermost brackets have an extra space padding the bracket
00148
00149     return theStr;
00150 }
00151
00152 // TENSOR ARITHMETIC: addition/subtraction of tensors, scalar multiplication/division
00153
00163 template <typename t, size_t TensorDim>
00164 std::array<t, TensorDim> operator+(const std::array<t, TensorDim> &a1, const std::array<t, TensorDim>
00165 &a2)
00166 {
00167     std::array<t, TensorDim> temp{a1};
00168     for (int i = 0; i < TensorDim; ++i)
00169         temp[i] = temp[i] + a2[i];
00170
00171     return temp;
00172 }
00173
00182 template <typename t, size_t TensorDim>
00183 std::array<t, TensorDim> operator-(const std::array<t, TensorDim> &a1, const std::array<t, TensorDim>
00184 &a2)
00185 {
00186     std::array<t, TensorDim> temp{a1};
00187     for (int i = 0; i < TensorDim; ++i)
00188         temp[i] = temp[i] - a2[i];
00189
00190     return temp;
00191 }

```

```

00201 template <typename t, size_t TensorDim>
00202 std::array<t, TensorDim> operator*(const std::array<t, TensorDim> &t1, real lambda)
00203 {
00204     std::array<t, TensorDim> temp{t1};
00205     for (int i = 0; i < TensorDim; ++i)
00206         temp[i] = static_cast<t>(temp[i] * lambda); // static_cast necessary if t is integral type!
00207     return temp;
00208 }
00209
00219 template <typename t, size_t TensorDim>
00220 std::array<t, TensorDim> operator*(real lambda, const std::array<t, TensorDim> &t1)
00221 {
00222     return t1 * lambda;
00223 }
00224
00234 template <typename t, size_t TensorDim>
00235 std::array<t, TensorDim> operator/(const std::array<t, TensorDim> &t1, real lambda)
00236 {
00237     return t1 * (1 / lambda);
00238 }
00239
00240 #endif

```

7.18 FOORT/src/InOutOutput.cpp File Reference

```

#include "InOutOutput.h"
#include <algorithm>
#include <filesystem>

```

Functions

- void [SetOutputLevel](#) ([OutputLevel](#) theLvl)
Set the Output Level.
- void [SetLoopMessageFrequency](#) ([largecounter](#) thefreq)
Set the Loop Message Frequency.
- [largecounter](#) [GetLoopMessageFrequency](#) ()
Get the Loop Message Frequency during integration loops.
- void [ScreenOutput](#) (std::string_view theOutput, [OutputLevel](#) lvl, bool newLine)
Outputs line to screen console, contingent on it being allowed by the set outputlevel.

7.18.1 Detailed Description

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.18.2 Function Documentation

7.18.2.1 GetLoopMessageFrequency()

```
largecounter GetLoopMessageFrequency ()
```

Get the Loop Message Frequency during integration loops.

Returns

largecounter

7.18.2.2 ScreenOutput()

```
void ScreenOutput (
    std::string_view theOutput,
    OutputLevel lvl,
    bool newLine)
```

Outputs line to screen console, contingent on it being allowed by the set outputlevel.

Defaults are lvl = `OutputLevel::Level_3_ALLDETAIL` and newLine = true

Parameters

<i>theOutput</i>	output to be put on the screen
<i>lvl</i>	the output level
<i>newLine</i>	indicates whether we want a new line after the output

7.18.2.3 SetLoopMessageFrequency()

```
void SetLoopMessageFrequency (
    largecounter thefreq)
```

Set the Loop Message Frequency.

Parameters

<i>thefreq</i>	the desired frequency
----------------	-----------------------

7.18.2.4 SetOutputLevel()

```
void SetOutputLevel (
    OutputLevel theLvl)
```

Set the Output Level.

Parameters

<i>theLvl</i>	The desired level
---------------	-------------------

7.19 FOORT/src/InOutOutput.h File Reference

Declarations of functions & classes that deal with the output to files and to the screen.

```
#include "Geometry.h"
#include <string_view>
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
```

Classes

- class [GeodesicOutputHandler](#)
// [GeodesicOutputHandler](#) handles all of the output to file. It gets passed all of the output strings for every [Geodesic](#), it then stores this data until it eventually writes all data to the appropriate files

Enumerations

- enum class [OutputLevel](#) {
[Level_0_WARNING](#) = 0 , [Level_1_PROC](#) = 1 , [Level_2_SUBPROC](#) = 2 , [Level_3_ALLDDETAIL](#) = 3 ,
[Level_4_DEBUG](#) = 4 , [MaxLevel](#) }

The output level determines at what priority level the output is generated to the console.

Functions

- void [SetOutputLevel](#) ([OutputLevel](#) theLvl)
Set the Output Level.
- void [ScreenOutput](#) (std::string_view theOutput, [OutputLevel](#) lvl=[OutputLevel::Level_3_ALLDDETAIL](#), bool newLine=true)
Outputs line to screen console, contingent on it being allowed by the set outputlevel.
- void [SetLoopMessageFrequency](#) ([largecounter](#) thefreq)
Set the Loop Message Frequency.
- [largecounter](#) [GetLoopMessageFrequency](#) ()
Get the Loop Message Frequency during integration loops.

7.19.1 Detailed Description

Declarations of functions & classes that deal with the output to files and to the screen.

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.19.2 Enumeration Type Documentation

7.19.2.1 OutputLevel

```
enum class OutputLevel [strong]
```

The output level determines at what priority level the output is generated to the console.

Level_0_WARNING: Only warnings are outputted. Level_1_PROC: Coarsest level output; only the major procedures produce output. Level_2_SUBPROC: Subprocedures can also produce output. Level_3_ALLDETAIL: Finest level output; all details are shown. Level_4_DEBUG: Finest level output AND debug messages as well.

Enumerator

Level_0_WARNING	
Level_1_PROC	
Level_2_SUBPROC	
Level_3_ALLDETAIL	
Level_4_DEBUG	
MaxLevel	

7.19.3 Function Documentation

7.19.3.1 GetLoopMessageFrequency()

```
largecounter GetLoopMessageFrequency ()
```

Get the Loop Message Frequency during integration loops.

Returns

largecounter

7.19.3.2 ScreenOutput()

```
void ScreenOutput (
    std::string_view theOutput,
    OutputLevel lvl,
    bool newLine)
```

Outputs line to screen console, contingent on it being allowed by the set outputlevel.

Defaults are lvl = `OutputLevel::Level_3_ALLDETAIL` and newLine = true

Parameters

<i>theOutput</i>	output to be put on the screen
<i>lvl</i>	the output level
<i>newLine</i>	indicates whether we want a new line after the output

7.19.3.3 SetLoopMessageFrequency()

```
void SetLoopMessageFrequency (
    largecounter thefreq)
```

Set the Loop Message Frequency.

Parameters

<i>thefreq</i>	the desired frequency
----------------	-----------------------

7.19.3.4 SetOutputLevel()

```
void SetOutputLevel (
    OutputLevel theLvl)
```

Set the Output Level.

Parameters

<i>theLvl</i>	The desired level
---------------	-------------------

7.20 InputOutput.h

[Go to the documentation of this file.](#)

```

00001 #ifndef _FOORT_INPUTOUTPUT_H
00002 #define _FOORT_INPUTOUTPUT_H
00003
00013 #include "Geometry.h" // for basic tensor objects
00014
00015 #include <string_view> // std::string_view
00016 #include <iostream>    // needed for file and console output
00017 #include <fstream>     // needed for file output
00018 #include <string>       // std::string used in various places
00019 #include <vector>       // needed to create vectors of strings
00020
00021 // SCREENOUTPUT
00022
00031 enum class OutputLevel
00032 {
00033     Level_0_WARNING = 0,
00034     Level_1_PROC = 1,
00035     Level_2_SUBPROC = 2,
00036     Level_3_ALLDDETAIL = 3,
00037     Level_4_DEBUG = 4,
00038
00039     MaxLevel // (unused)
00040 };
00041
00042 // Set the output level used; note: the output level itself is a static variable in InputOutput.cpp
00043 void SetOutputLevel(OutputLevel theLvl);
00044
00045 // Outputs line to screen console), contingent on it being allowed by the set outputlevel
00046 void ScreenOutput(std::string_view theOutput, OutputLevel lvl = OutputLevel::Level_3_ALLDDETAIL, bool
newLine = true);
00047
00048 // Set and Get for the loop message frequency (messages indicating progress during each integration
loop)
00049 void SetLoopMessageFrequency(largecounter thefreq);
00050 largecounter GetLoopMessageFrequency();
00051
00052 // FILE OUTPUT
00053
00059 class GeodesicOutputHandler
00060 {
00061 public:
00062     // No default constructor possible
00063     GeodesicOutputHandler() = delete;
00064     // Constructor must pass the following strings that are used to construct the file names of the
output file:
00065     // FilePrefix, TimeStamp, FileExtension, and a vector of strings DiagNames (the names of each of
the Diagnostics that will
00066     // be outputting). It must also specify how many outputs to cache before outputting to a file, and
00067     // how many geodesics are allowed per file created
00068     GeodesicOutputHandler(std::string FilePrefix, std::string TimeStamp, std::string FileExtension,
std::vector<std::string> DiagNames,
00069         largecounter nrouputstocache = LARGE_COUNTER_MAX - 1, // note -1,
// since we will
00070         actually cache one more than this number before outputting everything
00071         largecounter geodperfile = LARGE_COUNTER_MAX,
std::string firstlineinfo = "");
00072
00073     // This tells the OutputHandler to prepare for this many geodesic outputs to arrive;
00074     // the internal state needs to be prepared such that they can come in without providing a data
race
00075     void PrepareForOutput(largecounter nrOutputToCome);
00076
00077     // A new vector of output strings from a (single) Geodesic;
00078     // the length of the vector should be m_DiagNames.size()+1, since the first entry
00079     // is the screen index
00080     // NOTE: this procedure needs to be thread-safe!
00081     void NewGeodesicOutput(largecounter index, std::vector<std::string> theOutput);
00082
00083     // Calling this indicates that there is no further output to be expected;
00084     // this means we will write all remaining cached output to file
00085     void OutputFinished();
00086
00087     // Returns full description string of output handler
00088     std::string getFullDescriptionStr() const;
00089
00090 private:
00091     // Helper function: write everything that is cached to file now (clear the cache)
00092     void WriteCachedOutputToFile();
00093
00094     // Helper function: open the file with name filename for the first time, preparing it to write
00095     // This will effectively clear this file of any pre-existing content.
00096     void OpenForFirstTime(const std::string &filename);

```

```

00099
00100 // Helper function: return the full file name for the n-th output file
00101 // for the Diagnostic diagnr (this is an entry in m_DiagNames)
00102 std::string GetFileName(int diagnr, unsigned short filenr) const;
00103
00104 // The const strings that are used to construct the output file names
00106 const std::string m_FilePrefix;
00108 const std::string m_TimeStamp;
00110 const std::string m_FileExtension;
00112 const std::vector<std::string> m_DiagNames;
00113
00115 const bool m_PrintFirstLineInfo;
00117 const std::string m_FirstLineInfoString;
00118
00119 // Const setting the maximum number of outputs that can be cached before writing output to file.
00120 const largecounter m_nrOutputsToCache{};
00122 const largecounter m_nrGeodesicsPerFile{};
00123
00125 bool m_WriteToConsole{false};
00126
00128 largecounter m_PrevCached{0};
00129
00131 largecounter m_CurrentGeodesicsInFile{0};
00132
00134 unsigned short m_CurrentFullFiles{0};
00135
00137 std::vector<std::vector<std::string> m_AllCachedData{};
00138 };
00139
00140 #endif

```

7.21 FOORT/src/Integrators.cpp File Reference

```

#include "Integrators.h"
#include "Geodesic.h"
#include <algorithm>
#include <cmath>
#include <sstream>
#include <iostream>

```

7.21.1 Detailed Description

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.22 FOORT/src/Integrators.h File Reference

Declarations of integrator functions that integrate geodesic equation.

```
#include "Geometry.h"
#include "Metric.h"
#include <string>
```

Namespaces

- namespace [Integrators](#)

Typedefs

- using [GeodesicIntegratorFunc](#) = void (*)([Point](#), [OneIndex](#), [Point](#) &, [OneIndex](#) &, [real](#) &, const [Metric](#) *, const [Source](#) *)

Functions

- std::string [Integrators::GetFullIntegratorDescription](#) ()
Description string getter for the Integrator.
- [real](#) [Integrators::GetAdaptiveStep](#) ([Point](#) curpos, [OneIndex](#) curvel)
Determine the (affine parameter) step size to take.
- void [Integrators::IntegrateGeodesicStep_RK4](#) ([Point](#) curpos, [OneIndex](#) curvel, [Point](#) &nextpos, [OneIndex](#) &nextvel, [real](#) &stepsize, const [Metric](#) *theMetric, const [Source](#) *theSource)
Integrate the geodesic equation by one step using Runge-Kutta-4.
- void [Integrators::IntegrateGeodesicStep_Verlet](#) ([Point](#) curpos, [OneIndex](#) curvel, [Point](#) &nextpos, [OneIndex](#) &nextvel, [real](#) &stepsize, const [Metric](#) *theMetric, const [Source](#) *theSource)
Integrate the geodesic equation by one step using the Verlet algorithm.

Variables

- constexpr [real](#) [Integrators::delta_nodiv0](#) = 1e-20
This is used to avoid dividing by zero.
- [real](#) [Integrators::Derivative_hval](#) {1e-7}
The amount of any coordinate that we shift to calculate derivatives (using central difference)
- std::string [Integrators::IntegratorDescription](#) {"RK4"}
The name of the integrator selected.
- [real](#) [Integrators::epsilon](#) {0.03}
This is the base step size to be taken (the integrator will adapt this if necessary)
- [real](#) [Integrators::SmallestPossibleStepsize](#) {1e-12}
The affine parameter must always go forward by at least this amount.
- [real](#) [Integrators::VerletVelocityTolerance](#) {0.001}
Tolerance on the change in velocity in the iteration loop of the Verlet integration algorithm.

7.22.1 Detailed Description

Declarations of integrator functions that integrate geodesic equation.

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.22.2 Typedef Documentation

7.22.2.1 GeodesicIntegratorFunc

```
using GeodesicIntegratorFunc = void (*)(Point, OneIndex, Point &, OneIndex &, real &, const
Metric *, const Source *)
```

7.23 Integrators.h

[Go to the documentation of this file.](#)

```
00001 #ifndef _FOORT_INTEGRATORS_H
00002 #define _FOORT_INTEGRATORS_H
00003
00013 #include "Geometry.h" // for basic tensor objects
00014 #include "Metric.h" // for the metric
00015
00016 #include <string> // std::string
00017
00019 class Source;
00020
00021 // This is the structure of a function that integrates the geodesic equation one step
00022 // It takes as arguments:
00023 // - current position, current velocity
00024 // - references to: next position, next velocity, affine parameter step size (which the functions
    sets)
00025 // - pointers to: the Metric, the Source that are used to evaluate the geodesic equation
00026 using GeodesicIntegratorFunc = void (*)(Point, OneIndex, Point &, OneIndex &, real &, const Metric *,
    const Source *);
00027
00028 // Namespace for integrator constants and functions
00029 namespace Integrators
00030 {
00032     constexpr real delta_nodiv0 = 1e-20;
00033
00035     inline real Derivative_hval{1e-7};
00036
00038     inline std::string IntegratorDescription{"RK4"};
00039
00040     // Full descriptive string of integrator and all integrator options
00041     std::string GetFullIntegratorDescription();
00042 }
```

```

00044     inline real epsilon{0.03};
00045
00047     inline real SmallestPossibleStepsize{1e-12};
00048
00049     // Function to get (adaptive) step size
00050     real GetAdaptiveStep(Point curpos, OneIndex curvel);
00051
00052     // This is a GeodesicIntegratorFunc
00053     // Using the Runge-Kutta-4 algorithm to integrate the geodesic equation
00054     void IntegrateGeodesicStep_RK4(Point curpos, OneIndex curvel,
00055                                     Point &nextpos, OneIndex &nextvel, real &stepsize, const Metric
00056                                     *theMetric, const Source *theSource);
00057
00058     inline real VerletVelocityTolerance{0.001};
00059
00060     // This is a GeodesicIntegratorFunc
00061     // Using the velocity Verlet algorithm to integrate the geodesic equation
00062     void IntegrateGeodesicStep_Verlet(Point curpos, OneIndex curvel,
00063                                         Point &nextpos, OneIndex &nextvel, real &stepsize, const Metric
00064                                         *theMetric, const Source *theSource);
00065 }
00066 #endif

```

7.24 FOORT/src/Main.cpp File Reference

```

#include "Config.h"
#include "Diagnostics.h"
#include "Geodesic.h"
#include "Geometry.h"
#include "InputOutput.h"
#include "Integrators.h"
#include "Metric.h"
#include "Terminations.h"
#include "Utilities.h"
#include "ViewScreen.h"
#include <omp.h>
#include <iostream>

```

Functions

- void [LoadPrecompiledOptions](#) (std::unique_ptr< [Metric](#) > &theM, std::unique_ptr< [Source](#) > &theS, [DiagBitflag](#) &AllDiags, [DiagBitflag](#) &ValDiag, [TermBitflag](#) &AllTerms, std::unique_ptr< [ViewScreen](#) > &theView, [GeodesicIntegratorFunc](#) &theIntegrator, std::unique_ptr< [GeodesicOutputHandler](#) > &theOutputHandler)
- int [main](#) (int argc, char *argv[])

7.24.1 Function Documentation

7.24.1.1 LoadPrecompiledOptions()

```

void LoadPrecompiledOptions (
    std::unique_ptr< Metric > & theM,
    std::unique_ptr< Source > & theS,
    DiagBitflag & AllDiags,
    DiagBitflag & ValDiag,
    TermBitflag & AllTerms,
    std::unique_ptr< ViewScreen > & theView,
    GeodesicIntegratorFunc & theIntegrator,
    std::unique_ptr< GeodesicOutputHandler > & theOutputHandler)

```

7.24.1.2 main()

```
int main (  
    int argc,  
    char * argv[])
```

7.25 FOORT/src/Mesh.cpp File Reference

```
#include "Mesh.h"  
#include "Utilities.h"  
#include <algorithm>  
#include <limits>  
#include <iostream>
```

7.25.1 Detailed Description

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.26 FOORT/src/Mesh.h File Reference

Declarations of abstract base [Mesh](#) class and all its descendants.

```
#include "Geometry.h"  
#include "Diagnostics.h"  
#include "InputOutput.h"  
#include <cmath>  
#include <utility>  
#include <forward_list>  
#include <memory>  
#include <vector>  
#include <array>  
#include <string>
```

Classes

- class [Mesh](#)
Abstract base [Mesh](#) class.
- class [SimpleSquareMesh](#)
A simple square mesh that will integrate a square of evenly spaced pixels.
- class [InputCertainPixelsMesh](#)
[Mesh](#) which integrates only certain user-inputted pixels.
- class [SquareSubdivisionMesh](#)
Adaptive subdivision [Mesh](#).
- struct [SquareSubdivisionMesh::PixelInfo](#)
A struct the [Mesh](#) uses to keep all information about a given pixel.
- class [SquareSubdivisionMeshV2](#)
Adaptive subdivision [Mesh](#).
- struct [SquareSubdivisionMeshV2::PixelInfo](#)
A struct the [Mesh](#) uses to keep all information about a given pixel.

7.26.1 Detailed Description

Declarations of abstract base [Mesh](#) class and all its descendants.

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.27 Mesh.h

[Go to the documentation of this file.](#)

```
00001 #ifndef _FOORT_MESH_H
00002 #define _FOORT_MESH_H
00003
00013 #include "Geometry.h" // needed for basic tensor objects
00014 #include "Diagnostics.h" // needed for Diagnostic "value" and "distance" functions
00015 #include "InputOutput.h" // needed for ScreenOutput
00016
00017 #include <cmath> // needed for sqrt (only on Linux)
00018 #include <utility> // std::move
00019 #include <forward_list> // std::forward_list
00020 #include <memory> // std::unique_ptr
00021 #include <vector> // std::vector
00022 #include <array> // std::array
00023 #include <string> // for strings
00024
```

```

00028 class Mesh
00029 {
00030 public:
00031     // Basic constructor constructs Diagnostic that is used for determinines "values" and "distances"
    between values
00032     Mesh(DiagBitflag valdiag)
00033     // Calling CreateDiagnosticVector in this way will create a vector with exactly one element in
    it,
00034     // i.e. the Diagnostic we need!
    : m_DistanceDiagnostic{std::move((CreateDiagnosticVector(valdiag, valdiag, nullptr))[0])}
00035     {
00036     }
00037 }
00038 // virtual destructor to ensure correct destruction of descendants
00039 virtual ~Mesh() = default;
00040 // Getter for how many geodesics the Mesh currently wants to integrate in this iteration
00041 virtual largecounter getCurNrGeodesics() const = 0;
00042 // This sets a new initial conditions (in the form of a ScreenPoint and ScreenIndex)
00043 // for a next pixel to be integrated in the current iteration
00044 virtual void getNewInitConds(largecounter index, ScreenPoint &newunitpoint, ScreenIndex
&newscreenindex) const = 0;
00045 // When a geodesic is finished integrating, it tells the Mesh and passes on its final "value"
00046 // NOTE: despite being a non-const member function, this must be designed to be thread-safe!
00047 virtual void GeodesicFinished(largecounter index, std::vector<real> finalValues) = 0;
00048 // This is called when the current iteration is finished. The Mesh can now evaluate whether to
    continue or not
00049 virtual void EndCurrentLoop() = 0;
00050 // Returns false if the Mesh wants another iteration of pixels to integrate
00051 virtual bool IsFinished() const = 0;
00052 // Returns a string description of the Mesh (spaces allowed), describing its options
00053 virtual std::string getFullDescriptionStr() const;
00054 protected:
00055     const std::unique_ptr<const Diagnostic> m_DistanceDiagnostic;
00056 };
00057
00058 class SimpleSquareMesh final : public Mesh
00059 {
00060 public:
00061     // Default constructor not possible
00062     SimpleSquareMesh() = delete;
00063     // Constructor initializes total number of pixels and passes valdiag to base constructor
00064     // Note that we static_cast the sqrt() to round off the row/column size to an integer number
00065     SimpleSquareMesh(largecounter totalPixels, DiagBitflag valdiag)
    : m_TotalPixels{static_cast<pixelcoord>(sqrt(totalPixels)) *
    static_cast<pixelcoord>(sqrt(totalPixels))},
    m_RowColumnSize{static_cast<pixelcoord>(sqrt(totalPixels))},
    Mesh(valdiag)
00066     {
00067         if constexpr (dimension != 4)
00068             ScreenOutput("SimpleSquareMesh only defined in 4D!", OutputLevel::Level_0_WARNING);
00069     }
00070     // Declarations of overriding virtual functions
00071     largecounter getCurNrGeodesics() const final;
00072     void getNewInitConds(largecounter index, ScreenPoint &newunitpoint, ScreenIndex &newscreenindex)
    const final;
00073     void GeodesicFinished(largecounter index, std::vector<real> finalValues) final;
00074     void EndCurrentLoop() final;
00075     bool IsFinished() const final;
00076     // Description string getter
00077     std::string getFullDescriptionStr() const final;
00078 private:
00079     const largecounter m_TotalPixels;
00080     const pixelcoord m_RowColumnSize;
00081     bool m_Finished{false};
00082 };
00083
00084 class InputCertainPixelsMesh : public Mesh
00085 {
00086 public:
00087     // Default constructor not possible
00088     InputCertainPixelsMesh() = delete;
00089     // Copy constructor not possible

```

```

00119     InputCertainPixelsMesh(const InputCertainPixelsMesh &) = delete;
00120     // Constructor given in Mesh.cpp file; constructor asks for input of pixels
00121     InputCertainPixelsMesh(largecounter totalPixels, DiagBitflag valdiag);
00122
00123     // Declarations of overriding virtual functions
00124
00125     largecounter getCurNrGeodesics() const final;
00126
00127     void getNewInitConds(largecounter index, ScreenPoint &newunitpoint, ScreenIndex &newscreenindex)
const final;
00128
00129     void GeodesicFinished(largecounter index, std::vector<real> finalValues) final;
00130
00131     void EndCurrentLoop() final;
00132
00133     bool IsFinished() const final;
00134
00135     // Description string getter
00136     std::string getFullDescriptionStr() const final;
00137
00138 private:
00139     const pixelcoord m_RowColumnSize;
00140
00141     largecounter m_TotalPixels{0};
00142     std::vector<ScreenIndex> m_PixelsToIntegrate{};
00143     bool m_Finished{false};
00144 };
00145
00146 class SquareSubdivisionMesh : public Mesh
00147 {
00148 public:
00149     // default constructor not possible
00150     SquareSubdivisionMesh() = delete;
00151     // Constructor must be called with arguments:
00152     // - maxPixels: max. nr of pixels that can be integrated in TOTAL, over all iterations (if 0, then
this is infinite,
00153     // i.e. we keep integrating until all squares are maximally subdivided or have weight 0)
00154     // - initialPixels: initial number of pixels to integrate (spaced equally over the screen)
00155     // - maxSubdivide: maximum number of times that we can subdivide squares (note: 1 denotes the
00156     // initial grid, so 2 would be subdividing the squares once)
00157     // - iterationPixels: maximum number of pixels to subdivide in each integration iteration
00158     // (max number of pixels that will be integrates is then 5*iterationPixels)
00159     // - initialSubToFinal: once we decide to subdivide a square, do we automatically keep subdividing
it
00160     // until we reach maxSubdivision?
00161     // - valdiag: the "value" and "distance" Diagnostic to use
00162     SquareSubdivisionMesh(largecounter maxPixels, largecounter initialPixels, int maxSubdivide,
largecounter iterationPixels, bool initialSubToFinal,
00163     DiagBitflag valdiag)
00164     : m_InitialPixels{static_cast<pixelcoord>(sqrt(initialPixels)) *
static_cast<pixelcoord>(sqrt(initialPixels))},
00165     m_MaxSubdivide{maxSubdivide},
00166     m_RowColumnSize{static_cast<pixelcoord>((sqrt(initialPixels) - 1) * ExpInt(2, maxSubdivide -
1) + 1)},
00167     m_PixelsLeft{maxPixels}, m_MaxPixels{maxPixels}, m_InfinitePixels{maxPixels == 0},
00168     m_IterationPixels{iterationPixels},
00169     m_InitialSubDivideToFinal{initialSubToFinal}, Mesh(valdiag)
00170     {
00171         if constexpr (dimension != 4)
00172             ScreenOutput("SquareSubdivisionMesh only defined in 4D!", OutputLevel::Level_0_WARNING);
00173
00174         // DEBUG message for constructor (can delete)
00175         ScreenOutput("SquareSubdivisionMesh constructed: maxPixels: " + (m_InfinitePixels ? "infinite"
: std::to_string(maxPixels)) + "; m_InitialPixels: " + std::to_string(m_InitialPixels) + ";
m_RowColumnSize: " + std::to_string(m_RowColumnSize), OutputLevel::Level_4_DEBUG);
00176
00177         // Initialize the initial square, equally spaced grid
00178         InitializeFirstGrid();
00179     }
00180
00181     // Declarations of overriding virtual functions
00182
00183     largecounter getCurNrGeodesics() const final;
00184
00185     void getNewInitConds(largecounter index, ScreenPoint &newunitpoint, ScreenIndex &newscreenindex)
const final;
00186
00187     void GeodesicFinished(largecounter index, std::vector<real> finalValues) final;
00188
00189     void EndCurrentLoop() final;
00190
00191     bool IsFinished() const final;
00192
00193     // Description string getter
00194     std::string getFullDescriptionStr() const final;
00195
00196 private:

```

```

00208     const largecounter m_InitialPixels;
00210     const int m_MaxSubdivide;
00212     const pixelcoord m_RowColumnSize;
00214     const largecounter m_IterationPixels;
00216     const largecounter m_MaxPixels;
00218     const bool m_InitialSubDivideToFinal;
00220     const bool m_InfinitePixels;
00221
00223     largecounter m_PixelsLeft;
00224
00228     struct PixelInfo
00229     {
00230         // Constructor with its ScreenIndex and current subdivision level
00231         PixelInfo(ScreenIndex ind, int subdiv) : Index{ind}, SubdivideLevel{subdiv} {}
00232
00234         ScreenIndex Index{};
00235
00237         int SubdivideLevel{};
00238
00240         real Weight{-1};
00241
00243         std::vector<real> DiagValue{};
00244
00246         largecounter LowerNbrIndex{0};
00248         largecounter RightNbrIndex{0};
00249     };
00251     std::vector<PixelInfo> m_CurrentPixelQueue{};
00253     std::vector<bool> m_CurrentPixelQueueDone{};
00255     std::vector<PixelInfo> m_AllPixels{};
00256
00257     // Initializes the first nxn screen in m_CurrentPixelQueue
00258     void InitializeFirstGrid();
00259
00260     // Updates the neighbors of all pixels (that should have neighbors and don't yet) in m_AllPixels
00261     void UpdateAllNeighbors();
00262
00263     // Updates all weights of the pixels in m_AllPixels with weight < 0 and subdiv > 0 and subdiv <
00264     // m_MaxSubdivide
00265     // Assumes all squares have neighbors assigned correctly
00266     void UpdateAllWeights();
00267
00268     // This will take the pixel m_AllPixels[ind] and subdivide it,
00269     // adding up to <=5 pixels to the CurrentPixelQueue
00270     void SubdivideAndQueue(largecounter ind);
00271
00272     // Helper function to exponentiate an int to an int
00273     // Note: the result can be larger than fits in an int, but the base is always 2 and the exp is
00274     // always
00275     // a number <=m_MaxSubdivide (which is an int)
00276     pixelcoord ExpInt(int base, int exp);
00277 };
00278
00286 class SquareSubdivisionMeshV2 : public Mesh
00287 {
00288 public:
00289     // default constructor not possible
00290     SquareSubdivisionMeshV2() = delete;
00291     // Constructor must be called with arguments:
00292     // - maxPixels: max. nr of pixels that can be integrated in TOTAL, over all iterations (if 0, then
00293     // this is infinite,
00294     // i.e. we keep integrating until all squares are maximally subdivided or have weight 0)
00295     // - initialPixels: initial number of pixels to integrate (spaced equally over the screen)
00296     // - maxSubdivide: maximum number of times that we can subdivide squares (note: 1 denotes the
00297     // initial grid, so 2 would be subdividing the squares once)
00298     // - iterationPixels: maximum number of pixels to subdivide in each integration iteration
00299     // (max number of pixels that will be integrates is then 5*iterationPixels)
00300     // - initialSubToFinal: once we decide to subdivide a square, do we automatically keep subdividing
00301     // it
00302     // until we reach maxSubdivision?
00303     // - valdiag: the "value" and "distance" Diagnostic to use
00304     SquareSubdivisionMeshV2(largecounter maxPixels, largecounter initialPixels, int maxSubdivide,
00305         largecounter iterationPixels, bool initialSubToFinal,
00306         DiagBitflag valdiag)
00307     : m_InitialPixels{static_cast<pixelcoord>(sqrt(initialPixels)) *
00308         static_cast<pixelcoord>(sqrt(initialPixels))},
00309       m_MaxSubdivide{maxSubdivide},
00310       m_RowColumnSize{static_cast<pixelcoord>((sqrt(initialPixels) - 1) * ExpInt(2, maxSubdivide -
00311         1) + 1)},
00312       m_PixelsLeft{maxPixels}, m_MaxPixels{maxPixels}, m_InfinitePixels{maxPixels == 0},
00313       m_IterationPixels{iterationPixels},
00314       m_InitialSubDivideToFinal{initialSubToFinal}, Mesh(valdiag)
00315     {
00316         if constexpr (dimension != 4)
00317             ScreenOutput("SquareSubdivisionMeshV2 only defined in 4D!", OutputLevel::Level_0_WARNING);
00318
00319         // Initialize the initial square, equally spaced grid
00320         InitializeFirstGrid();
00321     }

```

```

00315     }
00316
00317     // Declarations of overriding virtual functions
00318
00319     largecounter getCurNrGeodesics() const final;
00320
00321     void getNewInitConds(largecounter index, ScreenPoint &newunitpoint, ScreenIndex &newscreenindex)
const final;
00322
00323     void GeodesicFinished(largecounter index, std::vector<real> finalValues) final;
00324
00325     void EndCurrentLoop() final;
00326
00327     bool IsFinished() const final;
00328
00329     // Description string getter
00330     std::string getFullDescriptionStr() const final;
00331
00332 private:
00333     const largecounter m_InitialPixels;
00334     const int m_MaxSubdivide;
00335     const pixelcoord m_RowColumnSize;
00336     const largecounter m_IterationPixels;
00337     const largecounter m_MaxPixels;
00338     const bool m_InitialSubDividideToFinal;
00339     const bool m_InfinitePixels;
00340
00341     largecounter m_PixelsLeft;
00342
00343     largecounter m_PixelsIntegrated{0};
00344
00345     struct PixelInfo
00346     {
00347         // Constructor with its ScreenIndex and current subdivision level
00348         PixelInfo(ScreenIndex ind, int subdiv) : Index{ind}, SubdivideLevel{subdiv} {}
00349
00350         const ScreenIndex Index{};
00351
00352         int SubdivideLevel{};
00353
00354         real Weight{-1};
00355
00356         std::vector<real> DiagValue{};
00357
00358         PixelInfo *LeftNbr{nullptr};
00359         PixelInfo *RightNbr{nullptr};
00360         PixelInfo *UpNbr{nullptr};
00361         PixelInfo *DownNbr{nullptr};
00362         PixelInfo *SEdiagNbr{nullptr};
00363     };
00364
00365     std::forward_list<std::unique_ptr<PixelInfo>> m_AllPixels{};
00366
00367     std::vector<PixelInfo *> m_ActivePixels{};
00368     std::vector<PixelInfo *> m_CurrentPixelQueue{};
00369     std::vector<bool> m_CurrentPixelQueueDone{};
00370     std::vector<PixelInfo *> m_CurrentPixelUpdating{};
00371
00372     // Initializes the first nxn screen and puts them in m_CurrentPixelQueue
00373     void InitializeFirstGrid();
00374
00375     // Updates all weights of the pixels in m_CurrentPixelUpdating;
00376     // these should have subdiv > 0 and subdiv < m_MaxSubdivide, and all their neighbors assigned
00377     // correctly
00378     // All pixels with weight > 0 will be added to m_ActivePixels
00379     void UpdateAllWeights();
00380
00381     // Helper functions that return the appropriate neighbor of p, ONLY if this neighbor exists at the
00382     // subdivision level specified
00383     // Returns nullptr otherwise; also return nullptr if p==nullptr
00384     PixelInfo *GetUp(PixelInfo *p, int subdiv) const;
00385     PixelInfo *GetDown(PixelInfo *p, int subdiv) const;
00386     PixelInfo *GetRight(PixelInfo *p, int subdiv) const;
00387     PixelInfo *GetLeft(PixelInfo *p, int subdiv) const;
00388
00389     // This will take the pixel m_AllPixels[ind] and subdivide it,
00390     // adding up to <=5 pixels to the CurrentPixelQueue
00391     void SubdivideAndQueue(largecounter ind);
00392
00393     // Helper function to exponentiate an int to an int
00394     // Note: the result can be larger than fits in an int, but the base is always 2 and the exp is
00395     // always
00396     // a number <=m_MaxSubdivide (which is an int)
00397     pixelcoord ExpInt(int base, int exp) const;
00398 };
00399
00400 #endif

```

7.28 FOORT/src/Metric.cpp File Reference

```
#include "Metric.h"
#include "InputOutput.h"
#include "Integrators.h"
#include <cmath>
#include <algorithm>
#include "Spline.h"
#include <sstream>
```

7.29 FOORT/src/Metric.h File Reference

Declarations of abstract base [Metric](#) class and all its descendants.

```
#include "Geometry.h"
#include "Spline.h"
#include <string>
#include <vector>
```

Classes

- class [Metric](#)
The abstract base class for all Metrics.
- class [SphericalHorizonMetric](#)
Abstract base class for a metric with a spherical horizon (i.e. horizon at constant radius r)
- class [KerrMetric](#)
The Kerr metric.
- class [FlatSpaceMetric](#)
Flat space metric in 4D.
- class [RasheedLarsenMetric](#)
Rasheed-Larsen black hole.
- class [JohannsenMetric](#)
Johannsen black hole metric.
- class [MankoNovikovMetric](#)
Mano-Novikov metric (with angular momentum and $M3$ parameter turned on)
- class [KerrSchildMetric](#)
Kerr metric in Kerr-Schild coordinates.
- class [SingularityMetric](#)
Abstract base class for a metric with an arbitrary number of singularities (of arbitrary codimension)
- class [ST3CrMetric](#)
Ring fuzzball metric.
- class [BosonStarMetric](#)
Boson star metric with solitonic potential.

7.29.1 Detailed Description

Declarations of abstract base [Metric](#) class and all its descendants.

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.30 Metric.h

[Go to the documentation of this file.](#)

```
00001 #ifndef _FOORT_METRIC_H
00002 #define _FOORT_METRIC_H
00003
00004 #include "Geometry.h" // Needed for basic tensor objects etc.
00005
00006 #include "Spline.h"
00007 #include <string> // for strings
00008 #include <vector> // needed for the (non-fixed size) vector of symmetries in the metric
00009
00023 class Metric
00024 {
00025 public:
00026     // Virtual destructor to ensure correct destruction of descendants
00027     virtual ~Metric() = default;
00028
00029     Metric(bool rlogscale = false);
00030
00031     // Basic functions that return the metric with indices down or up:
00032     // pure virtual as they must be defined in the descendant classes.
00033     //
00034     // Get the metric at Point p, indices down
00035     virtual TwoIndex getMetric_dd(const Point &p) const = 0;
00036     // Get the metric at Point p, indices up
00037     virtual TwoIndex getMetric_uu(const Point &p) const = 0;
00038
00039     // The following functions return the Christoffel and other derivative quantities of the metric.
00040     // They are implemented for this base class, BUT are left as virtual functions to allow for
00041     // other metrics to implement their own (more efficient)
00042     // way of calculating them, if so desired.
00043     //
00044     // Get the Christoffel symbol, indices up-down-down
00045     virtual ThreeIndex getChristoffel_udd(const Point &p) const;
00046     // Get the Riemann tensor, indices up-down-down-down
00047     virtual FourIndex getRiemann_uddd(const Point &p) const;
00048     // Get the Kretschmann scalar
00049     virtual real getKretschmann(const Point &p) const;
00050
00051     // Function to get the description of the metric
00052     // (used for outputting to the screen while running and possibly to the output files)
00053     // There is a base class implementation of this function returning an undescriptive string
00054     virtual std::string getFullDescriptionStr() const;
00055
00056     bool getrLogScale() const;
00057
00058 protected:
```

```

00060     std::vector<int> m_Symmetries{};
00062     const bool m_rLogScale;
00063 };
00064
00065 class SphericalHorizonMetric : public Metric
00066 {
00067 public:
00068     // No default construction allowed, must specify horizon radius
00069     SphericalHorizonMetric() = delete;
00070     // Constructor that initializes horizon radius
00071     SphericalHorizonMetric(real HorizonRadius, bool rLogScale);
00072
00073     // Getter functions for the two member variables
00074     real getHorizonRadius() const;
00075
00076 protected:
00077     const real m_HorizonRadius;
00078 };
00079
00080 class KerrMetric final : public SphericalHorizonMetric
00081 {
00082 private:
00083     const real m_aParam;
00084     const real m_mParam;
00085
00086 public:
00087     // No default constructor allowed, must specify a
00088     KerrMetric() = delete;
00089
00090     // Constructor setting parameter a
00091     KerrMetric(real aParam, bool rLogScale = false, real mParam = 1.);
00092
00093     // The override of the basic metric getter functions
00094     TwoIndex getMetric_dd(const Point &p) const final;
00095     TwoIndex getMetric_uu(const Point &p) const final;
00096
00097     // The override of the description string getter
00098     std::string getFullDescriptionStr() const final;
00099 };
00100
00101 class FlatSpaceMetric final : public Metric
00102 {
00103 public:
00104     // Simple (default) constructor is all that is needed
00105     FlatSpaceMetric(bool rlogscale = false);
00106
00107     // The override of the basic metric getter functions
00108     TwoIndex getMetric_dd(const Point &p) const final;
00109     TwoIndex getMetric_uu(const Point &p) const final;
00110
00111     // The override of the description string getter
00112     std::string getFullDescriptionStr() const final;
00113 };
00114
00115 class RasheedLarsenMetric final : public SphericalHorizonMetric
00116 {
00117 private:
00118     const real m_aParam;
00119     const real m_mParam;
00120     const real m_pParam;
00121     const real m_qParam;
00122
00123 public:
00124     // No default constructor allowed, must specify parameters
00125     RasheedLarsenMetric() = delete;
00126
00127     // Constructor setting parameter a
00128     RasheedLarsenMetric(real mParam, real aParam, real pParam, real qParam, bool rLogScale = false);
00129
00130     // The override of the basic metric getter functions
00131     TwoIndex getMetric_dd(const Point &p) const final;
00132     TwoIndex getMetric_uu(const Point &p) const final;
00133
00134     // The override of the description string getter
00135     std::string getFullDescriptionStr() const final;
00136 };
00137
00138 class JohannsenMetric final : public SphericalHorizonMetric
00139 {
00140 private:
00141     // Johannsen up to first order in deviation function is specified by five parameters (if M=1)
00142     const real m_aParam;
00143     const real m_alpha13Param;
00144     const real m_alpha22Param;
00145     const real m_alpha52Param;
00146     const real m_eps3Param;

```

```

00176
00177 public:
00178     // No default constructor allowed, must specify parameters
00179     JohannsenMetric() = delete;
00180
00181     // Constructor setting parameter a
00182     JohannsenMetric(real aParam, real alpha13Param, real alpha22Param, real alpha52Param, real
eps3Param, bool rLogScale = false);
00183
00184     // The override of the basic metric getter functions
00185     TwoIndex getMetric_dd(const Point &p) const final;
00186     TwoIndex getMetric_uu(const Point &p) const final;
00187
00188     // The override of the description string getter
00189     std::string getFullDescriptionStr() const final;
00190 };
00191
00196 class MankoNovikovMetric final : public SphericalHorizonMetric
00197 {
00198 private:
00199     // Manko-Novikov metric with only alpha3 as symmetry breaking parameter
00200     const real m_aParam;
00201     const real m_alpha3Param;
00202
00203     // These are convenient derived quantities from a
00204     const real m_alphaParam;
00205     const real m_kParam;
00206
00207 public:
00208     // No default constructor allowed, must specify parameters
00209     MankoNovikovMetric() = delete;
00210
00211     // Constructor setting parameter a and alpha3
00212     MankoNovikovMetric(real aParam, real alpha3Param, bool rLogScale = false);
00213
00214     // The override of the basic metric getter functions
00215     TwoIndex getMetric_dd(const Point &p) const final;
00216     TwoIndex getMetric_uu(const Point &p) const final;
00217
00218     // The override of the description string getter
00219     std::string getFullDescriptionStr() const final;
00220 };
00221
00225 class KerrSchildMetric final : public SphericalHorizonMetric
00226 {
00227 private:
00228     const real m_aParam;
00229
00230 public:
00231     // No default constructor allowed, must specify a
00232     KerrSchildMetric() = delete;
00233
00234     // Constructor setting parameter a
00235     KerrSchildMetric(real aParam, bool rLogScale = false);
00236
00237     // The override of the basic metric getter functions
00238     TwoIndex getMetric_dd(const Point &p) const final;
00239     TwoIndex getMetric_uu(const Point &p) const final;
00240
00241     // The override of the description string getter
00242     std::string getFullDescriptionStr() const final;
00243 };
00244
00249 class SingularityMetric : public Metric
00250 {
00251 public:
00252     // Constructor that initializes singularities
00253     SingularityMetric(std::vector<Singularity> thesings, bool rLogScale);
00254
00255     // Getter functions for the two member variables
00256     std::vector<Singularity> getSingularities() const;
00257
00258 protected:
00259     const std::vector<Singularity> m_AllSingularities;
00260 };
00261
00266 class ST3CrMetric final : public SingularityMetric
00267 {
00268 public:
00269     // Constructor which will be called to initialize all parameters of the metric
00270     ST3CrMetric(real P, real q0, real lambda, bool rlogscale = false);
00271
00272     // The basic getter functions
00273     TwoIndex getMetric_dd(const Point &p) const final;
00274     TwoIndex getMetric_uu(const Point &p) const final;
00275
00276     // The description string getter

```

```

00283     std::string getFullDescriptionStr() const final;
00284
00285 private:
00286     const real m_P;
00287     const real m_q0;
00288     const real m_lambda;
00289
00290     real get_omega(real r, real theta, real l) const;
00291     real f_phi(real phi, real r, real theta, real l, real R) const;
00292     real f_om_phi(real phi, real r, real theta, real l, real R) const;
00293 };
00294
00300 class BosonStarMetric final : public Metric
00301 {
00302 public:
00303     // Simple (default) constructor is all that is needed
00304     BosonStarMetric(double Phi_infinity, int num_lines, bool rLogScale = false,
00305                     std::string Phi_filename = "Phi.dat", std::string m_filename = "m.dat");
00306     // The override of the basic metric getter functions
00307     TwoIndex getMetric_dd(const Point &p) const final;
00308     TwoIndex getMetric_uu(const Point &p) const final;
00309     // The override of the description string getter
00310     std::string getFullDescriptionStr() const final;
00311
00312 protected:
00314     const double m_Phi_infinity;
00317     const int m_num_lines;
00318
00320     std::string m_Phi_filename;
00322     std::string m_m_filename;
00323
00325     tk::spline m_PhiSpline;
00327     tk::spline m_mSpline;
00328
00330     void read_data();
00331 };
00332
00334 // Declare your new Metric class here, publically inheriting from the base class Metric
00335 // (or SphericalHorizonMetric if your Metric has a horizon, or SingularityMetric if your Metric has
00336 // other, arbitrary singularities)
00337 // Give definitions (implementation) of these functions in Metric.cpp (or other source code file)
00338 // Don't forget to set m_Symmetries appropriately (in the constructor),
00339 // if your metric has any symmetry (e.g. stationarity, axisymmetry)!
00340 // Sample code:
00341 /*
00341 class MyMetric final : public Metric // good practice to make the class final unless descendant
00342 // classes are possible
00343 {
00344 public:
00344     // Constructor which will be called to initialize all parameters of the metric
00345     MyMetric(args...);
00346
00347     // The basic getter functions
00348     // These MUST be implemented
00349     TwoIndex getMetric_dd(const Point& p) const final;
00350     TwoIndex getMetric_uu(const Point& p) const final;
00351
00352     // The description string getter
00353     // This is optional (but recommended) to implement; if not implemented,
00354     // the base class Metric::getFullDescriptionStr() will be called instead
00355     std::string getFullDescriptionStr() const final;
00356
00357 private:
00358     // good practice to have all const params (initialized in the constructor)
00359     // since the metric cannot change after initialization
00360     // const params...;
00361
00362 };
00363 */
00365
00366 #endif

```

7.31 FOORT/src/Spline.cpp File Reference

Simple cubic spline interpolation library without external dependencies.

```

#include "Spline.h"
#include <cstdio>
#include <cassert>

```

```
#include <cmath>
#include <vector>
#include <algorithm>
```

7.31.1 Detailed Description

Simple cubic spline interpolation library without external dependencies.

This library is not made by the authors of FOORT, but implemented as found in the following link: <https://kluge.in-chemnitz.de/opensource/spline/>. The main body has been separated into [Spline.cpp](#), however.

Author

Tino Kluge

Date

2021

Copyright

GNU General Public License

7.32 FOORT/src/Spline.h File Reference

Simple cubic spline interpolation library without external dependencies.

```
#include <cstdio>
#include <cassert>
#include <cmath>
#include <vector>
#include <algorithm>
```

Classes

- class [tk::spline](#)
- class [tk::internal::band_matrix](#)

Namespaces

- namespace [tk](#)
- namespace [tk::internal](#)

7.32.1 Detailed Description

Simple cubic spline interpolation library without external dependencies.

This library is not made by the authors of FOORT, but implemented as found in the following link: <https://kluge.in-chemnitz.de/opensource/spline/>. The main body has been separated into `Spline.cpp`, however.

Author

Tino Kluge

Date

2021

Copyright

GNU General Public License

7.33 Spline.h

[Go to the documentation of this file.](#)

```
00001 /*
00002  * spline.h
00003  *
00004  * simple cubic spline interpolation library without external
00005  * dependencies
00006  *
00007  * -----
00008  * Copyright (C) 2011, 2014, 2016, 2021 Tino Kluge (ttk448 at gmail.com)
00009  *
00010  * This program is free software; you can redistribute it and/or
00011  * modify it under the terms of the GNU General Public License
00012  * as published by the Free Software Foundation; either version 2
00013  * of the License, or (at your option) any later version.
00014  *
00015  * This program is distributed in the hope that it will be useful,
00016  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00017  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00018  * GNU General Public License for more details.
00019  *
00020  * You should have received a copy of the GNU General Public License
00021  * along with this program. If not, see <http://www.gnu.org/licenses/>.
00022  * -----
00023  *
00024  */
00025
00026 #ifndef TK_SPLINE_H
00027 #define TK_SPLINE_H
00028
00029 #include <cstdio>
00030 #include <cassert>
00031 #include <cmath>
00032 #include <vector>
00033 #include <algorithm>
00034 #ifdef HAVE_SSTREAM
00035 #include <sstream>
00036 #include <string>
00037 #endif // HAVE_SSTREAM
00038
00039 // not ideal but disable unused-function warnings
00040 // (we get them because we have implementations in the header file,
00041 // and this is because we want to be able to quickly separate them
00042 // into a cpp file if necessary)
00043 #pragma GCC diagnostic push
00044 #pragma GCC diagnostic ignored "-Wunused-function"
00045 namespace tk
00046 {
```

```

00057 // spline interpolation
00058 class spline
00059 {
00060 public:
00061     // spline types
00062     enum spline_type
00063     {
00064         linear = 10,           // linear interpolation
00065         cspline = 30,          // cubic splines (classical C^2)
00066         cspline_hermite = 31 // cubic hermite splines (local, only C^1)
00067     };
00068
00069     // boundary condition type for the spline end-points
00070     enum bd_type
00071     {
00072         first_deriv = 1,
00073         second_deriv = 2
00074     };
00075
00076 protected:
00077     std::vector<double> m_x, m_y; // x,y coordinates of points
00078     // interpolation parameters
00079     // f(x) = a_i + b_i*(x-x_i) + c_i*(x-x_i)^2 + d_i*(x-x_i)^3
00080     // where a_i = y_i, or else it won't go through grid points
00081     std::vector<double> m_b, m_c, m_d; // spline coefficients
00082     double m_c0;                     // for left extrapolation
00083     spline_type m_type;
00084     bd_type m_left, m_right;
00085     double m_left_value, m_right_value;
00086     bool m_made_monotonic;
00087     void set_coeffs_from_b();          // calculate c_i, d_i from b_i
00088     size_t find_closest(double x) const; // closest idx so that m_x[idx]<=x
00089
00090 public:
00091     // default constructor: set boundary condition to be zero curvature
00092     // at both ends, i.e. natural splines
00093     spline();
00094
00095     spline(const std::vector<double> &X, const std::vector<double> &Y,
00096           spline_type type = cspline,
00097           bool make_monotonic = false,
00098           bd_type left = second_deriv, double left_value = 0.0,
00099           bd_type right = second_deriv, double right_value = 0.0);
00100
00101     // modify boundary conditions: if called it must be before set_points()
00102     void set_boundary(bd_type left, double left_value,
00103                     bd_type right, double right_value);
00104
00105     // set all data points (cubic_spline=false means linear interpolation)
00106     void set_points(const std::vector<double> &x,
00107                    const std::vector<double> &y,
00108                    spline_type type = cspline);
00109
00110     // adjust coefficients so that the spline becomes piecewise monotonic
00111     // where possible
00112     // this is done by adjusting slopes at grid points by a non-negative
00113     // factor and this will break C^2
00114     // this can also break boundary conditions if adjustments need to
00115     // be made at the boundary points
00116     // returns false if no adjustments have been made, true otherwise
00117     bool make_monotonic();
00118
00119     // evaluates the spline at point x
00120     double operator()(double x) const;
00121     double deriv(int order, double x) const;
00122
00123     // returns the input data points
00124     std::vector<double> get_x() const;
00125     std::vector<double> get_y() const;
00126     double get_x_min() const;
00127     double get_x_max() const;
00128
00129 #ifndef HAVE_SSTREAM
00130     // spline info string, i.e. spline type, boundary conditions etc.
00131     std::string info() const;
00132 #endif // HAVE_SSTREAM
00133 };
00134
00135 namespace internal
00136 {
00137
00138     // band matrix solver
00139     class band_matrix
00140     {
00141     private:
00142         std::vector<std::vector<double>> m_upper; // upper band
00143         std::vector<std::vector<double>> m_lower; // lower band

```

```

00144     public:
00145         band_matrix() {}; // constructor
00146         band_matrix(int dim, int n_u, int n_l); // constructor
00147         ~band_matrix() {}; // destructor
00148         void resize(int dim, int n_u, int n_l); // init with dim,n_u,n_l
00149         int dim() const; // matrix dimension
00150         int num_upper() const;
00151         int num_lower() const;
00152         // access operator
00153         double &operator()(int i, int j); // write
00154         double operator()(int i, int j) const; // read
00155         // we can store an additional diagonal (in m_lower)
00156         double &saved_diag(int i);
00157         double saved_diag(int i) const;
00158         void lu_decompose();
00159         std::vector<double> r_solve(const std::vector<double> &b) const;
00160         std::vector<double> l_solve(const std::vector<double> &b) const;
00161         std::vector<double> lu_solve(const std::vector<double> &b,
00162                                     bool is_lu_decomposed = false);
00163     };
00164
00165 } // namespace internal
00166 } // namespace tk
00167 #pragma GCC diagnostic pop
00168
00169 #endif /* TK_SPLINE_H */

```

7.34 FOORT/src/Terminations.cpp File Reference

```

#include "Terminations.h"
#include "Geodesic.h"
#include "InputOutput.h"
#include <cmath>

```

Functions

- [TerminationUniqueVector CreateTerminationVector](#) ([TermBitflag](#) termflags, [Geodesic](#) *const theGeodesic)

7.34.1 Detailed Description

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.34.2 Function Documentation

7.34.2.1 CreateTerminationVector()

```
TerminationUniqueVector CreateTerminationVector (
    TermBitflag termflags,
    Geodesic *const theGeodesic)
```

7.35 FOORT/src/Terminations.h File Reference

Declarations of abstract base [Termination](#) class and all its descendants.

```
#include "Geometry.h"
#include <stdint>
#include <memory>
#include <vector>
#include <utility>
```

Classes

- class [Termination](#)
Abstract base class for all Terminations.
- class [HorizonTermination](#)
HorizonTermination: Terminate geodesics if they get too close to the horizon.
- class [BoundarySphereTermination](#)
BoundarySphereTermination: Terminate geodesics if they reach outside of a boundary sphere.
- class [TimeOutTermination](#)
TimeOutTermination: Terminate geodesics if they take too many steps.
- class [ThetaSingularityTermination](#)
ThetaSingularityTermination: Terminate geodesics if they get too close to a polar coordinate singularity.
- class [NaNTermination](#)
NaNTermination: Terminate geodesics if they contain a NaN in position or velocity.
- class [GeneralSingularityTermination](#)
GeneralSingularityTermination: Terminate geodesics if they get too close to one of a number of singularities (of arbitrary codimension)
- struct [TerminationOptions](#)
Base class for all TerminationOptions structs. Other TerminationOptions can inherit from here if they require more options.
- struct [HorizonTermOptions](#)
Options for HorizonTermination.
- struct [BoundarySphereTermOptions](#)
Options for BoundarySphereTermination.
- struct [TimeOutTermOptions](#)
Options for TimeOutTermination.
- struct [ThetaSingularityTermOptions](#)
Options for ThetaSingularityTermination.
- struct [NaNTermOptions](#)
Options for NaNTermination.
- struct [GeneralSingularityTermOptions](#)
Options for GeneralSingularityTermination.

Typedefs

- using [TermBitflag](#) = std::uint16_t
Used for constructing vector of Terminations. Note that this means every [Termination](#) is either "on" or "off"; it is not possible to have a [Termination](#) "on" more than once.
- using [TerminationUniqueVector](#) = std::vector<std::unique_ptr<[Termination](#)>>
The owner vector of derived [Termination](#) classes.

Enumerations

- enum class [Term](#) {
 [Uninitialized](#) = -1 , [Continue](#) = 0 , [Horizon](#) , [BoundarySphere](#) ,
 [TimeOut](#) , [ThetaSingularity](#) , [NaN](#) , [GeneralSingularity](#) ,
 [Maxterms](#) }

Possible termination conditions that can be set by Terminations.

Functions

- [TerminationUniqueVector CreateTerminationVector](#) ([TermBitflag](#) termflags, [Geodesic](#) *const theGeodesic)

Variables

- constexpr [TermBitflag](#) [Term_None](#) {0b0000'0000'0000'0000}
No terminations bitflag.
- constexpr [TermBitflag](#) [Term_BoundarySphere](#) {0b0000'0000'0000'0001}
Boundary Sphere termination bitflag.
- constexpr [TermBitflag](#) [Term_TimeOut](#) {0b0000'0000'0000'0010}
Time Out termination bitflag.
- constexpr [TermBitflag](#) [Term_Horizon](#) {0b0000'0000'0000'0100}
Horizon termination bitflag.
- constexpr [TermBitflag](#) [Term_ThetaSingularity](#) {0b0000'0000'0000'1000}
Theta Singularity termination bitflag.
- constexpr [TermBitflag](#) [Term_NaN](#) {0b0000'0000'0001'0000}
NaN termination bitflag.
- constexpr [TermBitflag](#) [Term_GeneralSingularity](#) {0b0000'0000'0010'0000}
General Singularity termination bitflag.

7.35.1 Detailed Description

Declarations of abstract base [Termination](#) class and all its descendants.

Author

Daniel R. Mayerson

Version

0.1

Date

2025-01-14

Copyright

Copyright (c) 2025

7.35.2 Typedef Documentation

7.35.2.1 TermBitflag

```
using TermBitflag = std::uint16_t
```

Used for constructing vector of Terminations. Note that this means every [Termination](#) is either "on" or "off"; it is not possible to have a [Termination](#) "on" more than once.

7.35.2.2 TerminationUniqueVector

```
using TerminationUniqueVector = std::vector<std::unique_ptr<Termination>>>
```

The owner vector of derived [Termination](#) classes.

7.35.3 Enumeration Type Documentation

7.35.3.1 Term

```
enum class Term [strong]
```

Possible termination conditions that can be set by Terminations.

Enumerator

Uninitialized	Geodesic has not been properly initialized yet with initial position/velocity.
Continue	All is right, continue integrating geodesic.
Horizon	STOP, encountered horizon (set by HorizonTermination)
BoundarySphere	STOP, encountered boundary sphere (set by BoundarySphereTermination)
TimeOut	STOP, taken too many steps (set by TimeOutTermination)
ThetaSingularity	STOP, too close to polar coordinate singularity ($\theta = 0$ or $\theta = \pi/2$) (set by ThetaSingularityTermination)
NaN	STOP, NaN encountered in geodesic position or velocity (set by NaNTermination)
GeneralSingularity	STOP, singularity encountered (of any codimension) (set by GeneralSingularityTermination)
Maxterms	

7.35.4 Function Documentation

7.35.4.1 CreateTerminationVector()

```
TerminationUniqueVector CreateTerminationVector (
    TermBitflag termflags,
    Geodesic *const theGeodesic)
```

7.35.5 Variable Documentation

7.35.5.1 Term_BoundarySphere

```
TermBitflag Term_BoundarySphere {0b0000'0000'0000'0001} [constexpr]
```

Boundary Sphere termination bitflag.

7.35.5.2 Term_GeneralSingularity

```
TermBitflag Term_GeneralSingularity {0b0000'0000'0010'0000} [constexpr]
```

General Singularity termination bitflag.

7.35.5.3 Term_Horizon

```
TermBitflag Term_Horizon {0b0000'0000'0000'0100} [constexpr]
```

Horizon termination bitflag.

7.35.5.4 Term_NaN

```
TermBitflag Term_NaN {0b0000'0000'0001'0000} [constexpr]
```

NaN termination bitflag.

7.35.5.5 Term_None

```
TermBitflag Term_None {0b0000'0000'0000'0000} [constexpr]
```

No terminations bitflag.

7.35.5.6 Term_ThetaSingularity

```
TermBitflag Term_ThetaSingularity {0b0000'0000'0000'1000} [constexpr]
```

Theta Singularity termination bitflag.

7.35.5.7 Term_TimeOut

```
TermBitflag Term_TimeOut {0b0000'0000'0000'0010} [constexpr]
```

Time Out termination bitflag.

7.36 Terminations.h

[Go to the documentation of this file.](#)

```

00001 #ifndef _FOORT_TERMINATIONS_H
00002 #define _FOORT_TERMINATIONS_H
00003
00013 #include "Geometry.h" // for basic tensor objects
00014
00015 #include <cstdint> // for std::uint16_t
00016 #include <memory> // for std::unique_ptr
00017 #include <vector> // for std::vector
00018 #include <utility> // for std::pair
00019
00020 // Forward declaration of Geodesic class needed here, since Diagnostics are passed a pointer to their
    owner Geodesic
00021 // (note "Geodesic.h" is NOT included to avoid header loop, and we do not need Geodesic member
    functions here!)
00022 class Geodesic;
00023
00024 // TERMINATION BITFLAGS
00026 using TermBitflag = std::uint16_t;
00027
00028 // Define a bitflag per existing Termination
00030 constexpr TermBitflag Term_None{0b0000'0000'0000'0000};
00032 constexpr TermBitflag Term_BoundarySphere{0b0000'0000'0000'0001};
00034 constexpr TermBitflag Term_TimeOut{0b0000'0000'0000'0010};
00036 constexpr TermBitflag Term_Horizon{0b0000'0000'0000'0100};
00038 constexpr TermBitflag Term_ThetaSingularity{0b0000'0000'0000'1000};
00040 constexpr TermBitflag Term_NaN{0b0000'0000'0001'0000};
00042 constexpr TermBitflag Term_GeneralSingularity{0b0000'0000'0010'0000};
00043
00045 // Add a TermBitflag for your new Termination. Make sure you use a bitflag that has not been used
    before!
00046 // Sample code:
00047 /*
00048 constexpr TermBitflag Term_MyTerm { 0b0000'0000'0000'1000 };
00049 */
00051
00052 // TERMINATION CONDITIONS
00053
00057 enum class Term
00058 {
00060     Uninitialized = -1,
00062     Continue = 0,
00064     Horizon,
00066     BoundarySphere,
00068     TimeOut,
00070     ThetaSingularity,
00072     NaN,
00074     GeneralSingularity,
00075
00077     // Add a new Termination condition that your new Termination can set
00078     // Sample code:
00079     /*
00080     MyTermCond, // STOP, encountered (...)
00081     */
00083
00084     Maxterms // Number of termination conditions that exist
00085 };
00086
00087 // GENERAL DECLARATIONS OF ABSTRACT BASE CLASS
00088
00092 class Termination
00093 {
00094 public:
00095     // Constructor must initialize the pointer to its owner Geodesic
00096     Termination() = delete;
00097     Termination(Geodesic *const theGeodesic) : m_OwnerGeodesic{theGeodesic}
00098     {
00099     }
00100
00101     // Resets Termination object. This is called when the owner Geodesic is reset in order to start
    integrating
00102     // a new geodesic.
00103     // The base class implementation only resets m_StepsSinceUpdated
00104     // Descendants can override this if they need to reset additional internal variables
00105     virtual void Reset();
00106
00107     // virtual destructor to ensure correct destruction of descendants
00108     virtual ~Termination() = default;
00109
00110     // Function that is called to determine whether Termination wants to
00111     // terminate the Geodesic. Returns Term::Continue if no termination wanted,
00112     // otherwise it returns the appropriate Term condition
00113     virtual Term CheckTermination() = 0;

```

```

00114
00115     // This returns the full description of the Termination
00116     virtual std::string getFullDescriptionStr() const = 0;
00117
00118 protected:
00120     Geodesic *const m_OwnerGeodesic;
00121
00122     // Helper function to decide if the Termination should indeed update its status, based on
00123     // UpdateNSteps (which is set to 0 if we always update)
00124     bool DecideUpdate(largecounter UpdateNSteps);
00125
00127     largecounter m_StepsSinceUpdated{};
00128 };
00129
00131 using TerminationUniqueVector = std::vector<std::unique_ptr<Termination>>;
00132
00133 // Helper to create a new vector of Termination options, based on the bitflag
00134 TerminationUniqueVector CreateTerminationVector(TermBitflag termflags, Geodesic *const theGeodesic);
00135
00136 // DECLARATIONS FOR DERIVED TERMINATION CLASSES (DESCENDANTS)
00137
00138 // Forward declaration needed before Termination
00139 struct HorizonTermOptions;
00140
00144 class HorizonTermination final : public Termination
00145 {
00146 public:
00147     // Basic constructor only passes on Geodesic pointer to base class constructor
00148     HorizonTermination(Geodesic *const theGeodesic) : Termination(theGeodesic) {}
00149
00150     // Check if we are too close to the horizon
00151     Term CheckTermination() final;
00152
00153     // Description string
00154     std::string getFullDescriptionStr() const final;
00155
00157     static std::unique_ptr<HorizonTermOptions> TermOptions;
00158 };
00159
00160 // Forward declaration needed before Termination
00161 struct BoundarySphereTermOptions;
00162
00166 class BoundarySphereTermination final : public Termination
00167 {
00168 public:
00169     // Basic constructor only passes on Geodesic pointer to base class constructor
00170     BoundarySphereTermination(Geodesic *const theGeodesic) : Termination(theGeodesic) {}
00171
00172     // Check if we have passed the boundary sphere
00173     Term CheckTermination() final;
00174
00175     // Description string
00176     std::string getFullDescriptionStr() const final;
00177
00179     static std::unique_ptr<BoundarySphereTermOptions> TermOptions;
00180 };
00181
00182 // Forward declaration needed before Termination
00183 struct TimeOutTermOptions;
00184
00188 class TimeOutTermination final : public Termination
00189 {
00190 public:
00191     // Basic constructor only passes on Geodesic pointer to base class constructor
00192     TimeOutTermination(Geodesic *const theGeodesic) : Termination(theGeodesic) {}
00193
00194     // This descendant needs to override Reset in order to also reset m_CurNrSteps
00195     void Reset() final;
00196
00197     // Check if we have already taken too many steps
00198     Term CheckTermination() final;
00199
00200     // Description string
00201     std::string getFullDescriptionStr() const final;
00202
00204     static std::unique_ptr<TimeOutTermOptions> TermOptions;
00205
00206 private:
00208     largecounter m_CurNrSteps{0};
00209 };
00210
00211 // Forward declaration needed before Termination
00212 struct ThetaSingularityTermOptions;
00213
00217 class ThetaSingularityTermination final : public Termination
00218 {
00219 public:

```

```

00220     ThetaSingularityTermination(Geodesic *const theGeodesic) : Termination(theGeodesic) {}
00221
00222     // No override of Reset() necessary
00223
00224     // Check the specific termination condition
00225     Term CheckTermination() final;
00226
00227     // Description string
00228     std::string getFullDescriptionStr() const final;
00229
00230     static std::unique_ptr<ThetaSingularityTermOptions> TermOptions;
00231 };
00232
00233 // Forward declaration needed before Termination
00234 struct NaNTermOptions;
00235
00236 class NaNTermination final : public Termination
00237 {
00238 public:
00239     // Basic constructor only passes on Geodesic pointer to base class constructor
00240     NaNTermination(Geodesic *const theGeodesic) : Termination(theGeodesic) {}
00241
00242     // Check if we are too close to the horizon
00243     Term CheckTermination() final;
00244
00245     // Description string
00246     std::string getFullDescriptionStr() const final;
00247
00248     static std::unique_ptr<NaNTermOptions> TermOptions;
00249 };
00250
00251 // Forward declaration needed before Termination
00252 struct GeneralSingularityTermOptions;
00253
00254 class GeneralSingularityTermination final : public Termination
00255 {
00256 public:
00257     // Basic constructor only passes on Geodesic pointer to base class constructor
00258     GeneralSingularityTermination(Geodesic *const theGeodesic) : Termination(theGeodesic) {}
00259
00260     // Check if we are too close to the horizon
00261     Term CheckTermination() final;
00262
00263     // Description string
00264     std::string getFullDescriptionStr() const final;
00265
00266     static std::unique_ptr<GeneralSingularityTermOptions> TermOptions;
00267 private:
00268     std::string SingularityToString(int singnr) const;
00269 };
00270
00271 // Declare your Termination class here, inheriting from Termination.
00272 // Sample code:
00273 /*
00274 // Forward declaration needed before Termination
00275 struct TerminationOptions; // possibly will instead need to declare descendant options struct
00276 class MyTermination final : public Termination // good practice to make the class final unless
00277 descendant classes are possible
00278 {
00279 public:
00280     // Constructor must at least pass on Geodesic pointer to base class constructor
00281     MyTermination(Geodesic* const theGeodesic) : Termination(theGeodesic) {}
00282
00283     // Do you need to reset any internal variables specific to MyTermination? If so, override Reset()
00284     (This is not mandatory)
00285     // Note: make sure to call the base class implementation Termination::Reset()
00286     // from within your implementation of MyTermination::Reset(), so that the base class internal
00287     variable is also reset!
00288     void Reset() final;
00289
00290     // Check the specific termination condition
00291     Term CheckTermination() final;
00292
00293     // Description string
00294     std::string getFullDescriptionStr() const final;
00295
00296     // The options that the Termination keeps (will probably be a descendant struct instead, which
00297     specifies
00298     // any additional options the Termination needs)
00299     static std::unique_ptr<TerminationOptions> TermOptions;
00300 private:
00301     // any private member variables that are needed to keep track of things
00302 };
00303 */
00304

```

```

00314 // ALL TERMINATIONOPTIONS STRUCTS
00315
00319 struct TerminationOptions
00320 {
00321 public:
00322     // Basic constructor only sets the number of steps between updates
00323     TerminationOptions(largecounter Nsteps) : UpdateEveryNSteps{Nsteps}
00324     {
00325     }
00326
00327     // virtual destructor to ensure correct destruction of descendants
00328     virtual ~TerminationOptions() = default;
00329
00331     const largecounter UpdateEveryNSteps;
00332 };
00333
00338 struct HorizonTermOptions : public TerminationOptions
00339 {
00340 public:
00341     HorizonTermOptions(real theHorizonRadius, bool therLogScale, real theAtHorizonEps, largecounter
Nsteps) : HorizonRadius{theHorizonRadius}, AtHorizonEps{theAtHorizonEps}, rLogScale{therLogScale},
TerminationOptions(Nsteps)
00342     {
00343     }
00344
00346     const real HorizonRadius;
00348     const real AtHorizonEps;
00350     const bool rLogScale;
00351 };
00352
00357 struct BoundarySphereTermOptions : public TerminationOptions
00358 {
00359 public:
00360     BoundarySphereTermOptions(real theRadius, bool therLogScale, largecounter Nsteps) :
SphereRadius{theRadius}, rLogScale{therLogScale},
00361     TerminationOptions(Nsteps)
00362     {
00363     }
00364
00366     const real SphereRadius;
00368     const bool rLogScale;
00369 };
00370
00375 struct TimeOutTermOptions : public TerminationOptions
00376 {
00377 public:
00378     TimeOutTermOptions(largecounter MaxStepsAllowed, largecounter Nsteps) : MaxSteps{MaxStepsAllowed},
TerminationOptions(Nsteps)
00379     {
00380     }
00381
00383     const largecounter MaxSteps;
00384 };
00385
00386 // Options class for ThetaSingularityTermination
00391 struct ThetaSingularityTermOptions : public TerminationOptions
00392 {
00393 public:
00394     ThetaSingularityTermOptions(real epsilon, largecounter Nsteps) : ThetaSingEpsilon{epsilon},
TerminationOptions(Nsteps)
00395     {
00396     }
00397
00399     const real ThetaSingEpsilon;
00400 };
00401
00405 struct NaNTermOptions : public TerminationOptions
00406 {
00407 public:
00408     NaNTermOptions(bool consoleoutputon, largecounter Nsteps) : OutputToConsole{consoleoutputon},
TerminationOptions(Nsteps)
00409     {
00410     }
00411
00413     const bool OutputToConsole;
00414 };
00415
00420 struct GeneralSingularityTermOptions : public TerminationOptions
00421 {
00422 public:
00423     GeneralSingularityTermOptions(std::vector<Singularity> sings,
real eps, bool consoleoutputon, bool therlogscale, largecounter
00424     Nsteps)
00425     : Singularities{std::move(sings)}, Epsilon{eps}, OutputToConsole{consoleoutputon},
00426     rLogScale{therlogscale},
00427     TerminationOptions(Nsteps) {}

```

```

00428
00430     const std::vector<Singularity> Singularities;
00432     const real Epsilon;
00434     const bool OutputToConsole;
00436     const bool rLogScale;
00437 };
00438
00440 // Add your new TerminationOptions struct here, inheriting from TerminationOptions (if needed)
00441 // Sample code:
00442 /*
00443 struct MyTermOptions : public TerminationOptions
00444 {
00445 public:
00446     MyTermOptions(..., largecounter Nsteps) : TerminationOptions(Nsteps) //, other initialization
00447     {}
00448
00449     // member variables (const!) here
00450 };
00451 */
00453
00454 #endif

```

7.37 FOORT/src/Utilities.cpp File Reference

```

#include "Utilities.h"
#include <sstream>
#include <iomanip>

```

7.38 FOORT/src/Utilities.h File Reference

```

#include "Metric.h"
#include "Diagnostics.h"
#include "Terminations.h"
#include "Geodesic.h"
#include "ViewScreen.h"
#include "Integrators.h"
#include <chrono>
#include <string>
#include <vector>

```

Classes

- class [Utilities::Timer](#)

Namespaces

- namespace [Utilities](#)

Functions

- std::string [Utilities::GetTimeStampString](#) ()
Other functions in [Utilities](#).
- std::vector< std::string > [Utilities::GetDiagNameStrings](#) (DiagBitflag alldiags, DiagBitflag valdiag)
- std::string [Utilities::GetFirstLineInfoString](#) (const Metric *theMetric, const Source *theSource, DiagBitflag alldiags, DiagBitflag valdiag, TermBitflag allterms, const ViewScreen *theView)

7.39 Utilities.h

[Go to the documentation of this file.](#)

```

00001 #ifndef _FOORT_UTILITIES_H
00002 #define _FOORT_UTILITIES_H
00003
00009
00010 // We use Metric, Diagnostic, Termination, Geodesic, ViewScreen, and Integrator declarations here
00011 #include "Metric.h"
00012 #include "Diagnostics.h"
00013 #include "Terminations.h"
00014 #include "Geodesic.h"
00015 #include "ViewScreen.h"
00016 #include "Integrators.h"
00017
00018 #include <chrono> // for timer functionality
00019 #include <string> // for strings
00020 #include <vector> // for std::vector
00021
00022 // Namespace that contains our utility functions
00023 namespace Utilities
00024 {
00025     // Timer class to keep track of elapsed time
00026     class Timer
00027     {
00028     private:
00029         // Type aliases to make accessing nested type easier
00030         using Clock = std::chrono::steady_clock;
00031         using Second = std::chrono::duration<double, std::ratio<1>;
00032
00033         // begin time
00034         std::chrono::time_point<Clock> m_beg{Clock::now()};
00035
00036     public:
00037         // reset begin time
00038         void reset();
00039
00040         // returns time elapsed since begin time
00041         double elapsed() const;
00042     };
00043
00044     // Returns a string of the current time (in a format that can be used to append to file names)
00045     std::string GetTimeStampString();
00046
00047     // Helper function to get all Diagnostic Names (for outputting to files)
00048     std::vector<std::string> GetDiagNameStrings(DiagBitflag alldiags, DiagBitflag valdiag);
00049
00050     // This returns the full string to be written to every output file as its first line
00051     // It contains information about all the settings used to produce the output
00052     std::string GetFirstLineInfoString(const Metric *theMetric, const Source *theSource,
00053                                       DiagBitflag alldiags, DiagBitflag valdiag, TermBitflag
00054 allterms, const ViewScreen *theView);
00055 }
00056 #endif

```

7.40 FOORT/src/ViewScreen.cpp File Reference

```

#include "ViewScreen.h"
#include <algorithm>

```

7.41 FOORT/src/ViewScreen.h File Reference

```

#include "Geometry.h"
#include "Metric.h"
#include "Mesh.h"
#include <memory>
#include <utility>
#include <array>
#include <string>

```

Classes

- class [ViewScreen](#)

Enumerations

- enum class [GeodesicType](#) { [Null](#) = 0 , [Timelike](#) = -1 , [Spacelike](#) = 1 }

7.41.1 Enumeration Type Documentation

7.41.1.1 GeodesicType

```
enum class GeodesicType [strong]
```

Enumerator

Null	
Timelike	
Spacelike	

7.42 ViewScreen.h

[Go to the documentation of this file.](#)

```
00001 #ifndef _FOORT_VIEWSCREEN_H
00002 #define _FOORT_VIEWSCREEN_H
00003
00011
00012 #include "Geometry.h" // For basic tensor objects
00013 #include "Metric.h" // For the Metric object
00014 #include "Mesh.h" // For the Mesh object
00015
00016 #include <memory> // std::unique_ptr
00017 #include <utility> // std::move
00018 #include <array> // std::array
00019 #include <string> // strings
00020
00021 // Type of geodesic being integrated. NOTE: only Null supported/implemented at the moment!
00022 enum class GeodesicType
00023 {
00024     Null = 0,
00025     Timelike = -1,
00026     Spacelike = 1,
00027 };
00028
00029 // ViewScreen class: this class is in charge of converting a pixel on the screen (which the Mesh wants
// to integrate)
00030 // to physical initial conditions for the position and velocity of a geodesic. It owns a Mesh
// instance, which will tell it
00031 // which pixels to integrate etc.
00032 class ViewScreen
00033 {
00034 public:
00035     // No default constructor possible
00036     ViewScreen() = delete;
00037     // constructor must pass following arguments along:
00038     // - physical position and looking direction;
00039     // - screen dimensions (in dimensions of length)
00040     // - the Mesh used (ViewScreen must become a owner of this object!)
00041     // - the Metric used (ViewScreen is NOT the owner of the Metric)
00042     // - the geodesic type to be integrated (null, timelike, spacelike)
00043     ViewScreen(Point pos, OneIndex dir, ScreenPoint screensize, ScreenPoint screencenter,
00044                 std::unique_ptr<Mesh> theMesh, const Metric *const theMetric, GeodesicType thegeodtype
= GeodesicType::Null)
00045     : m_Pos{pos}, m_Direction{dir}, m_ScreenSize{screensize}, m_ScreenCenter{screencenter},
00046       m_theMesh{std::move(theMesh)},
```

```

00047         m_theMetric{theMetric}, m_GeodType{thegeodtype},
00048         m_rLogScale{theMetric->getrLogScale()}
00049     {
00050         // At the moment, we don't even use the direction; we are always pointed towards the origin
00051         if (m_Direction != Point{0, -1, 0, 0})
00052         {
00053             ScreenOutput("ViewScreen is only supported pointing inwards at the moment; Direction = {0,
-1, 0, 0} will be used",
00054                         OutputLevel::Level_0_WARNING);
00055         }
00056         // At the moment, we are only integrating null geodesics
00057         if (m_GeodType != GeodesicType::Null)
00058         {
00059             ScreenOutput("ViewScreen only supports null geodesics at the moment; geodesics integrated
will be null.",
00060                         OutputLevel::Level_0_WARNING);
00061         }
00062
00063         // Construct the vielbein now
00064         ConstructVielbein();
00065     }
00066
00067     // Heart of the ViewScreen: here, the ViewScreen is asked to provide initial conditions
00068     // for the geodesic nr index of the current iteration; based on the screen index
00069     // that the Mesh gives, it sets up these physical initial conditions.
00070     void SetNewInitialConditions(largecounter index, Point &pos, OneIndex &vel, ScreenIndex &scrIndex)
const;
00071
00072     // These member functions essentially pass on information to/from the Mesh
00073     bool IsFinished() const; // Does the ViewScreen (i.e. the Mesh) want to integrate
more geodesics or not?
00074     largecounter getCurNrGeodesics() const; // Current number of geodesics in this iteration
00075     void EndCurrentLoop(); // The current iteration of geodesics is finished;
prepare the next one
00076     // NOTE: despite not being const, this function has been designed to be threadsafe!
00077     void GeodesicFinished(largecounter index, std::vector<real> finalValues); // This geodesic has
been integrated, returning its final "values"
00078
00079     // Description string getter (spaces allowed), also will contain information about the Mesh
00080     std::string getFullDescriptionStr() const;
00081
00082 private:
00083     // The metric at the position of the viewscreen (we only need indices down)
00084     TwoIndex m_Metric_dd{};
00085     // The vielbein used to transform from the curved spacetime at the viewscreen to a locally flat
frame
00086     TwoIndex m_Vielbein{};
00087
00088     // Helper function to construct the vielbein given the metric
00089     void ConstructVielbein();
00090
00091     // The position and looking direction of the camera
00092     const Point m_Pos;
00093     const OneIndex m_Direction;
00094     // The screensize (in physical units of length)
00095     const ScreenPoint m_ScreenSize;
00096     // The screen center
00097     const ScreenPoint m_ScreenCenter;
00098
00099     // Whether the metric uses a logarithmic r coordinate or not
00100     const bool m_rLogScale;
00101
00102     // const pointer to const Metric
00103     const Metric *const m_theMetric;
00104
00105     // The geodesic type to be integrated
00106     const GeodesicType m_GeodType{GeodesicType::Null};
00107
00108     // The const pointer to the Mesh we are using to determine pixels to be integrated
00109     const std::unique_ptr<Mesh> m_theMesh;
00110 };
00111
00112 #endif

```


Index

- ~Diagnostic
 - D diagnostic, 48
- ~DiagnosticOptions
 - D diagnosticOptions, 51
- ~EmissionModel
 - E emissionModel, 52
- ~FluidVelocityModel
 - F fluidVelocityModel, 68
- ~Mesh
 - M mesh, 122
- ~Metric
 - M metric, 124
- ~Source
 - S source, 146
- ~Termination
 - T termination, 174
- ~TerminationOptions
 - T terminationOptions, 176
- ~band_matrix
 - tk::internal::band_matrix, 22
- AtHorizonEps
 - H horizonTermOptions, 102
- band_matrix
 - tk::internal::band_matrix, 21
- bd_type
 - tk::spline, 150
- BosonStarMetric, 24
 - B bosonStarMetric, 25
 - getFullDescriptionStr, 26
 - getMetric_dd, 26
 - getMetric_uu, 26
 - m_m_filename, 27
 - m_mSpline, 27
 - m_num_lines, 27
 - m_Phi_filename, 27
 - m_Phi_infinity, 27
 - m_PhiSpline, 27
 - read_data, 26
- BoundarySphere
 - T terminations.h, 248
- BoundarySphereTermination, 28
 - B boundarySphereTermination, 29
 - C checkTermination, 29
 - getFullDescriptionStr, 29
 - T termOptions, 29
- BoundarySphereTermOptions, 30
 - B boundarySphereTermOptions, 30
 - rLogScale, 31

- SphereRadius, 31
- CheckTermination
 - B boundarySphereTermination, 29
 - G generalSingularityTermination, 78
 - H horizonTermination, 100
 - N NaNTermination, 128
 - T termination, 175
 - T thetaSingularityTermination, 178
 - T timeOutTermination, 181
- Clock
 - U utilities::Timer, 184
- ClosestRadiusDiagnostic, 31
 - C closestRadiusDiagnostic, 32
 - D diagOptions, 34
 - F finalDataValDistance, 32
 - getFinalDataVal, 33
 - getFullDataStr, 33
 - getFullDescriptionStr, 33
 - getNameStr, 33
 - m_closestRadius, 34
 - R reset, 34
 - U updateData, 34
- ClosestRadiusOptions, 35
 - C closestRadiusOptions, 35
 - R rLogScale, 36
- Config, 11
 - C configCollection, 12
 - G getGeodesicIntegrator, 12
 - G getMesh, 12
 - G getMetric, 13
 - G getOutputHandler, 13
 - G getSource, 13
 - G getViewScreen, 14
 - I initializeDiagnostics, 14
 - I initializeScreenOutput, 14
 - I initializeTerminations, 15
 - O output_important_default, 15
 - O output_other_default, 15
 - S settingError, 12
- Config.h
 - C CONFIGURATION_MODE, 191
- ConfigCollection
 - C config, 12
- ConfigReader, 15
 - M maxStreamSize, 16
- ConfigReader::ConfigCollection, 36
 - C configSettingValue, 37
 - D displayCollection, 38
 - D displaySetting, 38

- DisplayTabs, 38
- Exists, 38
- GetSettingIndex, 39
- IsCollection, 39
- LookupValue, 40
- LookupValueInteger, 41–43
- m_Settings, 46
- NrSettings, 43
- operator[], 43, 44
- ReadCollection, 44
- ReadFile, 44
- ReadSettingName, 44
- ReadSettingSpecificChar, 45
- ReadSettingValue, 45
- ConfigReader::ConfigCollection::ConfigSetting, 47
 - SettingName, 47
 - SettingValue, 47
- ConfigReader::ConfigReaderException, 46
 - ConfigReaderException, 46
 - m_settingtrace, 47
 - trace, 46
- ConfigReaderException
 - ConfigReader::ConfigReaderException, 46
- ConfigSettingValue
 - ConfigReader::ConfigCollection, 37
- CONFIGURATION_MODE
 - Config.h, 191
- ConstructVielbein
 - ViewScreen, 187
- Continue
 - Terminations.h, 248
- CreateDiagnosticVector
 - Diagnostics.cpp, 196
 - Diagnostics.h, 199
- CreateTerminationVector
 - Terminations.cpp, 246
 - Terminations.h, 248
- cspline
 - tk::spline, 151
- cspline_hermite
 - tk::spline, 151
- DecideUpdate
 - Diagnostic, 48
 - Termination, 175
- delta_nodiv0
 - Integrators, 18
- deriv
 - tk::spline, 151
- Derivative_hval
 - Integrators, 18
- Diag_ClosestRadius
 - Diagnostics.h, 199
- Diag_EquatorialEmission
 - Diagnostics.h, 199
- Diag_EquatorialPasses
 - Diagnostics.h, 199
- Diag_FourColorScreen
 - Diagnostics.h, 199
- Diag_GeodesicPosition
 - Diagnostics.h, 200
- Diag_None
 - Diagnostics.h, 200
- DiagBitflag
 - Diagnostics.h, 198
- Diagnostic, 47
 - ~Diagnostic, 48
 - DecideUpdate, 48
 - Diagnostic, 48
 - FinalDataValDistance, 49
 - getFinalDataVal, 49
 - getFullDataStr, 49
 - getFullDescriptionStr, 49
 - getNameStr, 49
 - m_OwnerGeodesic, 50
 - m_StepsSinceUpdated, 50
 - Reset, 50
 - UpdateData, 50
- DiagnosticOptions, 51
 - ~DiagnosticOptions, 51
 - DiagnosticOptions, 51
 - theUpdateFrequency, 51
- Diagnostics.cpp
 - CreateDiagnosticVector, 196
- Diagnostics.h
 - CreateDiagnosticVector, 199
 - Diag_ClosestRadius, 199
 - Diag_EquatorialEmission, 199
 - Diag_EquatorialPasses, 199
 - Diag_FourColorScreen, 199
 - Diag_GeodesicPosition, 200
 - Diag_None, 200
 - DiagBitflag, 198
 - DiagnosticUniqueVector, 198
- DiagnosticUniqueVector
 - Diagnostics.h, 198
- DiagOptions
 - ClosestRadiusDiagnostic, 34
 - EquatorialEmissionDiagnostic, 57
 - EquatorialPassesDiagnostic, 63
 - GeodesicPositionDiagnostic, 95
- DiagValue
 - SquareSubdivisionMesh::PixelInfo, 133
 - SquareSubdivisionMeshV2::PixelInfo, 135
- dim
 - tk::internal::band_matrix, 22
- dimension
 - Geometry.h, 219
- DisplayCollection
 - ConfigReader::ConfigCollection, 38
- DisplaySetting
 - ConfigReader::ConfigCollection, 38
- DisplayTabs
 - ConfigReader::ConfigCollection, 38
- DownNbr
 - SquareSubdivisionMeshV2::PixelInfo, 135
- elapsed

- Utilities::Timer, 184
- EmissionModel, 52
 - ~EmissionModel, 52
 - GetEmission, 52
 - getFullDescriptionStr, 52
- EndCurrentLoop
 - InputCertainPixelsMesh, 105
 - Mesh, 122
 - SimpleSquareMesh, 141
 - SquareSubdivisionMesh, 156
 - SquareSubdivisionMeshV2, 163
 - ViewScreen, 187
- Epsilon
 - GeneralSingularityTermOptions, 80
- epsilon
 - Integrators, 18
- EquatorialEmissionDiagnostic, 53
 - DiagOptions, 57
 - EquatorialEmissionDiagnostic, 55
 - FinalDataValDistance, 55
 - getFinalDataVal, 55
 - getFullDataStr, 55
 - getFullDescriptionStr, 56
 - getNameStr, 56
 - m_Intensity, 57
 - Reset, 56
 - UpdateData, 56
- EquatorialEmissionOptions, 57
 - EquatorialEmissionOptions, 58
 - EquatPassUpperBound, 59
 - GeometricFudgeFactor, 59
 - RedShiftPower, 59
 - RLogScale, 59
 - TheEmissionModel, 59
 - TheFluidVelocityModel, 59
- EquatorialPassesDiagnostic, 59
 - DiagOptions, 63
 - EquatorialPassesDiagnostic, 61
 - FinalDataValDistance, 61
 - getFinalDataVal, 61
 - getFullDataStr, 61
 - getFullDescriptionStr, 62
 - getNameStr, 62
 - m_EquatPasses, 63
 - m_PrevTheta, 63
 - Reset, 62
 - UpdateData, 62
- EquatorialPassesOptions, 63
 - EquatorialPassesOptions, 64
 - Threshold, 64
- EquatPassUpperBound
 - EquatorialEmissionOptions, 59
- Exists
 - ConfigReader::ConfigCollection, 38
- Explnt
 - SquareSubdivisionMesh, 156
 - SquareSubdivisionMeshV2, 163
- f_om_phi
 - ST3CrMetric, 171
- f_phi
 - ST3CrMetric, 171
- FinalDataValDistance
 - ClosestRadiusDiagnostic, 32
 - Dagnostic, 49
 - EquatorialEmissionDiagnostic, 55
 - EquatorialPassesDiagnostic, 61
 - FourColorScreenDiagnostic, 70
 - GeodesicPositionDiagnostic, 93
- find_closest
 - tk::spline, 151
- FindISCO
 - GeneralCircularRadialFluid, 74
- first_deriv
 - tk::spline, 150
- FlatSpaceMetric, 65
 - FlatSpaceMetric, 66
 - getFullDescriptionStr, 66
 - getMetric_dd, 66
 - getMetric_uu, 66
- FluidVelocityModel, 67
 - ~FluidVelocityModel, 68
 - FluidVelocityModel, 68
 - GetFourVelocityd, 68
 - getFullDescriptionStr, 68
 - m_theMetric, 68
- FOORT/src/Config.cpp, 189
- FOORT/src/Config.h, 189, 191
- FOORT/src/ConfigReader.cpp, 192
- FOORT/src/ConfigReader.h, 193, 194
- FOORT/src/Diagnostics.cpp, 196
- FOORT/src/Diagnostics.h, 197, 200
- FOORT/src/DiagnosticsEmission.cpp, 205
- FOORT/src/DiagnosticsEmission.h, 206, 207
- FOORT/src/Geodesic.cpp, 208
- FOORT/src/Geodesic.h, 209, 210
- FOORT/src/Geometry.h, 211, 219
- FOORT/src/InputOutput.cpp, 221
- FOORT/src/InputOutput.h, 223, 226
- FOORT/src/Integrators.cpp, 227
- FOORT/src/Integrators.h, 228, 229
- FOORT/src/Main.cpp, 230
- FOORT/src/Mesh.cpp, 231
- FOORT/src/Mesh.h, 231, 232
- FOORT/src/Metric.cpp, 237
- FOORT/src/Metric.h, 237, 238
- FOORT/src/Spline.cpp, 241
- FOORT/src/Spline.h, 242, 243
- FOORT/src/Terminations.cpp, 245
- FOORT/src/Terminations.h, 246, 250
- FOORT/src/Utilities.cpp, 254
- FOORT/src/Utilities.h, 254, 255
- FOORT/src/ViewScreen.cpp, 255
- FOORT/src/ViewScreen.h, 255, 256
- FourColorScreenDiagnostic, 69
 - FinalDataValDistance, 70
 - FourColorScreenDiagnostic, 70

- getFinalDataVal, 70
- getFullDataStr, 70
- getFullDescriptionStr, 71
- getNameStr, 71
- m_quadrant, 72
- Reset, 71
- UpdateData, 71
- FourIndex
 - Geometry.h, 213
- GeneralCircularRadialFluid, 72
 - FindISCO, 74
 - GeneralCircularRadialFluid, 73
 - GetChrstrRaisedDer, 74
 - GetCircularVelocityd, 74
 - GetFourVelocityd, 74
 - getFullDescriptionStr, 75
 - GetInsideISCOCircularVelocityd, 75
 - GetRadialVelocityd, 75
 - m_betaPhi, 76
 - m_betaR, 76
 - m_ISCOexists, 76
 - m_ISCOpphi, 76
 - m_ISCOpt, 76
 - m_ISCOr, 76
 - m_subKeplerParam, 76
- GeneralSingularity
 - Terminations.h, 248
- GeneralSingularityTermination, 77
 - CheckTermination, 78
 - GeneralSingularityTermination, 78
 - getFullDescriptionStr, 78
 - SingularityToString, 78
 - TermOptions, 79
- GeneralSingularityTermOptions, 79
 - Epsilon, 80
 - GeneralSingularityTermOptions, 80
 - OutputToConsole, 80
 - rLogScale, 80
 - Singularities, 80
- Geodesic, 81
 - Geodesic, 82
 - getAllOutputStr, 83
 - getCurrentLambda, 83
 - getCurrentPos, 83
 - getCurrentVel, 83
 - getDiagnosticFinalValue, 84
 - getScreenIndex, 84
 - getTermCondition, 84
 - m_AllDiagnostics, 85
 - m_AllTerminations, 85
 - m_curLambda, 85
 - m_CurrentPos, 85
 - m_CurrentVel, 85
 - m_ScreenIndex, 86
 - m_TermCond, 86
 - m_theIntegrator, 86
 - m_theMetric, 86
 - m_theSource, 86
 - operator=, 84
 - Reset, 84
 - Update, 85
- GeodesicFinished
 - InputCertainPixelsMesh, 105
 - Mesh, 122
 - SimpleSquareMesh, 141
 - SquareSubdivisionMesh, 157
 - SquareSubdivisionMeshV2, 164
 - ViewScreen, 187
- GeodesicIntegratorFunc
 - Integrators.h, 229
- GeodesicOutputHandler, 86
 - GeodesicOutputHandler, 88
 - GetFileName, 88
 - getFullDescriptionStr, 88
 - m_AllCachedData, 90
 - m_CurrentFullFiles, 90
 - m_CurrentGeodesicsInFile, 90
 - m_DiagNames, 90
 - m_FileExtension, 90
 - m_FilePrefix, 90
 - m_FirstLineInfoString, 91
 - m_nrGeodesicsPerFile, 91
 - m_nrOutputsToCache, 91
 - m_PrevCached, 91
 - m_PrintFirstLineInfo, 91
 - m_TimeStamp, 91
 - m_WriteToConsole, 91
 - NewGeodesicOutput, 89
 - OpenForFirstTime, 89
 - OutputFinished, 89
 - PrepareForOutput, 89
 - WriteCachedOutputToFile, 90
- GeodesicPositionDiagnostic, 92
 - DiagOptions, 95
 - FinalDataValDistance, 93
 - GeodesicPositionDiagnostic, 93
 - getFinalDataVal, 94
 - getFullDataStr, 94
 - getFullDescriptionStr, 94
 - getNameStr, 94
 - m_AllSavedPoints, 95
 - Reset, 95
 - UpdateData, 95
- GeodesicPositionOptions, 96
 - GeodesicPositionOptions, 96
 - OutputNrSteps, 97
- GeodesicType
 - ViewScreen.h, 256
- GeometricFudgeFactor
 - EquatorialEmissionOptions, 59
- Geometry.h
 - dimension, 219
 - FourIndex, 213
 - largecounter, 213
 - LARGECOUNTER_MAX, 213
 - OneIndex, 214

- operator+, 216
- operator-, 216
- operator/, 217
- operator*, 215
- pi, 219
- PIXEL_MAX, 213
- pixelcoord, 214
- Point, 214
- real, 214
- ScreenIndex, 214
- ScreenPoint, 214
- Singularity, 214
- SingularityCoord, 214
- ThreeIndex, 215
- toString, 217, 218
- TwoIndex, 215
- get_omega
 - ST3CrMetric, 172
- get_x
 - tk::spline, 151
- get_x_max
 - tk::spline, 151
- get_x_min
 - tk::spline, 151
- get_y
 - tk::spline, 152
- GetAdaptiveStep
 - Integrators, 16
- getAllOutputStr
 - Geodesic, 83
- getChristoffel_udd
 - Metric, 124
- GetChrstrRaisedDer
 - GeneralCircularRadialFluid, 74
- GetCircularVelocityd
 - GeneralCircularRadialFluid, 74
- getCurNrGeodesics
 - InputCertainPixelsMesh, 105
 - Mesh, 122
 - SimpleSquareMesh, 142
 - SquareSubdivisionMesh, 157
 - SquareSubdivisionMeshV2, 164
 - ViewScreen, 187
- getCurrentLambda
 - Geodesic, 83
- getCurrentPos
 - Geodesic, 83
- getCurrentVel
 - Geodesic, 83
- GetDiagNameStrings
 - Utilities, 19
- getDiagnosticFinalValue
 - Geodesic, 84
- GetDown
 - SquareSubdivisionMeshV2, 164
- GetEmission
 - EmissionModel, 52
 - GLMJohnsonSUEmission, 98
- GetFileName
 - GeodesicOutputHandler, 88
- getFinalDataVal
 - ClosestRadiusDiagnostic, 33
 - Dagnostic, 49
 - EquatorialEmissionDiagnostic, 55
 - EquatorialPassesDiagnostic, 61
 - FourColorScreenDiagnostic, 70
 - GeodesicPositionDiagnostic, 94
- GetFirstLineInfoString
 - Utilities, 19
- GetFourVelocityd
 - FluidVelocityModel, 68
 - GeneralCircularRadialFluid, 74
- getFullDataStr
 - ClosestRadiusDiagnostic, 33
 - Dagnostic, 49
 - EquatorialEmissionDiagnostic, 55
 - EquatorialPassesDiagnostic, 61
 - FourColorScreenDiagnostic, 70
 - GeodesicPositionDiagnostic, 94
- getFullDescriptionStr
 - BosonStarMetric, 26
 - BoundarySphereTermination, 29
 - ClosestRadiusDiagnostic, 33
 - Dagnostic, 49
 - EmissionModel, 52
 - EquatorialEmissionDiagnostic, 56
 - EquatorialPassesDiagnostic, 62
 - FlatSpaceMetric, 66
 - FluidVelocityModel, 68
 - FourColorScreenDiagnostic, 71
 - GeneralCircularRadialFluid, 75
 - GeneralSingularityTermination, 78
 - GeodesicOutputHandler, 88
 - GeodesicPositionDiagnostic, 94
 - GLMJohnsonSUEmission, 98
 - HorizonTermination, 100
 - InputCertainPixelsMesh, 105
 - JohannsenMetric, 109
 - KerrMetric, 113
 - KerrSchildMetric, 116
 - MankoNovikovMetric, 119
 - Mesh, 122
 - Metric, 125
 - NaNTermination, 128
 - NoSource, 131
 - RasheedLarsenMetric, 138
 - SimpleSquareMesh, 142
 - Source, 146
 - SquareSubdivisionMesh, 157
 - SquareSubdivisionMeshV2, 164
 - ST3CrMetric, 172
 - Termination, 175
 - ThetaSingularityTermination, 178
 - TimeOutTermination, 181
 - ViewScreen, 187
- GetFullIntegratorDescription

- Integrators, 17
- GetGeodesicIntegrator
 - Config, 12
- getHorizonRadius
 - SphericalHorizonMetric, 149
- GetInsideISCOCircularVelocityd
 - GeneralCircularRadialFluid, 75
- getKretschmann
 - Metric, 125
- GetLeft
 - SquareSubdivisionMeshV2, 165
- GetLoopMessageFrequency
 - InputOutput.cpp, 222
 - InputOutput.h, 224
- GetMesh
 - Config, 12
- GetMetric
 - Config, 13
- getMetric_dd
 - BosonStarMetric, 26
 - FlatSpaceMetric, 66
 - JohannsenMetric, 109
 - KerrMetric, 113
 - KerrSchildMetric, 116
 - MankoNovikovMetric, 119
 - Metric, 125
 - RasheedLarsenMetric, 138
 - ST3CrMetric, 172
- getMetric_uu
 - BosonStarMetric, 26
 - FlatSpaceMetric, 66
 - JohannsenMetric, 110
 - KerrMetric, 113
 - KerrSchildMetric, 117
 - MankoNovikovMetric, 120
 - Metric, 125
 - RasheedLarsenMetric, 139
 - ST3CrMetric, 173
- getNameStr
 - ClosestRadiusDiagnostic, 33
 - Diagnostic, 49
 - EquatorialEmissionDiagnostic, 56
 - EquatorialPassesDiagnostic, 62
 - FourColorScreenDiagnostic, 71
 - GeodesicPositionDiagnostic, 94
- getNewInitConds
 - InputCertainPixelsMesh, 106
 - Mesh, 122
 - SimpleSquareMesh, 142
 - SquareSubdivisionMesh, 157
 - SquareSubdivisionMeshV2, 165
- GetOutputHandler
 - Config, 13
- GetRadialVelocityd
 - GeneralCircularRadialFluid, 75
- getRiemann_uddd
 - Metric, 126
- GetRight
 - SquareSubdivisionMeshV2, 165
- getrLogScale
 - Metric, 126
- getScreenIndex
 - Geodesic, 84
- GetSettingIndex
 - ConfigReader::ConfigCollection, 39
- getSingularities
 - SingularityMetric, 145
- GetSource
 - Config, 13
- getSource
 - NoSource, 131
 - Source, 146
- getTermCondition
 - Geodesic, 84
- GetTimeStampString
 - Utilities, 20
- GetUp
 - SquareSubdivisionMeshV2, 166
- GetViewScreen
 - Config, 14
- GLMJohnsonSUEmission, 97
 - GetEmission, 98
 - getFullDescriptionStr, 98
 - GLMJohnsonSUEmission, 98
 - m_gamma, 99
 - m_mu, 99
 - m_sigma, 99
- Horizon
 - Terminations.h, 248
- HorizonRadius
 - HorizonTermOptions, 102
- HorizonTermination, 99
 - CheckTermination, 100
 - getFullDescriptionStr, 100
 - HorizonTermination, 100
 - TermOptions, 101
- HorizonTermOptions, 101
 - AtHorizonEps, 102
 - HorizonRadius, 102
 - HorizonTermOptions, 102
 - rLogScale, 102
- Index
 - SquareSubdivisionMesh::PixelInfo, 133
 - SquareSubdivisionMeshV2::PixelInfo, 135
- InitializeDiagnostics
 - Config, 14
- InitializeFirstGrid
 - SquareSubdivisionMesh, 158
 - SquareSubdivisionMeshV2, 166
- InitializeScreenOutput
 - Config, 14
- InitializeTerminations
 - Config, 15
- InputCertainPixelsMesh, 103
 - EndCurrentLoop, 105

- GeodesicFinished, [105](#)
- getCurNrGeodesics, [105](#)
- getFullDescriptionStr, [105](#)
- getNewInitConds, [106](#)
- InputCertainPixelsMesh, [104](#)
- IsFinished, [106](#)
- m_Finished, [106](#)
- m_PixelsToIntegrate, [106](#)
- m_RowColumnSize, [107](#)
- m_TotalPixels, [107](#)
- InputOutput.cpp
 - GetLoopMessageFrequency, [222](#)
 - ScreenOutput, [222](#)
 - SetLoopMessageFrequency, [222](#)
 - SetOutputLevel, [222](#)
- InputOutput.h
 - GetLoopMessageFrequency, [224](#)
 - Level_0_WARNING, [224](#)
 - Level_1_PROC, [224](#)
 - Level_2_SUBPROC, [224](#)
 - Level_3_ALLDDETAIL, [224](#)
 - Level_4_DEBUG, [224](#)
 - MaxLevel, [224](#)
 - OutputLevel, [224](#)
 - ScreenOutput, [224](#)
 - SetLoopMessageFrequency, [225](#)
 - SetOutputLevel, [225](#)
- IntegrateGeodesicStep_RK4
 - Integrators, [17](#)
- IntegrateGeodesicStep_Verlet
 - Integrators, [17](#)
- IntegratorDescription
 - Integrators, [18](#)
- Integrators, [16](#)
 - delta_nodiv0, [18](#)
 - Derivative_hval, [18](#)
 - epsilon, [18](#)
 - GetAdaptiveStep, [16](#)
 - GetFullIntegratorDescription, [17](#)
 - IntegrateGeodesicStep_RK4, [17](#)
 - IntegrateGeodesicStep_Verlet, [17](#)
 - IntegratorDescription, [18](#)
 - SmallestPossibleStepsize, [18](#)
 - VerletVelocityTolerance, [18](#)
- Integrators.h
 - GeodesicIntegratorFunc, [229](#)
- IsCollection
 - ConfigReader::ConfigCollection, [39](#)
- IsFinished
 - InputCertainPixelsMesh, [106](#)
 - Mesh, [123](#)
 - SimpleSquareMesh, [142](#)
 - SquareSubdivisionMesh, [158](#)
 - SquareSubdivisionMeshV2, [166](#)
 - ViewScreen, [187](#)
- JohannsenMetric, [107](#)
 - getFullDescriptionStr, [109](#)
 - getMetric_dd, [109](#)
 - getMetric_uu, [110](#)
 - JohannsenMetric, [109](#)
 - m_alpha13Param, [110](#)
 - m_alpha22Param, [110](#)
 - m_alpha52Param, [110](#)
 - m_aParam, [110](#)
 - m_eps3Param, [111](#)
- KerrMetric, [111](#)
 - getFullDescriptionStr, [113](#)
 - getMetric_dd, [113](#)
 - getMetric_uu, [113](#)
 - KerrMetric, [113](#)
 - m_aParam, [114](#)
 - m_mParam, [114](#)
- KerrSchildMetric, [114](#)
 - getFullDescriptionStr, [116](#)
 - getMetric_dd, [116](#)
 - getMetric_uu, [117](#)
 - KerrSchildMetric, [116](#)
 - m_aParam, [117](#)
- I_solve
 - tk::internal::band_matrix, [22](#)
- largecounter
 - Geometry.h, [213](#)
- LARGECOUNTER_MAX
 - Geometry.h, [213](#)
- LeftNbr
 - SquareSubdivisionMeshV2::PixelInfo, [135](#)
- Level_0_WARNING
 - InputOutput.h, [224](#)
- Level_1_PROC
 - InputOutput.h, [224](#)
- Level_2_SUBPROC
 - InputOutput.h, [224](#)
- Level_3_ALLDDETAIL
 - InputOutput.h, [224](#)
- Level_4_DEBUG
 - InputOutput.h, [224](#)
- linear
 - tk::spline, [151](#)
- LoadPrecompiledOptions
 - Main.cpp, [230](#)
- LookupValue
 - ConfigReader::ConfigCollection, [40](#)
- LookupValueInteger
 - ConfigReader::ConfigCollection, [41–43](#)
- LowerNbrIndex
 - SquareSubdivisionMesh::PixelInfo, [133](#)
- lu_decompose
 - tk::internal::band_matrix, [22](#)
- lu_solve
 - tk::internal::band_matrix, [22](#)
- m_ActivePixels
 - SquareSubdivisionMeshV2, [167](#)
- m_AllCachedData
 - GeodesicOutputHandler, [90](#)

- m_AllDiagnostics
 - Geodesic, [85](#)
- m_AllPixels
 - SquareSubdivisionMesh, [159](#)
 - SquareSubdivisionMeshV2, [167](#)
- m_AllSavedPoints
 - GeodesicPositionDiagnostic, [95](#)
- m_AllSingularities
 - SingularityMetric, [145](#)
- m_AllTerminations
 - Geodesic, [85](#)
- m_alpha13Param
 - JohannsenMetric, [110](#)
- m_alpha22Param
 - JohannsenMetric, [110](#)
- m_alpha3Param
 - MankoNovikovMetric, [120](#)
- m_alpha52Param
 - JohannsenMetric, [110](#)
- m_alphaParam
 - MankoNovikovMetric, [120](#)
- m_aParam
 - JohannsenMetric, [110](#)
 - KerrMetric, [114](#)
 - KerrSchildMetric, [117](#)
 - MankoNovikovMetric, [120](#)
 - RasheedLarsenMetric, [139](#)
- m_b
 - tk::spline, [152](#)
- m_beg
 - Utilities::Timer, [185](#)
- m_betaPhi
 - GeneralCircularRadialFluid, [76](#)
- m_betaR
 - GeneralCircularRadialFluid, [76](#)
- m_c
 - tk::spline, [152](#)
- m_c0
 - tk::spline, [153](#)
- m_ClosestRadius
 - ClosestRadiusDiagnostic, [34](#)
- m_curLambda
 - Geodesic, [85](#)
- m_CurNrSteps
 - TimeOutTermination, [182](#)
- m_CurrentFullFiles
 - GeodesicOutputHandler, [90](#)
- m_CurrentGeodesicsInFile
 - GeodesicOutputHandler, [90](#)
- m_CurrentPixelQueue
 - SquareSubdivisionMesh, [159](#)
 - SquareSubdivisionMeshV2, [167](#)
- m_CurrentPixelQueueDone
 - SquareSubdivisionMesh, [159](#)
 - SquareSubdivisionMeshV2, [167](#)
- m_CurrentPixelUpdating
 - SquareSubdivisionMeshV2, [167](#)
- m_CurrentPos
 - Geodesic, [85](#)
- m_CurrentVel
 - Geodesic, [85](#)
- m_d
 - tk::spline, [153](#)
- m_DiagNames
 - GeodesicOutputHandler, [90](#)
- m_Direction
 - ViewScreen, [187](#)
- m_DistanceDiagnostic
 - Mesh, [123](#)
- m_eps3Param
 - JohannsenMetric, [111](#)
- m_EquatPasses
 - EquatorialPassesDiagnostic, [63](#)
- m_FileExtension
 - GeodesicOutputHandler, [90](#)
- m_FilePrefix
 - GeodesicOutputHandler, [90](#)
- m_Finished
 - InputCertainPixelsMesh, [106](#)
 - SimpleSquareMesh, [143](#)
- m_FirstLineInfoString
 - GeodesicOutputHandler, [91](#)
- m_gamma
 - GLMJohnsonSUEmission, [99](#)
- m_GeodType
 - ViewScreen, [187](#)
- m_HorizonRadius
 - SphericalHorizonMetric, [149](#)
- m_InfinitePixels
 - SquareSubdivisionMesh, [159](#)
 - SquareSubdivisionMeshV2, [168](#)
- m_InitialPixels
 - SquareSubdivisionMesh, [159](#)
 - SquareSubdivisionMeshV2, [168](#)
- m_InitialSubDividideToFinal
 - SquareSubdivisionMesh, [159](#)
 - SquareSubdivisionMeshV2, [168](#)
- m_Intensity
 - EquatorialEmissionDiagnostic, [57](#)
- m_ISCOexists
 - GeneralCircularRadialFluid, [76](#)
- m_ISCOpphi
 - GeneralCircularRadialFluid, [76](#)
- m_ISCOpt
 - GeneralCircularRadialFluid, [76](#)
- m_ISCOR
 - GeneralCircularRadialFluid, [76](#)
- m_IterationPixels
 - SquareSubdivisionMesh, [160](#)
 - SquareSubdivisionMeshV2, [168](#)
- m_kParam
 - MankoNovikovMetric, [120](#)
- m_lambda
 - ST3CrMetric, [173](#)
- m_left
 - tk::spline, [153](#)

- m_left_value
 - tk::spline, 153
- m_lower
 - tk::internal::band_matrix, 23
- m_m_filename
 - BosonStarMetric, 27
- m_made_monotonic
 - tk::spline, 153
- m_MaxPixels
 - SquareSubdivisionMesh, 160
 - SquareSubdivisionMeshV2, 168
- m_MaxSubdivide
 - SquareSubdivisionMesh, 160
 - SquareSubdivisionMeshV2, 168
- m_Metric_dd
 - ViewScreen, 188
- m_mParam
 - KerrMetric, 114
 - RasheedLarsenMetric, 139
- m_mSpline
 - BosonStarMetric, 27
- m_mu
 - GLMJohnsonSUEmission, 99
- m_nrGeodesicsPerFile
 - GeodesicOutputHandler, 91
- m_nrOutputsToCache
 - GeodesicOutputHandler, 91
- m_num_lines
 - BosonStarMetric, 27
- m_OwnerGeodesic
 - Diagnostic, 50
 - Termination, 175
- m_P
 - ST3CrMetric, 173
- m_Phi_filename
 - BosonStarMetric, 27
- m_Phi_infinity
 - BosonStarMetric, 27
- m_PhiSpline
 - BosonStarMetric, 27
- m_PixelsIntegrated
 - SquareSubdivisionMeshV2, 168
- m_PixelsLeft
 - SquareSubdivisionMesh, 160
 - SquareSubdivisionMeshV2, 168
- m_PixelsToIntegrate
 - InputCertainPixelsMesh, 106
- m_Pos
 - ViewScreen, 188
- m_pParam
 - RasheedLarsenMetric, 139
- m_PrevCached
 - GeodesicOutputHandler, 91
- m_PrevTheta
 - EquatorialPassesDiagnostic, 63
- m_PrintFirstLineInfo
 - GeodesicOutputHandler, 91
- m_q0
 - ST3CrMetric, 173
- m_qParam
 - RasheedLarsenMetric, 139
- m_quadrant
 - FourColorScreenDiagnostic, 72
- m_right
 - tk::spline, 153
- m_right_value
 - tk::spline, 153
- m_rLogScale
 - Metric, 126
 - ViewScreen, 188
- m_RowColumnSize
 - InputCertainPixelsMesh, 107
 - SimpleSquareMesh, 143
 - SquareSubdivisionMesh, 160
 - SquareSubdivisionMeshV2, 169
- m_ScreenCenter
 - ViewScreen, 188
- m_ScreenIndex
 - Geodesic, 86
- m_ScreenSize
 - ViewScreen, 188
- m_Settings
 - ConfigReader::ConfigCollection, 46
- m_settingtrace
 - ConfigReader::ConfigReaderException, 47
- m_sigma
 - GLMJohnsonSUEmission, 99
- m_StepsSinceUpdated
 - Diagnostic, 50
 - Termination, 175
- m_subKeplerParam
 - GeneralCircularRadialFluid, 76
- m_Symmetries
 - Metric, 126
- m_TermCond
 - Geodesic, 86
- m_theIntegrator
 - Geodesic, 86
- m_theMesh
 - ViewScreen, 188
- m_theMetric
 - FluidVelocityModel, 68
 - Geodesic, 86
 - Source, 147
 - ViewScreen, 188
- m_theSource
 - Geodesic, 86
- m_TimeStamp
 - GeodesicOutputHandler, 91
- m_TotalPixels
 - InputCertainPixelsMesh, 107
 - SimpleSquareMesh, 143
- m_type
 - tk::spline, 153
- m_upper
 - tk::internal::band_matrix, 23

- m_Vielbein
 - ViewScreen, 188
- m_WriteToConsole
 - GeodesicOutputHandler, 91
- m_x
 - tk::spline, 153
- m_y
 - tk::spline, 153
- main
 - Main.cpp, 230
- Main.cpp
 - LoadPrecompiledOptions, 230
 - main, 230
- make_monotonic
 - tk::spline, 152
- MankoNovikovMetric, 117
 - getFullDescriptionStr, 119
 - getMetric_dd, 119
 - getMetric_uu, 120
 - m_alpha3Param, 120
 - m_alphaParam, 120
 - m_aParam, 120
 - m_kParam, 120
 - MankoNovikovMetric, 119
- MaxLevel
 - InputOutput.h, 224
- MaxSteps
 - TimeOutTermOptions, 183
- MaxStreamSize
 - ConfigReader, 16
- Maxterms
 - Terminations.h, 248
- Mesh, 121
 - ~Mesh, 122
 - EndCurrentLoop, 122
 - GeodesicFinished, 122
 - getCurNrGeodesics, 122
 - getFullDescriptionStr, 122
 - getNewInitConds, 122
 - IsFinished, 123
 - m_DistanceDiagnostic, 123
 - Mesh, 122
- Metric, 123
 - ~Metric, 124
 - getChristoffel_udd, 124
 - getFullDescriptionStr, 125
 - getKretschmann, 125
 - getMetric_dd, 125
 - getMetric_uu, 125
 - getRiemann_uddd, 126
 - getrLogScale, 126
 - m_rLogScale, 126
 - m_Symmetries, 126
 - Metric, 124
- NaN
 - Terminations.h, 248
- NaNTermination, 127
 - CheckTermination, 128
 - getFullDescriptionStr, 128
 - NaNTermination, 128
 - TermOptions, 129
- NaNTermOptions, 129
 - NaNTermOptions, 130
 - OutputToConsole, 130
- NewGeodesicOutput
 - GeodesicOutputHandler, 89
- NoSource, 130
 - getFullDescriptionStr, 131
 - getSource, 131
 - NoSource, 131
- NrSettings
 - ConfigReader::ConfigCollection, 43
- Null
 - ViewScreen.h, 256
- num_lower
 - tk::internal::band_matrix, 22
- num_upper
 - tk::internal::band_matrix, 22
- OneIndex
 - Geometry.h, 214
- OpenForFirstTime
 - GeodesicOutputHandler, 89
- operator()
 - tk::internal::band_matrix, 22
 - tk::spline, 152
- operator+
 - Geometry.h, 216
- operator-
 - Geometry.h, 216
- operator/
 - Geometry.h, 217
- operator=
 - Geodesic, 84
- operator[]
 - ConfigReader::ConfigCollection, 43, 44
- operator*
 - Geometry.h, 215
- Output_Important_Default
 - Config, 15
- Output_Other_Default
 - Config, 15
- OutputFinished
 - GeodesicOutputHandler, 89
- OutputLevel
 - InputOutput.h, 224
- OutputNrSteps
 - GeodesicPositionOptions, 97
- OutputToConsole
 - GeneralSingularityTermOptions, 80
 - NaNTermOptions, 130
- pi
 - Geometry.h, 219
- PIXEL_MAX
 - Geometry.h, 213
- pixelcoord

- Geometry.h, 214
- PixelInfo
 - SquareSubdivisionMesh::PixelInfo, 133
 - SquareSubdivisionMeshV2::PixelInfo, 135
- Point
 - Geometry.h, 214
- PrepareForOutput
 - GeodesicOutputHandler, 89
- r_solve
 - tk::internal::band_matrix, 23
- RasheedLarsenMetric, 136
 - getFullDescriptionStr, 138
 - getMetric_dd, 138
 - getMetric_uu, 139
 - m_aParam, 139
 - m_mParam, 139
 - m_pParam, 139
 - m_qParam, 139
 - RasheedLarsenMetric, 138
- read_data
 - BosonStarMetric, 26
- ReadCollection
 - ConfigReader::ConfigCollection, 44
- ReadFile
 - ConfigReader::ConfigCollection, 44
- ReadSettingName
 - ConfigReader::ConfigCollection, 44
- ReadSettingSpecificChar
 - ConfigReader::ConfigCollection, 45
- ReadSettingValue
 - ConfigReader::ConfigCollection, 45
- real
 - Geometry.h, 214
- RedShiftPower
 - EquatorialEmissionOptions, 59
- Reset
 - ClosestRadiusDiagnostic, 34
 - Diagnostic, 50
 - EquatorialEmissionDiagnostic, 56
 - EquatorialPassesDiagnostic, 62
 - FourColorScreenDiagnostic, 71
 - Geodesic, 84
 - GeodesicPositionDiagnostic, 95
 - Termination, 175
 - TimeOutTermination, 182
- reset
 - Utilities::Timer, 184
- resize
 - tk::internal::band_matrix, 23
- RightNbr
 - SquareSubdivisionMeshV2::PixelInfo, 135
- RightNbrIndex
 - SquareSubdivisionMesh::PixelInfo, 133
- RLogScale
 - ClosestRadiusOptions, 36
 - EquatorialEmissionOptions, 59
- rLogScale
 - BoundarySphereTermOptions, 31
- GeneralSingularityTermOptions, 80
- HorizonTermOptions, 102
- saved_diag
 - tk::internal::band_matrix, 23
- ScreenIndex
 - Geometry.h, 214
- ScreenOutput
 - InputOutput.cpp, 222
 - InputOutput.h, 224
- ScreenPoint
 - Geometry.h, 214
- Second
 - Utilities::Timer, 184
- second_deriv
 - tk::spline, 150
- SEdiagNbr
 - SquareSubdivisionMeshV2::PixelInfo, 135
- set_boundary
 - tk::spline, 152
- set_coeffs_from_b
 - tk::spline, 152
- set_points
 - tk::spline, 152
- SetLoopMessageFrequency
 - InputOutput.cpp, 222
 - InputOutput.h, 225
- SetNewInitialConditions
 - ViewScreen, 187
- SetOutputLevel
 - InputOutput.cpp, 222
 - InputOutput.h, 225
- SettingError
 - Config, 12
- SettingName
 - ConfigReader::ConfigCollection::ConfigSetting, 47
- SettingValue
 - ConfigReader::ConfigCollection::ConfigSetting, 47
- SimpleSquareMesh, 140
 - EndCurrentLoop, 141
 - GeodesicFinished, 141
 - getCurNrGeodesics, 142
 - getFullDescriptionStr, 142
 - getNewInitConds, 142
 - IsFinished, 142
 - m_Finished, 143
 - m_RowColumnSize, 143
 - m_TotalPixels, 143
 - SimpleSquareMesh, 141
- Singularities
 - GeneralSingularityTermOptions, 80
- Singularity
 - Geometry.h, 214
- SingularityCoord
 - Geometry.h, 214
- SingularityMetric, 143
 - getSingularities, 145
 - m_AllSingularities, 145
 - SingularityMetric, 144

- SingularityToString
 - GeneralSingularityTermination, 78
- SmallestPossibleStepsize
 - Integrators, 18
- Source, 145
 - ~Source, 146
 - getFullDescriptionStr, 146
 - getSource, 146
 - m_theMetric, 147
 - Source, 146
- Spacelike
 - ViewScreen.h, 256
- SphereRadius
 - BoundarySphereTermOptions, 31
- SphericalHorizonMetric, 147
 - getHorizonRadius, 149
 - m_HorizonRadius, 149
 - SphericalHorizonMetric, 148
- spline
 - tk::spline, 151
- spline_type
 - tk::spline, 150
- SquareSubdivisionMesh, 154
 - EndCurrentLoop, 156
 - Explnt, 156
 - GeodesicFinished, 157
 - getCurNrGeodesics, 157
 - getFullDescriptionStr, 157
 - getNewInitConds, 157
 - InitializeFirstGrid, 158
 - IsFinished, 158
 - m_AllPixels, 159
 - m_CurrentPixelQueue, 159
 - m_CurrentPixelQueueDone, 159
 - m_InfinitePixels, 159
 - m_InitialPixels, 159
 - m_InitialSubDivideToFinal, 159
 - m_IterationPixels, 160
 - m_MaxPixels, 160
 - m_MaxSubdivide, 160
 - m_PixelsLeft, 160
 - m_RowColumnSize, 160
 - SquareSubdivisionMesh, 156
 - SubdivideAndQueue, 158
 - UpdateAllNeighbors, 158
 - UpdateAllWeights, 158
- SquareSubdivisionMesh::PixelInfo, 132
 - DiagValue, 133
 - Index, 133
 - LowerNbrIndex, 133
 - PixelInfo, 133
 - RightNbrIndex, 133
 - SubdivideLevel, 133
 - Weight, 133
- SquareSubdivisionMeshV2, 161
 - EndCurrentLoop, 163
 - Explnt, 163
 - GeodesicFinished, 164
 - getCurNrGeodesics, 164
 - GetDown, 164
 - getFullDescriptionStr, 164
 - GetLeft, 165
 - getNewInitConds, 165
 - GetRight, 165
 - GetUp, 166
 - InitializeFirstGrid, 166
 - IsFinished, 166
 - m_ActivePixels, 167
 - m_AllPixels, 167
 - m_CurrentPixelQueue, 167
 - m_CurrentPixelQueueDone, 167
 - m_CurrentPixelUpdating, 167
 - m_InfinitePixels, 168
 - m_InitialPixels, 168
 - m_InitialSubDivideToFinal, 168
 - m_IterationPixels, 168
 - m_MaxPixels, 168
 - m_MaxSubdivide, 168
 - m_PixelsIntegrated, 168
 - m_PixelsLeft, 168
 - m_RowColumnSize, 169
 - SquareSubdivisionMeshV2, 163
 - SubdivideAndQueue, 166
 - UpdateAllWeights, 167
- SquareSubdivisionMeshV2::PixelInfo, 134
 - DiagValue, 135
 - DownNbr, 135
 - Index, 135
 - LeftNbr, 135
 - PixelInfo, 135
 - RightNbr, 135
 - SEdiagNbr, 135
 - SubdivideLevel, 135
 - UpNbr, 136
 - Weight, 136
- ST3CrMetric, 169
 - f_om_phi, 171
 - f_phi, 171
 - get_omega, 172
 - getFullDescriptionStr, 172
 - getMetric_dd, 172
 - getMetric_uu, 173
 - m_lambda, 173
 - m_P, 173
 - m_q0, 173
 - ST3CrMetric, 171
- SubdivideAndQueue
 - SquareSubdivisionMesh, 158
 - SquareSubdivisionMeshV2, 166
- SubdivideLevel
 - SquareSubdivisionMesh::PixelInfo, 133
 - SquareSubdivisionMeshV2::PixelInfo, 135
- Term
 - Terminations.h, 248
- Term_BoundarySphere
 - Terminations.h, 249

- Term_GeneralSingularity
 - Terminations.h, [249](#)
- Term_Horizon
 - Terminations.h, [249](#)
- Term_NaN
 - Terminations.h, [249](#)
- Term_None
 - Terminations.h, [249](#)
- Term_ThetaSingularity
 - Terminations.h, [249](#)
- Term_TimeOut
 - Terminations.h, [249](#)
- TermBitflag
 - Terminations.h, [248](#)
- Termination, [174](#)
 - ~Termination, [174](#)
 - CheckTermination, [175](#)
 - DecideUpdate, [175](#)
 - getFullDescriptionStr, [175](#)
 - m_OwnerGeodesic, [175](#)
 - m_StepsSinceUpdated, [175](#)
 - Reset, [175](#)
 - Termination, [174](#)
- TerminationOptions, [176](#)
 - ~TerminationOptions, [176](#)
 - TerminationOptions, [176](#)
 - UpdateEveryNSteps, [177](#)
- Terminations.cpp
 - CreateTerminationVector, [246](#)
- Terminations.h
 - BoundarySphere, [248](#)
 - Continue, [248](#)
 - CreateTerminationVector, [248](#)
 - GeneralSingularity, [248](#)
 - Horizon, [248](#)
 - Maxterms, [248](#)
 - NaN, [248](#)
 - Term, [248](#)
 - Term_BoundarySphere, [249](#)
 - Term_GeneralSingularity, [249](#)
 - Term_Horizon, [249](#)
 - Term_NaN, [249](#)
 - Term_None, [249](#)
 - Term_ThetaSingularity, [249](#)
 - Term_TimeOut, [249](#)
 - TermBitflag, [248](#)
 - TerminationUniqueVector, [248](#)
 - ThetaSingularity, [248](#)
 - TimeOut, [248](#)
 - Uninitialized, [248](#)
- TerminationUniqueVector
 - Terminations.h, [248](#)
- TermOptions
 - BoundarySphereTermination, [29](#)
 - GeneralSingularityTermination, [79](#)
 - HorizonTermination, [101](#)
 - NaNTermination, [129](#)
 - ThetaSingularityTermination, [179](#)
 - TimeOutTermination, [182](#)
- TheEmissionModel
 - EquatorialEmissionOptions, [59](#)
- TheFluidVelocityModel
 - EquatorialEmissionOptions, [59](#)
- ThetaSingEpsilon
 - ThetaSingularityTermOptions, [180](#)
- ThetaSingularity
 - Terminations.h, [248](#)
- ThetaSingularityTermination, [177](#)
 - CheckTermination, [178](#)
 - getFullDescriptionStr, [178](#)
 - TermOptions, [179](#)
 - ThetaSingularityTermination, [178](#)
- ThetaSingularityTermOptions, [179](#)
 - ThetaSingEpsilon, [180](#)
 - ThetaSingularityTermOptions, [180](#)
- theUpdateFrequency
 - DiagnosticOptions, [51](#)
- ThreeIndex
 - Geometry.h, [215](#)
- Threshold
 - EquatorialPassesOptions, [64](#)
- Timelike
 - ViewScreen.h, [256](#)
- TimeOut
 - Terminations.h, [248](#)
- TimeOutTermination, [180](#)
 - CheckTermination, [181](#)
 - getFullDescriptionStr, [181](#)
 - m_CurNrSteps, [182](#)
 - Reset, [182](#)
 - TermOptions, [182](#)
 - TimeOutTermination, [181](#)
- TimeOutTermOptions, [182](#)
 - MaxSteps, [183](#)
 - TimeOutTermOptions, [183](#)
- tk, [19](#)
- tk::internal, [19](#)
- tk::internal::band_matrix, [21](#)
 - ~band_matrix, [22](#)
 - band_matrix, [21](#)
 - dim, [22](#)
 - l_solve, [22](#)
 - lu_decompose, [22](#)
 - lu_solve, [22](#)
 - m_lower, [23](#)
 - m_upper, [23](#)
 - num_lower, [22](#)
 - num_upper, [22](#)
 - operator(), [22](#)
 - r_solve, [23](#)
 - resize, [23](#)
 - saved_diag, [23](#)
- tk::spline, [149](#)
 - bd_type, [150](#)
 - cspline, [151](#)
 - cspline_hermite, [151](#)

- deriv, 151
- find_closest, 151
- first_deriv, 150
- get_x, 151
- get_x_max, 151
- get_x_min, 151
- get_y, 152
- linear, 151
- m_b, 152
- m_c, 152
- m_c0, 153
- m_d, 153
- m_left, 153
- m_left_value, 153
- m_made_monotonic, 153
- m_right, 153
- m_right_value, 153
- m_type, 153
- m_x, 153
- m_y, 153
- make_monotonic, 152
- operator(), 152
- second_deriv, 150
- set_boundary, 152
- set_coeffs_from_b, 152
- set_points, 152
- spline, 151
- spline_type, 150
- toString
 - Geometry.h, 217, 218
- trace
 - ConfigReader::ConfigReaderException, 46
- TwoIndex
 - Geometry.h, 215
- Uninitialized
 - Terminations.h, 248
- Update
 - Geodesic, 85
- UpdateAllNeighbors
 - SquareSubdivisionMesh, 158
- UpdateAllWeights
 - SquareSubdivisionMesh, 158
 - SquareSubdivisionMeshV2, 167
- UpdateData
 - ClosestRadiusDiagnostic, 34
 - Diagnostic, 50
 - EquatorialEmissionDiagnostic, 56
 - EquatorialPassesDiagnostic, 62
 - FourColorScreenDiagnostic, 71
 - GeodesicPositionDiagnostic, 95
- UpdateEveryNSteps
 - TerminationOptions, 177
- UpdateFinish
 - UpdateFrequency, 185
- UpdateFrequency, 185
 - UpdateFinish, 185
 - UpdateNSteps, 185
 - UpdateStart, 185
- UpdateNSteps
 - UpdateFrequency, 185
- UpdateStart
 - UpdateFrequency, 185
- UpNbr
 - SquareSubdivisionMeshV2::PixelInfo, 136
- Utilities, 19
 - GetDiagNameStrings, 19
 - GetFirstLineInfoString, 19
 - GetTimeStampString, 20
- Utilities::Timer, 184
 - Clock, 184
 - elapsed, 184
 - m_beg, 185
 - reset, 184
 - Second, 184
- VerletVelocityTolerance
 - Integrators, 18
- ViewScreen, 186
 - ConstructVielbein, 187
 - EndCurrentLoop, 187
 - GeodesicFinished, 187
 - getCurNrGeodesics, 187
 - getFullDescriptionStr, 187
 - IsFinished, 187
 - m_Direction, 187
 - m_GeodType, 187
 - m_Metric_dd, 188
 - m_Pos, 188
 - m_rLogScale, 188
 - m_ScreenCenter, 188
 - m_ScreenSize, 188
 - m_theMesh, 188
 - m_theMetric, 188
 - m_Vielbein, 188
 - SetNewInitialConditions, 187
 - ViewScreen, 186
- ViewScreen.h
 - GeodesicType, 256
 - Null, 256
 - Spacelike, 256
 - Timelike, 256
- Weight
 - SquareSubdivisionMesh::PixelInfo, 133
 - SquareSubdivisionMeshV2::PixelInfo, 136
- WriteCachedOutputToFile
 - GeodesicOutputHandler, 90